

ПРИМЕНЕНИЕ ТЕХНОЛОГИЙ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА И ПОДДЕРЖКИ ПРИНЯТИЯ РЕШЕНИЙ В ЗАДАЧАХ ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ



Ростов–на–Дону
2022

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ПРИМЕНЕНИЕ ТЕХНОЛОГИЙ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА И ПОДДЕРЖКИ ПРИНЯТИЯ РЕШЕНИЙ В ЗАДАЧАХ ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ

Коллективная монография

*Под общей редакцией профессора кафедры «Кибербезопасность
информационных систем», д.ф.–м.н. Л.В. Черкесовой*

Ростов–на–Дону
ДГТУ
2022

УДК 004.056.5

ББК 32.97

П 54

Рецензенты:

доктор технических наук, профессор, заслуженный деятель науки РФ,
заместитель директора по научной работе Ростовского филиала
Российской таможенной Академии *Д.А. Безуглов*

доктор технических наук, профессор, заместитель директора по науке
Ростовского научно–исследовательского института радиосвязи *Д.Д. Габриэльян*

Авторы:

Короченцев Д.А., Черкесова Л.В., Болдырихин Н.В., Костоглотов А.А.,
Куликова О.В., Ревякина Е.А., Савельев В.А., Сафарьян О.А., и другие

**П 54 Применение технологий искусственного интеллекта и поддержки
 принятия решений в задачах информационной безопасности:**

Коллективная монография / Д.А. Короченцев, Л.В. Черкесова,
Н.В. Болдырихин, А.А. Костоглотов, О.В. Куликова, Е.А. Ревякина,
В.А. Савельев, О.А. Сафарьян [и др.]; под общей редакцией Л.В.
Черкесовой; Донской государственный технический университет. –
Ростов–на–Дону: ДГТУ, 2022. – 570 с.

ISBN ??? – ? – ???? – ???? – ?

Представлены проекты решения и компьютерной реализации задач
информационной безопасности и защиты данных, с применением технологий
искусственного интеллекта и методов поддержки принятия решений.

Коллективная монография предназначена для студентов, дипломников,
магистрантов, аспирантов и докторантов кафедры «Кибербезопасность
информационных систем» факультета «Информатика и вычислительная
техника» ДГТУ, а также для широкого круга заинтересованных читателей.

УДК 004.056.5

ББК 32.97

Печатается по решению редакционно–издательского совета
Донского государственного технического университета

© Короченцев Д.А., Черкесова Л.В., Болдырихин Н.В.,
Костоглотов А.А., Куликова О.В., Ревякина Е.А.,
Савельев В.А., Сафарьян О.А., и др. 2022 г.

© Донской государственный
технический университет, 2022

ВВЕДЕНИЕ

На практике, современный специалист по информационной безопасности, работающий даже в небольшой организации, физически не может выполнить все необходимые защитные мероприятия уже на этапе мониторинга, когда ему необходимо, практически в реальном времени, изучать и анализировать огромное количество логов (файлов регистрации, протоколов или журналов). При этом на остальные действия у него времени уже не остаётся, а файлы логов поступают постоянно. Рабочий день длится не круглосуточно, и даже если специалисты по защите информации работают в несколько смен (а это может позволить себе далеко не каждая организация), их силы – не бесконечны. Вручную анализировать логи, с большой эффективностью, специалист может не более 15–20% рабочего времени, а значит, его задача на 100% не выполняется. Конечно, существует программное обеспечение, помогающие в этой нелегкой работе. Это ПО собирает в одно окно логи от различных систем, и отбрасывает логи, не представляющие интереса. Такие программы могут провести примитивную аналитику, и помочь специалисту, обратив внимание на подозрительные события информационной безопасности. Однако для устранения всё возрастающих угроз безопасности информации, и для решения задач по защите данных, возникающих в России в XXI веке, их явно недостаточно.

Применение технологий машинного обучения в кибербезопасности сегодня очень востребовано, поскольку на специалиста в области информационной безопасности и защиты данных (который должен анализировать состояние защищённости информационной системы, обнаруживать атаки и своевременно их нейтрализовывать, производить анализ возможных уязвимостей и устранять их, наполнять базы знаний о *киберугрозах* и *кибератаках*, заниматься, в том числе, *киберразведкой*, чтобы быть подготовленным к *кибертеррористическим атакам* заранее и эффективно их отражать), в настоящее время обрушивается практически непосильный груз работы и ответственности.

Давно не секрет, что многие интеллектуальные технологии (например, изменение изображения лица одного человека на лицо другого, изменение тембра голоса, имитация голоса другого человека и др.) давно используются злоумышленниками для достижения своих преступных целей путём обмана своих жертв – доверчивых людей, которыми они манипулируют, применяя психологические методы социальной инженерии, чтобы ввести их в заблуждение.

Актуальность научных исследований авторов обусловлена необходимостью разработки систем информационной безопасности с функциями машинного (глубинного) обучения для *противодействия* таким *кибертеррористическим действиям* и для организации *кибероборонительных мероприятий*. Выполнять свой необходимый функционал операций нейронные сети могут благодаря предварительному обучению на заранее размеченных банках данных – это могут быть тексты, изображения или определённые сигналы, полученные с помощью датчиков. Нейронная сеть, проходя обучение, способна научиться выполнять заданную операцию для дальнейшего самостоятельного выполнения.

Технология машинного обучения позволит автоматизировать часть работы специалиста–кибербезопасника: анализ защищённости информационной системы, логов событий информационной безопасности, исходных кодов информационных систем, больших данных (big data); данных, полученных в ходе киберразведки, и многих других задач, став незаменимым подспорьем для специалистов в области информационной безопасности и защите данных.

Нужно отметить, что применение интеллектуальных технологий не станет окончательным решением в области кибербезопасности. Одна нейронная сеть не способна устранить все проблемы, и для выполнения этой крайне объёмной и сложной задачи её необходимо разбить на подзадачи. Если нужно, разбивать задачу следует до тех пор, пока не станет возможным автоматизировать её целиком, применяя весь спектр технологий машинного обучения. Комплекс самообучающихся нейронных сетей призван работать, по аналогии с целым отделом специалистов в области кибербезопасности, оставляя человеку только функцию контроля за корректностью работы нейронных сетей.

В предлагаемой коллективной монографии показан ряд проектов, связанных с применением современных технологий искусственного интеллекта и теории поддержки принятия решений в задачах информационной безопасности.

Монография предназначена как для студентов – дипломников ДГТУ: бакалавров, магистрантов и специалистов, так и для аспирантов и докторантов кафедры «Кибербезопасность информационных систем» факультета ИВТ. Она может быть полезна и для студентов ДГТУ других факультетов, желающих познакомиться с примерами использования интеллектуальных информационных технологий и методов теории поддержки принятия решений в области информационной безопасности и защиты данных, а также для студентов других технических университетов Российской Федерации и дружественных ей стран.

1. КРИПТОГРАФИЧЕСКИЙ ПРОТОКОЛ С ОТКРЫТЫМ КЛЮЧОМ ДИФФИ–ХЕЛЛМАНА С ПРИМЕНЕНИЕМ НЕЙРОННОЙ СЕТИ

В криптоанализе часто используется способность нейронных сетей к исследованию пространства решений, что даёт, в том числе, возможность создавать новые типы атак на существующие алгоритмы шифрования. Как известно, любая функция может быть представлена нейронной сетью. При этом используются такие свойства нейронных сетей, как самообучение, взаимное обучение, стохастическое поведение, а также низкая чувствительность к шуму, неточностям и помехам (искажению данных, весовых коэффициентов, ошибок в программе). Использование нейронных сетей позволяет решить многие проблемы, существующие в традиционной криптографии с открытым ключом: при распределении ключей, хешировании и генерации псевдослучайных чисел.

Одной из областей применения нейронных сетей является обмен ключами, как важнейший этап конфиденциальной передачи информации. Протокол Диффи–Хеллмана, основанный на теории чисел, сможет сохранить свою криптостойкость лишь до тех пор, пока у атакующего хакера не обнаружится вычислительная мощь квантового компьютера. В отличие от известных алгоритмов, нейронная сеть работает иначе – в её основе лежат многоуровневые перцептроны, находящиеся на двух концах незащищенного канала связи. Закрытый ключ в этом случае представлен каждой стороны в виде начальных весов связей перцептрона. Новая область криптографии, *нейрокриптография*, имеет достоинства и перспективы по сравнению с традиционными методами.

Объектом исследования здесь является нейрокриптография, *предметом* – криптографические алгоритмы, использующие искусственные нейронные сети. *Целью исследования* является программная реализация алгоритма обмена ключами с использованием искусственных нейронных сетей. Для достижения поставленной цели следует решить *следующие задачи*: анализ предметной области; изучить уже существующие алгоритмы обмена ключами, выявить достоинства и недостатки; рассмотреть основные принципы построения протокола обмена закрытым ключом; разработать программную реализацию протокола Диффи–Хеллмана на основе древовидной машины чётности (ДМЧ).

1.1. Теоретические основы создания нейронных сетей

Искусственная нейронная сеть (ИНС) представляет собой математическую модель, а также её программное или аппаратное воплощение, построенные по принципу организации и функционирования биологических нейронных сетей — сетей нервных клеток живого организма. ИНС является системой соединённых и взаимодействующих между собой простых процессоров (искусственных нейронов). Эти процессоры достаточно просты, особенно в сравнении с процессорами, используемыми в персональных компьютерах, и каждый процессор подобной сети имеет дело только с сигналами, которые он периодически получает, и сигналами, которые он сам периодически посылает другим процессорам. И, тем не менее, будучи соединёнными в достаточно большую сеть с управляемым взаимодействием, такие по отдельности простые процессоры вместе способны выполнять довольно сложные задачи.

Нейронные сети не программируются в привычном смысле этого слова — они обучаются. Возможность глубокого обучения — это одно из главных преимуществ нейронных сетей перед традиционными алгоритмами. Технически обучение заключается в нахождении коэффициентов связей между нейронами. В процессе обучения нейронная сеть способна выявлять сложные зависимости между входными и выходными данными, а также выполнять их обобщение. Это значит, что, в случае успешного обучения, сеть сможет вернуть верный результат на основании данных, которые отсутствовали в обучающей выборке, а также неполных и / или «зашумленных», частично искажённых данных.

Нейронная сеть представляет собой связь нейронов как вычислительных единиц, которые получают информацию, производят над ней простые вычисления и передают её дальше. Они делятся на три основных типа: входные, скрытые и выходные. Существуют также нейроны смещения и контекстные нейроны. В том случае, когда нейросеть состоит из большого количества нейронов, вводится понятие *слоя*. Соответственно, есть входной слой (зелёный), который получает информацию, несколько скрытых слоев (голубой цвет), которые ее обрабатывают, и выходной слой (жёлтый цвет), который выводит результат (рис. 1.1). У каждого из нейронов имеется два основных параметра: входные и выходные данные. На вход попадает сумма произведения всех нейронов с предыдущего слоя и их веса, после чего она нормализуется, с помощью функции активации и попадает на выход.

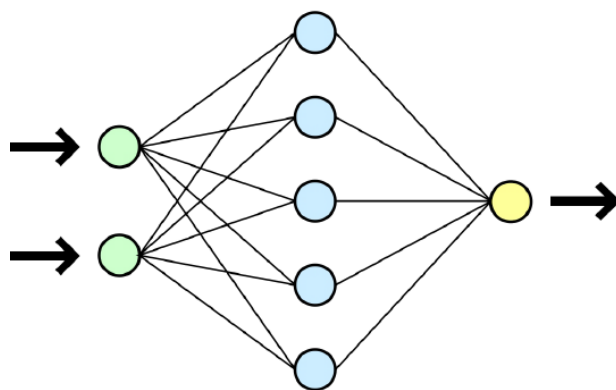


Рисунок 1.1. Схема нейронной сети

Искусственные нейронные сети используются для решения сложных интеллектуальных задач, требующих выполнения аналитических вычислений, подобных тем, которые происходят в человеческом мозге [1].

Самыми распространенными применениями нейронных сетей является:

1) *Распознавание образов и классификация.* В качестве образов могут выступать различные по своей природе объекты: символы текста, изображения, образцы звуков и т. д. При обучении сети предлагаются различные образцы таких образов, с указанием того, к какому классу они относятся. Образец, как правило, представляется как вектор значений соответствующих признаков. При этом совокупность всех признаков должна однозначно определять класс, к которому относится образец. В случае, если признаков недостаточно, сеть может соотнести один и тот же образец с несколькими классами, что неверно. По окончании обучения сети ей можно предъявлять неизвестные ранее образы и получать ответ о принадлежности к определённому классу.

Топология такой сети характеризуется тем, что количество нейронов в выходном слое, как правило, равно количеству определяемых классов. При этом устанавливается соответствие между выходом нейронной сети и классом, который он представляет. Когда сети предъявляется некий образ, на одном из её выходов должен появиться признак того, что образ принадлежит этому классу. В то же время на других выходах должен быть признак того, что образ данному классу не принадлежит. Если на двух или более выходах есть признак принадлежности к классу, считается, что сеть «не уверена» в своём ответе.

2) *Прогнозирование.* Способности нейронной сети к прогнозированию напрямую следуют из её способности к обобщению и выделению скрытых зависимостей между входными и выходными данными.

После обучения сеть способна предсказать будущее значение некой последовательности на основе нескольких предыдущих значений и (или) каких-то существующих в настоящий момент факторов. Следует отметить, что прогнозирование возможно только тогда, когда предыдущие изменения действительно в какой-то степени предопределяют будущие. Например, прогнозирование котировок акций, на основе котировок за прошлую неделю, может оказаться успешным (а может и не оказаться), тогда как прогнозирование результатов завтрашней лотереи на основе данных за последние 50 лет – почти наверняка не даст никаких результатов.

3) *Кластеризация*. Под этим термином понимается разбиение множества входных сигналов на классы, при том, что ни количество, ни признаки классов заранее не известны. После обучения, такая сеть способна определять, к какому классу относится входной сигнал. Сеть также может сигнализировать о том, что входной сигнал не относится ни к одному из выделенных классов — это является признаком новых, отсутствующих в обучающей выборке, данных. Таким образом, подобная нейронная сеть может выявлять новые, неизвестные ранее классы сигналов. Соответствие между классами, выделенными сетью, и классами, существующими в предметной области, устанавливается человеком.

4) *Аппроксимация*. Нейронные сети могут аппроксимировать непрерывные функции. Доказана обобщённая аппроксимационная теорема: с помощью линейных операций и каскадного соединения можно из произвольного нелинейного элемента получить устройство, вычисляющее любую непрерывную функцию, с некоторой, наперёд заданной точностью. Это означает, что нелинейная характеристика нейрона может быть произвольной: от *сигмоидальной* до *произвольного волнового пакета*, или *вейвлета*, *синуса* или *многочлена*. От выбора нелинейной функции может зависеть сложность конкретной сети, но с любой нелинейностью сеть остаётся универсальным аппроксиматором, и при правильном выборе структуры может достаточно точно аппроксимировать функционирование любого непрерывного автомата.

5) *Сжатие данных и ассоциативная память*. Способность нейросетей к выявлению взаимосвязей между различными параметрами дает возможность выразить данные большой размерности более компактно, если они тесно взаимосвязаны друг с другом. Обратный процесс – восстановление исходного набора данных из части информации – называется (авто)ассоциативной памятью.

Ассоциативная память позволяет также восстанавливать исходный сигнал / образ из зашумленных / поврежденных входных данных. Решение задачи гетероассоциативной памяти реализует память, адресуемую по содержимому.

б) *Нейрокриптография*. В криптоанализе используется способность нейронных сетей исследовать пространство решений. Имеется возможность создавать новые типы атак на существующие алгоритмы шифрования, основанные на том, что любая функция может быть представлена нейронной сетью. Взломав алгоритм, теоретически можно найти решение. При этом используются свойства нейронных сетей – самообучение, взаимное обучение, стохастическое поведение, низкая чувствительность к неточностям, шуму и помехам (искажениям данных, весовых коэффициентов, ошибок в программе). Они позволяют решать проблемы криптографии с открытым ключом, распределения ключей, хеширования и генерации псевдослучайных чисел.

Искусственные нейронные сети имеют различную структуру, которая составляется в зависимости от типа поставленной задачи [2]:

- *сеть прямого распространения* (все нейронные связи направлены строго в направлении от входных нейронов к выходным);
- *рекуррентная сеть* (сигнал с выходных нейронов или нейронов скрытого слоя частично передается обратно, на входы нейронов входного слоя);
- *радиально–базисная функция* (единственный скрытый слой; где только нейроны скрытого слоя имеют нелинейную активационную функцию; синаптические веса связей входного и скрытого слоев равны единице);
- *самоорганизующаяся карта* (соревновательная нейронная сеть с обучением без учителя, выполняющая задачу визуализацию и кластеризацию).

1.2. Взаимодействующие нейронные сети

Искусственные нейронные сети обучаются на примерах. Это означает, что сеть «преподаватель» представляет пары *вход–выход* высокоразмерных данных, а сеть «студент» обучается на этих данных. Синаптические веса адаптируются простыми правилами парам входа–выхода. Когда сети — «преподаватель», так же, как и «студент» — имеют N весовых коэффициентов, для получения способности обобщения, процесс обучения требует N примеров. После стадии машинного обучения, сеть «студент» достигает некоторого «перекрытия» сети «преподаватель», и их векторы весовых коэффициентов коррелируются.

Как следствие, сеть «студент» может классифицировать входной образец, не принадлежащий к обучающей выборке. Средняя ошибка классификации входных образцов будет уменьшаться с возрастанием числа примеров обучения.

Обучение может быть выполнено в двух различных режимах: *пакетном* и *интерактивном*. В первом случае все примеры сохраняются и используются для минимизации общей ошибки обучения. Во втором случае только один новый пример используется на каждом шаге и затем удаляется. Поэтому интерактивное обучение можно рассматривать как динамический процесс: на каждом шаге преподаватель создает новый пример, который студент использует, чтобы произвести малые изменения весовых коэффициентов.

Используя динамическое обучение, нейронные сети могут произвести секретный ключ, обмениваясь битами по секретному каналу связи, и обучая друг друга. На каждом шаге обучения две сети получают общий случайный входной вектор и обучаются по их взаимным битам выхода (происходит синхронизация взаимным обучением). Если скорость обучения η является большой, и векторы весовых коэффициентов остаются нормализованными, то эти две сети стремятся к антипараллельному положению.

Два персептрона А и В получают общий случайный входной N – мерный вектор x и изменяют свой N –мерный нормализованный вектор веса w согласно их взаимному биту σ . Персептроны показаны на рис. 1.2.

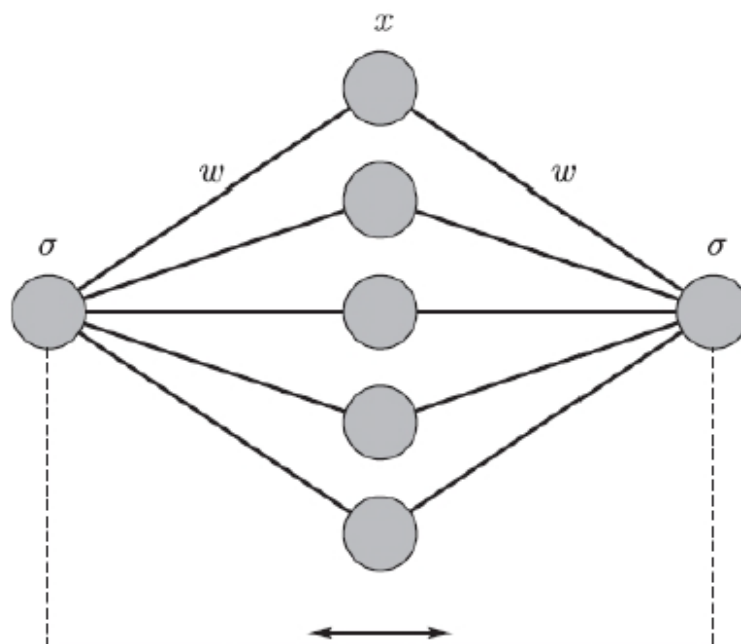


Рисунок 1.2. Процесс обучения персептрона

Бит выхода σ отдельного персептрона задается уравнением:

$$\sigma_i = \text{sign}(w_i \cdot x_i). \quad (1.1)$$

На каждом шаге обучения t общий случайный входной вектор предоставляется двум сетям, которые генерируют два бита выхода σ^A и σ^B согласно формуле (1). Векторы весовых коэффициентов модифицируются следующим правилом обучения персептрона:

$$w^A(t+1) = w^A(t) + \frac{\eta}{N} x \sigma^B \theta(-\sigma^A \sigma^B). \quad (1.2)$$

$$w^B(t+1) = w^B(t) + \frac{\eta}{N} x \sigma^A \theta(-\sigma^A \sigma^B), \quad (1.3)$$

где $\theta(x)$ —функция шага. Таким образом, шаг обучения выполняется со скоростью η , если биты выходов персептронов различны. После каждого шага обучения векторы весов нормализуются. Две нейронных сети могут синхронизировать друг друга взаимным обучением. Обе сети обучаются на примерах, сгенерированных их партнёром, и, наконец, получают антипараллельное выравнивание (4).

$$\sigma^A = -\sigma^B. \quad (1.4)$$

Противник, который знает набор пар входа – выхода, может получить веса двух партнёров после синхронизации. Очевидно, один персептрон передаёт слишком много информации. Поэтому каждый из абонентов должен скрыть как можно больше информации от противника, но и передать достаточное количество данных для другого абонента, чтобы синхронизировать веса.

1.3. Протокол Диффи–Хеллмана на базе синхронизации двух ДМЧ

Для обмена ключами между двумя абонентами часто используется алгоритм (протокол) Диффи–Хеллмана. Синхронизация двух древовидных машин чётности (ДМЧ) позволяет заменить его замену на более безопасную и криптостойкую версию. Синхронизация этих машин чётности похожа на синхронизацию двух хаотических осцилляторов в теории хаотических связей.

Древовидная машина чётности—это особый вид многоуровневой нейронной сети прямого распространения. Её структура показана на рис. 1.3.

ДМЧ состоит из K скрытых нейронов, которые являются *персептронами*. Каждый из них имеет N входных нейронов и один выходной нейрон.

Входные нейроны принимают следующие двоичные значения:

$$x_{i,j} \in \{-1, +1\}, \quad (1.5)$$

а матрица весов w размерностью $K \times N$ принимает дискретные значения двоичных величин входных нейронов в диапазоне от $-L$ до $+L$,

$$w_{i,j} \in \{-L, -L+1, \dots, +L\}. \quad (1.6)$$

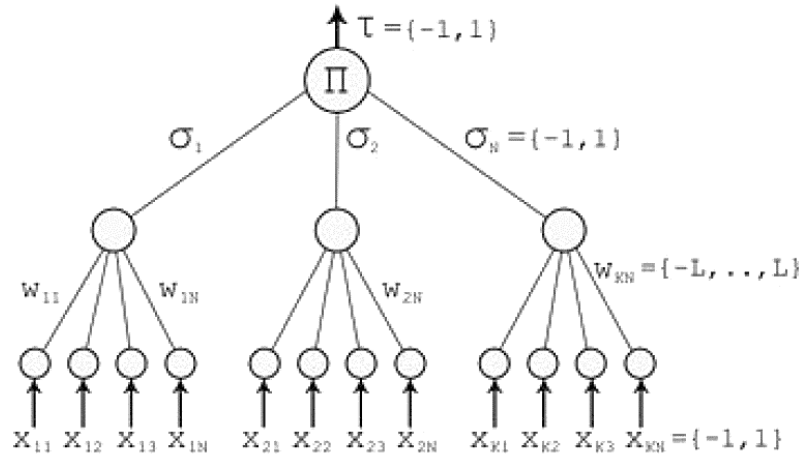


Рисунок 1.3. Структура древовидной машины чётности

Значение каждого скрытого нейрона есть σ (сигма) от суммы произведений входного значения и весового коэффициента:

$$\sigma_i = \text{sgn} \left(\sum_{j=1}^N w_{i,j} x_{i,j} \right). \quad (1.7)$$

Сигма равна 1, если сумма больше нуля и -1 , если меньше нуля. При случае, когда сумма равно нулю, сигма принимает значение -1 для обеспечения двоичности выходного значения. Значение выходного нейрона вычисляется, как произведение всех его скрытых слоёв:

$$\tau = \prod_{i=1}^k \sigma_i, \quad (1.8)$$

где τ принимает значение -1 при нечётном количестве $\sigma_i = -1$, и значение 1 при чётном количестве выходов скрытых нейронов, равных -1 .

В начале процесса синхронизации абоненты начинают со случайно заданными весами ДМЧ. На каждом этапе генерируется случайный входной вектор x_i размерностью $K \times N$ и подаётся на входы обеим древовидным машинам чётности, после чего вычисляются выходы машин. Абоненты передают выходные биты друг другу. Если они не равны, то веса ДМЧ не обновляются. В противном случае применяется правило обучения Хеббiana, где обе нейронные сети учатся друг у друга [1, 2]:

$$w_{i,j}^+ = g(w_{i,j} + x_{i,j} \cdot \tau \cdot \theta(\sigma_i \tau) \cdot \theta(\tau^A \tau^B)). \quad (1.9)$$

Функция $g(w)$ гарантирует, что веса ДМЧ останутся в допустимом диапазоне от $-L$ до $+L$. Если какой-либо из весов покинет эту область, функция сбросит его до ближайшего граничного значения $\pm L$. Операции повторяются до полной синхронизации двух абонентов.

Если к синхронизации подключится злоумышленник, имеющий такими же параметры ДМЧ, что и у абонентов, и при условии, что он может подслушивать сообщения, но не изменять их, то возможны три ситуации:

1) выходы ДМЧ не равны, и абоненты не обновляют веса; 2) все три выходных бита равны — все трое обновляют веса; 3) выходы равны только у абонентов, в этом случае они обновляют веса своих ДМЧ, а злоумышленник нет. Из-за этого он обучается медленнее, чем происходит синхронизация. Таким образом, злоумышленник может определить ключ лишь с очень малой вероятностью.

1.4. Безопасность протокола Диффи–Хеллмана, основанного на синхронизации двух древовидных машин чётности

Безопасный протокол обмена ключами должен иметь следующие свойства: любой атакующий хакер, которому известны все детали протокола и вся информации об обмене ключами между абонентами A и B , не должен иметь достаточной вычислительной мощности для нахождения секретного ключа.

Предполагается, что нападающему злоумышленнику E известна работа алгоритма, последовательность входных векторов и последовательности выходных битов. Атакующий хакер E может начать со всех $(2L+1)^{3N}$ начальных векторов весовых коэффициентов, и вычислить те, которые не противоречат последовательности входа / выхода. Как показано, все эти начальные состояния движутся к одному конечному вектору весов — уникальному ключу. Однако эта задача не представляется вычислительно осуществимой.

Следовательно, нужно найти алгоритм, который пытается адаптироваться к известным входам / выходам. Обучающее правило абонентов A и B имеет свойство: если пара элементов синхронизирована, то остается такой навсегда. Состояние синхронизации — аттрактор динамики обучения. Любой алгоритм, используемый нападающим E , также должен обладать данным свойством.

Очевидная предполагаемая атака: хакер E использует тот же самый алгоритм, что и один из абонентов, например, B . Если $\tau^A = \tau^B$, то изменяются векторы весовых коэффициентов E , для которых элементы σ_i^E одинаковы с τ^A .

Численное моделирование и аналитические вычисления показывают, что, фактически, атакующая сторона E синхронизируется с абонентами A и B после некоторого времени обучения t_{Learn} . Однако время обучения намного дольше, чем время синхронизации. В среднем, процесс обучения проходит в 1000 раз медленнее, чем процесс синхронизации легитимных абонентов с противником.

Поэтому, если процесс обучения будет остановлен сразу после процесса синхронизации, то нападающий противник не будет иметь возможности вычислить ключ. Ключ является безопасным для этого алгоритма атаки.

У злоумышленника не получится синхронизироваться с абонентами протокола, так как абонент *B* может влиять на сеть *A*, тогда как хакер *E* может только слушать. Синхронизация, как и обучение – это соревнование между притягивающим и отталкивающим управлениями битов выхода [1, 2]. Для параллельной машины вероятность отталкивания является намного большей для атакующего противника *E*, чем для абонентов *A* и *B*, очень близких к взаимной синхронизации. Но для простого персептрона это не так.

1.5. Обзор существующих протоколов обмена ключами

Использование одного ключа шифрования для зашифровки большого объёма информации приводит к очень большой нагрузке на ключ, что существенно облегчает задачу криптоаналитику – злоумышленнику (хакеру) *E*. Поэтому между абонентами следует периодически обновлять ключ. Для этих целей используются протоколы обмена ключами. Протоколы генерирует общий секретный ключ. Он используется для шифрования дальнейшего обмена с помощью алгоритмов симметричного шифрования. Существует несколько подобных алгоритмов, которые будут рассмотрены далее.

1.5.1. Протокол Диффи–Хеллмана

Протокол Диффи–Хеллмана представляет собой криптографический алгоритм, позволяющий двум и более сторонам получить общий секретный ключ, используя незащищенный от прослушивания канал связи. Полученный ключ используется для шифрования дальнейшего обмена с помощью алгоритмов симметричного шифрования. Для генерации секретного ключа необходимо, чтобы у каждого из абонентов был свой секретный элемент. Далее, по открытым каналам связи, передаётся некий общий параметр. Прodelывая одинаковые манипуляции со своим секретным и общим параметром вычисляется новый элемент. Стороны передают его друг другу, так же по открытым каналам связи, и с помощью своего секрета генерируют уже закрытый ключ, который будет одинаковым для обоих абонентов. Схема алгоритма, применённая в протоколе Диффи–Хеллмана, показана на рис. 1.4.

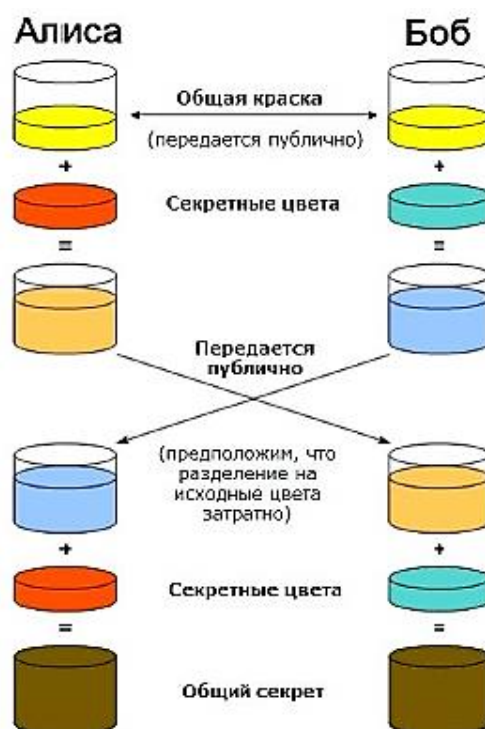


Рисунок 1.4. Схема протокола Диффи–Хеллмана

Криптостойкость алгоритма протокола Диффи–Хеллмана базируется на чрезвычайной трудности решения задачи дискретного логарифмирования в поле вычетов по большому простому модулю, а также на отсутствии у злоумышленника необходимой для этого решения вычислительной мощности.

Пусть легальных абонентов сети зовут Алиса и Боб, как это принято в традиционной криптографии. Описание протокола выглядит следующим образом: обоим абонентам известны некоторые два числа g и p , которые не являются секретными и могут быть известны также другим заинтересованным лицам. Для того, чтобы создать неизвестный более никому секретный ключ, оба абонента генерируют большие случайные числа: Алиса получает число a , а Боб – число b . Затем Алиса, по следующей формуле (10), вычисляет остаток от деления по модулю p , а затем пересылает полученный результат Бобу:

$$A = g^a \bmod p. \quad (1.10)$$

Боб, в свою очередь, по формуле (11) вычисляет остаток от деления по модулю p , а затем передаёт полученный результат Алисе:

$$B = g^b \bmod p. \quad (1.11)$$

Предполагается, что злоумышленник (Ева) теоретически может получить оба этих значения, но не сможет модифицировать их (так как у хакера нет возможности вмешаться в процесс передачи данных) [3].

На втором этапе Алиса, на основе имеющихся у неё значений A и a , а также полученных от Боба значений B и b , вычисляет значение по формуле:

$$B^a \bmod p = g^{ab} \bmod p. \quad (1.12)$$

Далее Боб, на основе известных ему значений B и b , а полученных от Алисы по сети значений A и a , вычисляет значение, используя формулу:

$$A^b \bmod p = g^{ab} \bmod p. \quad (1.13)$$

В итоге, у Алисы и Боба получается одно и то же число, которое они могут использовать в качестве общего (совместного) секретного ключа.

1.5.2. Трёхэтапный протокол Ади Шамира

Трёхэтапный протокол Шамира – это первый из криптографических трёхэтапных протоколов, разработанных Ади Шамиром в 1980 году. Его называют "протокол с тремя замками". Он позволяет двум сторонам безопасно обмениваться сообщениями, без обмена ключами шифрования. Отправитель и получатель обмениваются тремя зашифрованными сообщениями.

Существует стандартный безопасный шифр, который применяют в трёхэтапном протоколе Шамира. Выбирают достаточно большое число p , для которого $p-1$ имеет большой простой множитель. Абонент A выбирает число e , взаимно простое с числом $p-1$. Также существует число d такое, что

$$de = 1 \bmod (p-1), \quad (1.14)$$

где n – целое число. Шифруют сообщение M , с помощью функции:

$$E(e, M) = M^e \bmod p. \quad (1.15)$$

Получатель B аналогично выбирает целое число b , взаимно простое с $p-1$, и число c должно быть такое, что выполняется соотношение:

$$cb = 1 \bmod (p-1). \quad (1.16)$$

После этого он производит вычисление и отправляет обратно:

$$E(b, E(e, M)) = (M^e)^b \bmod p. \quad (1.17)$$

Сторона абонента A , получив сообщение, вычисляет функцию по формуле (18) и отправляет результат получателю:

$$D(d, M^{eb}) = (M^{eb})^d = M^{b(1+n(p-1))} = M^b M^{n(p-1)b} = M^b \bmod p. \quad (1.18)$$

Абонент B расшифровывает сообщение с помощью следующей формулы:

$$D(c, M^b) = M^{bc} \bmod p = M. \quad (1.19)$$

Таким образом, у обеих абонентов, Алисы и Боба, имеется секретный ключ. Схема протокола изображена на рис. 1.5.

1.5.3. Протокол защищенных переговоров

В отличие от описанных протоколов Диффи–Хеллмана и Шамира, протокол *защищенных переговоров* направлен на обеспечение конфиденциальности при передаче сведений о ключе легитимных абонентов, на защиту информации от некачественных паролей и атаки «человек в середине» [4].

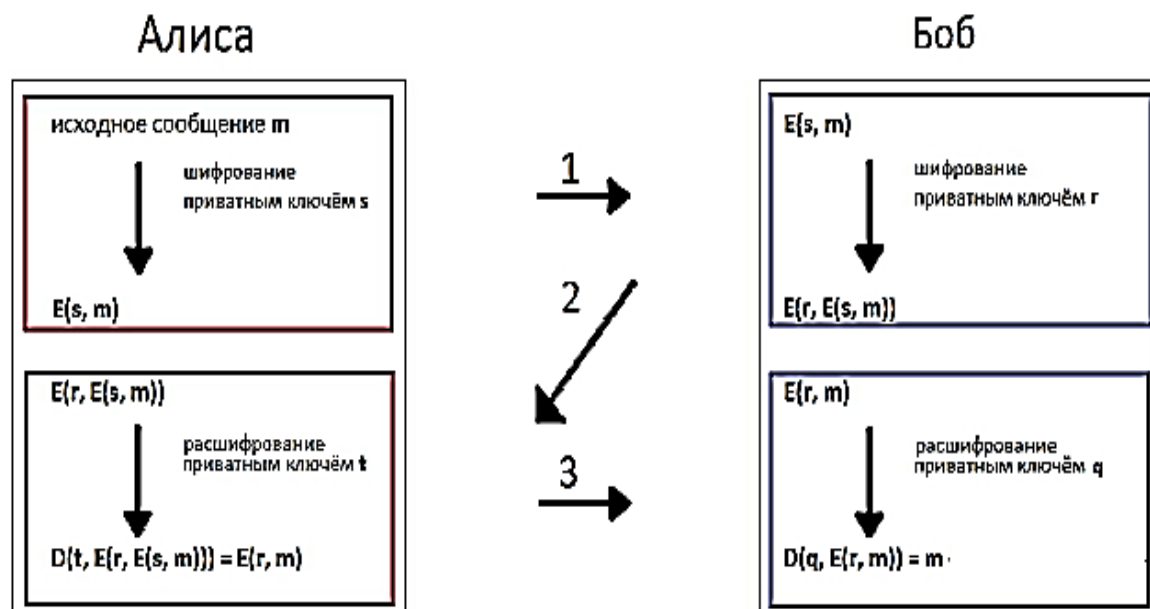


Рисунок 1.5. Схема трёхэтапного протокола Шамира

В этом протоколе используется хэш–функция (20) от двух переменных, обладающая необычным свойством – она часто приводит к коллизиям по первой переменной и почти никогда не приводит к коллизиям по второй переменной:

$$H'(x, y) = H(H(k, x) \bmod 2^m, x), \quad (1.20)$$

где $H(k, x)$ – обычная функция от двух переменных – k и x .

Предполагается, что Алиса и Боб используют общий секретный пароль P и уже обменялись секретным ключом K по протоколу Диффи–Хеллмана. Пароль P используется для проверки идентичности их сеансовых ключей, не позволяя злоумышленнику Еве проводить атаку «человек в середине».

Алиса вычисляет значение $H'(P, K)$ и отправляет его Бобу. В тоже время Боб также вычисляет значение $H'(P, K)$, и, если оно совпадает с тем, который прислала Алиса, отвечает ей сообщением $H'(H(P, K))$. Алиса вычисляет значение $H'(H(P, K))$ и сравнивает результат с тем, который прислал Боб.

При попытке реализовать атаку «человек в середине», злоумышленник Ева вынужден использовать разные ключи K_1 и K_2 для Алисы и Боба соответственно. При использовании предполагаемой хэш-функции, многие пароли будут давать одинаковый результат на ключе K_1 . Поэтому, когда злоумышленник находит совпадение, то, вероятнее всего, это неправильный пароль, и поэтому обмануть Боба Еве будет чрезвычайно трудно.

1.5.4. Сравнение протоколов обмена ключами

Передача ключа по открытым каналам связи стала большой проблемой криптографии XX века. Однако эту проблему удалось решить после появления протокола Диффи–Хеллмана. Этот алгоритм позволил дать ответ на главный вопрос: «Каким образом, при обмене зашифрованными посланиями, можно уйти от необходимости передачи секретного кода расшифровки, который, как правило, по объёму не меньше самого послания?» Открытое распространение ключей Диффи–Хеллмана позволяет любой паре пользователей компьютерной системы самим выработать общий секретный ключ, без необходимости обмениваться друг с другом секретными данными при его пересылке.

Криптографическая стойкость алгоритма Диффи–Хеллмана основана на предполагаемой сложности вычисления дискретного логарифма. Однако вычислительные мощности современных компьютеров с каждым годом всё увеличиваются. Ожидается скорое появление квантовых компьютеров, которые смогут вычислять сложнейшие задачи, основанные на проблемах теории чисел.

Нейронный протокол обмена ключей не основан на теории чисел. Он базируется на различии между однонаправленной и двунаправленной синхронизацией нейронных сетей. Таким образом, для протокола Диффи – Хеллмана, построенном на древовидных машинах чётности, увеличение вычислительных мощностей компьютеров не опасно. Тем самым, среди других протоколов он является весьма перспективным алгоритмом [5].

Рассмотрев возможности нейронных сетей и их использование в протоколе обмена ключами, а также протоколы обмена ключами, генерирующие общий секретный ключ между пользователями для шифрования дальнейшего обмена данными с помощью алгоритмов симметричного шифрования, авторы пришли к выводу о целесообразности использования искусственных нейронных сетей для реализации протокола Диффи–Хеллмана на древовидных машинах чётности.

1.6. Алгоритмическое конструирование программного средства создания общего закрытого ключа

1.6.1. Анализ входных и выходных данных программного средства

Для определения структуры древовидной машины чётности необходимо выделить её входные данные. В качестве параметров программы используются три переменные. Каждая из них имеет ограничения: переменные должны быть целочисленными, беззнаковыми и входить в указанный диапазон [6].

Количество нейронов скрытого слоя обозначается как k и имеет диапазон от 1 до 100. Количество нейронов, входящих в каждый нейрон скрытого слоя, обозначим n . Оно ограничивается также диапазоном от 1 до 100. Параметром r обозначим диапазон переменных весовой матрицы. Каждый элемент матрицы входит в диапазон от $-r$ до $+r$. Входное ограничение для r варьируется от 1 до 50. Рекомендуемые параметры для ДМЧ обозначаются как (3, 100, 3). При таких входных данных, количество комбинаций весовой матрицы равно

$$(2r + 1)^{nk} = 7^{300} \approx 3 \cdot 10^{253}. \quad (1.21)$$

На сегодняшний день взлом с помощью *грубой силы*, при таком количестве комбинаций, невозможен. Выходная информация в приложении будет представлена в виде графиков, текстовых сообщений, всплывающими окнами, подсказками и предупреждениями.

1.6.2. Структура работы ПО создания общего закрытого ключа

Разрабатываемое программное обеспечение архитектурно состоит из двух областей активности: клиентской части, которую запускает каждый пользователь на своём устройстве, и серверной части, которая должна быть развернута на одном из устройств. Взаимодействие этих частей происходит с помощью Интернет-соединения. Архитектура ПО показана на рис. 1.6.

Приложение выполняет аутентификацию пользователей; работу с учётными записями в базе данных; настройку параметров протокола обмена ключами; установку соединения между пользователями; генерацию общего секретного ключа; вывод информации об этом процессе; добавление информации о противнике, пытающемся синхронизироваться с участниками протокола генерации общего секретного ключа; вывод графиков синхронизации абонентов протокола и синхронизации противника с абонентами протокола.



Рисунок 1.6. Архитектура ПО создания общего закрытого ключа

В задачи серверной части ПО входят авторизация и аутентификация; проверка на равенство параметров двух ДМЧ; обеспечение обмена данными между двумя пользователями; а также информирование об ошибках.

Для работы приложения нужно запустить клиентскую часть, которая выступает в качестве абонента для создания общего закрытого ключа, и войти под заранее созданными логином и паролем, или зарегистрироваться заново.

После этого открывается главное окно, которое включает в себя поля для ввода параметров k , n , r , имени получателя, а также контроля и выбора одного из трёх правил обучения древовидной машины чётности (ДМЧ). С помощью верхней вкладки, содержащей имя конкретного пользователя, можно изменить логин и пароль, удалить пользователя или заменить его. Для создания общего секретного ключа с заданным абонентом нужно нажать на кнопку «Начать». При этом создаётся дополнительное окно, отвечающее за вывод информации о работе протокола, а именно за статус его выполнения, затраченное время на синхронизацию весов ДМЧ и сгенерированный общий секретный ключ. Далее, на сервер происходит запрос на выполнение протокола. Если второй абонент также посылает запрос, то после успешного сравнения сервером параметров древовидной машины чётности начинается выполнение протокола. Кроме того, в программе используется модель злоумышленника, который пытается синхронизировать свою ДМЧ с двумя другими. При полной синхронизации абонентов, используя веса ДМЧ, создается общий секретный ключ. Затем выводятся графики синхронизации весов абонентов: не только синхронизации абонентов между собой, но и синхронизации злоумышленника с абонентами.

Схема работы программы представлена на рис. 1.7.

1.6.3. Аутентификация пользователей

Для обеспечения более надёжной системы безопасности, при генерации общего секретного ключа используется аутентификация на сервере.

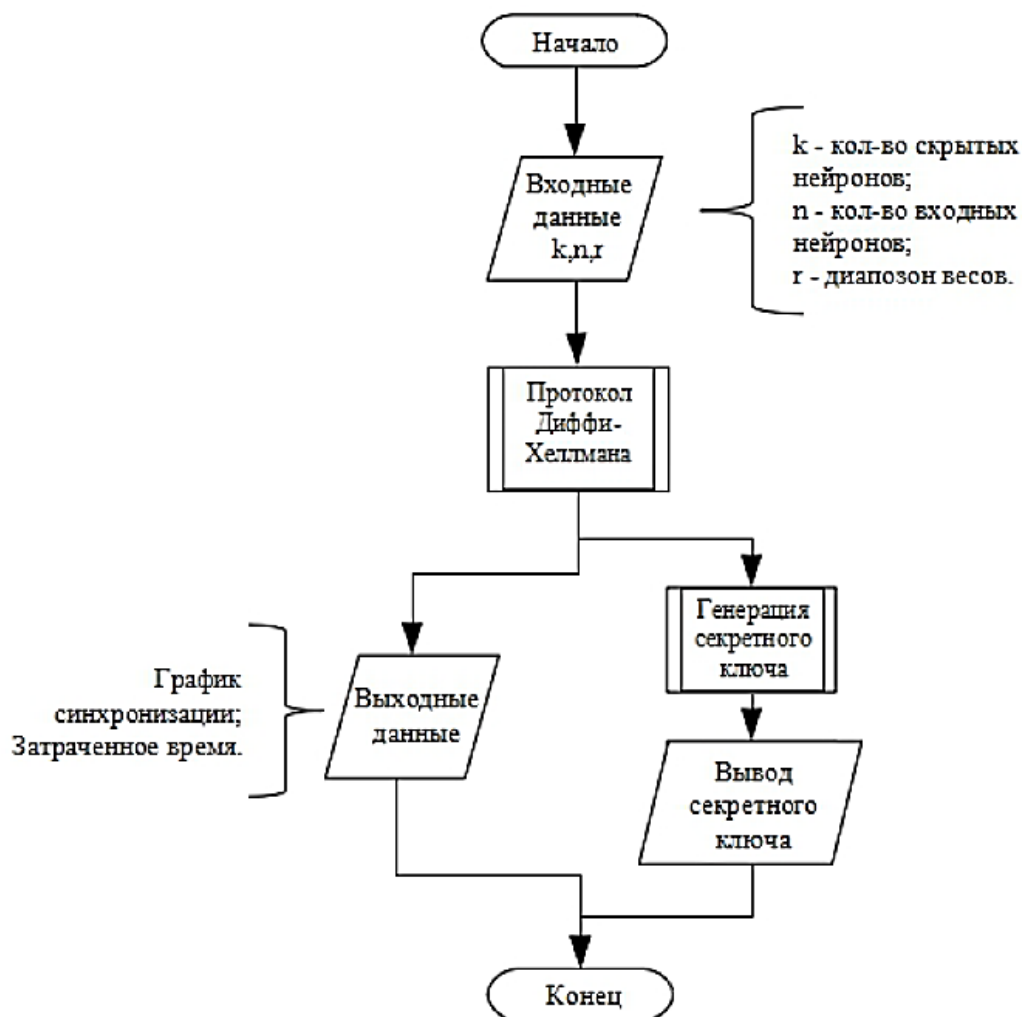


Рисунок 1.7. Алгоритм работы программного средства

На рис. 1.8 представлена блок-схема разрабатываемой программы, показывающей процедуру аутентификации пользователя по логину и паролю. На ней отображен процесс ввода данных, и проверка на корректность ввода.

Вход возможен как от имени обычного пользователя, так и от имени администратора. У пользователя есть возможность изменить логин и пароль на новый, ещё не зарегистрированный в системе, и удалить свою учётную запись из базы данных. У администратора появляется функционал для удаления любого пользователя из базы данных и назначения учётной записи любого пользователя в качестве администратора всей информационной системы.

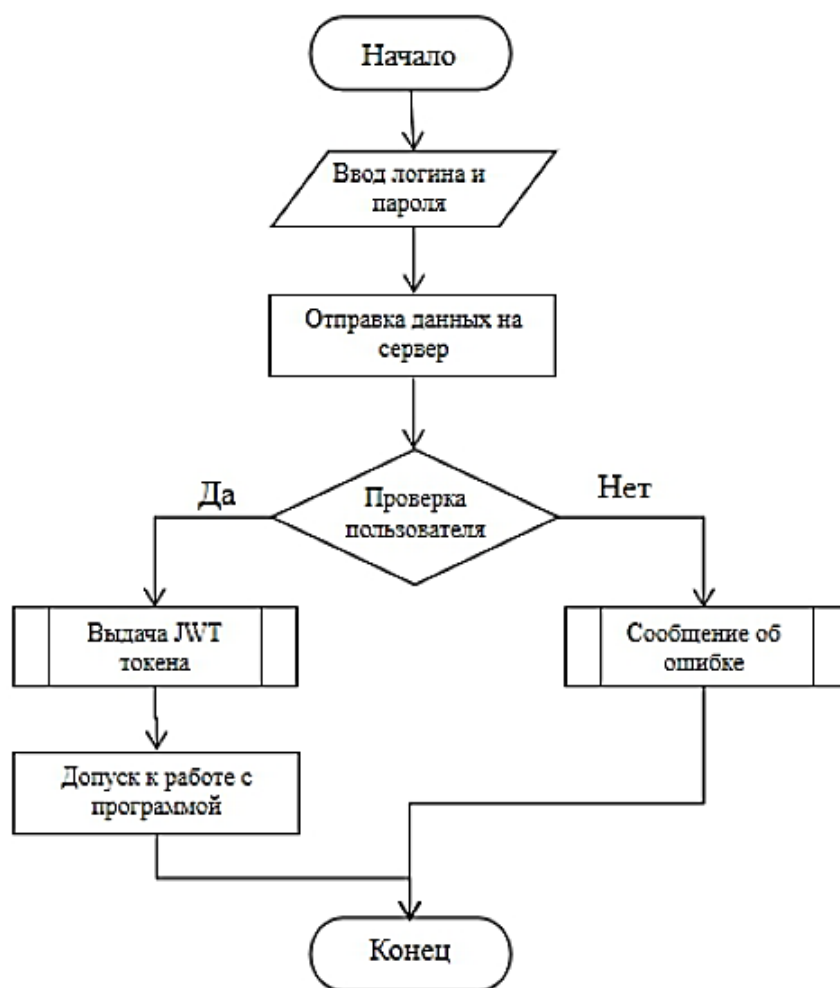


Рисунок 1.8. Аутентификация пользователей

При вводе некорректного имени или пароля пользователя сервер возвращает сообщение об ошибке. Обработав эту ошибку, пользовательское приложение выводит сообщение о том, что такого пользователя не существует.

Для аутентификации сервер использует JSON Web Token (JWT). Это открытый стандарт (RFC 7519) для создания токенов доступа, основанный на формате JSON. Токены создаются сервером, подписываются секретным ключом и передаются клиенту, который в дальнейшем использует конкретный токен для подтверждения своей личности в процессе аутентификации [7].

JWT–токен разделен на три основные группы:

- 1) Header – объект формата JSON, который содержит информацию о разновидности токена и алгоритме его шифрования.
- 2) Payload–объект JSON, содержащий данные для авторизации пользователя.
- 3) Signature – строка, которая создается с помощью секретного кода Header и Payload. Эта строка служит для проверки и верификации токена.

1.6.4. Реализация протокола Диффи–Хеллмана на базе ДМЧ

Протокол Диффи–Хеллмана, основанный на работе древовидной машине чётности, реализован через класс. После создания экземпляра вызывается метод, устанавливающий соединение с сервером через сокеты, и метод выполнения протокола, который принимает четыре входных параметра: три параметра ДМЧ (k , n , r) и правило обучения. Блок–схемы протокола на сторонах клиента и сервера представлена на рис. 1.9 и 1.10.

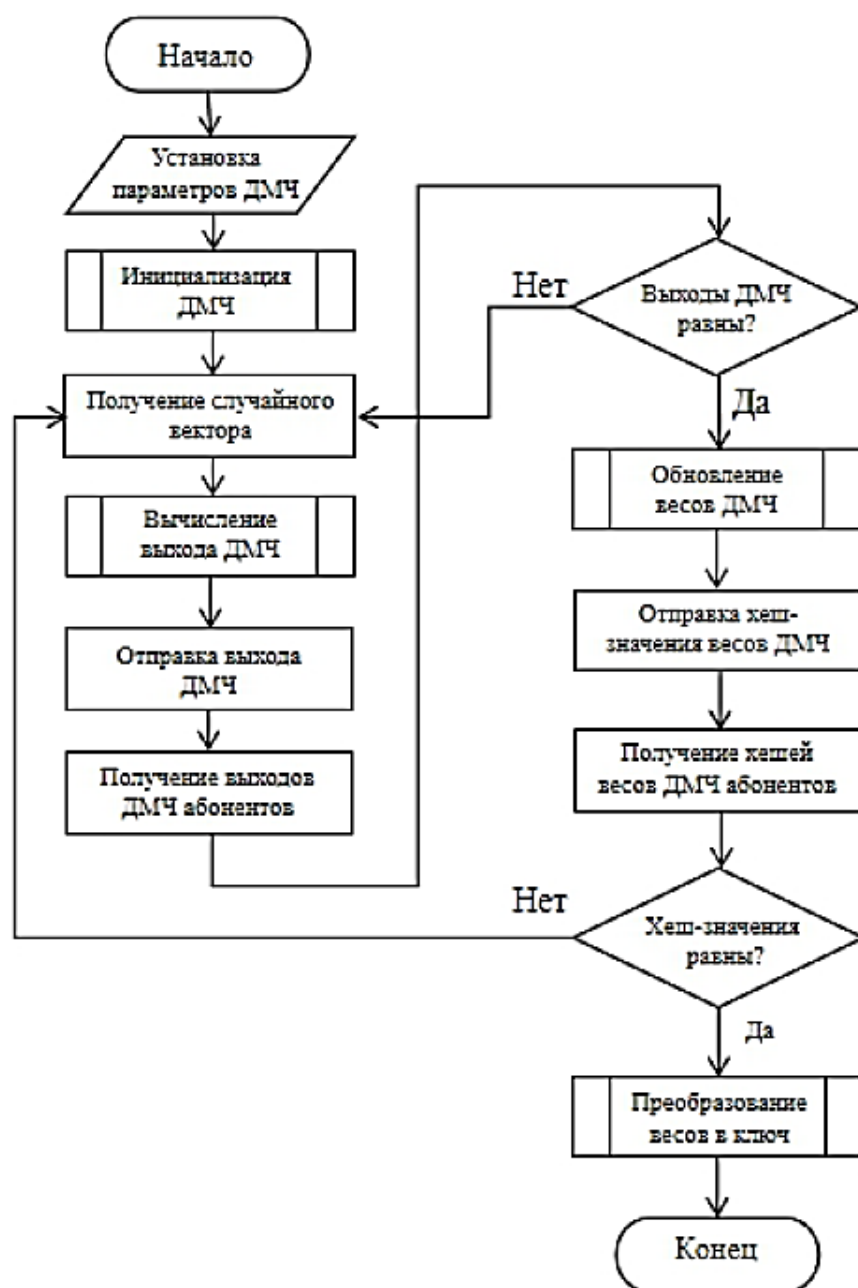


Рисунок 1.9. Блок–схема протокола на стороне клиента

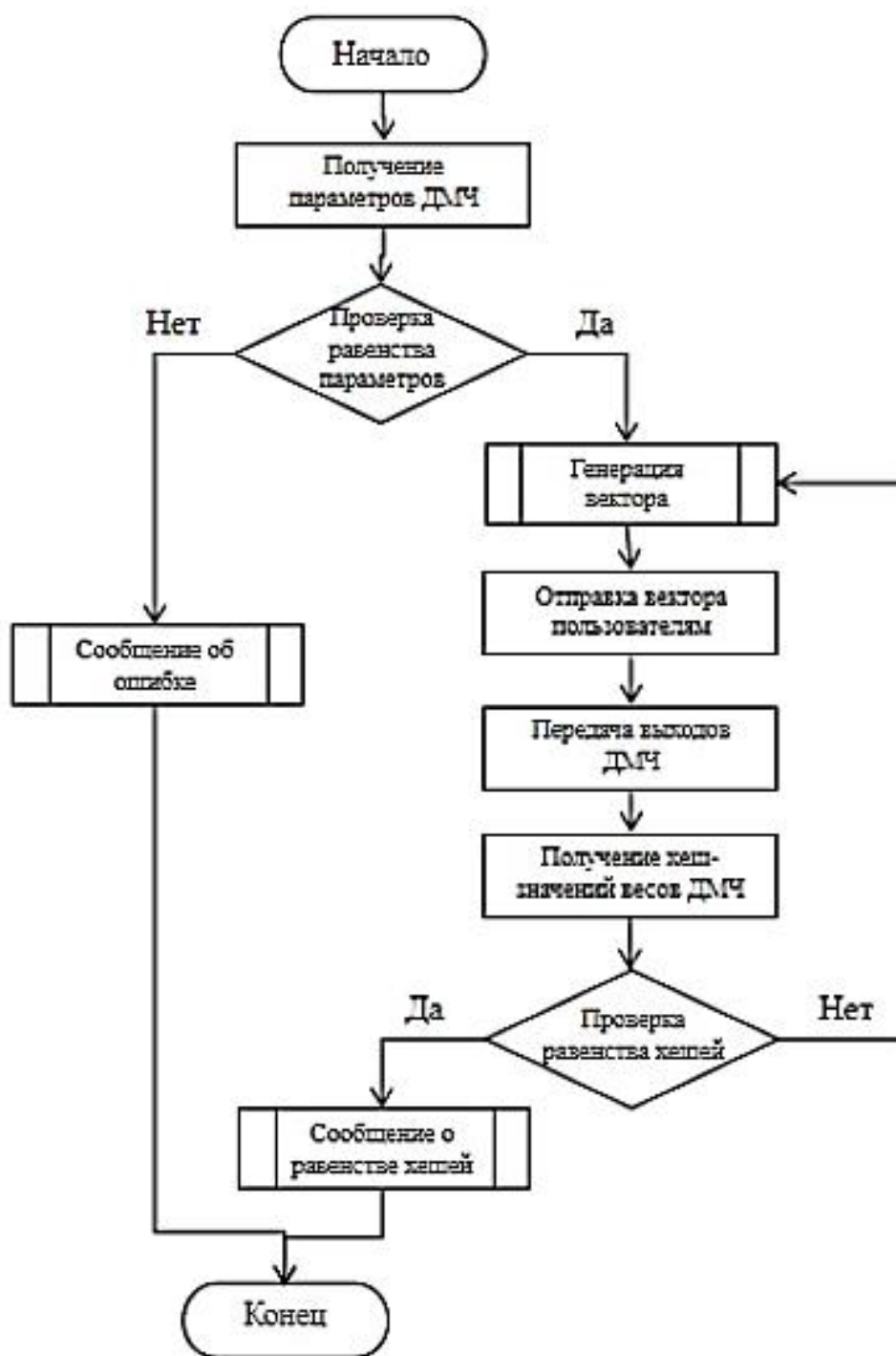


Рисунок 1.10. Блок–схема протокола на стороне сервера

После получения сервером двух запросов на генерацию общего секретного ключа происходит сравнение параметров двух ДМЧ. В случае равенства параметров сервер создаёт случайный *бинарный массив* размерностью $k \times n$ и передаёт обоим абонентам. Этот же вектор принимает и злоумышленник. Далее вычисляются выходы древовидной машины чётности [8].

Если результаты двух пользователей равны, то они обновляют свои веса по выбранному правилу. Злоумышленник обновляет свои веса, если выход его ДМЧ равен выходам абонентов. После этого вычисляется процент синхронизации между двумя пользователями, а также синхронизация злоумышленника с абонентами. Вычисление происходят по формуле:

$$1 - \text{average} \left(\sum_{i=1}^k \sum_{j=1}^n \frac{w_{i,j}^1 - w_{i,j}^2}{2r} \right) \quad (1.22)$$

где функция *average()* – это среднее арифметическое значение; а переменная $w_{i,j}$ – матрица весов древовидной машины чётности.

На основании данных о синхронизации строятся два графика. Один график выводит информацию о процессе работы протокола между двумя абонентами, второй — между злоумышленником и абонентами. На горизонтальной оси x расположено количество пройденных этапов, а на вертикальной оси y – синхронизация в процентах [9].

После полной синхронизации с помощью хеширования весов ДМЧ получается общий секретный ключ. Он выводится на экран пользователя вместе с графиками синхронизации весов. Если в процессе выполнения протокола произойдёт ошибка, то будет выведено соответствующее сообщение.

1.7. Программная реализация протокола обмена ключами

1.7.1. Выбор языка программирования ПО и его обоснование

Следует определиться с языком программирования, на котором будет разработано программное средство, а также выбрать среду его разработки, где будет проводиться написание самого кода. Разберёмся, какими особенностями должен обладать язык программирования для реализации средства обмена сообщениями. Выделим следующие ключевые особенности языка: высокая скорость вычислений; простота работы с многомерными массивами; возможности создания графического интерфейса и визуализации графических данных; возможность создания сервера, а также простота взаимодействия с ним.

Наличие условия высокой скорости вычислений обуславливается большим объёмом данных, а именно искусственных нейронов, количество которых может достигать до сотен, в зависимости от введенных пользователем параметров древовидной машины чётности. Выполнение этого условия могут обеспечить такие языки программирования, как C++, C#, Python.

Критерий простоты работы с массивами необходим, так как веса нейронов хранятся и обрабатываются в виде многомерного массива. Язык C++ не подходит под это условие, так как в нём реализовано недостаточное количество функций операций над массивом. В языках Python и C# имеются библиотеки с большим количеством функций для работы с массивами данных.

Возможность создания графического интерфейса и построения графиков обеспечивают практически все современные кроссплатформенные языки программирования, но наиболее удобный функционал обеспечивает язык C#, где в среде разработки Microsoft Visual Studio имеется возможность работать с графическим интерфейсом с помощью визуального редактора, и язык C++, который имеет тот же инструмент в среде разработки Qt.

Условие создания сервера и взаимодействия с ним обуславливается необходимостью передачи данных между абонентами протокола. Этому критерию более соответствует язык C#, так как имеет интуитивно понятный фреймворк ASP.NET. Содержание платформы показано на рис. 1.11.

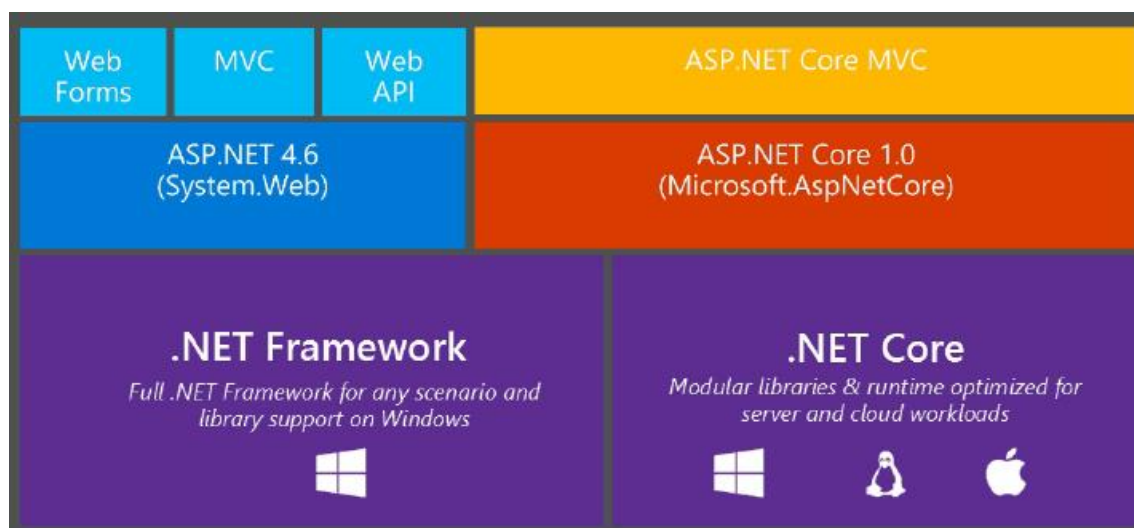


Рисунок 1.11. Структура фреймворков

Из всего перечисленного следует, что наиболее оптимальным вариантом будет выбор языка программирования C#. Он удовлетворяет всем условиям, имеет знакомый синтаксис языка C, и удобен для разработки больших проектов.

В этом проекте используется версия 7.3 языка программирования C#. Интерфейс написан с помощью компонента WinForms платформы .NET Framework версии 4.6. Для серверной части из вариантов платформ .NET Framework и .NET Core используем язык ASP.NET Core версии 2.2, так как он обладает большим быстродействием и является кроссплатформенным языком.

1.7.2. Выбор и обоснование интегрированной среды разработки

На сегодняшний день существует довольно много сред разработки, поддерживающих разработку на языке C#. Рассмотрим наиболее известные из них. Как и в случае с выбором языка программирования, для начала важно определить возможности, которые должна предоставлять среда разработки. Выделим следующие особенности сред разработки: наличие системы контроля версий \ github; проверка программного кода на наличие синтаксических неточностей и ошибок; возможность работы с несколькими проектами; бесплатное распространение; надёжность и стабильность работы.

Рассмотрим Microsoft Visual Studio, ProjectRider, Eclipse, Code::Blocks. Под критерий бесплатного распространения не подходит среда ProjectRider. Кроме того, она имеет урезанный функционал. Все остальные интегрированные среды разработки (IDE – Integrated Development Environment) являются бесплатными или имеют бесплатные версии. В случае выполнения условия стабильности работы среды следует исключить IDE Code::Blocks. Критерий удобства работы с несколькими проектами обуславливается необходимостью реализовывать клиентскую (desktopную) и серверную часть программы. Под данное условие подходят первые три среды.

Среди оставшихся IDE можно выделить Microsoft Visual Studio. Эта среда имеет плагин для работы с системами контроля версий, в том числе и github, создана той же компанией, что и выбранный язык программирования, в связи с чем имеет высокий функционал, удобный для работы с .NET платформами. В рассматриваемом проекте использована IDE версии 2017.

1.7.3. Выбор системы управления базами данных

Для хранения данных пользователей используется СУБД MySQL Server. Это кроссплатформенная система, которая способна работать в различных операционных системах и имеет ряд характеристик: многопоточность; многопользовательский доступ; быстродействие; масштабируемость.

В настоящее время MySQL представляет собой одну из самых надёжных, быстрых и качественных из всех существующих систем управления базами данных. Основными причинами этого является её бесплатное распространение вместе с исходными кодами и высокое быстродействие СУБД MySQL.

Стоит отметить, что СУБД MySQL Server имеет клиент–серверную архитектуру, одно из направлений которого связано с удаленным взаимодействием пользовательских приложений с сервером MySQL.

К преимуществам выбранного хранилища данных можно отнести:

- удобство при настройке системным администратором;
- простоту использования при разработке программного обеспечения;
- снижение рисков потери данных, в виду сбоя при работе сервера, поскольку MySQL Server позволяет создавать резервную копию.

В качестве недостатков стоит отметить низкую надёжность при работе с данными, а также невысокую скорость при разработке.

1.7.4. Проекты программного средства по обмену ключами

ПО для обмена ключами между пользователями представлено в сорока двух исходных файлах, рассредоточенных между тремя проектами:

- проект с библиотекой содержит 4 исходных файла и 4 класса, реализующие протокол Диффи–Хеллмана на базе древовидной машины чётности;
- проект, отвечающий за интерфейс программы, реализует 6 классов и содержит 12 исходных файлов. Точка входа в программу находится в файле «Program.cs». В проекте реализовано 5 форм взаимодействия с пользователем;
- серверный проект включает в себя 28 классов и 26 файлов. В корневом узле находятся 3 файла, включая файл запуска, и 4 папки с исходниками. Папка «Controllers» включает в себя 3 файла, папка «Models» – 5 файлов, «Util» – 1 файл. Папка «Services», в свою очередь, содержит следующие папки: «Auth» – 4 файла, «Hashing» – 2 файла, «Hubs» – 4 файла и «ProxyDb» – 4 файла.

1.7.5. Классы программного средства по обмену ключами

Программное средство представлено 38 классами. Выделим 4 основных класса: TreeParityMachine, DiffieHellman, DiffieHellmanWithSyncPoints, который наследуется от DiffieHellman, а также DiffieHellmanWithInterceptor, который наследуется от класса DiffieHellmanWithSyncPoints.

Класс TreeParityMachine реализует древовидную машину чётности. На рис. 1.12 представлена структура этого класса. Опишем публичные методы:

Метод TreeParityMachine является конструктором класса. В качестве входных параметров используется: количество входных нейронов, количество нейронов скрытого слоя, диапазон весов ДМЧ и правило обучения. По умолчанию установлено правило обучения Хеббиана.

Class TreeParityMachine
<pre> public TreeParityMachine(uint , uint, int, LearningRuleNeurons) public GetTau(int[][]) : int public UpdateWeighs(int) : void public GetHashWeights() : string public GetSecretKey() : string public GetWeights() : int[][] private GetRandomArray(int, int) : int[] private GenerationWeights(uint, uint, int) : int[][] private GetHash(string) : string private Signum(int) : int private ArrayProduct(int[]) : int private EqualsInt(int, int) : int private ClipWeight(int) : int private HebbianLearningRule(int) : void private AntiHebbianLearningRule(int) : void private RandomWalkLearningRule(int) : void </pre>

Рисунок 1.12. Структура класса TreeParityMachine

Метод GetTau возвращает идентификатор чётности, вычисленный через входные значения. Принимает случайный многомерный (в простейшем случае, двумерный) массив размерностью k на n состоящих из $(-1, 1)$ для нейронов.

Метод UpdateWeighs обновляет веса по заданному правилу. Он принимает чётность древовидной машины второго абонента протокола.

Метод GetHashWeights возвращает хеш-значение весов древовидной машины чётности, используя алгоритм SHA–256.

Метод GetSecretKey возвращает секретный ключ, сгенерированный из весов древовидной машины чётности.

Класс DiffieHellman реализует протокол обмена секретного ключа Диффи–Хеллмана, основанный на древовидной машине чётности. Кроме того, этот класс обеспечивает связь с сервером через сокеты и запросы. На рис. 1.13 представлена структура данного класса. Отметим, что методы, которые оканчиваются на Async являются асинхронными.

Метод DiffieHellman является конструктором класса. В нём настраивается http соединение с сервером и соединение через сокеты.

Метод GetTokenAsync отвечает за получение токена с сервера для аутентификации. В качестве входных элементов метод используется имя пользователя, зарегистрированного на сервере, и пароль этого пользователя.

Class DiffieHellman
<pre> public DiffieHellman() private GetTokenAsync(string, string) : string protected SetHubConnectionOn(): void public ConnectAsync(string, string, string) : void public RunProtocolAsync(uint, uint, int, LearningRuleNeurons) : void public ReceiveEqualsParamsAsync(bool) : void public ReceiveValuesInputAsync(int[][]) : void public ReceiveTauAsync(int) : void public ReceiveHashWeightsAsync(string) : void </pre>

Рисунок 1.13. Структура класса DiffieHellman

Метод `SetHubConnectionOn` устанавливает необходимые методы для вызова их сервером через соответствующие сокеты.

Метод `ConnectAsync` отвечает за установку соединения с сервером. Он принимает имя и пароль пользователя, а также имя второго абонента.

Метод `RunProtocolAsync` запускает протокол Диффи–Хеллмана, послав запрос на сервер. Принимает количество входных нейронов, количество скрытых нейронов, диапазон весов и правило обучения нейронов.

Дальнейшие методы вызываются самим сервером. Их установка была заложена ранее, в методе `SetHubConnectionOn`.

Метод `ReceiveEqualsParamsAsync` получает информацию о равенстве переданных параметров древовидной машины чётности. В случае равенства происходит запрос на сервер на получение случайной общей матрицы, иначе посылается сигнал о прекращении протокола.

Метод `ReceiveValuesInputAsync` получает случайную матрицу от сервера, вычисляет четность древовидной машины и отправляет её обратно.

Метод `ReceiveTauAsync` получает чётность второго абонента, обновляет веса древовидной машины чётности и отправляет хеш весов на сервер.

Метод `ReceiveHashWeightsAsync` получает хеш весов второй стороны и сравнивает их. Если оба хэша равны, то вызывается событие успешного завершения работы протокола. В противном случае посылается запрос на сервер, для получения случайной матрицы.

Класс `DiffieHellmanWithSyncPoints` наследуется от класса `DiffieHellman`. Он является дополнением к протоколу Диффи–Хеллмана и отвечает за вывод графика синхронизации между обоими абонентами протокола. Структура данного класса представлена на рис. 1.14.

DiffieHellmanWithSyncPoints : DiffieHellman
<pre> public DiffieHellmanWithSyncPoints() : base() protected override SetHubConnectionOn() : void public override async ReceiveTauAsync(int) : Task protected ComputeAverageDifferenceWeights(int[][], int[][]) : double public virtual ReceiveValueSyncWeightsAsync(int[][]) : void </pre>

Рисунок 1.14. Структура класса DiffieHellmanWithSyncPoints

Метод DiffieHellmanWithSyncPoints является конструктором класса. Перед созданием экземпляра класса вызывается конструктор родителя. В этом методе создается список для хранения информации о синхронизации абонентов протокола, а также устанавливается вызов события передачи этих значений после завершения синхронизации.

Метод SetHubConnectionOn устанавливает соответствующий метод класса для его последующего вызова со стороны сервера.

Метод ReceiveTauAsync получает чётность второго пользователя и отправляет запрос на получение его весовой матрицы ДМЧ.

Метод ComputeAverageDifferenceWeights вычисляет схожесть весов двух древовидных машин четности в процентном соотношении.

Метод ReceiveValueSyncWeightsAsync получает значение весов ДМЧ второго абонента и вызывает метод ComputeAverageDifferenceWeights.

Класс DiffieHellmanWithInterceptor наследуется от класса–родителя DiffieHellmanWithSyncPoints. Он расширяет предыдущий класс, добавляя симуляцию противника, который пытается синхронизироваться с абонентами. На рис. 1.15 представлена структура данного класса.

DiffieHellmanWithInterceptor : DiffieHellmanWithSyncPoints
<pre> public DiffieHellmanWithInterceptor() : base() public override async RunProtocolAsync(uint, uint, int, LearningRuleNeurons) : Task public override async ReceiveValuesInputAsync(int[][]) : Task public override async ReceiveTauAsync(int) : Task public override async ReceiveHashWeightsAsync(string) : Task public override ReceiveValueSyncWeightsAsync(int[][]) : void </pre>

Рисунок 1.15. Структура класса DiffieHellmanWithInterceptor

Метод DiffieHellmanWithInterceptor является конструктором класса. Перед созданием экземпляра класса вызывается конструктор родителя. В данном методе создаются два списка для хранения информации о синхронизации абонентов протокола, а также устанавливается вызов события передачи этих значений после завершения синхронизации [10].

Метод `RunProtocolAsync` создает древовидную машину чётности на основе введенных пользователем параметров, для использования её противником. После этого продолжает выполнения протокола пользователями.

Метод `ReceiveValuesInputAsync` получает вектор случайных значений $(-1, 1)$ от сервера для их дальнейшей обработки.

Метод `ReceiveTauAsync` получает значения, вычисленные древовидной машиной чётности для обоих абонента протокола.

Метод `ReceiveHashWeightsAsync` получает значения весов ДМЧ второго пользователя и сравнивает их со своими значениями. Если они равны, то это значит, что перехватчик смог синхронизироваться с одним из абонентом.

Метод `ReceiveValueSyncWeightsAsync` получает значения обоих абонентов, вызывает метод `ReceiveHashWeightsAsync` для проверки успешности синхронизации с абонентами протокола и сравнивает схожесть весов противника с весами пользователей для дальнейшего построения графиков.

1.8. Тестирование разработанного программного обеспечения

Для работы с приложением нужно начать работу с сервером. После успешного запуска, путём отправки http запроса с помощью API, доступного без аутентификации пользователя, стартует десктопное приложение. На экране появляется форма авторизации, изображенная на рис. 1.16.

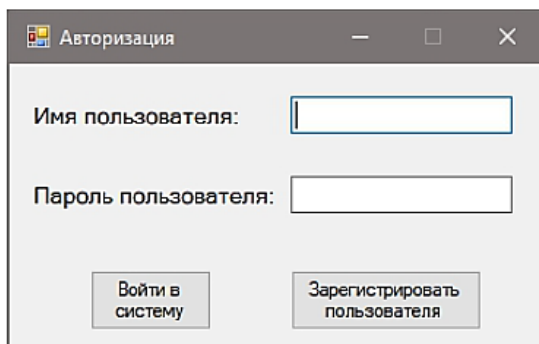


Рисунок 1.16. Вход в программу

После корректного входа или создания нового пользователя, открывается окно, представленное на рис. 1.17. Оно включает в себя параметры древовидной машины чётности – количество скрытых нейронов, входных нейронов, максимальный вес элементов матрицы ДМЧ, выбор правила обучения из трёх возможных вариантов (правило Хеббиана, правило анти-Хеббиана и правило случайного блуждания) [11, 12].

Имеется поле ввода имени получателя, т.е. второго абонента протокола. Поля ввода параметров древовидной машины чётности имеют свои рекомендованные значения, используемые по умолчанию [13].

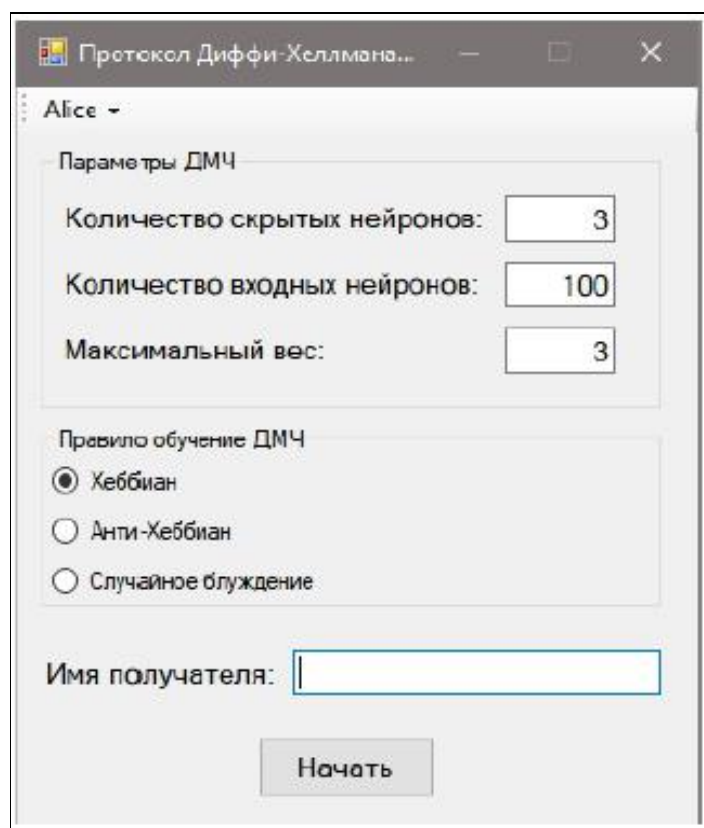


Рисунок 1.17. Главное окно

Главное окно, в левом верхнем углу, имеет вкладку, содержащую имя пользователя. При её нажатии появляются три пункта. При активизации кнопки «Изменить» открывается форма для ввода нового имени и пароля конкретного пользователя. Данная форма показана на рис. 1.18.

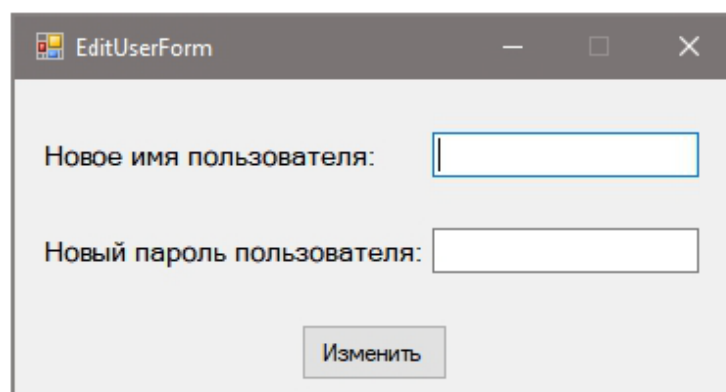


Рисунок 1.18. Окно изменения пользователя

При нажатии кнопки «Удалить» открывается диалоговое окно, требующее подтверждения действия. Пункт «Выйти» отвечает за вывод формы входа в программу. Введя имя получателя в главном окне, необходимо нажать на кнопку «Начать» для запуска протокола, после чего открывается форма, содержащая информацию о процессе выполнения протокола между обоими абонентами (пользователями). Такая форма изображена на рис. 1.19.

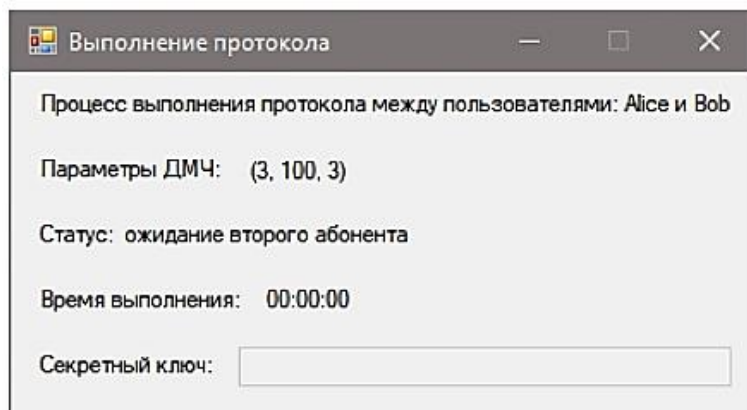


Рисунок 1.19. Начало выполнения протокола

Окно включает в себя демонстрацию введенных параметров древовидной машины чётности, статус выполнения протокола, время выполнения синхронизации ДМЧ абонентов и пустое поле для секретного ключа [14].

После того, как будет проделан тот же набор действий и для второго абонента (пользователя), начинается процесс обучения обоих древовидных машин чётности. При их полной синхронизации статус процесса изменится, счётчик времени остановится, и в поле с секретным ключом будет вставлен общий секретный ключ, что и продемонстрировано на рис. 1.20. Будет выведен график синхронизации весов древовидных машин чётности обоих абонентов и перехватчика, изображённый на рис. 1.21.

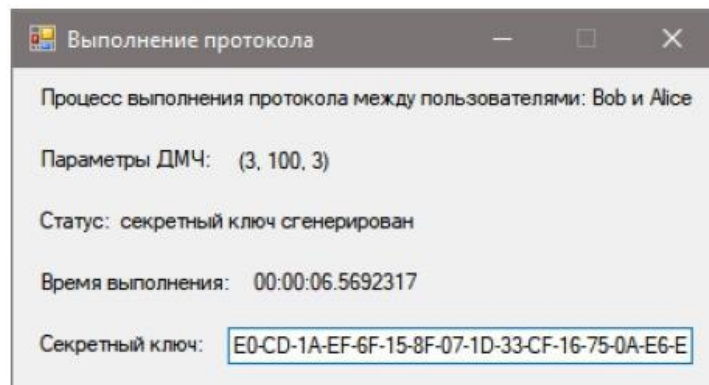
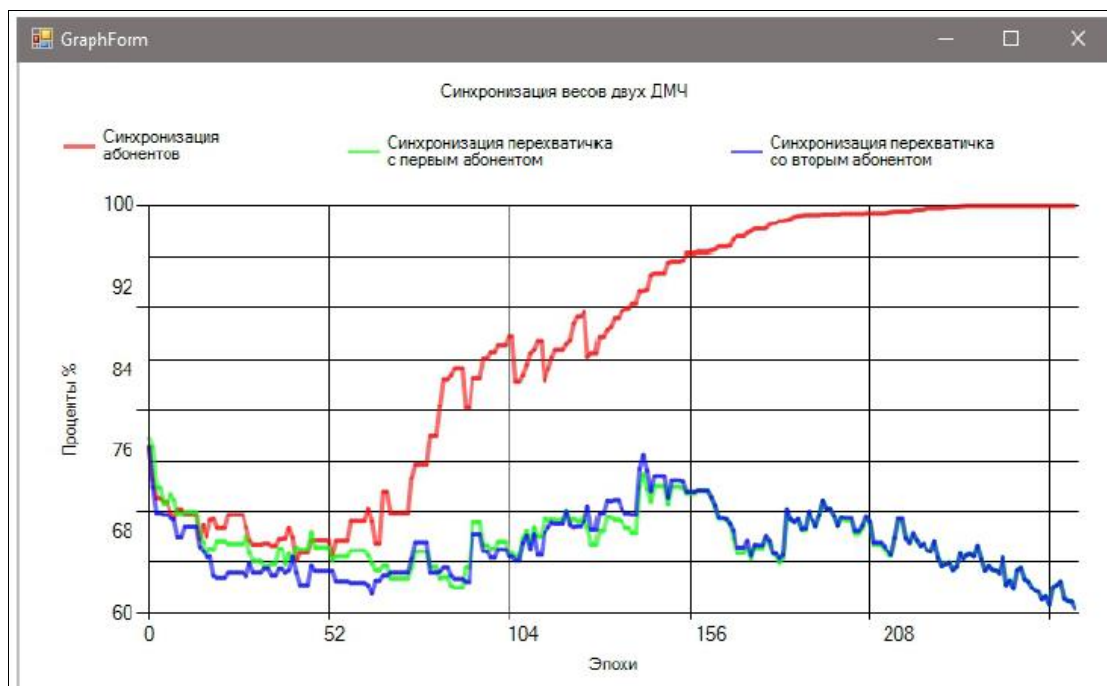


Рисунок 1.20. Окончание выполнения протокола



Рисунок

1.21. График синхронизации

Для тестирования программы нужно запустить десктопное приложение. Если в форме входа попытаться войти в качестве незарегистрированного пользователя, то результат данной попытки можно увидеть на рис. 1.22.

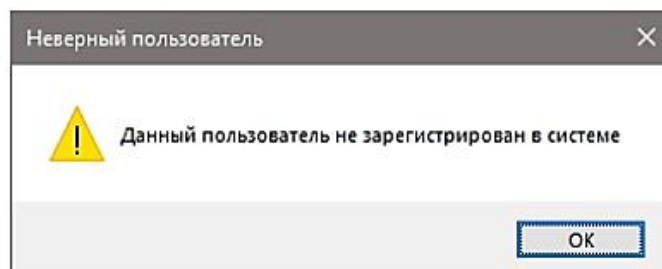


Рисунок 1.22. Неверный пользователь

В случае попытки регистрации под уже существующим именем, приходит сообщение, проиллюстрированное на рис. 1.23.

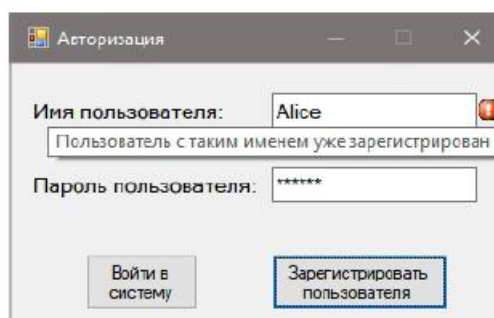


Рисунок 1.23. Ошибка регистрации

После входа в систему от имени пользователя, если он попытается ввести символы в поля ввода параметров ДМЧ, то они не будут вводиться. При вводе слишком больших значений появится восклицательный знак с информацией о недопустимом значении. Такое сообщение представлено на рис. 1.24.

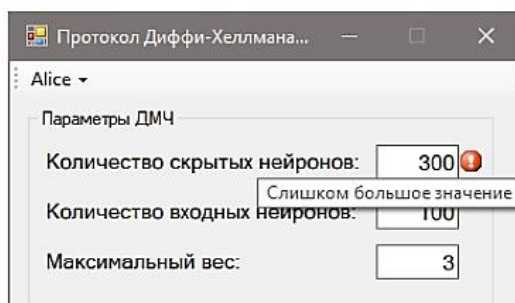


Рисунок 1.24. Ошибка ввода параметров

Проверим вариант, когда имя получателя указано неверно. В этом случае также появляется восклицательный знак, информирующий о неверно введенном имени получателя, что показано на рис. 1.25.

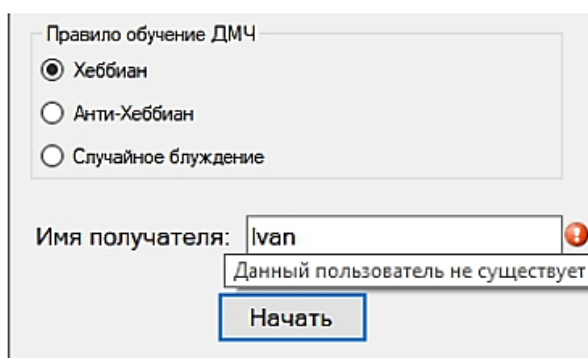


Рисунок 1.25. Ошибка ввода получателя

Если в процессе синхронизации весов двух абонентов один из них отключится, то в информационной строке статуса данного процесса появится соответствующая ошибка. Ее можно увидеть на рис. 1.26.

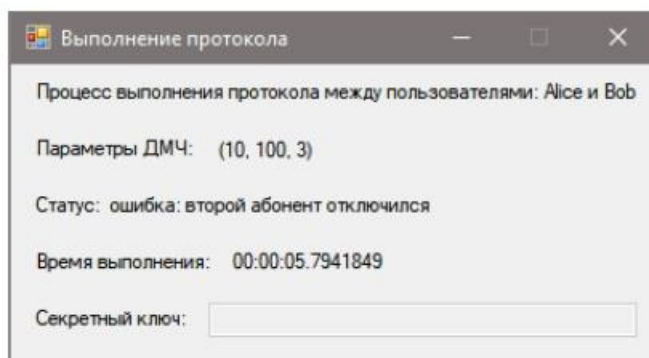


Рисунок 1.26. Ошибка синхронизации весов

При задании различных параметров древовидной машины чётности, в строке «Параметры ДМЧ» появляется соответствующая информация об ошибке, что и продемонстрировано на рис. 1.27.

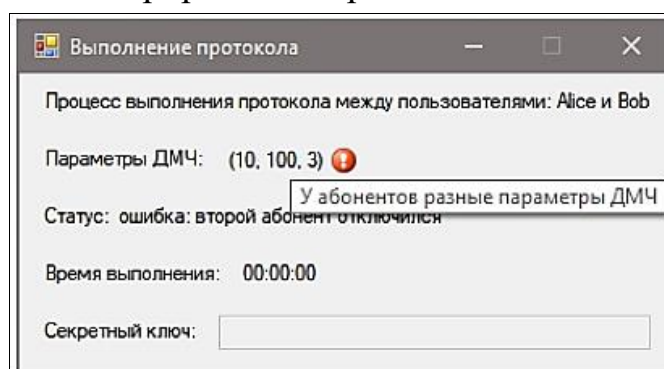


Рисунок 1.27. Ошибка равенства параметров

Если в процессе выполнения протокола Диффи–Хеллмана на базе ДМЧ сервер не отвечает, то соответствующее сообщение выводится в строке статуса синхронизации весов, что показано на рис. 1.28.

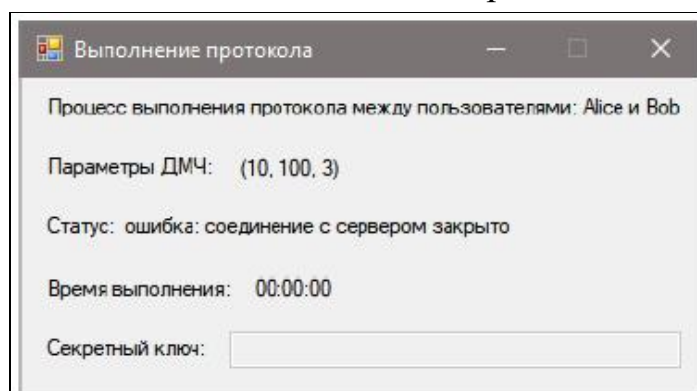


Рисунок 1.28. Ошибка соединения с сервером

В результате проведённого исследования создано программное средство, реализующее работу протокола Диффи–Хеллмана, основанного на синхронизации двух древовидных машин чётности. Результаты исследования вносят свой вклад в развитие нейрокриптографии, нового и перспективного направления традиционной криптографии. Если классический протокол Диффи–Хеллмана, основанный на теории чисел, оказывается незащищённым перед квантовыми компьютерами, то модификация протокола с помощью ДМЧ избавляет от угрозы квантовых кибератак, значительно повышая его криптостойкость [15]. Реализованное программное средство обеспечивает безопасный обмен закрытым ключом между абонентами. Тестирование программного средства показало его надёжную и устойчивую работу.

2. АЛГОРИТМ ХЕШИРОВАНИЯ ИЗОБРАЖЕНИЙ В БАЗЕ ДАННЫХ НА ОСНОВЕ ИСКУССТВЕННОЙ НЕЙРОННОЙ СЕТИ

Объём информации в мире возрастает ежегодно на 30 %. В среднем, на одного человека в год в мире производится $2,5 \cdot 10^8$ байт. Как ожидается, к началу третьего тысячелетия основными ресурсами общества станут не промышленные или сельскохозяйственные, а информационные ресурсы.

Безопасность информации стала важнейшей составляющей жизни современного «оцифрованного» общества. Наряду с другими важнейшими задачами, особое место занимает необходимость хеширования.

Хеширование применяется в самых разнообразных ИТ-областях: в СУБД, для ускорения поиска по ключу, в криптосистемах, в высокоуровневых языках программирования для реализации разнообразных структур данных и алгоритмов и пр. В информационной безопасности хеширование основой таких задач, как хранение паролей, создание уникальных криптографических ключей и электронной цифровой подписи, аудит подлинности и целостности документов в ПК. Хеширование остаётся весьма востребованной областью информационных технологий, а благодаря недавним достижениям в индустрии нанoeлектроники и росту вычислительных мощностей компьютеров появилась возможность модификации и улучшения алгоритмов хеширования.

С ростом общего объёма визуальной информации, передаваемой по сети, возросла необходимость в эффективном хранении и поиске изображений в больших базах данных. Для решения этой проблемы идеально подходит хеширование изображений, с сохранением их семантических признаков. Данная область научного исследования сравнительно молода и развивается всего на протяжении последних десяти лет, поэтому требует дальнейших усовершенствований и модификаций используемых алгоритмов.

2.1. Обзор существующих программных решений

В процессе исследования было проведено теоретическое исследование. Рассмотрены работы основоположников области искусственного интеллекта – «Организация поведения» Дональда Хебба, «Принципы нейродинамики», «Перцептроны и теория механизмов мозга» Фрэнка Розенблатта, а также труды современных исследователей нейронных сетей: «Машинное обучение» Тома Митчелла, «Верховный алгоритм» Петро Домингоса, «Методы обработки и распознавания изображений лиц в задачах биометрии» Георгия Кухарева, и др.

Проведен анализ узконаправленной и специализированной литературы, а именно работы «ImageNet Classification with Deep Convolutional Neural Networks» Алекса Крижевского, «Deep Learning of Binary Hash Codes for Fast Image Retrieval» Кевина Лина, «Deep Semantic Ranking Based Hashing for Multi-Label Image Retrieval» Жао Фэнг, «Deep Visual–Semantic Quantization for Efficient Image Retrieval» Као Ю, «HashNet: Deep Learning to Hash by Continuation» Цзяньминь Вана и многие другие. Разработка предлагаемого программного средства ведется на основании документов [16 – 20].

Объектом исследования являются алгоритмы масштабно–инвариантной трансформации признаков изображения. *Предмет изучения* – хеширование изображений с сохранением их семантических признаков. *Цель работы* – реализация программного средства для хеширования изображений с использованием *сверточных нейронных сетей* (СНС). В список поставленных задач входит: обзор существующих программных решений задачи масштабно–инвариантной трансформации признаков изображения; сравнительный анализ методов получения семантических признаков изображения; определение и реализация подходящей для достижения поставленной цели архитектуры нейронной сети; реализация алгоритма преобразования семантических признаков изображения в хеш–пространство; определение качества сохранения семантических признаков изображения в его хеш–значении.

Существует ряд программных средств для хеширования изображений с использованием свёрточных нейронных сетей. На основании проведённого исследования эффективности различных подходов машинного обучения к решению задачи семантического хеширования было решено использовать методы глубокого машинного обучения на основе СНС. Сравнительный анализ эффективности применения методов машинного обучения и свёрточных нейронных сетей для решения задачи семантического хеширования проведён в работе [17]. Существующие практические приложения для семантического хеширования изображений опубликованы в работах [18 – 20].

2.1.1. Глубокое обучение на основе двоичных хеш–кодов

В работе [21] её авторы Кевин Лин и Хью-Фенг Янг предложили программную библиотеку для реализации быстрого поиска изображений, на основе применения технологии свёрточных нейронных сетей, с применением алгоритма «приблизительного ближайшего соседа».

Метод, предложенный в этой работе, осуществляется в три этапа: *на первом этапе* происходит предварительное обучение нейронной сети на тренировочных данных (например, используются датасеты ImageNet, CIFAR-10 или любой другой пользовательский датасет, приведенный в бинарный вид); *на втором этапе* реализуется настройка нейронной сети для одновременного её обучения семантическим признакам изображения и аппроксимирующей хеш-подобной функции для вычисления хеш-кодов; и *на третьем этапе* выполняется извлечение всех изображений, схожих с изображением, заданным в запросе к базе данных, с помощью иерархического алгоритма глубокого поиска. На рис. 2.1 показан процесс предварительного обучения нейронной сети на тренировочных данных, а также процесс настройки нейронной сети для обучения семантическим признакам входных изображений и вычисления их хеш-кодов. На этом же рисунке также продемонстрировано, каким образом предварительно обученная на первом этапе свёрточная нейронная сеть настраивается на втором этапе на вычисление хеш-кодов.

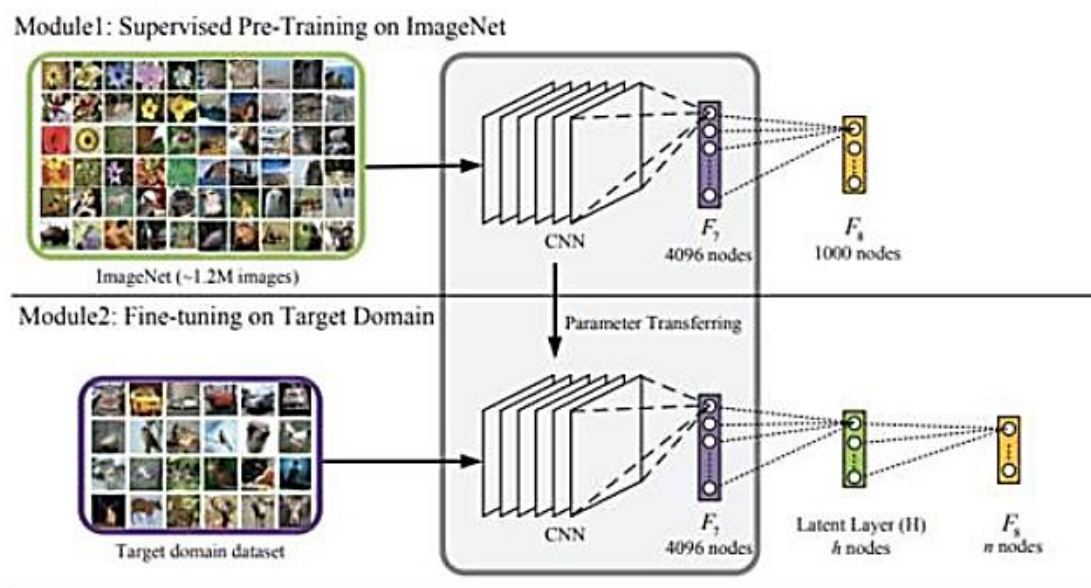


Рисунок 2.1. Первый и второй этапы работы СНС для вычисления хеш-кодов

На рис. 2.2 показан процесс глубокого иерархического поиска похожих изображений с помощью алгоритма «приблизительного ближайшего соседа». Для извлечения семантических признаков изображения использовалась свёрточная нейронная сеть, предложенная Алексеем Крижевским в работе [16]. Сеть предварительно была обучена с помощью миллиона изображений из тысячи разных классов, содержащихся в библиотеке Caffe CNN.

Module3: Image Retrieval via Hierarchical Deep Search

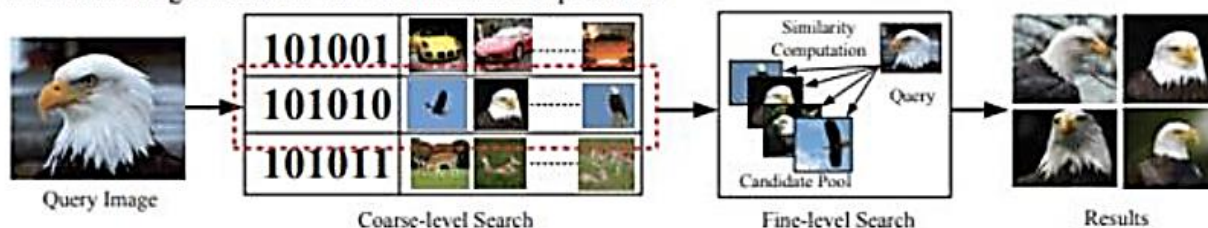


Рисунок 2.2. Третий этап работы свёрточной нейронной сети для вычисления хеш-кодов

На рис. 2.3 показан результат работы предложенной модели нейронной сети на тренировочных данных, полученных от компании Yahoo–1M.



Рисунок 2.3. Результаты работы СНС на тренировочных данных Yahoo–1M

Несмотря на высокую точность предложенной модели и эффективность алгоритма глубокого иерархического поиска, такая модель предъявляет высокие требования к вычислительным ресурсам обучаемой машины, а это нежелательный фактор при необходимости переобучения использованной модели под конкретные технические спецификации.

Предлагаемый подход направлен на изучение нелинейных параметров проекции входного изображения, которые квантуют входные биты в двоичный дескриптор, с сохранением семантических признаков. Работа предъявляет особое требование к искомому хеш-значению: инвариантность по отношению к аффинным преобразованиям входного изображения. Это требование является основным при работе с аугментированными входными изображениями.

2.1.2. Глубокое обучение на основе компактных дескрипторов

Метод глубокого обучения, предложенный в работе Дживен Лу и Чу-Сон Чена [22], предлагает сверточную нейронную сеть, обучаемую без учителя. В работе выдвигаются следующие требования к двоичным дескрипторам изображения: необходимость квантования с минимальными потерями, равномерного распределения и некоррелированность битов в двоичных кодах.

Предлагаемый подход направлен на изучение нелинейных параметров проекции входного изображения, которые квантуют входные биты в двоичный дескриптор с сохранением семантических признаков.

На первом этапе нейронная сеть обучается сохранению локальной структуры входных данных в выходном двоичном дескрипторе, что достигается с помощью минимизации функции квантования.

На втором этапе нейронная сеть обучается равномерному распределению битов в выходном двоичном дескрипторе, что делается для увеличения количества информации, которое несет в себе хеш–значение.

На третьем этапе нейронная сеть обучается инвариантности по отношению к вращению и шумам входного изображения, для обеспечения возможности дескриптора фиксировать больше некоррелированных данных.

Точность предлагаемого подхода основана на минимизации функции квантования, которая обеспечивает стремительное уменьшение погрешности распознавания между извлеченным вектором главных компонент и вычисляемым дескриптором изображения. В исследовании предъявляется важное требование к искомому хеш–значению: его биты должны быть равномерно распределены. Такое условие предъявляется с целью максимизации полезной информации, которую несет в себе хеш–значение, что позволяет сохранить большее количество информации из входного изображения.

На рис. 2.4 схематично показан алгоритм работы предложенной модели и основные требования к её обучению. Сперва нейронная сеть обучается извлечению двоичных дескрипторов из входного изображения.

Основными требованиями к обучению является минимальная функция потерь квантования и равномерное распределение битов в выходном дескрипторе; затем, нейронная сеть обучается инвариантности к аффинным преобразованиям входного изображения, а к требованиям к обучению добавляется инвариантность по вращению. Результаты работы предложенной модели свёрточной нейронной сети представлены на рис. 2.5.

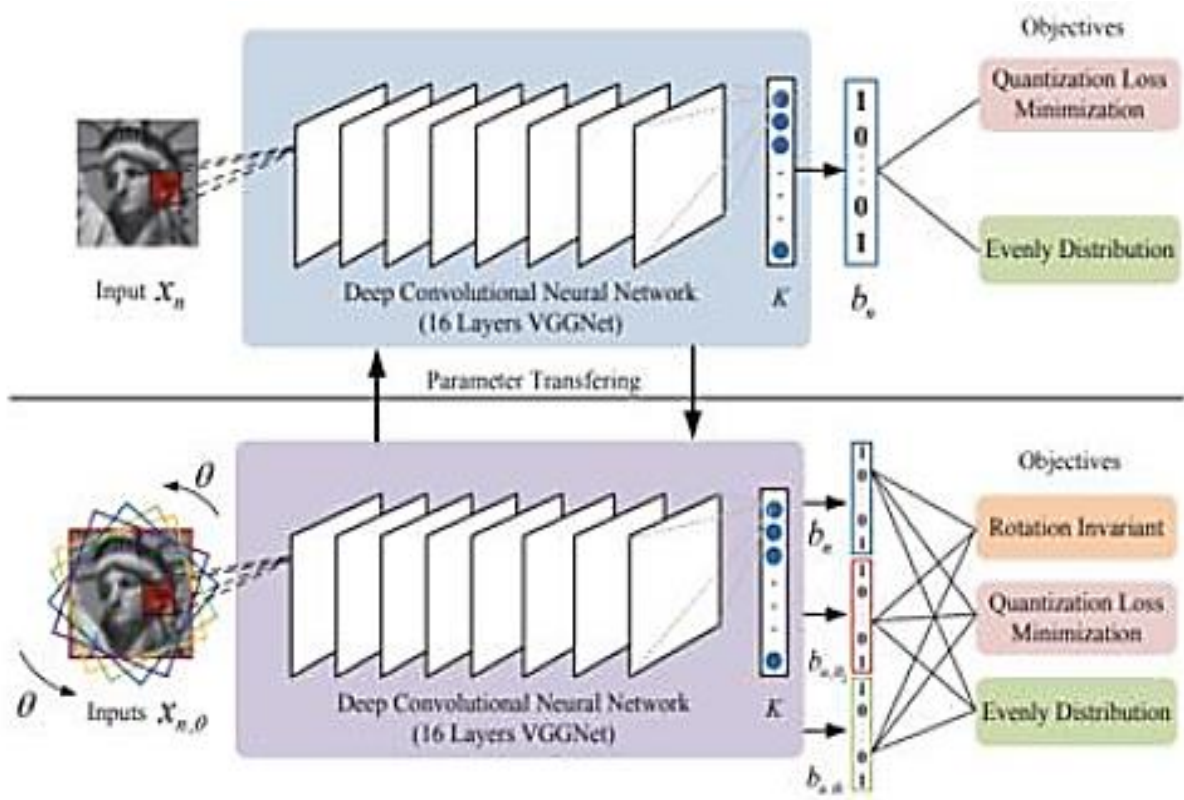


Рисунок 2.4. Входные данные и основные задачи для обучения СНС

На рис. 2.5 можно видеть, что несмотря на удовлетворительный результат поиска изображений, в большинстве случаев предложенная модель даёт большую погрешность, что отражается на качестве картинок.



Рисунок 2.5. Результат работы обученной нейронной сети

На рис. 2.6 для сравнения представлены изображения, извлеченные корректно и не корректно (с большой погрешностью и с низким качеством).

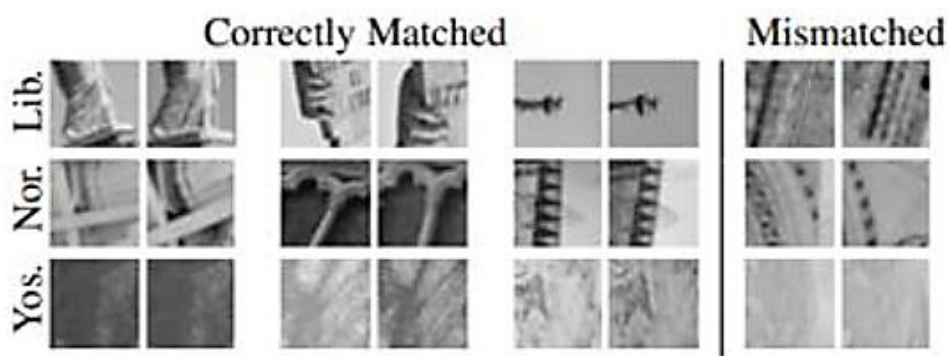


Рисунок 2.6. Корректно и не корректно извлеченные изображения

На основании проведенного обзора аналогов существующих программных решений для семантического хеширования и сравнительного анализа их эффективности можно сделать вывод о том, что проанализированные программные решения предъявляют завышенные требования к обучаемой машине, делая процесс переобучения предлагаемых моделей практически невозможным на устройствах с недостаточной вычислительной мощностью. Некоторые из рассмотренных моделей дают слишком большую погрешность в извлечении похожих картинок по хеш-кодам.

2.1.3. Поиск изображений в виде хэш-функций

В работе [23] Артем Бабенко и его коллеги предлагают использовать высокую производительность свёрточной нейронной сети при решении задач классификации изображений. Представлены качественные доказательства возможности поиска изображений на основе результатов работы СНС. Авторы используют главные компоненты изображения, извлеченные в верхних слоях СНС для задачи хеширования. В этой работе свёрточные нейронные сети использовались для создания дескрипторов, нужных для поиска изображений. Архитектура такой СНС схематично показана на рис. 2.7.

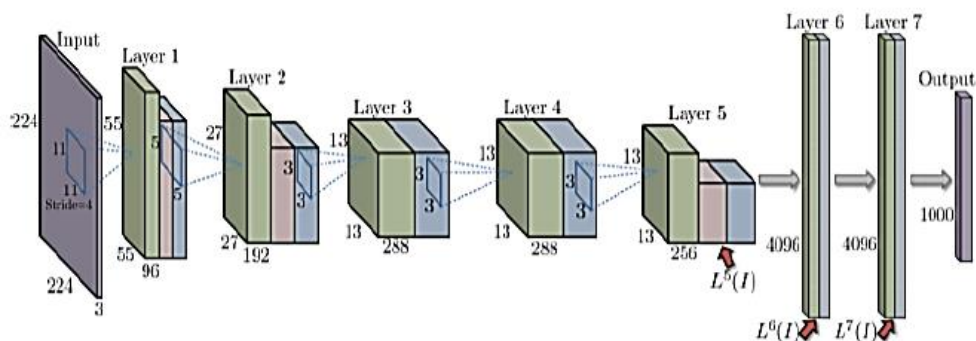


Рисунок 2.7. Архитектура СНС для извлечения нейронных кодов

Приведенная архитектура свёрточной нейронной сети является одной из многочисленных вариаций нейронной сети AlexNet, предложенной Алексом Крижевским в работе [16]. Зеленые узлы на рисунке отвечают за результаты свертки, красные узлы – за результаты объединения свертки, а голубые узлы – за результаты ReLu («Rectified linear» – линейный выпрямитель) преобразования.

Предложенная свёрточная нейронная сеть использует входной нейронный слой для обработки входного изображения, 5 скрытых слоёв свертки для извлечения главных признаков изображения (узлы 1–5), 2 полносвязных узла для вычисления нейронных кодов и обучения СНС, направленного на сохранение главных признаков входного изображения в пространстве нейронных кодов (узлы 6 и 7). Выходной узел содержит нейронный код для входного изображения, который состоит из 1000 бит.

Многочисленные исследования показали универсальность использования нейронных кодов для хранения и поиска изображений. Поэтому разработка посвящена повышению эффективности свёрточной нейронной сети в применении к задачам хеширования изображений. Основная идея повторяет ранее рассмотренные подходы к хешированию изображений. С помощью свёрточной нейронной сети, специально подобранной для решения этой задачи, извлекаются главные признаки входного изображения, а затем, на основе извлеченных главных признаков, вычисляется хеш-значение, на основании которого изображение сохраняется в базе данных. Работа [24] отличается тем, что в ней шестой полносвязный узел используется для уменьшения размерности вектора главных компонент, извлеченного в результате работы пяти скрытых узлов свертки, а седьмой полносвязный узел используется для вычисления кода. Предложенная СНС обучалась и тестировалась с помощью следующих наборов данных: Oxford Buildings, INRIA Holidays dataset и University of Kentucky Benchmark dataset.

Набор данных Oxford Buildings содержит 5062 фотографии, собранных с помощью Flickr, и соответствующих основным достопримечательностям Оксфорда. Изображения из данного набора имеют сложную структуру и состоят из нескольких зданий, что позволяет полностью протестировать возможности свёрточной нейронной сети. Пример тренировочного набора данных Oxford Buildings продемонстрирован на рис. 2.8.

Набор данных Oxford Buildings предъявляет высокие требования к эффективности СНС и снижает точность результатов при малейшем упрощении архитектуры применяемой свёрточной нейронной сети.

Methods	Test Error (%)
2-Layer CNN + 2-Layer NN [31]	0.53
Stochastic Pooling [31]	0.47
NIN + Dropout [17]	0.47
Conv. maxout + Dropout [9]	0.45
Ours w/ 48 nodes latent layer	0.47
Ours w/ 128 nodes latent layer	0.50

Рисунок 2.10. Погрешность работы модели

На рис. 2.10 в левой колонке перечислены применяемые методы для обучения свёрточной нейронной сети, а в правой колонке показана соответствующая каждому методу погрешность распознавания изображения по его хеш-значению. Последние две записи относятся к модели СНС, предложенной в работе [27], и отвечают за минимальные значения величины погрешности среди всех применяемых методов обучения.

В этом исследовании было использовано 10 выходных нейронов в предпоследнем скрытом полносвязном узле для распознавания 10 классов изображений различных арабских цифр. Количество выходных нейронов, использованных в такой модели, равняется 48 или 128, что соответствует количеству бит хеш-значения для каждого изображения.

Описанный метод глубокого машинного обучения двоичных хеш-кодов показывает уровень ошибки, равный 0.47 %, что показано на рисунке 2.10. Модель отличается тем, что применяемая в ней свёрточная нейронная сеть сконструирована для поиска изображений на основе полученных хеш-значений, в то время, когда большинство аналогичных моделей СНС сконцентрированы только на извлечении главных признаков входного изображения и на вычислении соответствующего хеш-значения.

Общий график точности извлечения изображения на основе его хеш-значения для различных методов обучения представлен на рис. 2.11. На этом рисунке красным цветом выделен график, показывающий точность извлечения изображения на основе его хеш-значения для описываемой модели. Очевидно, что этот график дает наилучшую точность извлечения изображения.

Работы [21, 26] Кевина Лина, по обучению компактных двоичных дескрипторов с использованием СНС, заключалась в тройном обучении с потерями, задачей которого являлась минимизация функции квантования.

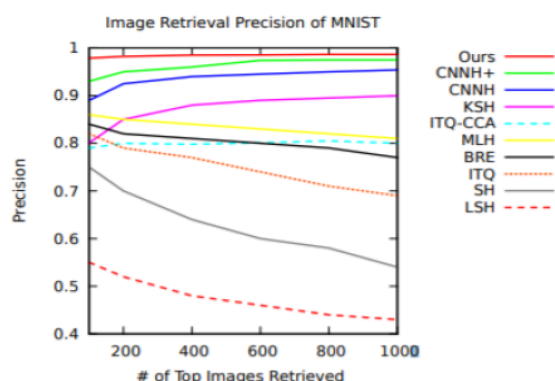


Рисунок 2.11. Точность извлечения изображения на основе его хеш-значения

Метод, предложенный в работе [28], заключается в обучении проекций входного изображения на двоичную строку с сохранением дискретной информации, содержащейся во входных данных. Идея данной работы заключается в том, чтобы сделать двоичный дескриптор информативным, а этого можно добиться с помощью минимизации функции квантования. Чем меньше потеря квантования, тем лучше двоичный дескриптор сохраняет информацию, которая содержится во входных данных. Цель этих исследований состоит в минимизации потерь квантования между двоичным дескриптором изображения и самим входным изображением. Сравнение степени точности предложенного метода и его аналогов можно увидеть на рис. 2.12.

Здесь приведены графики точности извлечения изображения на основе его хеш-значения, для различных глубоких методов машинного обучения. График, обозначенный красным цветом, отвечает за метод, предложенный Кевином Лином. Из рис. 2.12 можно понять, что в разных ситуациях наблюдаются значительные колебания точности распознавания. Это связано с тем, что дескрипторы изображений, предлагаемые в работе [29], инвариантны по отношению к аффинным преобразованиям входных изображений, однако неустойчивы к аугментированным входным изображениям, что приводит к значительным потерям процентов точности распознавания изображений.

В работе [30] предложен метод построения двоичных дескрипторов для изображений, которые были бы инвариантны по отношению как к аффинным преобразованиям входных данных, так и к их аугментации. Это достигается с помощью предварительного аугментирования входных изображений из тренировочного набора, что позволяет обучить свёрточную нейронную сеть распознавать аугментированные входные данные и корректно извлекать из них вектор главных признаков. Результаты работы модели, предложенной в этой работе, продемонстрированы на рисунке (таблице) 2.13.

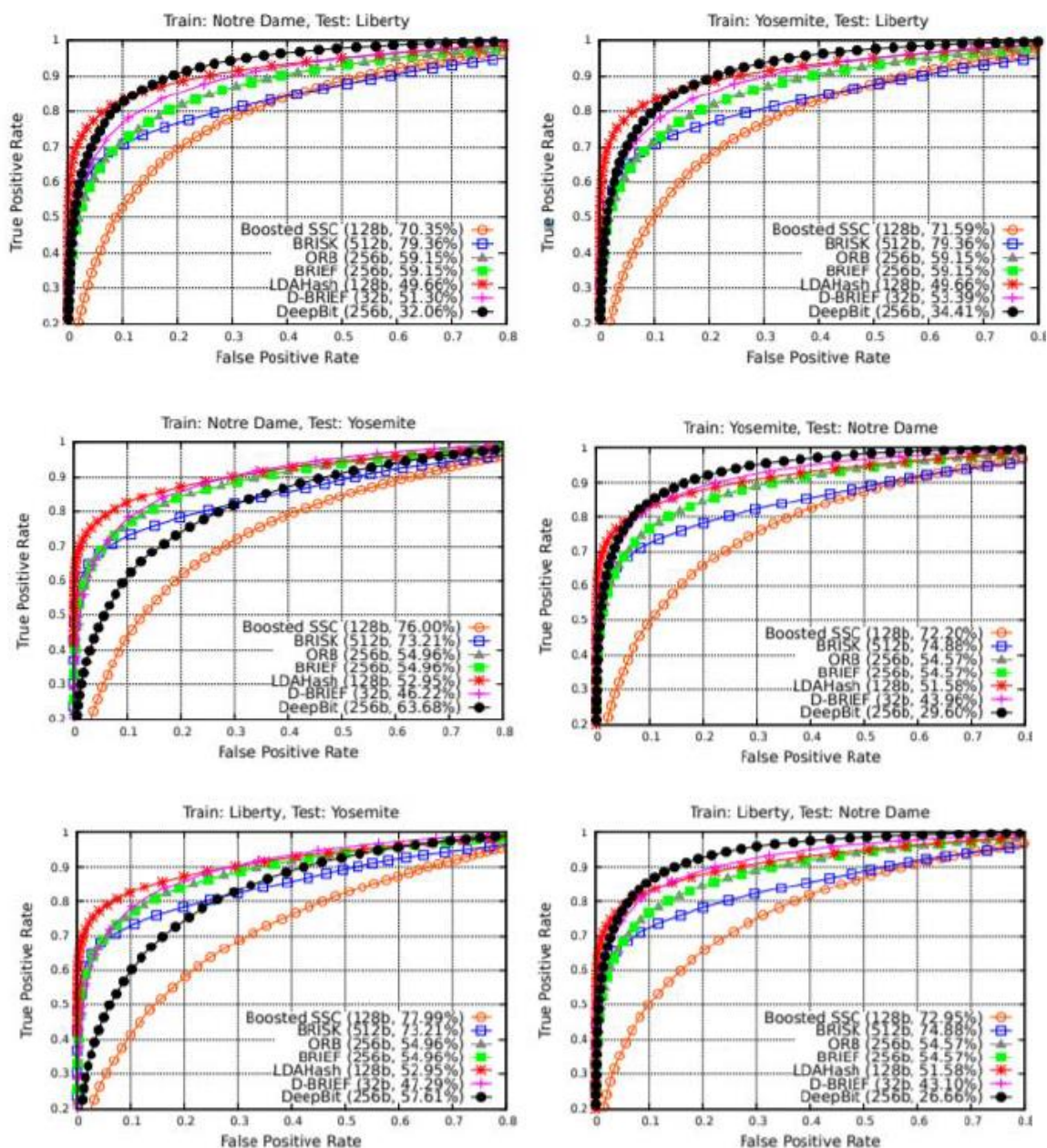


Рисунок 2.12. Сравнение точности предложенной модели и её аналогов

На рис. 2.13 в первом столбце представлены номер выходного узла, задачей которого является вычисление двоичного дескриптора или хеш- значения, а также применяемый метод дообучения или переобучения, далее следуют столбцы, которые соответствуют 16-, 32-, 64-, 128-, 256- и 512-битным хеш- значениям. В каждой ячейке содержится оценка эффективности модели СНС, применяемой в работе [31]. Данные рисунка 2.13 показывают, что точность работы предложенной модели увеличивается после применения дообучения в зависимости от входных данных свёрточной нейронной сети.

Инвариантность по отношению к аффинным преобразованиям входных изображений и их аугментации достигается за счёт процесса общего снижения производительности предложенной модели свёрточной нейронной сети.

Dimensions	16	32	64	128	256	512
Oxford						
Layer 6	0.328	0.390	0.421	0.433	0.435	0.435
Layer 6 + landmark retraining	0.418	0.515	0.548	0.557	0.557	0.557
Layer 6 + turntable retraining	0.289	0.349	0.377	0.391	0.392	0.393
Oxford 105K						
Layer 6	0.260	0.330	0.370	0.388	0.392	0.392
Layer 6 + landmark retraining	0.354	0.467	0.508	0.523	0.524	0.522
Layer 6 + turntable retraining	0.223	0.293	0.331	0.348	0.350	0.351
Holidays						
Layer 6	0.591	0.683	0.729	0.747	0.749	0.749
Layer 6 + landmark retraining	0.609	0.729	0.777	0.789	0.789	0.789
Layer 6 + turntable retraining	0.587	0.702	0.741	0.756	0.756	0.756
UKB						
Layer 6	2.630	3.130	3.381	3.416	3.423	3.426
Layer 6 + landmark retraining	2.410	2.980	3.256	3.297	3.298	3.300
Layer 6 + turntable retraining	2.829	3.302	3.526	3.552	3.556	3.557

Рисунок 2.13. Сравнение точности предложенного метода и его аналогов

Таким образом, проведён краткий обзор и сравнительный анализ эффективности существующих программных решений задачи хеширования изображений с сохранением их семантических признаков. На основании проведенного сравнительного анализа сделан вывод: чем выше заявленная точность предлагаемой модели, тем более высокие требования предъявляются к вычислительным ресурсам обучаемой машины. По итогам исследования определены требования к искомому хеш–значению изображения [32]:

- необходимость получения и сохранения максимально возможного количества информации из входного изображения;
- сокращение потерь информации, до самых минимальных значений, из входного изображения при извлечении главных признаков;
- обеспечение инвариантности относительно аффинных преобразований и аугментации входных данных изображения при извлечении главных признаков;
- необходимость равномерного распределения битов хеш–значений для увеличения полезной информации при извлечении главных признаков.

2.2. Сравнение методов извлечения главных признаков изображения

Опишем существующие методы машинного обучения, применительно к решению задачи семантического хеширования изображений, и выполним сравнительный анализ эффективности традиционных алгоритмов извлечения семантических признаков на основе свёрточных нейронных сетей.

2.2.1. Традиционные алгоритмы извлечения главных признаков

Алгоритмы масштабно-инвариантной трансформации признаков были разработаны для решения задач компьютерного зрения. Это обнаружение (детектирование) объектов, оценка их местоположения, нахождение точек их перекрытия, фиксация их перемещения и др. Подобные алгоритмы основаны на применении *дескрипторов*—уникальных идентификаторов для каждого входного изображения. Существует целое множество подобных дескрипторов [33].

Простейшим дескриптором изображения является вектор его пикселей, а для определения схожести изображений с таким дескриптором используется взаимнокорреляционная функция. Однако высокая размерность подобных дескрипторов повышает вычислительную сложность процесса распознавания.

Дескрипторы на основе распределения пикселей были впервые предложены Алексом Джонсоном и Майклом Хебертом. Их подход использует матрицу интенсивности (яркости) каждого пиксела изображения. Такие дескрипторы используются для генерации вращательного изображения, состоящего из матрицы интенсивности и гистограммы положения соседних точек, относительно интересующей точки в трёхмерном пространстве. Подобные алгоритмы применяют уникальные идентификаторы, созданные для каждого входного изображения. Таких дескрипторов существует целое множество.

Непараметрическое преобразование— это особый подход к решению задачи обнаружения объектов, отличающийся устойчивостью к изменениям освещения. Разработан Рамином Забихом и Джоном Вудфиллом. Их подход полагается на локальные преобразования, основанные на непараметрической статистике, использующие информацию о взаимных отношениях между данными, а не сами значения этих данных. Область изображения описывается с помощью упорядоченных бинарных отношений интенсивностей в соседних точках.

Пространственно–частотные методы описывают частотное содержание изображения. Так, преобразование Фурье разбивает содержимое изображения на базовые функции. Однако в этом представлении пространственные отношения между точками не являются явными, а базисные функции бесконечны, поэтому их сложно адаптировать к локальному подходу. Преобразование Габора решает эту проблему, но для его реализации требуется большое количество фильтров для улавливания небольших изменений частоты и ориентации, то есть снова появляется проблема высокой размерности данных и вычислительной сложности алгоритма. Фильтры Габора и *вейвлеты* (семейства функций, локальных во времени и по частоте, в которых все функции получаются из одной, посредством сдвигов и растяжений её по оси времени так, что они следуют друг за другом) часто исследуются в контексте классификации текстур.

Дифференциальные дескрипторы используют производные изображения, вычисленные до заданного порядка, что аппроксимирует функцию соседства точки на изображении. Дифференциальные инварианты объединяют компоненты локальной области для получения инвариантности вращения. Свойства локальных производных исследовали Макс Кендеринк и Люк Флорак. Роберт Фримен и Шелдон Адельсон разработали регулируемые фильтры, которые указывают производным определенное направление, определяющее компоненты области изображения. Управляющие производные в направлении градиента делают их инвариантными к вращению.

В дальнейшем были разработаны алгоритмы для генерации дескрипторов изображений, которые являются инвариантными относительно и вращения, и масштабирования, и колебаний освещения, и локальных геометрических искажений. Метод для получения таких дескрипторов предложил Дэвид Лоу [24], суть метода состоит в детектировании локальных семантических признаков входного изображения, известных также как SIFT–дескрипторы, их сохранении и сопоставлении извлеченных дескрипторов с признаками изображений, которые хранятся в иерархической базе данных. Инвариантность SIFT–дескрипторов относительно масштабирования и изменения освещённости достигается с помощью применения различных дифференциальных операторов, таких, как оператор Собеля, детектор Харриса, детектор Моравеца, детектор Ши–Томаса и др. Подробный обзор и анализ вышеописанных методов дан в работе [29].

2.2.2. Метод k -ближайших соседей

Метод представляет собой алгоритм, в котором функция аппроксимируется только локально, а все вычисления откладываются до момента классификации образа. Это один из простейших и фундаментальный метод классификации, когда предварительных знаний о распределении данных мало, или они вообще отсутствуют. В применении к задаче классификации образов обычно используют kNN -классификатор (k Nearest Neighbors, т.е. *ближайшие соседи*), который присваивает объект тому классу, который повторяется чаще всего среди k ближайших соседей данного элемента, классы которых уже известны. kNN -классификатор устойчив к шумам и легко обучается на сложных моделях. Его начальное состояние показано на рис. 2.14. kNN -классификатор присваивает объект тому классу, который повторяется чаще всего среди k ближайших соседей данного элемента, классы которых уже известны.



Рисунок 2.14. Начальное состояние kNN -классификатора

Результат работы kNN -классификатора можно увидеть на рис. 2.15.



Рисунок 2.15. Результат работы kNN -классификатора

2.2.3. Линейный классификатор

Линейный классификатор, или метод линейной регрессии, чаще всего применяется для бинарной классификации, когда имеется два класса объектов, причём точностью разделения этих объектов можно пренебречь. Этот метод классификации часто используется для линейно разделимых практических приложений. Линейный классификатор эффективен в пространствах больших размеров или в пространствах, где количество измерений больше, чем количество образцов. Недостаток линейного классификатора состоит в том, что усложнение аппроксимирующей функции и недостаток обучающих данных ведёт к потере производительности. Пример работы линейного классификатора продемонстрирован на рис. 2.16.

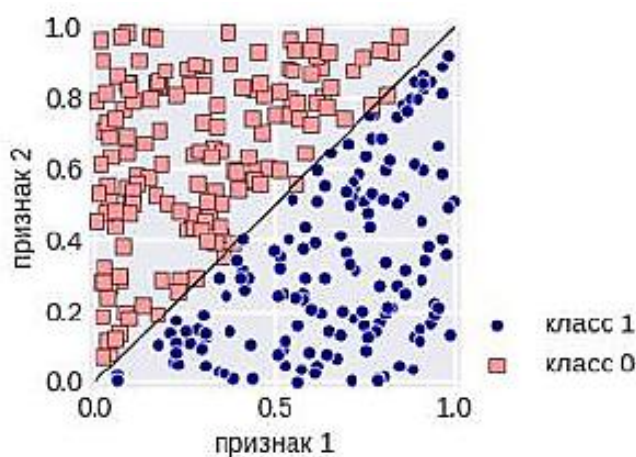


Рисунок 2.16. Результат работы линейного классификатора

2.2.4. Логистическая регрессия

Традиционное применение логистической регрессии относится к задаче прогнозирования (предсказания) появления конкретного события с помощью логистической кривой. Применительно к задаче бинарной классификации изображений, логистическая регрессия используется для определения фактора принадлежности объекта к конкретному классу.

Логистическая регрессия хорошо выделяет границы раздела различных классов объектов. Этот метод хорошо справляется с аппроксимацией выпуклых и непрерывно возрастающих функций, что необходимо, когда при обучении свёрточных нейронных сетей используются функции активации, такие, как гиперболический тангенс или степенная функция любого типа.

Пример работы логистического классификатора показан на рис. 2.17.

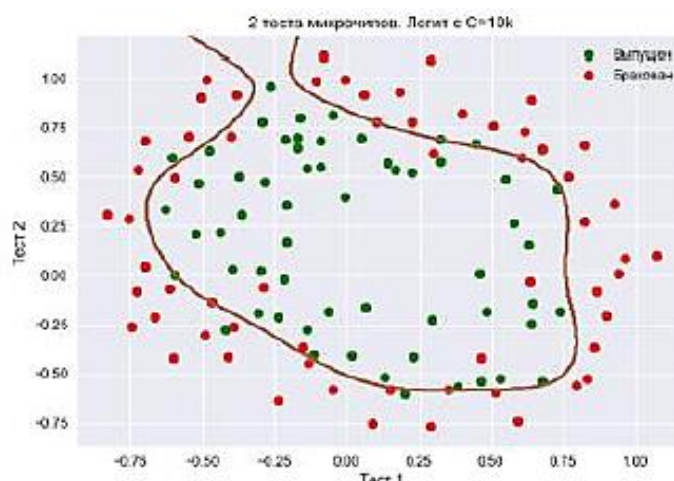


Рисунок 2.17. Результат работы логистического классификатора

Логистическую регрессию часто предпочитают линейной регрессии, по причине того, что она хорошо сохраняет промежуточные значения по кривой, распределяя значения случайной величины по всей области определения логистической функции. Вследствие этого, логистическая классификация хорошо сохраняет границы разделения объектов.

2.2.5. Методы сверточных нейронных сетей

Простейшие многослойные нейронные сети хорошо справляются с задачами бинарной классификации и линейной регрессии, но тривиальность архитектуры подобных структур не позволяет решать задачи с большими размерностями данных. В 2012 году Алекс Крижевский с коллегами опубликовал работу по распознаванию образов с помощью технологии свёрточных нейронных сетей. С тех пор технология СНС получила известность среди специалистов по машинному обучению. В работе [16] содержится архитектура СНС, показавшая высокую эффективность в решении задач классификации. Технология СНС неоднократно побеждала в соревнованиях по распознаванию изображений из набора данных ImageNet.

В отличие от традиционных подходов машинного обучения к решению задачи извлечения семантических признаков изображения, при которых процесс классификации объектов на изображении происходит независимо от процесса извлечения семантических признаков этого изображения, свёрточные нейронные сети автоматически генерируют карты признаков и используют их сразу в процессе классификации объектов при распознавании образов.

Архитектура СНС состоит из трёх типов узлов: входного, скрытого и выходного. Входной узел свёрточной нейронной сети представляет собой слой входных нейронов для обработки входных данных (изображения). Скрытый узел СНС представляет собой слой свертки, дополненный слоем объединения результатов свёртки, так, что входные данные уменьшенной размерности выбираются с помощью конкретного алгоритма (например, максимального объединения) и передаются на следующий узел СНС. Выходной узел свёрточной нейронной сети представляет собой результат классификации, или распознавания входного объекта.

На рис. 2.18 можно увидеть, что размерность входного изображения, представленного слева, изначально равная $64 \times 64 \times 32$ пикселей, уменьшается на каждом шаге работы СНС. В результате работы свёрточной нейронной сети для входного изображения получается одномерный вектор его семантических признаков, размерностью 4096 бит. Такое уменьшение размерности является одной из основных особенностей технологии СНС.

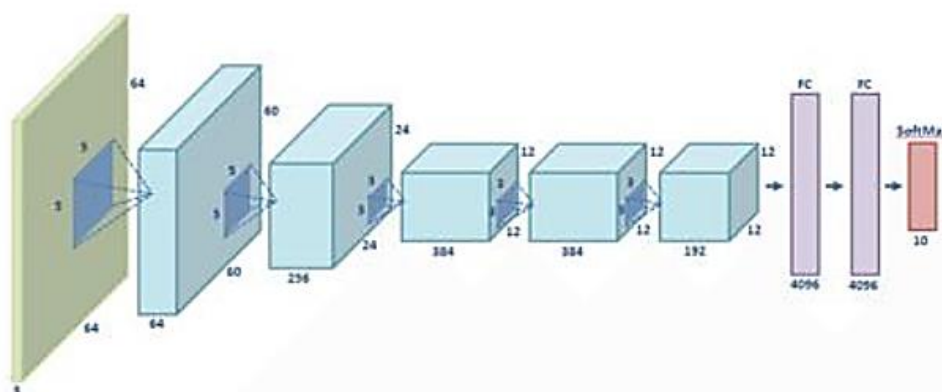


Рисунок 2.18. Извлечение признаков изображения с использованием СНС

Такое существенное уменьшение размерности входных данных и стало главной особенностью применения технологии свёрточных нейронных сетей при решении задач компьютерного зрения. Данная особенность возможна благодаря алгоритмам свёртки, которые получают на вход данные большей размерности, а на выходе выдают данные намного меньшей размерности. Алгоритмы свёртки основаны на применении различных функций активации нейронов, таких как ступенчатая, логистическая (сигмоидальная) функции, арктангенс (обратный тангенс), линейный выпрямитель (Rectified Linear Unit, ReLU), адаптивная кусочно–линейная функция, и множество других.

На рис. 2.19 показан исчерпывающий перечень нейронных функций активации, которые также применяются в свёрточных нейронных сетях. Такая СНС способна представить любое изображение в виде информативного вектора, который описывает семантический смысл изображения таким образом, что, в сочетании с соответствующей метрикой, информативный вектор может обеспечить хорошие результаты поиска изображений.

Название	Формула	Область значений
Линейная	$f(s) = k s$	$(-\infty, \infty)$
Полулинейная	$f(s) = \begin{cases} k s, & s > 0, \\ 0, & s \leq 0 \end{cases}$	$(0, \infty)$
Логистическая (сигмоидальная)	$f(s) = \frac{1}{1 + e^{-as}}$	$(0, 1)$
Гиперболический тангенс (сигмоидальная)	$f(s) = \frac{e^{as} - e^{-as}}{e^{as} + e^{-as}}$	$(-1, 1)$
Экспоненциальная	$f(s) = e^{-as}$	$(0, \infty)$
Синусоидальная	$f(s) = \sin(s)$	$(-1, 1)$
Сигмоидальная (рациональная)	$f(s) = \frac{s}{a + s }$	$(-1, 1)$
Шаговая (линейная с насыщением)	$f(s) = \begin{cases} -1, & s \leq -1, \\ s, & -1 < s < 1, \\ 1, & s \geq 1 \end{cases}$	$(-1, 1)$
Пороговая	$f(s) = \begin{cases} 0, & s < 0 \\ 1, & s \geq 0 \end{cases}$	$(0, 1)$
Модульная	$f(s) = s $	$(0, \infty)$
Знаковая (сигнатурная)	$f(s) = \begin{cases} 1, & s > 0, \\ -1, & s \leq 0 \end{cases}$	$(-1, 1)$
Квадратичная	$f(s) = s^2$	$(0, \infty)$

Рисунок 2.19. Извлечение признаков изображения с использованием СНС

Свёрточные нейронные сети часто используются для многомерной классификации объектов, и особенно эффективны для обработки больших наборов данных изображений. На этапе обучения СНС изучает иерархию изображения. Обученная СНС способна представить изображение в виде информативного вектора, описывающего семантический смысл изображения. В сочетании с соответствующей метрикой, обученная свёрточная нейронная сеть может обеспечить хорошие результаты поиска изображений.

2.3. Алгоритмическое конструирование программного средства

2.3.1. Анализ входных и выходных данных

Разработанное программное обеспечение корректно работает как с известными наборами тренировочных данных (MNIST, CIFAR–10 и Yahoo–1M), так и с пользовательскими наборами. Единственное требование, предъявляемое к подобным наборам тренировочных данных – это их соответствие формату входных данных, подходящего для обучения свёрточной нейронной сети.

Набор тренировочных данных MNIST состоит из 10 категорий рукописных цифр от 0 до 9 и содержит 60000 тренировочных и 10000 тестовых изображений.

Пример тренировочных данных из набора MNIST показан на рис. 2.20. Все рукописные цифры были нормализованы к оттенкам серого цвета и представлены на изображении в разрешении 28×28 пикселей.

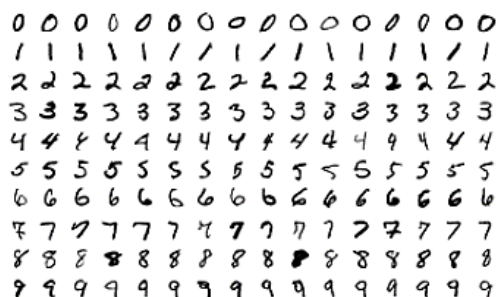


Рисунок 2.20. Набор тренировочных данных MNIST

Набор тренировочных данных CIFAR–10 состоит из 10 категорий объектов, причём каждый класс объектов содержит по 6000 изображений, что в сумме составляет 60000 картинок. CIFAR–10 разбит на 50000 тренировочных и 10000 тестовых изображений. Пример тренировочных данных из набора CIFAR–10 продемонстрирован на рис. 2.21.



Рисунок 2.21. Набор тренировочных данных CIFAR–10

Пример тренировочных данных из набора Yahoo-1М представлен на рис. 2.22. Набор содержит 1124087 изображений различных торговых продуктов, которые разбиты на 116 различных классов одежды. Датасет собран на основе данных Интернет–магазинов одежды, проиндексированных Yahoo. Все изображения помечены конкретной категорией.



Рисунок 2.22. Набор тренировочных данных Yahoo–1М

Наборы тренировочных изображений переведены в бинарный вид и записаны в файлы с расширением .ldb, что облегчает обработку больших объёмов входных данных. Пример набора входных данных для реализованного программного средства представлен на рис. 2.23.

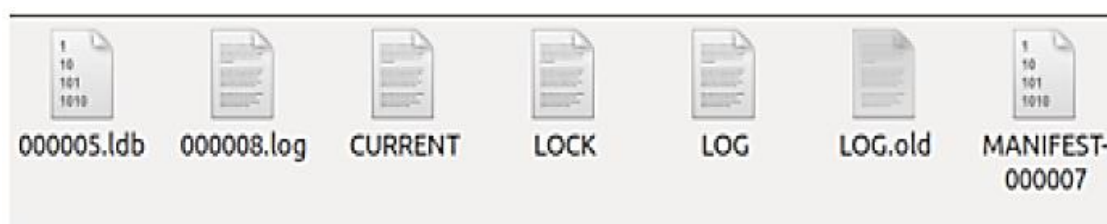


Рисунок 2.23. Набор входных данных для реализованного программного средства

Выходные данные разработанного программного средства представляют собой 48–битные хеш–значения для каждого входного изображения, которые сохраняются в файл, формата .hdf5, предназначенный для хранения большого объёма информации. Выходной файл формата .hdf5 содержит информацию из двух основных типов: Datasets, представляющие собой наборы данных, т.е. многомерные массивы объектов одного типа; и Groups – группы, которые являются контейнерами для наборов данных, а также других групп.

В данном исследовании в качестве групп выступают классы изображений, а в качестве наборов данных – вычисленные хеш–значения для каждого изображения из конкретного класса. На рис. 2.23 можно увидеть, что входные тренировочные изображения сохраняются в файл .ldb.

Наборы тренировочных изображений переведены в бинарный вид и записаны в файлы с расширением .ldb, что облегчает обработку больших объёмов входных данных. На рис. 2.23 изображена структура входного набора тренировочных изображений, а также все загруженные изображения. Тренировочные наборы состоят из 100 изображений.

На рис. 2.24 представлен один из тренировочных классов изображений, который подается на вход программного средства. На рис. 2.25 показана часть хеш–кодов для тренировочного класса изображений из рис. 2.24.

```

DATASET "image_lds" {
  DATATYPE HST_STRING {
    STRSIZE HST_VARIABLE;
    STRPAD HST_STR_NULLTERM;
    CSET HST_CSET_UTF8;
    CTYPE HST_C_S1;
  }
  DATASPACE SIMPLE ( ( 576 ) / ( H55_UNLIMITED ) )
  DATA {
    (0): "36603010.jpg", "77407295.jpg", "62515922.jpg", "31846290.jpg",
    (4): "02725022.jpg", "69772057.jpg", "51222478.jpg", "61062818.jpg",
    (8): "69747271.jpg", "85556800.jpg", "77404936.jpg", "94983569.jpg",
    (12): "52160159.jpg", "90426681.jpg", "13390501.jpg", "67412308.jpg",
    (16): "34224647.jpg", "32362498.jpg", "88154873.jpg", "48611354.jpg",
    (20): "46893705.jpg", "20727020.jpg", "14535856.jpg", "82160698.jpg",
    (24): "75352615.jpg", "27386937.jpg", "35157228.jpg", "44505817.jpg",
    (28): "96542947.jpg", "12545153.jpg", "18452727.jpg", "05720118.jpg",
    (32): "37752877.jpg", "91969825.jpg", "38247521.jpg", "47066893.jpg",
    (36): "94678761.jpg", "94363774.jpg", "43993331.jpg", "47921452.jpg",
    (40): "34714551.jpg", "40740785.jpg", "01194298.jpg", "52406698.jpg",
    (44): "86830945.jpg", "80663701.jpg", "89510415.jpg", "82223319.jpg",
    (48): "69926107.jpg", "88838732.jpg", "33931743.jpg", "35366019.jpg",
    (52): "08434888.jpg", "85296108.jpg", "33778526.jpg", "18524137.jpg",
    (56): "41050206.jpg", "91518131.jpg", "59553390.jpg", "75060010.jpg",
    (60): "78740534.jpg", "64906693.jpg", "97223705.jpg", "90694910.jpg",
    (64): "39660954.jpg", "15047138.jpg", "16041991.jpg", "56921659.jpg",
    (68): "07029053.jpg", "08114089.jpg", "11820501.jpg", "09777776.jpg",
    (72): "62239343.jpg", "41626252.jpg", "66663213.jpg", "83213901.jpg",
    (76): "34394238.jpg", "59163587.jpg", "00206614.jpg", "63915315.jpg",
    (80): "06999509.jpg", "55625963.jpg", "69216532.jpg", "56871699.jpg",
    (84): "44525948.jpg", "74175055.jpg", "89462560.jpg", "87424475.jpg",
    (88): "09594814.jpg", "98196405.jpg", "93404590.jpg", "80285336.jpg",
    (92): "89989352.jpg", "85954385.jpg", "58577883.jpg", "48970072.jpg",
    (96): "66034014.jpg", "61087521.jpg", "01969460.jpg", "48618253.jpg",
    (100): "55018290.jpg", "79503689.jpg", "52289756.jpg", "47766750.jpg"
  }
}

```

Рисунок 2.24. Пример тренировочного класса изображений

```

DATASET "Index" {
  DATATYPE HST_STD_I64LE
  DATASPACE SIMPLE ( ( 576, 2 ) / ( H55_UNLIMITED, 2 ) )
  DATA {
    (0,0): 0, 0,
    (1,0): 1, 1,
    (2,0): 2, 2,
    (3,0): 3, 3,
    (4,0): 4, 4,
    (5,0): 5, 5,
    (6,0): 6, 6,
    (7,0): 7, 7,
    (8,0): 8, 8,
    (9,0): 9, 9,
    (10,0): 10, 10,
    (11,0): 11, 11,
    (12,0): 12, 12,
    (13,0): 13, 13,
    (14,0): 14, 14,
    (15,0): 15, 15,
    (16,0): 16, 16,
    (17,0): 17, 17,
    (18,0): 18, 18,
    (19,0): 19, 19,
    (20,0): 20, 20,
  }
}

```

Рисунок 2.25. Вычисленные хеш–значения для входного изображения

2.3.2. Структура программного средства хеширования изображений на основе свёрточных нейронных сетей

Общая схема разработанного программного средства представлена на рис. 2.26. После загрузки пользовательского набора данных выполняется процесс их бинаризации и приведения к формату .ldb, описанному ранее.

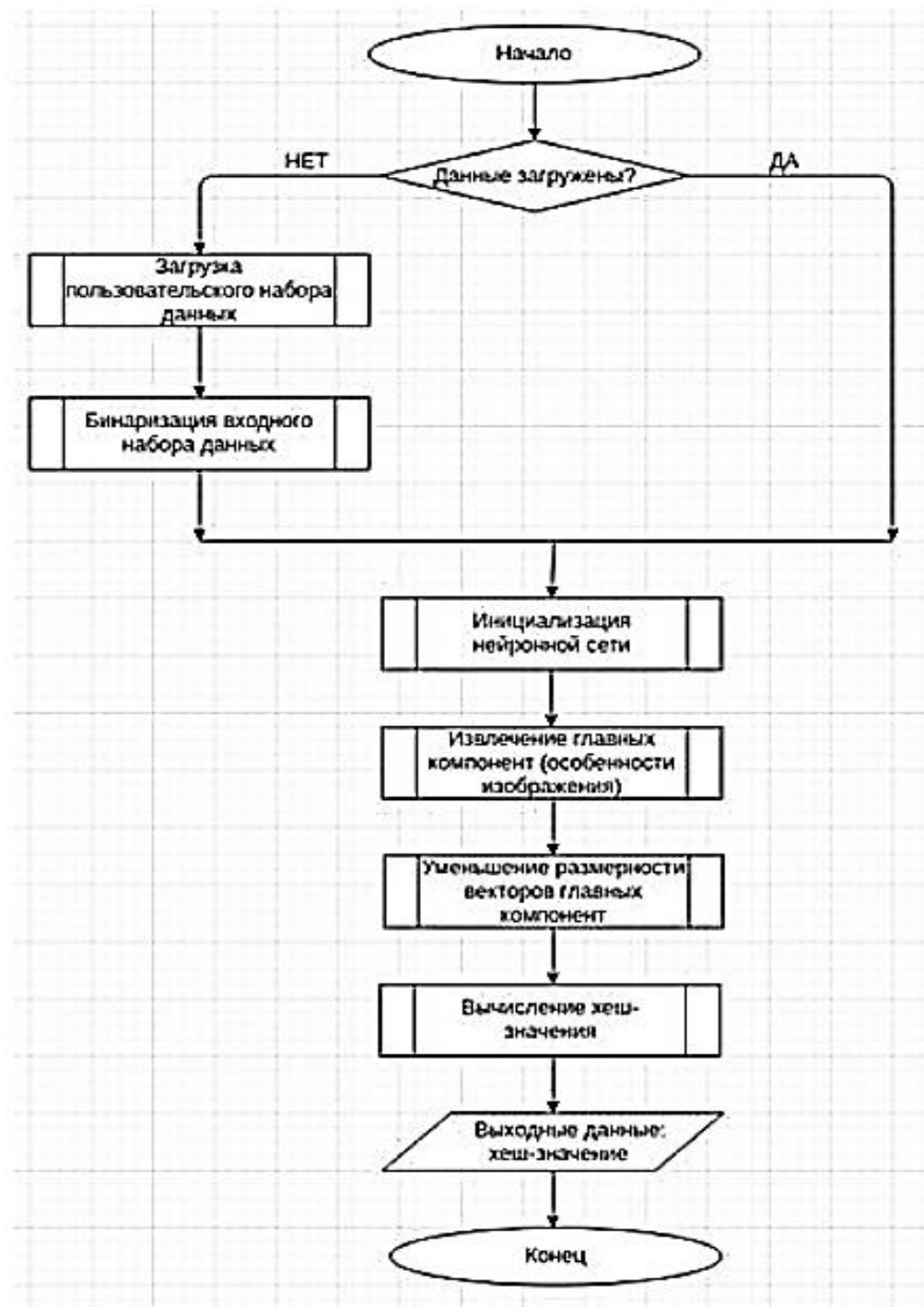


Рисунок 2.26. Схема работы программного средства

Затем выполняется процесс инициализации свёрточной нейронной сети, на основе входных данных, и дальнейшее вычисление хеш-значений для всех входных изображений. На схеме видно, что если перед началом работы ПО и загрузки входных данных в СНС тренировочные данные отсутствуют, то пользователь может загрузить собственный набор тренировочных данных.

Разработанное ПО представлено в виде пользовательской библиотеки и работает в качестве консольной утилиты. Выбор в пользу консольной утилиты и отсутствия графического интерфейса был сделан намеренно, с целью упрощения взаимодействия пользователя с ПО. Если графический интерфейс является слишком сложным и многоуровневым, то это значительно увеличивает время знакомства пользователя с программным средством, и усложняет его понимание. ПО предназначено для свободного использования, и консольная утилита является оптимальной формой для его распространения, которая ускоряет работу, так как не требует дополнительных визуальных окон.

2.3.3. Описание реализованных алгоритмов

Для функционирования программы необходимо описать алгоритмы бинаризации входных данных, обучения нейронной сети, регуляризации и нормализации нейронной сети, а также алгоритм получения хеш-значения и поиск похожих изображений на основе вычисленного хеш-значения.

А) Бинаризация входных данных

Независимо от того, с какими данными работает программное средство (с пользовательскими данными или тренировочными изображениями одного из обучающих датасетов), входные данные необходимо привести к формату .ldb для того, чтобы передать их на инициализацию нейронной сети. Схема алгоритма бинаризации входных данных и их упаковки в файл .ldb показана на рис. 2.27. Сначала тренировочный набор данных считывается из файла, а затем построчно преобразовывается в двоичный формат.

Для хранения тренировочного набора изображений используется формат данных .ldb, позволяющий хранить большие объёмы информации и поддерживает иерархическую структуру данных. После создания базы данных, тренировочный набор изображений преобразовывается в формат .ldb и упаковывается в созданную ранее базу данных.

Формат баз данных .ldb является оптимальным для хранения больших наборов данных с иерархической структурой, что позволяет не тратить время на разработку высокоэффективных алгоритмов обработки изображений в классических базах данных. База данных .ldb представляет универсальный метод хранения любых входных данных, которые имеют иерархическую структуру, что позволяет эффективно хранить входные изображения.



Рисунок 2.27. Схема алгоритма бинаризации входных данных

Б) Инициализация нейронной сети

Перед тем, как запускать свёрточную нейронную сеть и алгоритмы хеширования, нужно проинициализировать все слои и весовые коэффициенты СНС. Схема алгоритма инициализации СНС представлена на рис. 2.28.

На схеме алгоритма инициализации видно, что сначала нужно создать объект класса Solver, а затем, в цикле, пройти по всем слоям свёрточной нейронной сети и случайным образом сгенерировать все весовые коэффициенты.

Случайная генерация всех весовых коэффициентов нейронных слоев нужна, чтобы предотвратить предвзятость обучения свёрточной нейронной сети к определенному типу данных, и необходимость её переобучения.



Рисунок 2.28. Схема алгоритма инициализации свёрточной нейронной сети

В) Обучение свёрточной нейронной сети

Схема алгоритма обучения свёрточной нейронной сети представлена на рис. 2.29. На схеме алгоритма обучения показано, что в каждой итерации (эпохе) обучения СНС выполняет прямой проход, в котором вычисляет значения функций активации для каждого слоя и сохраняет состояния весовых коэффициентов и параметров сдвига, а затем реализует обратный проход, в котором вычисляет значения производных функций активации.

С помощью полученных значений производных, свёрточная нейронная сеть корректирует значения весовых коэффициентов на каждом слое, что позволяет минимизировать функцию ошибки. Этот метод обучения сети называется *градиентным спуском* и является одним из основных методов обучения любых нейронных сетей, в том числе, свёрточных.



Рисунок 2.29. Схема алгоритма обучения нейронной сети

Г) Вычисление хеш-значения для входного изображения

Недавние исследования [18–20] показали, что значения функций активации последних слоёв свёрточной нейронной сети может служить визуальным дескриптором входного изображения. Использование таких визуальных дескрипторов демонстрирует впечатляющие результаты, нужные для задач классификации изображений, поиска и других задач компьютерного зрения.

Однако такие визуальные дескрипторы являются многомерными, и требуют слишком больших затрат вычислительных ресурсов.

Авторы предлагают, наряду с использованием свёрточной нейронной сети для извлечения семантических признаков изображения, использовать дополнительный скрытый слой, задача обучения которого – аппроксимация хеш-функции для заданного вектора семантических признаков.

Чтобы облегчить эффективный поиск изображений, используется практичный способ преобразования результатов функций активации в двоичные коды. Такие двоичные компактные коды могут быть использованы для хеширования, а затем, для вычисления расстояния Хэмминга.

Обучение скрытого слоя для вычисления хеш-значения происходит благодаря имеющимся помеченным тренировочным данным и минимизации функции потерь от погрешности работы свёрточной нейронной сети. Структура скрытого слоя показана на рис. 2.30.

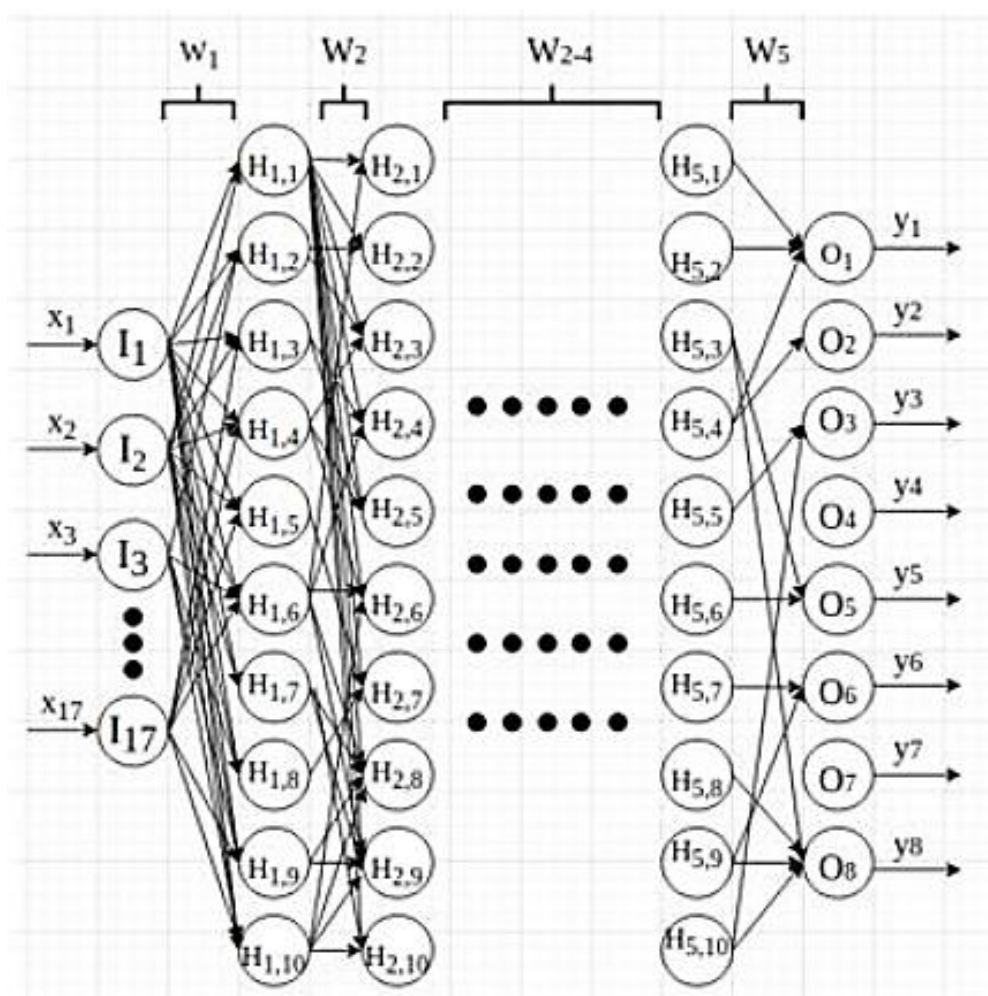


Рисунок 2.30. Структура скрытого слоя СНС для вычисления хеш-значения изображения

На рис. 2.30 видно, что скрытый слой свёрточной нейронной сети имеет параметры (17, 50, 8). Это означает, что данный слой использует для вычисления двоичных кодов 17 входных семантических признаков, 50 скрытых нейронов и 8 выходных нейронов, чьи значения функций активации используются для составления хеш–значения. Указанные параметры скрытого слоя свёрточной нейронной сети являются компромиссом между требуемой точностью извлечения изображений из базы данных и вычислительными ресурсами обучаемой машины. На рис. 2.31 представлена архитектура СНС, применяемая при реализации программного обеспечения.

```

name: "AlexNet"
layer {
  name: "data"
  type: "Input"
  top: "data"
  input_param { shape: { dim: 10 dim: 3 dim: 227 dim: 227 } }
}
layer {
  name: "conv1"
  type: "Convolution"
  bottom: "data"
  top: "conv1"
  param {
    lr_mult: 1
    decay_mult: 1
  }
  param {
    lr_mult: 2
    decay_mult: 0
  }
  convolution_param {
    num_output: 96
    kernel_size: 11
    stride: 4
  }
}
layer {
  name: "relu1"
  type: "ReLU"
  bottom: "conv1"
  top: "conv1"
}

```

Рисунок 2.31. Архитектура свёрточной нейронной сети для хеширования изображений

Настройки модели свёрточной нейронной сети сохраняются в специальном proto-файле, структура которого показана на рис. 2.31. Лаконичность такого формата позволяет эффективно создавать и сохранять новые архитектуры СНС.

Д) Поиск изображения на основе его хеш-значения

Поиск изображения, на базе его хеш-значения, основан на алгоритме нахождения приблизительных ближайших соседей (kNN – классификаторе). Схема алгоритма поиска продемонстрирована на рис. 2.32.



Рисунок 2.32. Схема алгоритма вычисления хеш-значения для изображения на основании вектора его семантических признаков

Извлечение изображения из базы данных происходит в несколько этапов:

- получение списка возможных кандидатов, подходящих на роль искомого изображения, с помощью алгоритма приблизительного ближайшего соседа;
- определение наилучших кандидатов с помощью минимизации расстояния Хэмминга между векторами главных признаков изображений;
- сортировка списка наилучших кандидатов среди похожих изображений, в порядке возрастания характеристики расстояния Хэмминга.

На основании многочисленных исследований, опубликованных в работах [25–27], сделан вывод о том, что расстояние Хэмминга является оптимальной характеристикой определения похожих изображений.

Поэтому минимальное расстояние Хэмминга между хеш–значениями изображений указывает на тот факт, что изображения должны быть схожи, при условии корректной работы свёрточной нейронной сети и метода извлечения главных признаков изображения.

2.4. Программное конструирование разрабатываемого средства

2.4.1. Выбор языка программирования и средств разработки

Перед началом разработки программного средства следует определиться с выбором языка программирования и среды разработки, так как именно это влияет на средства разработки, методы хранения данных и их использование.

Разрабатываемое ПО должно иметь следующий функционал:

- использование программного средства в консольном режиме;
- возможность изменения архитектуры свёрточной нейронной сети;
- инициализация работы самой свёрточной нейронной сети;
- обучение СНС семантическому хешированию изображений;
- вычисление расстояния Хэмминга между векторами семантических признаков;
- отображение информации о состоянии нейронной сети и процессе её обучения;
- сохранение текущей сессии в файл;
- загрузка уже сохраненной сессии.

Учитывая перечисленные выше функции, которые нужно реализовать в программном средстве, язык программирования должен обладать:

- возможностью использования средств параллельного программирования для ускорения обучения свёрточной нейронной сети;

- эффективными алгоритмами работы со стандартными контейнерами для вычисления расстояния Хэмминга и сортировки списка кандидатов;
- качественными алгоритмами работы с файлами .hdf5 и .ldb.

На данный момент существует много языков программирования, которые обладают перечисленными выше возможностями:

- С-подобные языки: C, C++, C#, которые можно использовать совместно с различными средствами разработки графического интерфейса пользователя, к примеру, Direct 3D, Windows Forms, Qt и со многими другими. С-подобные языки имеют как множество преимуществ (корректная работа с байт-кодом, компилируемость программного кода, высокая скорость выполнения программного кода), так и множество недостатков (статическая типизация, неудобство работы со строковым типом, а также с потоком ввода-вывода);
- скриптовые языки: Lua, PHP, Python, Ruby, также имеющие возможность работы с различными средствами разработки графического интерфейса типа PyQt (аналога Qt с поддержкой языка Python), Tkinter, gtk, vxruby и др. Скриптовые языки также имеют свои преимущества (динамическая типизация, высокая скорость разработки, небольшой вес итогового файла ПО, удобство работы со строковым типом данных) и недостатки (невысокая скорость выполнения программного кода, интерпретируемость программного кода).

После сравнительного анализа языков программирования, для разработки программного средства было решено использовать язык программирования C++ (для написания частей программы, взаимодействующих со средствами параллельного программирования и программирования CUDA), а также язык программирования Python. Комбинирование средств разработки позволяет извлечь максимальную пользу как из низкоуровневых интерфейсов C++ (для написания многопоточного кода и взаимодействия с ядрами CUDA), так и из простоты синтаксиса Python (для написания высокоуровневого интерфейса для подключения реализованной библиотеки). Программирование на языке C++ предоставляет большую гибкость в настройке ядер CUDA.

C++ представляет собой высокоуровневый язык программирования, поддерживающий большинство существующих на сегодняшний день парадигм программирования, включая объектно-ориентированное, функциональное, логическое, визуальное, модульное программирование и др. По сравнению с вышеперечисленными языками, C++ имеет ряд преимуществ:

- высокая типобезопасность и компилируемость программного кода;
- низкоуровневые интерфейсы для программирования ядер CUDA;
- высокая эффективность стандартных алгоритмов.

Python является высокоуровневым ЯП, поддерживающим большинство существующих на сегодняшний день парадигм программирования, включая объектно–ориентированное, функциональное, логическое программирование и др. Python – стремительно развивающийся язык, имеющий библиотеки для работы практически во всех сферах информационных технологий, начиная со строгих математически обоснованных вычислений, и заканчивая широко распространёнными в последнее время свёрточными нейронными сетями. По сравнению с перечисленными выше языками, Python имеет ряд преимуществ:

- удобство работы со скриптами: Python – изначально скриптовый язык программирования, поэтому разработка скриптов на нём выполняется быстрее;
- читаемость программного кода, что означает невысокий порог вхождения для пользователей, которые хотели бы получше разобраться с предлагаемым программным средством и помочь в его поддержке и развитии;
- интерпретируемость и кроссплатформенность разрабатываемого ПО;
- динамическая типизация для удобства работы со структурами данных;
- простота разработки, по сравнению с более низкоуровневыми ЯП.

Для разработки выбрана интегрированная среда Visual Studio Code. Алгоритмы, рассмотренные ранее, реализованы в программных модулях:

- модуль `tools/convert_imageset.cpp` – бинаризация входного набора изображений;
- модуль `caffe/solver.cpp` – инициализация свёрточной нейронной сети;
- модуль `tools/train_net.cpp` – обучение и настройка нейронной сети;
- модуль `libs/cprw15/DeepFeaturesExtractor.py` – извлечение семантических признаков изображения, необходимых для его распознавания;
- модуль `libs/cprw15/BinHashExtractor.py` – вычисление хеш–значения для изображения на основе его семантических признаков;
- модули `libs/indexer/BaseIndexer.py` и `libs/indexer/DeepIndexer.py` – алгоритмы глубокого иерархического поиска в базе данных `.ldb`.

2.4.2. Модули для разработки программного средства

Для полноценной работы ПО требуются следующие модули языка Python:

- `os` (работа с возможностями операционной системой);
- `sys` (работа с потоком ввода вывода и аргументами командной строки);

– sklearn (sklearn.metrics.hamming_loss – для вычисления расстояния Хэмминга, sklearn.metrics.pairwise.pairwise_distances для вычисления расстояния между векторами семантических признаков каждого изображения);

– scikit-learn (scikit-learn.quantization для вычисления расстояния Хи-квадрат, sklearn.metrics.pairwise.triple_distances – для вычисления расстояния между векторами хеш-значений, необходимых для поиска изображения);

– numpy (модуль работы с матричными типами array);

– scipy (модуль scipy.io для сохранения хеш-значений в двоичном формате);

– h5py (модуль h5py.File для сохранения хеш-значений для всех классов тренировочных изображений, в специальном формате hdf5).

Для корректной работы ПО также требуются модули языка C++:

– boost (boost/thread.hpp для параллельного программирования);

– glog (модуль glog/logging.h предназначен для гибкого логирования и сохранения данных в процессе обучения свёрточной нейронной сети);

– caffe (модуль caffe/common.hpp предназначен для контроля использования предварительно обученной свёрточной нейронной сети AlexNet);

– cublas (модуль cublas_v2.h необходим для программирования ядер CUDA);

– opencv2 (opencv2/core/core.hpp нужен для обработки входного изображения).

2.4.3. Функция модуля *src/caffe/layers*

Модуль *src/caffe/layers* предоставляет иерархию классов для различных слоёв свёрточной нейронной сети. Примерная структура иерархии классов модуля *layers* показана на рис. 2.33. На диаграмме представлена общая архитектура свёрточной нейронной сети, которая состоит из различных слоев (класс *ConvLayer*), а те, в свою очередь, состоят из свёрточных ядер (класс *ConvKernel*). Слои отличаются друг от друга типом свёрточных ядер, которые, в свою очередь, отличаются друг от друга функцией активации нейронов и функцией выборки. Подобная иерархия классов позволяет использовать преимущества объектно-ориентированного программирования и объектной архитектуры программного средства, что предоставляет большую гибкость в настройке и изменении структуры СНС и алгоритмов ее обучения. Для создания своей архитектуры свёрточной нейронной сети пользователю достаточно указать типы свёрточных узлов и вид объединяющей функции.

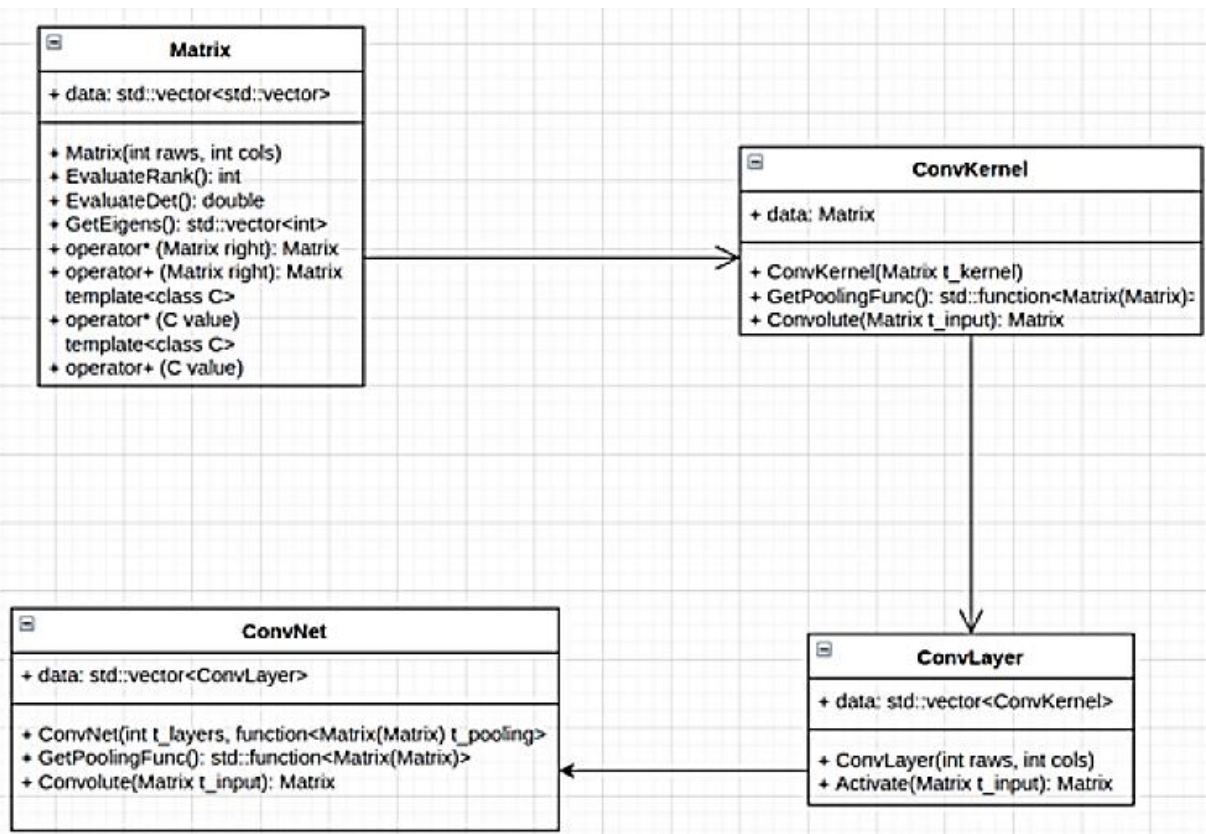


Рисунок 2.33. Иерархия классов модуля src/caffe/layers

Описанная гибкость иерархии классов `src/caffe/layers` делает библиотеку `caffe` незаменимым средством в разработке программного обеспечения, использующих возможности свёрточных нейронных сетей. С помощью этой библиотеки можно писать эффективные алгоритмы обучения и дообучения СНС на определенном пользовательском наборе тренировочных данных.

Особого внимания заслуживают функции обработки входных данных `ConvNet.Convolute(Matrix t_input)` и `ConvLayer.Activate(Matrix t_input)`.

Метод `ConvNet.Convolute(Matrix t_input)`. Входные данные: матрица пикселей входного изображения. Задача данного метода заключается в уменьшении размерности входных данных посредством применения функции свертки и функции pooling-узла для извлечения наиболее значащих бит. Выходные данные: уменьшенная матрица пикселей входного изображения.

Метод `ConvLayer.Activate(Matrix t_input)`. Входные данные: матрица весовых коэффициентов нейронного слоя. Задачей метода является вычисление функций активации нейронов и сохранение весовых коэффициентов в кэш. Выходные данные: вектор значений функции активации.

Метод `ConvLayer.Deactivate(Matrix t_input)`. Входные данные: матрица весовых коэффициентов нейронного слоя. Задачей метода является вычисление функций активации нейронов и сохранение весовых коэффициентов в кэш. Выходные данные: вектор значений функции активации.

2.4.4. Основные методы модуля `libs/cprw15/BinHashExtractor.py`

Модуль `libs/cprw15/BinHashExtractor.py` содержит функции для вычисления хеш–значения изображения на основе вектора его семантических признаков.

Метод `BinHashExtractor._init_(use_gpu, feat_len, model_prototxt, model_file)`. Входные данные: булево значение `use_gpu`, которое содержит значение 0, если пользователь не желает использовать мощности видеокарты, и значение 1 в противном случае; `feat_len` – размер пакета нормализации, `model_prototxt` – proto–файл, содержащий конфигурацию модели свёрточной нейронной сети, `model_file` – файл, содержащий настройки обучения этой СНС. Представляет собой конструктор класса `BinHashExtractor`. Предназначение функции `_init_` – создание объектов класса `BinHashExtractor`.

Метод `BinHashExtractor.Extract(list_im, layer_n)`. Входные данные: список весовых коэффициентов скрытого слоя нейронной сети для вычисления хеш–значения и номер скрытого слоя. Функция предназначена с целью добавления в структуру СНС дополнительного скрытого слоя, который в дальнейшем будет обучаться уменьшению погрешности работы свёрточной нейронной сети, таким образом, минимизируя расстояние Хэмминга между хеш–значениями схожих изображений. Выходные данные: 48–битное хеш–значение.

Суть работы модуля `BinHashExtractor.py` заключается в том, чтобы принять в качестве входных данных вектор главных признаков входного изображения, а затем запустить процесс дообучения свёрточной нейронной сети таким образом, чтобы СНС минимизировала погрешность сохранения информации из входного изображения в хеш–значении. Запуск функции обучения СНС выполняется с помощью вызова функции `caffe_batch_feat` модуля `libs/cvprw15/feat_tools.py`, которая отвечает за запуск обучения СНС.

2.4.5. Основные методы модуля `libs/indexer/DeepIndexer.py`

Модуль `libs/indexer/DeepIndexer.py` содержит скрипты для создания иерархической базы данных изображений, а также добавления новых изображений в базу данных по их хеш–значениям. Модуль содержит классы `DeepIndexer` и `DeepSearcher`, описанные ниже.

Метод `DeepIndexer.add (imageID, feature_vector, binary_code)`. Входные данные: `imageID` – ID входного изображения, `feature_vector` – вектор семантических признаков изображения, `binary_code` – хеш–значение изображения. Предназначением функции `add`, как следует из её названия, является добавление нового изображения в базу данных на основании вектора его семантических признаков и хеш–значения. Выходные данные отсутствуют.

Метод `DeepIndexer._createDatasets()`. Входные данные отсутствуют. Вспомогательный метод создания набора данных для размещения изображения в базе `.ldb`. Выходные данные отсутствуют.

Метод `DeepIndexer.finish()`. Входные данные отсутствуют. Этот метод предназначен для завершения инициализации и заполнения базы изображений `.ldb`. После заполнения метод закрывает базу данных с помощью функции `db.close()`. Выходные данные отсутствуют.

Метод `DeepSearcher.search(binary_code, fvector, maxCandidates=200)`. Входные данные: `binary_code` – хеш–значение входного изображения, `fvector` – вектор семантических признаков изображения, `maxCandidates` – необязательный параметр, задающий максимально допустимое число найденных кандидатов. Предназначение функции `search`, как следует из её названия, поиск приблизительных ближайших соседей на основании вычисления расстояния Хэмминга между хеш–значениями входного изображения и изображений, содержащихся в базе `.ldb`. Метод сортирует список найденных кандидатов в порядке возрастания расстояния Хэмминга между хеш–кодами изображений, тем самым выдавая наиболее подходящие результаты первыми. Выходные данные: список подходящих кандидатов (изображений).

Метод `DeepSearcher.findCandidates(binary_code, maxCandidates)`. Входные данные: `binary_code` – хеш–значение входного изображения. Метод выполняет эффективный поиск подходящих кандидатов. Выходные данные отсутствуют.

2.4.6. Основные методы модуля `libs/cprw15/feat_tools.py`

Модуль `libs/cprw15/feat_tools.py` содержит глобальные методы подготовки входных данных для свёрточной нейронной сети, инициализации и обучения СНС с применением пакетной нормализации. Рассмотрим методы подробнее:

Метод `caffe_init_feat (use_gpu, model_prototxt, model_file)`. Входные данные: `use_gpu` – булево значение, указывающее, применяет ли пользователь возможности вычисления на видеокарте или использует исключительно процессор, `model_prototxt` – абсолютный путь к файлу с настройками обучения СНС, `model_file` – абсолютный путь к файлу с конфигурацией модели СНС.

Метод `caffe_init_feat` инициализирует переданные параметры, а также создает объект класса `caffe.Net` с указанием файлов `model_prototxt` и `model_file`.

Метод `prepare_batch (image_files, transformer)`. Входные данные: `image_files` – список входных изображений, `transformer` – функция трансформации для обработки входных изображений. Метод `prepare_batch` загружает переданный список изображений и передает его функции трансформации с помощью вызова метода `transformer.preprocess ()` с указанием загруженного изображения.

Метод `caffe_batch_feat (list_im, net, feat_len, ilsvrc_2012_mean_pn, layer_n)`. Входные данные: `list_im` – список изображений, `net` – созданный ранее объект класса `caffe.Net`, `feat_len` – размер пакета нормализации, `ilsvrc_2012_mean_pn` – функция для вычисления среднего, `layer_n` – номер узла для хеширования изображений. Предназначение метода `caffe_batch_feat` – запуск обучения свёрточной нейронной сети, переданной в объекте `net`, с пакетной нормализацией длины `feat_len`. Цель обучения – минимизация погрешности трансформации главных признаков входного изображения в хеш–пространство. Параметр `layer_n` указывает на номер узла, результатом выполнения которого является хеш–значение, соответствующие входному изображению.

2.4.7. Функция модуля `tools/index_bin48_feat.py`

Модуль `tools/index_bin48_feat.py` содержит скрипт для запуска всего программного обеспечения. Скрипт эксплуатирует все рассмотренные ранее классы `BinHashExtractor` для запуска обучения свёрточной нейронной сети и классы `DeepIndexer` для сохранения изображений с помощью хеш–значений.

Модуль `tools/index_bin48_feat.py` выполняется в несколько этапов:

- парсинг аргументов командной строки и получение параметров `use_gpu`, `model_file`, `feat_proto`, `bin_proto`, `products_dir`, `deep_db`;
- создание объекта класса `DeepIndexer`, описанного выше;
- создание объекта класса `BinHashExtractor`, также описанного ранее;
- вызов метода `BinHashExtractor.extract()` для запуска обучения свёрточной нейронной сети и вычисления хеш–значений для входных изображений;
- создание объекта класса `DeepFeaturesExtractor` для извлечения главных признаков изображения, на основе ранее вычисленного хеш–значения;
- добавление входных изображений в базу данных `.ldb` на основе ранее вычисленных хеш–значений изображений, необходимых при их поиске.

2.4.8. Функция модуля *tools/deep_search.py*

Функция модуля *tools/deep_search.py* очень похожа на функцию модуля *tools/index_bin48_feat.py*. Этот скрипт запускает программу и эксплуатирует все ранее рассмотренные классы за единственным различием: *DeepIndexer* используется уже для поиска изображений в ранее сформированной базе данных *.ldb* на основе соответствующих хеш–значений.

Модуль *tools/deep_search.py* выполняется в несколько этапов:

- парсинг аргументов командной строки и получение параметров *use_gpu*, *model_file*, *feat_proto*, *bin_proto*, *products_dir*, *query*;
- создание объекта класса *BinHashExtractor*, описанного ранее;
- вызов метода *BinHashExtractor.extract()* для запуска обучения свёрточной нейронной сети и вычисления хеш–значений для входных изображений;
- создание нового объекта класса *DeepSearcher*, предназначенного для поиска изображения в базе данных *.ldb*;
- вызов метода *DeepSearcher.search()* для поиска изображения или похожих на него в базе данных *.ldb* на основе хеш–значения входного изображения;
- сортировка списка лучших кандидатов и формирование выходных данных.

Задачей модуля *tools/deep_search.py* является загрузка входного изображения, которое представляет собой запрос к базе данных *.ldb*, вычисление хеш–значения для изображения–запроса и поиск изображений в базе данных *.ldb*, которые будут схожи с изображением, содержащимся в запросе. Этот модуль использует алгоритмы поиска приблизительного ближайшего соседа и вычисления расстояния Хэмминга, которые были описаны ранее.

2.5. Демонстрация работы программного средства

Реализованное ПО можно подключать к исходным программным файлам на языках C++ и Python, а также запускать в виде консольной утилиты.

2.5.1. Запуск консольной утилиты для обучения нейронной сети

Рассмотрим пример запуска скрипта для начала процесса обучения свёрточной нейронной сети на основании загруженный тренировочных данных. Пример запуска консольной утилиты показан на рис. 2.34.

```

root@quickbrains:~/opt/cbir_binary_code# sudo bash tools/indexing.sh 0 /opt/cbir_binary_code/examples/foods25/RomanyNet_foods25_48_iter_30000.caffemodel /opt/cbir_binary_code/examples/main/deploy_fc7.prototxt /opt/cbir_binary_code/examples/foods25/RomanyNet_foods25_48_deploy.prototxt /opt/cbir_binary_code/data/foods25/lnqs /opt/cbir_binary_code/data/foods25/foods25_48_deepDB.hdf5 foods25
(sudo) password for quickbrains:
# cat > e
# export PYTHONUNBUFFERED=True
# PYTHONUNBUFFERED=True
# USE_GPU=0
# MODEL_FILE=/opt/cbir_binary_code/examples/foods25/RomanyNet_foods25_48_iter_30000.caffemodel
# FEAT_PROTO=/opt/cbir_binary_code/examples/main/deploy_fc7.prototxt
# BIN_PROTO=/opt/cbir_binary_code/examples/foods25/RomanyNet_foods25_48_deploy.prototxt
# PROD_DIR=/opt/cbir_binary_code/data/foods25/lnqs
# DEEP_DB=/opt/cbir_binary_code/data/foods25/foods25_48_deepDB.hdf5
# DATASET=foods25
# date +%Y-%m-%d_%H-%M-%S
# LOG=logs/foods25_indexing_bin48_feat.txt.2021-01-23_11:55:51
# #noc
# tee -a logs/foods25_indexing_bin48_feat.txt.2021-01-23_11:55:51
tee: logs/foods25_indexing_bin48_feat.txt.2021-01-23_11:55:51: No such file or directory
# echo logging output to logs/foods25_indexing_bin48_feat.txt.2021-01-23_11:55:51
logging output to logs/foods25_indexing_bin48_feat.txt.2021-01-23_11:55:51
# python tools/index_bin48_feat.py --use_gpu 0 --model_file /opt/cbir_binary_code/examples/foods25/RomanyNet_foods25_48_iter_30000.caffemodel --feat_proto /opt/cbir_binary_code/examples/main/deploy_fc7.prototxt --bin_proto /opt/cbir_binary_code/examples/foods25/RomanyNet_foods25_48_deploy.prototxt --products_dir /opt/cbir_binary_code/data/foods25/lnqs --deep_db /opt/cbir_binary_code/data/foods25/foods25_48_deepDB.hdf5
model_prototxt: /opt/cbir_binary_code/examples/foods25/RomanyNet_foods25_48_deploy.prototxt
model_file: /opt/cbir_binary_code/examples/foods25/RomanyNet_foods25_48_iter_30000.caffemodel
Using CPU Mode

```

Рисунок 2.34. Инициализация аргументов командной строки

Показан пример запуска скрипта tools/indexing.sh с параметрами:

- use_gpu 0 – означает, что используются только возможности процессора;
- model_file – файл с конфигурацией модели нейронной сети;
- feat_proto – прото-файл с настройками обучения нейронной сети;
- bin_proto – файл с настройками характеристик выходного хеш–значения;
- products_dir – папка (директорий) с тренировочными данными
- deep_db – файл для сохранения вычисленных хеш–значений.

После запуска скрипта можно увидеть, что все аргументы, полученные из командной строки, были проинициализированы, и все нужные лог–файлы для логирования процесса обучения свёрточной нейронной сети созданы. На рисунке 2.35 показан вывод прото–файла, содержащего конфигурацию СНС.

```

name: "RomanyNet_foods25"
state {
  phase: TEST
  level: 0
}
layer {
  name: "input"
  type: "Input"
  top: "data"
  input_param {
    shape {
      dim: 25
      dim: 3
      dim: 227
      dim: 227
    }
  }
}
layer {
  name: "conv1"
  type: "Convolution"
  bottom: "data"
  top: "conv1"
  convolution_param {
    num_output: 96
    kernel_size: 11
    stride: 4
  }
}

```

Рисунок 2.35. Вывод информации о заданной архитектуре свёрточной нейронной сети

На рис. 2.36 показан вывод информации об инициализации первого слоя СНС. Продемонстрирован весь процесс создания нейронного узла, передача входных данных на вход узла, вычисление необходимого объёма памяти, инициализация свёрточного ядра и создание активационных узлов.

Такая информация необходима для корректной работы программного средства и своевременного реагирования пользователя на ошибки. Вывод информации об ошибке является полезным при исправлении ошибок.

```

[0123 11:56:04.130457 25907 layer_factory.hpp:77] Creating layer input
[0123 11:56:04.130589 25907 net.cpp:100] Creating Layer input
[0123 11:56:04.130626 25907 net.cpp:408] input -> data
[0123 11:56:04.130807 25907 net.cpp:150] Setting up input
[0123 11:56:04.130859 25907 net.cpp:157] Top shape: 25 3 227 227 (3864675)
[0123 11:56:04.130900 25907 net.cpp:165] Memory required for data: 15458700
[0123 11:56:04.130931 25907 layer_factory.hpp:77] Creating layer conv1
[0123 11:56:04.131024 25907 net.cpp:100] Creating Layer conv1
[0123 11:56:04.131060 25907 net.cpp:434] conv1 <- data
[0123 11:56:04.131134 25907 net.cpp:408] conv1 -> conv1
[0123 11:56:04.132079 25907 net.cpp:150] Setting up conv1
[0123 11:56:04.132159 25907 net.cpp:157] Top shape: 25 96 55 55 (7260000)
[0123 11:56:04.132197 25907 net.cpp:165] Memory required for data: 44498700
[0123 11:56:04.132344 25907 layer_factory.hpp:77] Creating layer relu1
[0123 11:56:04.132431 25907 net.cpp:100] Creating Layer relu1
[0123 11:56:04.132469 25907 net.cpp:434] relu1 <- conv1
[0123 11:56:04.132527 25907 net.cpp:395] relu1 -> conv1 (in-place)
[0123 11:56:04.132591 25907 net.cpp:150] Setting up relu1
[0123 11:56:04.132617 25907 net.cpp:157] Top shape: 25 96 55 55 (7260000)
[0123 11:56:04.132654 25907 net.cpp:165] Memory required for data: 73538700
[0123 11:56:04.132678 25907 layer_factory.hpp:77] Creating layer pool1
[0123 11:56:04.132740 25907 net.cpp:100] Creating Layer pool1
[0123 11:56:04.132771 25907 net.cpp:434] pool1 <- conv1
[0123 11:56:04.132820 25907 net.cpp:408] pool1 -> pool1
[0123 11:56:04.132926 25907 net.cpp:150] Setting up pool1
[0123 11:56:04.132963 25907 net.cpp:157] Top shape: 25 96 27 27 (1749600)
[0123 11:56:04.132989 25907 net.cpp:165] Memory required for data: 80537100
[0123 11:56:04.133011 25907 layer_factory.hpp:77] Creating layer norm1
[0123 11:56:04.133074 25907 net.cpp:100] Creating Layer norm1
[0123 11:56:04.133107 25907 net.cpp:434] norm1 <- pool1
[0123 11:56:04.133173 25907 net.cpp:408] norm1 -> norm1
[0123 11:56:04.133260 25907 net.cpp:150] Setting up norm1
[0123 11:56:04.133298 25907 net.cpp:157] Top shape: 25 96 27 27 (1749600)
[0123 11:56:04.133334 25907 net.cpp:165] Memory required for data: 87535500

```

Рисунок 2.36. Вывод информации о процессе инициализации свёрточной нейронной сети

На рис. 2.37 продемонстрирован запуск хеширующего слоя СНС. Показано, как запускается процесс поиска изображений в базе данных .ldb.

```

=====
[2] Extraction of deep features at layer fc7
=====
issuing batches of 10 images rest will be filled with zeros
[c7, pycaffe_batch_feat: 100% [#####] Time: 0:04:56
[PROGRESS] saved 10 images - 2021-01-24 12:12:23.559317
[PROGRESS] saved 20 images - 2021-01-24 12:12:23.577714
[PROGRESS] saved 30 images - 2021-01-24 12:12:23.579499
[PROGRESS] saved 40 images - 2021-01-24 12:12:23.582238
[PROGRESS] saved 50 images - 2021-01-24 12:12:23.586598
[PROGRESS] saved 60 images - 2021-01-24 12:12:23.590552
[PROGRESS] saved 70 images - 2021-01-24 12:12:23.596046
[PROGRESS] saved 80 images - 2021-01-24 12:12:23.600997
[PROGRESS] saved 90 images - 2021-01-24 12:12:23.606669

```

Рисунок 2.37. Запуск скрытого нейронного слоя для вычисления хеш-значения

На рис. 2.38 продемонстрирован процесс завершения обучения свёрточной нейронной сети и сохранения семантических признаков, хеш-значений и индексов базы данных .ldb для тренировочных изображений. Из рис. 2.38 можно понять, что процесс обучения завершен.


```

[INFO] compacting datasets... - 2021-01-24 12:12:25.420050
[INFO] old size of 'image_ids': 1,152; new size: 576 - 2021-01-24 12:12:25.420311
[INFO] old size of 'index': 1,152; new size: 576 - 2021-01-24 12:12:25.420443
[INFO] old size of 'deepfeatures': 1,152; new size: 576 - 2021-01-24 12:12:25.420677
[INFO] old size of 'binarycode': 1,152; new size: 576 - 2021-01-24 12:12:25.420827

real    10m33,275s
user    9m39,255s
sys      3m8,292s

```

Рисунок 2.38. Завершение обучения свёрточной нейронной сети

2.5.2 Запуск скрипта для извлечения похожих изображений

Рассмотрим пример запуска скрипта `tools/deep_search.py` для нахождения поиска похожих изображений в базе данных `.ldb`, на основании тестовых изображений и их хеш–значений. На рис. 2.39 продемонстрирован запуск программы `tools/deep_search.py`. Скрипт запускается с параметрами:

- `use_gpu 0` – означает, что используются только возможности процессора;
- `model_file` – файл с конфигурацией модели нейронной сети;
- `feat_proto` – прото–файл с настройками обучения нейронной сети;
- `bin_proto` – файл с настройками характеристик выходного хеш–значения;
- `products_dir` – папка (директорий) с тренировочными данными
- `deer_db` – файл для сохранения вычисленных хеш–значений.

При реализации программного средства использовалось нулевое значение `use_gpu` с целью использовать исключительно мощности процессора обучаемой машины. Суть подхода к реализации обучения СНС заключается в нагрузочном тестировании эффективности алгоритма обучения свёрточной нейронной сети.

Файлы `feat_proto` и `bin_proto` являются гибкими инструментами при построении архитектуры СНС и настройке её обучения.

```

with open('tools/deep_search.py') as f:
    python tools/deep_search.py --use_gpu 0 --model_file /opt/cbtr_binary_code/examples/pr
ducts20/BonnyNet_products20_48_iter_20000.caffemodel --feat_proto /opt/cbtr_binary_code/examples/naia/deploy_fc7.prototxt --bin_proto /
opt/cbtr_binary_code/examples/products20/BonnyNet_products20_48_deploy.prototxt --products_dir /opt/cbtr_binary_code/data/products20/img
s --deep_db /opt/cbtr_binary_code/data/products20/products20_48_deep00.hdfs --query /opt/cbtr_binary_code/data/products20/imgs/moelinex/0
7072015.jpg
model_prototxt: /opt/cbtr_binary_code/examples/products20/BonnyNet_products20_48_deploy.prototxt
model_files: /opt/cbtr_binary_code/examples/products20/BonnyNet_products20_48_iter_20000.caffemodel
Using CPU Mode
WARNING: Logging before InitGoogleLogging() is written to STDERR
@0124 18:09:50.316672 32126 _caffe.cpp:122] DEPRECATION WARNING - deprecated use of Python interface
@0124 18:09:50.316727 32126 _caffe.cpp:123] Use this instead (with the named "weights" parameter):
@0124 18:09:50.316733 32126 _caffe.cpp:125] net(['/opt/cbtr_binary_code/examples/products20/BonnyNet_products20_48_deploy.prototxt', 1, w
eights='/opt/cbtr_binary_code/examples/products20/BonnyNet_products20_48_iter_20000.caffemodel')
@0124 18:09:50.316806 32126 upgrade_proto.cpp:53] Attempting to upgrade input file specified using deprecated VllayerParameter: /opt/cbtr
_binary_code/examples/products20/BonnyNet_products20_48_deploy.prototxt
@0124 18:09:50.316521 32126 upgrade_proto.cpp:61] Successfully upgraded file specified using deprecated VllayerParameter
@0124 18:09:50.316508 32126 upgrade_proto.cpp:67] Attempting to upgrade input file specified using deprecated input fields: /opt/cbtr_bin
ary_code/examples/products20/BonnyNet_products20_48_deploy.prototxt
@0124 18:09:50.316475 32126 upgrade_proto.cpp:70] Successfully upgraded file specified using deprecated input fields.
@0124 18:09:50.316499 32126 upgrade_proto.cpp:72] Note that future Caffe releases will only support input layers and not input fields.
@0124 18:09:50.316501 32126 net.cpp:158] Initializing net from parameters:
name: "BonnyNet_products20"
state {
  phase: TEST
  level: 0
}
layer {
  name: "Input"
  type: "Input"
  top: "data"
  input_param {
    shape {
      dim: 25
      dim: 3
      dim: 227
      dim: 227
    }
  }
}

```

Рисунок 2.39. Запуск скрипта `tools/deep_search.py`

На рис. 2.40 показан результат работы скрипта tools/deep_search.py. Можно увидеть, что поиск занял по времени 0.85 секунд, и что было найдено 20 наиболее подходящих кандидатов – похожих изображений.

```
[2] Extraction of deep features at layer fc7
.....
assuming batches of 10 images rest will be filled with zeros
fc7, pycaffe_batch_feat: 100x [.....] Time: 0:00:00
DeepSearcher.search: findcandidate_time on 0.039544 s
DeepSearcher.search: realsearch_time on 0.207162 s
[INFO] search took: 0.85s
[RESULT] 1. 23552365.jpg - 0.1653
[RESULT] 2. 39139590.jpg - 0.1829
[RESULT] 3. 50435870.jpg - 0.1857
[RESULT] 4. 57326630.jpg - 0.1857
[RESULT] 5. 30575518.jpg - 0.2100
[RESULT] 6. 26643652.jpg - 0.2154
[RESULT] 7. 51191712.jpg - 0.2159
[RESULT] 8. 27142455.jpg - 0.2190
[RESULT] 9. 36515816.jpg - 0.2227
[RESULT] 10. 00000012.jpg - 0.2381
[RESULT] 11. 00555931.jpg - 0.2331
[RESULT] 12. 63001510.jpg - 0.2375
[RESULT] 13. 89412944.jpg - 0.2407
[RESULT] 14. 02704179.jpg - 0.2573
[RESULT] 15. 82043261.jpg - 0.2610
[RESULT] 16. 06625991.jpg - 0.2745
[RESULT] 17. 01917127.jpg - 0.2767
[RESULT] 18. 64319000.jpg - 0.2772
[RESULT] 19. 35325527.jpg - 0.2851
[RESULT] 20. 00085403.jpg - 0.2918
```

Рисунок 2.40. Результат работы скрипта tools/deep_search.py

Можно сделать вывод, что программное средство эффективно использует ресурсы компьютера, выполняя поиск за 0.85 секунд, а также корректно находит 20 кандидатов. Такое и указывалось при запуске разработанного ПО.

На рис. 2.41 показано то изображение, которое было введено в программное средство в качестве запроса к базе данных изображений.



Рисунок 2.41. Изображение – запрос к базе данных изображений

На рис. 2.42 продемонстрирован пример найденных картинок – кандидатов, которые наилучшим образом подходят под входное изображение. Все 20 отобранных кандидатов похожи на изображение-запрос, а значит, можно заключить, что реализованное программное средство обрабатывает корректно и реализует все требуемые функции.



Рисунок 2.42. Пример кандидатов, отобранных по минимальному расстоянию Хэмминга

Результаты демонстрации работы реализованного программного средства показывают, что ранее описанные алгоритмы и методы реализованы корректно и эффективно, программное средство реализует заявленный функционал и отвечает всем требованиям, определенным техническим заданием.

Тестирование разработанного ПО доказало возможность его применения при реализации алгоритмов масштабно-инвариантной трансформации признаков изображения и глубокого поиска изображения в базе данных. Оно представлено в виде консольной утилиты, которую можно запустить с помощью простой команды консоли, что значительно облегчает работу с ПО. В процессе работы программы на экран консоли выводится вся информация, характеризующая основные этапы работы программного обеспечения.

Таким образом, реализовано программное средство для хеширования изображений на базе сверточных нейронных сетей. Цели и задачи исследования, включающие масштабно-инвариантную трансформацию изображения; анализ методов получения его семантических признаков; определение и реализацию архитектуры нейронной сети и алгоритма преобразования семантических признаков в хеш-пространство; а также осуществление метода определения качества сохранения семантических признаков изображения в его хеш-значении, были выполнены авторами в полном объеме.

3. РАСПОЗНАВАНИЕ ДЕСТРУКТИВНЫХ ЭМОЦИЙ ЧЕЛОВЕКА ПО ЕГО ИЗОБРАЖЕНИЮ НА ОСНОВЕ НЕЙРОСЕТЕВЫХ АЛГОРИТМОВ

В современном, высокотехнологичном мире, на смену механизации процессов приходит новая главенствующая тенденция – их автоматизация. С развитием искусственного интеллекта, в особенности нейросетевых технологий, стало возможным автоматизировать процессы, не поддающиеся обычной алгоритмизации, например, определение тональности текста и распознавание речи. В то же время, умение эффективно общаться с людьми является неотъемлемой частью успешности многих видов профессиональной или бытовой деятельности. Поэтому задача автоматизации процесса распознавания эмоций актуальна для многих областей, в которых взаимодействие с людьми и понимание испытываемых ими чувств является необходимым фактором.

В начале 20-го века была разработана достаточно фундаментальная теория о том, что универсальных эмоций не существует. Теория опиралась на зависимость носителя эмоций от типа культуры, которую он представляет. Но исследования Пола Экмана и Уоллеса Фризена доказало, что существует семь универсальных эмоций, которые не зависят от культурной и национальной принадлежности человека. Это: удовольствие, страдание, страх, гнев, радость, удивление и отвращение. В рассматриваемой системе используется сверточная нейронная сеть для решения задачи автоматического распознавания эмоций. Основным предназначением особой архитектуры свёрточных нейронных сетей является эффективное распознавание образов (изображений).

В сверточной нейронной сети выходы промежуточных слоёв образуют матрицу, то есть изображение, или набор матриц – несколько слоёв изображения. На вход сверточной нейронной сети можно подавать три слоя изображения (R-, G-, B- каналы изображения), которые обрабатываются отдельно и сравниваются с изображениями, находящимися в базах данных.

Объект исследования – информационная система распознавания эмоций по изображениям. *Предметом исследования* являются информационные процессы, протекающие в системе распознавания эмоций по изображениям. *Целью* является разработка программного средства распознавания эмоций человека по изображению с помощью обучения нейронной сети. *Для достижения поставленной цели* нужно выполнить анализ предметной области; синтез и обучение модели свёрточной нейронной сети; разработку программного средства, а также тестирование работы разработанных программ.

3.1. Анализ предметной области анализа эмоций

3.1.1. Исследование предметной области

На сегодняшний день существуют различные анализаторы эмоций. Часть из них позволяет определить только базовые 7 эмоций по изображениям, другие же способны определять эмоции в реальном времени, с помощью анализа видеопотока с видеокамер. Существуют и программные средства, способные определять эмоции по движению мышц на лице, что даёт достаточно высокую точность. Некоторые программы могут определять комбинации эмоций, например, грустный–злой, радостный–удивленный и т.д., а иные программные средства даже могут выйти за рамки семи базовых эмоций.

Самая очевидная сфера применения технологий распознавания эмоций – маркетинг. С их помощью можно определить, как рекламный ролик действует на человека (возможного покупателя). Для этого можно, например, установить конструкцию с камерой, которая будет менять рекламу в зависимости от настроения, пола и возраста проходящих мимо людей.

Системы распознавания эмоций можно использовать для того, чтобы узнать реакцию аудитории на фильм, спектакль, шоу или другие мероприятия. Компания Disney, в 2017 году, в сотрудничестве с учёными, провела эксперимент: установила камеры в кинотеатре и подключила алгоритмы глубокого обучения, для оценки эмоций зрителей. Система смогла предсказать реакцию публики, наблюдая за людьми всего несколько минут.

Распознавание эмоций применимо и в сфере образования. С его помощью можно изучать настроение и внимание учеников во время занятий. Так, в школах Петербурга уже пытались внедрить подобную технологию.

Интеллектуальные программы распознавания эмоций могут быть полезны и в работе с персоналом. Они помогают определить эмоциональное состояние сотрудника, своевременно заметить его усталость или недовольство, и эффективнее перераспределить задачи, заранее устраняя возможные конфликты.

Система интеллектуального распознавания эмоций может быть полезной и для автомобилистов. Она сможет определять эмоциональное состояние водителя и пассажиров транспортных средств. Система будет не только следить за уровнем стресса, но и пытаться снизить его – например, выбрать менее загруженный маршрут, включить спокойную расслабляющую музыку, включить отопление или кондиционер, регулируя температуру в салоне транспортного средства в зависимости от времени года.

На рис. 3.1 изображен пример работы такой программы.

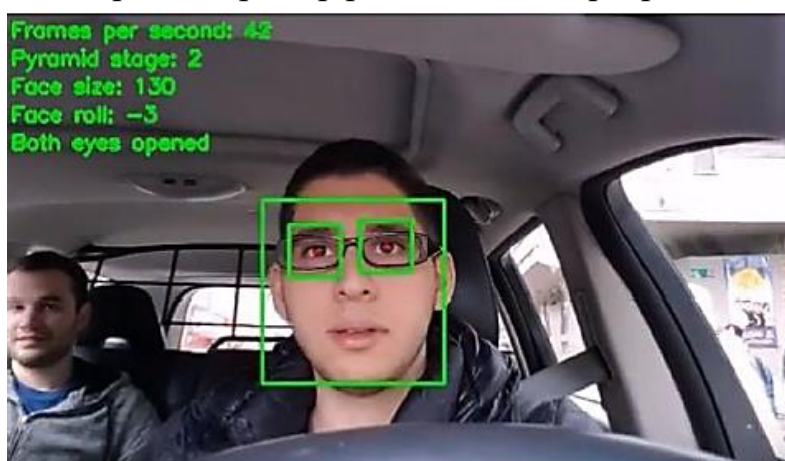


Рисунок 3.1. Анализ эмоций водителя автомобиля

Подобные программные средства могут пользоваться большим успехом в медицине, при реабилитации больных. Например, очки с искусственным интеллектом смогут помочь детям и взрослым с диагнозом аутизма, или других заболеваний, распознавать чужие эмоции и нарабатывать социальные навыки.

Разрабатываемое в этом исследовании программное средство, при некоей доработке, сможет стать защитой для каждого человека и его дома. Например, человека будят ночью стуком в дверь. Он спросонья спешит её открыть, но программа, которая подключена к камере на двери, проанализировав эмоции гостя, сообщает о том, что человек за дверью крайне агрессивно настроен. Вследствие – хозяин дома уже подумает, а не бездумно отворит двери.

Похожая система в настоящее время внедрена в умные города Китая, позволяя следить за эмоциями различных людей. Полиция пользуется очками дополненной реальности, в которые встроена система распознавания эмоций, а также система определения личности человека, что намного упрощает работу полиции. Китайский полицейский в подобных очках показан на рис. 3.2.



Рисунок 3.2. Очки с дополненной реальностью китайской полиции

Программный код и реализация китайских технологий дополненной реальности хранятся в секрете, и актуальны только для одной страны [34].

Еще один альтернативный вариант использования интеллектуальных программ распознавания эмоций – применение в робототехнике. Установка их в программное обеспечение роботов действие даёт возможность понимать человеческие чувства немного лучше, что должно повысить *эмпатию* (как осознанное *сопереживание* текущему эмоциональному состоянию другого человека) между роботами и человечеством. На рис. 3.3 показан робот, разработанный Сбербанком для обслуживания клиентов банка.



Рисунок 3.3. Работа программы распознавания эмоций у робота Сбербанка

3.1.2. Принцип работы сверточной сети

Сверточная нейронная сеть (СНС, от англ. Convolutional Neural Network, CNN) – это специальная архитектура нейронных сетей, предложенная Яном Лекуном [35]. Она нацелена на эффективное распознавание изображений.

Свёртка – это действие над двумя матрицами A (размерности $n_x \times n_y$) и B (размерности $m_x \times m_y$). Результатом этих действий является матрица $C=A*B$ размерности $(n_x-m_x+1) \times (n_y-m_y+1)$. Каждый элемент результата вычисляется как скалярное произведение матрицы B и некоторой подматрицы A этого же размера. Графическое представление операции свёртки показано на рис. 3.4.

Логический смысл свёртки заключается в следующем: чем больше величина элемента свёртки, тем больше эта часть матрицы A будет похожа на матрицу B (похожа в смысле скалярного произведения). По этим причинам матрица A – это изображение, а матрица B – фильтр или образец. *Свёрточный слой нейронной сети* является применением операции свертки к выходам с предыдущего слоя, где веса ядра свертки являются обучаемыми параметрами.

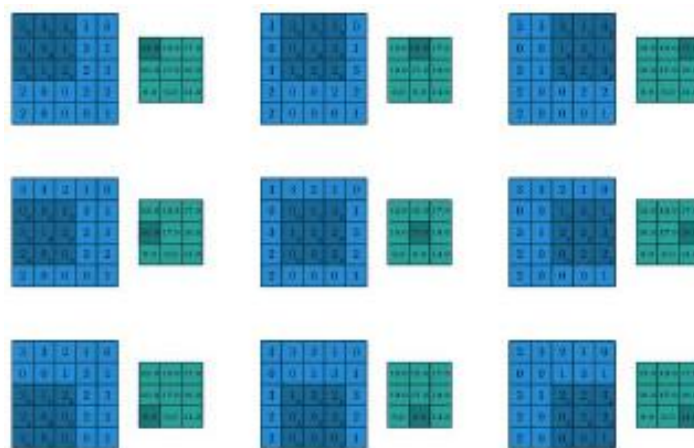


Рисунок 3.4. Операция свёртки как действие над матрицами

Еще один обучаемый вес используется в качестве константного сдвига (англ. *bias*). При этом существует несколько важных деталей, описанных далее.

В одном свёрточном слое может быть несколько сверток. В этом случае для каждой свертки на выходе получится своё изображение. Например, если вход имел размерность $w \times h$, а в слое было n сверток с ядром размерности $k_x \times k_y$, то выход будет иметь размерность $n \times (w - k_x + 1) \times (h - k_y + 1)$.

Ядра свертки могут быть трёхмерными. Свёртка трёхмерного входа с трёхмерным ядром происходит аналогично. Скалярное произведение считается ещё и по всем слоям изображения. Так, для усреднения информации о цветах исходного изображения, на первом слое используется свертка размерности $3 \times w \times h$. На выходе такого слоя будет одно изображение (вместо трёх).

Можно заметить, что в результате операции свертки изображение уменьшается. Пикселы, которые находятся на границе изображения, участвуют в меньшем количестве свёрток, чем внутренние. В связи с этим, в свёрточных слоях используется дополнение изображения (англ. *padding*). Чтобы сохранить размер изображения после свёртки, выходные данные предыдущего слоя дополняются пикселями. Такие свёртки называются одинаковыми (англ. *Same Convolution*), а свёртки без дополнения изображения называются правильными (англ. *Valid Convolution*). Среди способов, которыми можно заполнить новые пикселы, можно выделить следующие:

- Zero shift: 00[ABC] 00;
- Border extension: AA [ABC] CC;
- Mirror shift: BA [ABC] CB;
- Cyclic shift: BC [ABC] AB.

Еще одним параметром свёрточного слоя является сдвиг (англ. Stride). Хотя обычно свёртка применяется для каждого пиксела подряд, иногда используется сдвиг, отличный от единицы. Скалярное произведение считается не со всеми возможными положениями ядра, а только с положениями, кратными некоторому сдвигу s . Тогда, если вход имел размерность $w \times h$, а ядро свертки имело размерность $k_x \times k_y$ и использовался сдвиг s , то выход будет иметь размерность $\lfloor w - k_x/s + 1 \rfloor \times \lfloor h - k_y/s + 1 \rfloor$.

3.1.3. Обнаружение лиц с помощью метода Виолы–Джонса

Для определения лиц на изображении используются каскады Хаара (Haar Cascades), использующие метод Виолы–Джонса, разработанный в 2001 году.

Метод применяет технологию скользящего окна: рамка, меньшего размера, чем исходное изображение, движется, с определенным шагом, по изображению, и с помощью каскадов слабых классификаторов Хаара определяет, есть ли в рассматриваемом окне лицо человека. Метод скользящего окна применяется в различных задачах компьютерного зрения и распознавания образов.

Метод состоит из двух алгоритмов: обучения и распознавания. На практике скорость работы алгоритма обучения не так важна, скорость работы алгоритма распознавания. Метод является статистическим, структурным и нейронным.

Преимущества метода Виолы–Джонса [36]:

- использование простых классификаторов показывает хорошую скорость обработки изображения, и позволяет использовать этот метод в видеопотоке.
- возможно обнаружение более одного лица (группы лиц) на изображении.

На рис. 3.5 изображена схема работы алгоритма распознавания лица человека методом Виолы–Джонса в обобщённом виде.

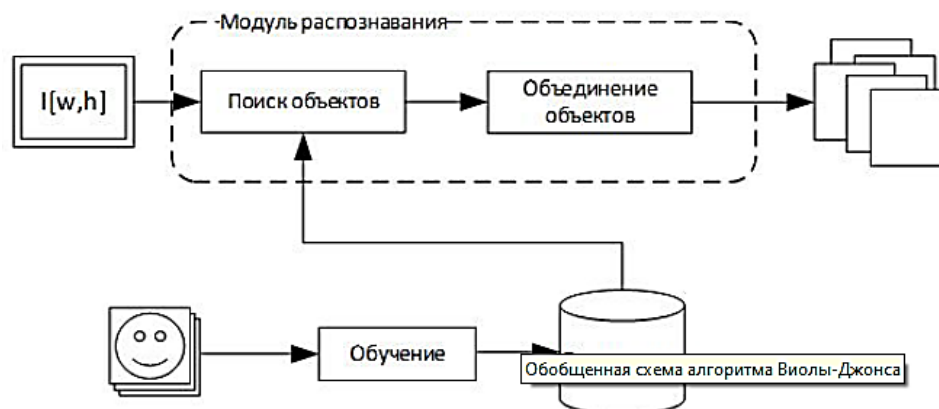


Рисунок 3.5. Обобщённая схема распознавания в алгоритме Виолы–Джонса

Обобщенная схема алгоритма выглядит так: перед началом распознавания алгоритм обучения на основе тестовых изображений обучает базу данных, состоящую из признаков, их границы и паритета. Далее алгоритмом распознавания используется созданная база данных для поиска объектов с различными масштабами изображения. На выходе алгоритм Виолы-Джонса возвращает всё множество найденных необъединенных объектов на разных масштабах. Следующая задача – принять решение о том, какие из найденных объектов действительно присутствуют в кадре, а какие – дубли.

В качестве признаков для алгоритма распознавания авторами были предложены признаки Хаара, сформированные на основе вейвлетов Хаара, предложенных венгерским математиком Альфредом Хааром в 1909 году.

Общим наблюдением в задаче распознавания человеческих лиц является тот факт, что области лица вокруг глаз темнее области щек. Рассмотрим маски, состоящие из темных и светлых областей. На рис. 3.6 представлены маски, которые использует алгоритм Виолы-Джонса.



Рисунок 3.6. Маски из темных и светлых областей

Каждая маска характеризуется пропорциями, минимальным размером и размером темных и светлых областей. Совместно с другими наблюдениями, такими, как пространство признаков в задаче распознавания для класса лиц, были предложены следующие признаки Хаара. Рис. 3.7 показывает виды масок, используемых для обнаружения лиц.

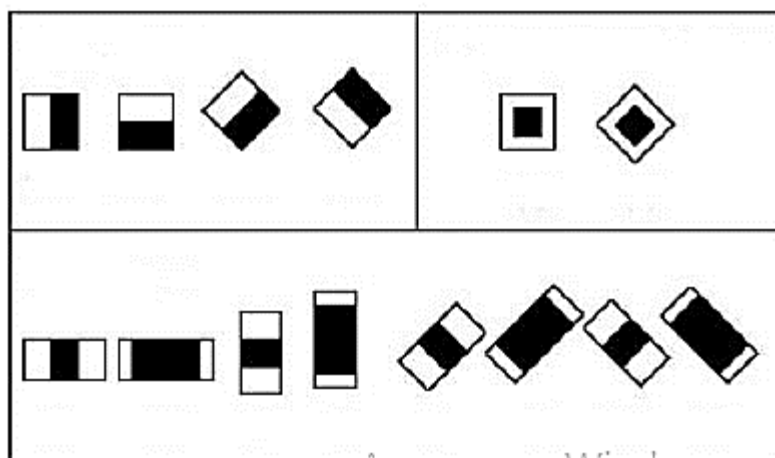


Рисунок 3.7. Разновидности масок

Признаки Хаара дают точечное значение перепада яркости по осям X и Y соответственно. Поэтому общий признак Хаара для распознавания лиц представляет собой набор двух смежных прямоугольников, которые лежат выше глаз и на щеках. Значение признака вычисляется по формуле:

$$F = X - Y, \quad (1)$$

где Y – сумма значений яркостей точек, закрываемых темной частью признака, а X – сумма значений яркостей точек, закрываемых светлой частью признака.

При распознавании, для каждого классификатора известны: положение в окне 24×24 пикселей, признак Хаара, используемый в классификаторе и порог E .

Изображение поступает на вход алгоритма $I(r, c)$ размером $W \times H$, где $I(r, c)$ – яркостная составляющая изображения. Результатом работы алгоритма служит множество прямоугольников $R(x, y, w, h)$, определяющих положение лиц в исходном изображении I . Алгоритм сканирует изображение I в нескольких масштабах, начиная с базовой шкалы: 11 масштабов, размер окна 24×24 пикселей. Каждый последующий уровень в 1,25 раза больше предыдущего.

Базовый алгоритм Виолы-Джонса имеет ряд недостатков [36]:

- большое количество близко расположенных друг к другу результатов из-за применения различных масштабов и скользящего окна;
- длительное время работы алгоритма обучения, в процессе чего нужно проанализировать большое количество тестовых изображений.

3.1.4. Обзор существующих программ-аналогов

Программы распознавания эмоций в настоящее время уже существуют. У каждой есть свои плюсы и минусы, которые нужно проанализировать [37]. Рассмотрим 4 программных средства: WeSee, FaceReader, EmoDetect, Magic.

WeSee (Великобритания). Эта технология используется для доступа к эмоциональному состоянию людей во время телевизионных съёмок (интервью) в прямом эфире. Некоторые *стартапы* предлагают программные средства *обнаружения эмоций* в качестве меры безопасности. Британская компания WeSee утверждает, что искусственный интеллект выявляет подозрительное поведение людей, считывая незаметные невооруженному глазу подсказки. Такие эмоции, как сомнение и злость, часто могут скрываться под маской сдержанности, и резко контрастировать с тем, что говорит человек. Компания WeSee утверждает, что работает с высокопоставленной правоохранительной организацией, анализируя моральное состояние и поведение интервьюируемых людей, не давая им наговорить лишнего, находясь на телевидении в прямом эфире.

Код и алгоритмы WeSee хранятся в закрытом доступе, что делает невозможным их свободное использование. Для разработчиков это является минусом, мешающим им сравнить собственную реализацию с этой.

На рис. 3.8 показан пример работы программы WeSee.



Рисунок 3.8. Окно работы программы WeSee

FaceReader компании *Noldus Information Technology* (Нидерланды) может верно распознавать по выражению лица такие эмоции, как радость, грусть, злость, удивление, страх, отвращение и спокойствие. На рис. 3.9 показан пример работы этого программного приложения.



Рисунок 3.9. Главное окно программного средства FaceReader

Преимущества программы *FaceReader*:

- средний процент распознавания эмоций человека составляет 89%, независимо от мимики лица, наклона и поворота головы на плоскости;
- программа работает с большим количеством форматов видеофайлов с кодеками MPEG1, MPEG2, DivX4, DivX5, DivX6, DV-AVI. При этом распознавание эмоций может происходить покадрово, либо полностью при просмотре всего видео. Приложение не ограничивается работой с загружаемым видео, FaceReader работает и со статичными изображениями, и в реальном времени, считывая данные с камеры устройства пользователя;
- возможности визуализации: наличие гистограмм, диаграмм, отображение накладываемой сетки, процентов выражаемых эмоций.

Недостатки: процент распознавания эмоций и возраста человека сильно падает при работе с лицами детей возрастом меньше 5 лет; распознавание может происходить неточно, если человек носит очки, усы, бороду, парик, маску; программа не определяет лица, повёрнутые в профиль.

EmoDetect (Нейроботикс, Россия). Это программное средство позволяет определить психоэмоциональное состояние человека по выборке изображений (видео или набор файлов). Разработанный классификатор эмоций позволяет определить основные эмоции: спокойствие, радость, удивление, грусть, злость, страх, отвращение. На рис. 3.10 показан пример работы этой программы.

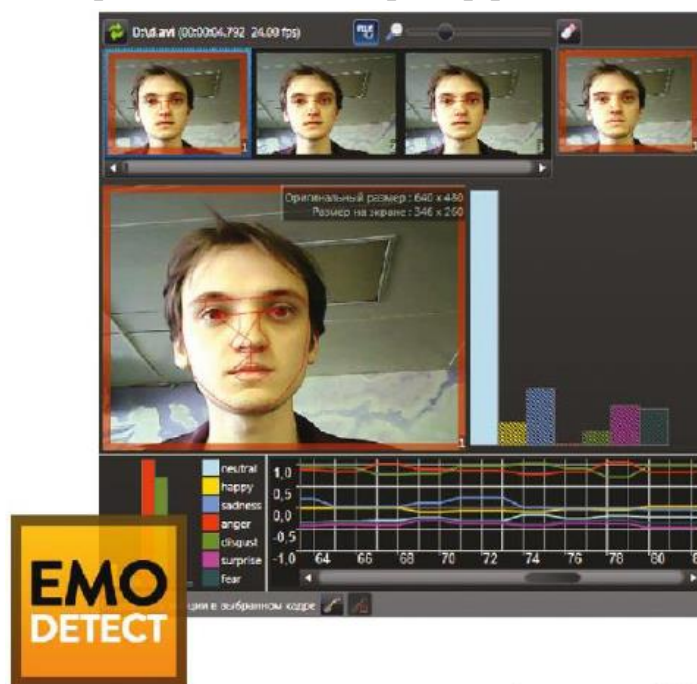


Рисунок 3.10. Главное окно программы EmoDetect

Достоинства программы *EmoDetect*:

- классификация эмоций происходит с помощью нескольких независимых классификаторов: нейронной сети, классификатора на основе взвешенных сумм признаков и системы решающих правил, что даёт точный результат;
- построение графика динамического изменения интенсивности эмоций человека во времени, и формирование отчёта о результатах обработки видео;
- возможность записи видеофайлов с веб-камеры, анализ всех форматов, поддерживаемых операционной системой.

Недостатки: программа не определяет лица, повёрнутые в профиль.

Magic. Это приложение способно распознавать человеческую мимику и накладывать на неё маски. Само приложение носит развлекательный характер и способно распознавать улыбку, злость, грусть, удивление, а также некоторые простые выражения лица, вроде поцелуя. Все нужные расчёты выполняются на мобильном устройстве пользователя. По словам разработчиков, даже в условиях разной освещённости и произвольного наклона головы человека точность распознавания эмоций превышает 85% [38].

Алгоритмы глубокого обучения могут достичь такой точности потому что обученная модель учитывает не только геометрию ключевых точек, но и текстуру распознаваемого лица. При этом модель не требует больших вычислительных ресурсов, что и позволяет использовать её в приложении на мобильном устройстве, как объясняют создатели программы *Magic*.

На рис. 3.11 продемонстрирована работа программы *Magic*, где можно увидеть отсутствие точных выводов данных.

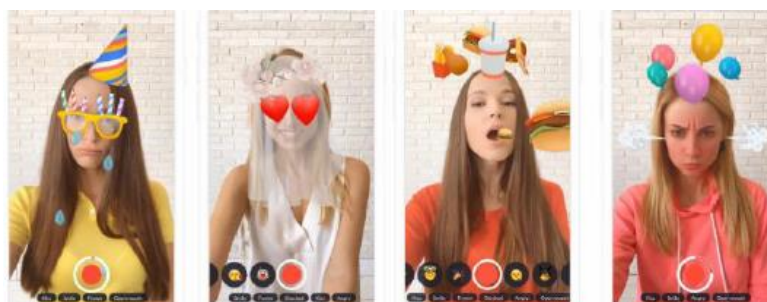


Рисунок 3.11. Пример работы программы *Magic*

Достоинства программы *Magic*: программа учитывает текстуру лица человека; способна распознавать уникальную мимику, например, поцелуй; тратит мало ресурсов при работе; способна работать в реальном времени.

Недостатки: программа рассчитана только на развлекательные цели; никаких точных данных о выражаемой эмоции [38].

3.1.5. Обзор существующих данных для обучения нейронной сети

AffectNet. Это один из самых больших набора данных с лицами. Этот набор собран на основе 1250 тегов относящимся к эмоциям на 6 разных языках: английский, немецкий, испанский, португальский, арабский и персидский. В нём содержится около миллиона изображений. Объем – 120GB.

Для получения доступа к данной базе необходимо заполнить данные о том, как и кем эти данные будут использоваться. После этого необходимо прождать около 14 дней, прежде чем придёт ответ. На рис. 3.12 показан пример изображений лиц с соответствующими эмоциями, хранящихся в датасете.



Рисунок 3.12. Пример изображений, хранящихся в датасете AffectNet

Ascertain. Это первый датасет, который соединяет воедино персональность, привычки и эмоциональное состояние. Имеет 5 шкал персональности, и данные о 58 людях с их электроэнцефалограммами, электрокардиограммами, кожно–гальваническими реакциями и данными о лицевой активности. Данных о количестве изображений, их качестве нет. Но в этом датасете, помимо изображений, находятся данные с психологических сенсоров.

Для получения доступа к данным необходимо распечатать бланк, вручную заполнить, отсканировать и отправить на проверку. Рис. 3.13 показывает схему данных в датасете, которая была взята с официального сайта.

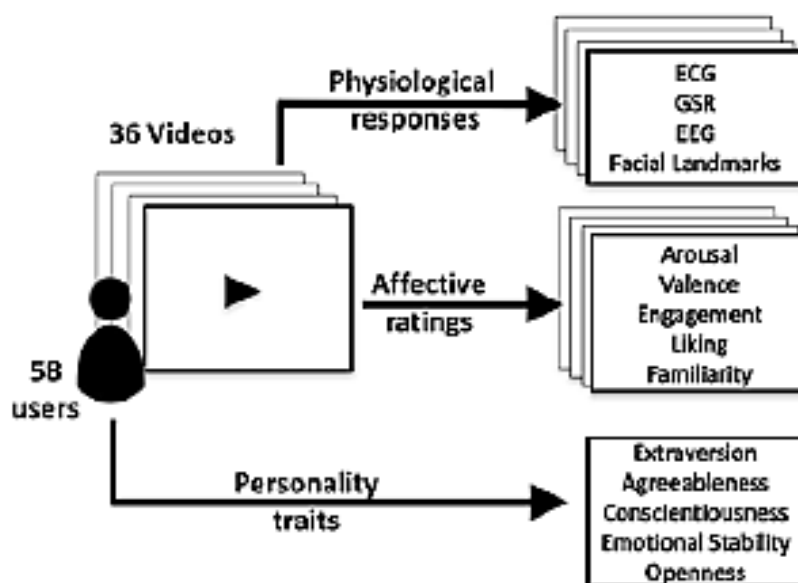


Рисунок 3.13. Схема данных в датасете по эмоциям в Ascertain

Dreamer. Это мультимодальная база данных, состоящая из сигналов электроэнцефалограммы (ЭЭГ) и электрокардиограммы (ЭКГ), записанных во время выявления аффекта с помощью аудиовизуальной стимуляции. Сигналы от 23 участников были записаны, вместе с самооценкой участниками своего эмоционального состояния. Все сигналы были получены с помощью портативного, носимого, беспроводного стандартного оборудования, которое может позволить использовать эффективные вычислительные методы в повседневных приложениях. Для ЭЭГ использовалась беспроводная ЭЭГ-гарнитура Emotiv EPOC, а для ЭКГ – датчик ЭКГ Shimmer2. В процессе использования необходимо запрашивать доступ к данным.

Extended Cohn-Kanade Dataset (CK+). Этот набор данных представляет собой 5876 изображений 123х человек. У всех изображений имеется одинаковый серый фон, их размер – 640×490 пикселей. Набор находится в открытом доступе.

Emotic. Это набор из 23571 изображений людей в различной обстановке, демонстрирующих 26 вариаций эмоций. Сами изображения были собраны с помощью программного средства Google Search Engine. Датасет ориентирован на обучение нейронной сети определению эмоций человека по его положению в пространстве и позе, в которой он находится. Данные – в открытом доступе.

Список 26 категорий эмоций включает:

- 1) спокойствие: хорошее самочувствие и расслабленность, отсутствие беспокойства, удовлетворённость, наличие положительных мыслей и ощущений;
- 2) привязанность: нежные чувства; любовь; нежность, ласка;
- 3) оценка состояния: чувство положительного мнения или суждения; уважение; восхищение; доверие, благодарность, преданность;
- 4) состояние ожидания: надежда, готовность к будущим событиям;
- 5) вовлеченность в события: внимание к чему-то; поглощение чем-то; любопытство; заинтересованность; переживание за исход событий;
- 6) чувство уверенности: убежденность в благоприятном исходе; гордость;
- 7) счастье: чувство восторга; чувство удовольствия или веселья;
- 8) удовольствие: чувство восторга и победы, проявляемое в эмоциях;
- 9) волнение: энтузиазм; мотивированность, стимулированность; энергичность;
- 10) сюрприз: внезапное открытие чего-то хорошего, приятная неожиданность;
- 11) сочувствие: состояние разделённости своих собственных чувств с эмоциями других людей, их целей или проблем; поддержка; сострадательность;
- 12) сомнение / замешательство: трудность понимания или решения; размышление о разных вариантах возможного поведения;
- 13) отстраненность: чувство незаинтересованности в окружающих событиях; скука; отсутствие сосредоточенности, равнодушие;
- 14) утомлённость: усталость; сонное состояние;
- 15) смущение: чувство неуверенности, стыда или вины;
- 16) стремление: сильное желание что-то получить; ревность; зависть; похоть;
- 17) неодобрение: ощущение, что что-то идёт не так, предосудительность; презрение; враждебность, желание всё переделать;
- 18) чувство отвращения: неприязнь, отвращение; чувство ненависти;
- 19) раздражение: беспокойность, нетерпеливость; расстроенность;
- 20) гнев: сильное неудовольствие или ярость; оскорблённость, обиженность;
- 21) чувствительность: ощущение физической или эмоциональной травмы; чувство деликатности или уязвимости, переживание, тревога;
- 22) печаль: чувство несчастья, печали, разочарования или уныния;
- 23) беспокойство: нервозность; волнение; огорчение; тревога; давление;
- 24) страх: подозрительность или боязнь опасности, угрозы, зла или боли;
- 25) боль: ощущение физических страданий, невозможность терпеть боль;
- 26) страдание: психологическая / эмоциональная боль; огорчение; мучение.

FER–2013. Это набор данных из 28709 изображений, разделённых на 7 базовых эмоций. В этом датасете предусмотрен некоторый процент изображений, необходимых для случайного определения эмоций. Данные собраны с помощью Google Search Engine и хранятся в свободном доступе.

Google Facial Expression Comparison Dataset. Этот набор данных представляет собой крупномасштабный набор данных по выражениям лиц, состоящий из триплетов изображений. В нём указаны человеческие аннотации, указывающие, какие два лица в каждом триплете образуют наиболее похожую пару, с точки зрения выражения лица. Каждый триплет в этом наборе данных аннотирован шестью или более экспертами–оценщиками. Этот набор данных отличается от наборов данных по выражению лиц, которые сосредоточены на дискретной классификации эмоций или обнаружении единиц действия. Такой датасет предназначен для помощи исследователям, работающим над темами, связанными с анализом выражений лиц, такими, как поиск изображений на основе выражений, обобщение фотоальбомов на основе выражений, классификация эмоций, синтез выражений и т.д. [39].

3.2. Средства реализации программного продукта

Для разработки программного средства следует использовать программное оборудование, поддерживающее работу с нейронными сетями [40].

TensorFlow. Это разработанная командой Google Brain, как продолжение закрытой системы машинного обучения DistBelief, комплексная платформа для машинного обучения с открытым исходным кодом. С 2015 года фреймворк открыт для свободного доступа. TensorFlow, как и большинство фреймворков глубокого обучения, имеет API на Python поверх механизма C и C ++, что ускоряет его работу. Разработана гибкая экосистема библиотек, инструментов и ресурсов сообщества. Это позволяет разработчикам использовать самые современные технологии машинного обучения, развёртывать и создавать новые приложения на этой базе. Платформа предоставляет интуитивно понятные высокоуровневые API–интерфейсы, например, Keras, с быстрым выполнением, что обеспечивает немедленную итерацию модели и простую отладку. За счёт кроссплатформенности, решение позволяет обучать и разворачивать модели в облаке и локально, независимо от используемого языка. Если нужно запустить модель машинного обучения на смартфоне или IoT–устройстве, то следует использовать среду глубокого обучения с открытым исходным кодом, с названием TensorFlow Lite.

Если нужно создать и обучить модель машинного обучения, написанную на JavaScript, и развернуть её в своем браузере или Node.js, то можно использовать библиотеку TensorFlow.js. Этот фреймворк постоянно развивается за счёт открытого исходного кода и многочисленного сообщества энтузиастов. Уже существует множество решённых задач, что существенно упрощает работу новым разработчикам. Библиотека TensorFlow.js известна и популярна.

Достоинства: это отличный фреймворк для создания нейронных сетей, работающих в Production, для оптимизации вычислительных ресурсов.

Недостатки: сложность в использовании и освоении; необходимость постоянного контроля используемой видеопамати; наличие собственных стандартов, отличающихся от известных; плохо разработанная документация.

OpenCV. Это библиотека, использующая технологии компьютерного зрения с открытым исходным кодом, для обработки численных алгоритмов и изображений. Реализована на языках C/C++, поддерживается языками Java, Lua, Ruby, Python, системой компьютерной математики MatLab и др. Может свободно использоваться в академических и коммерческих целях, так как распространяется в условиях лицензии BSD (Berkely Software Distribution).

На рис. 3.14 показан пример работы библиотеки.

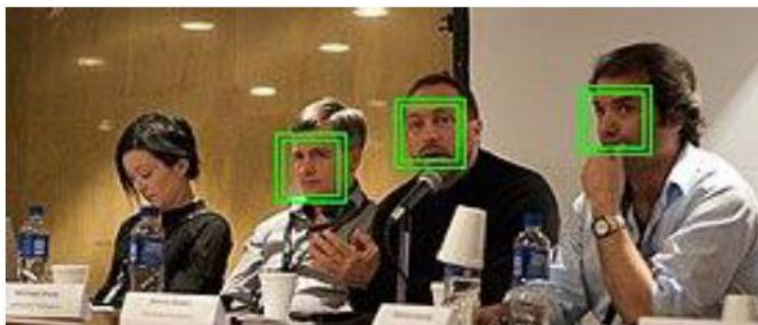


Рисунок 3.14. Пример работы библиотеки

3.2.1. Язык программирования C++

C++ представляет собой статически типизированный, компилируемый язык программирования высокого уровня. Им поддерживаются основные парадигмы программирования: объектно–ориентированное, процедурное и обобщённое программирование. В C++ сочетаются свойства как языков программирования низкого уровня, так и высокого уровня. Этот язык имеет богатые стандартные библиотеки и поддержку многопоточности. В сравнении с его предшественником, языком C, в C++ наибольшее внимание уделено поддержке объектно-ориентированного и обобщённого программирования.

Существует множество реализаций языка C++, как коммерческих, так и бесплатных, для различных платформ. Например, на платформе x86 это Embarcadero (Borland) C++ Builder, Intel C++ Compiler, GCC, Visual C++ и др. Язык C++ оказал огромное влияние на другие ЯП, в первую очередь на C# и Java. C++ является одним из самых известных языков программирования и широко используется в разработке программных средств. Область его применения включает создание высокопроизводительных серверов, операционных систем, драйверов внешних устройств, прикладных программ, приложений для встраиваемых систем, а также компьютерных игр [41].

Синтаксис C++ унаследован от языка C. Одним из принципов его разработки было сохранение совместимости с языком C. Тем не менее C++ не является в строгом смысле надмножеством C; множество программ, которые могут одинаково успешно транслироваться как компиляторами C, так и компиляторами C++, довольно велико, но не включает все возможные программы на C. На рис. 3.15 показан код программы «Hello world».

```
#include <iostream>
int main() {
    std::cout << "Hello, world!"
    << std::endl;
    return 0;
}
```

Рисунок 3.15. Пример синтаксиса языка C++

Для разработки программного средства нужна библиотека OpenCV [42].

Установка OpenCV для C++. Скачав последний стабильный релиз OpenCV с официального сайта, следует распаковать файлы в отдельную папку и добавить двоичные файлы в системный путь, чтобы было можно обращаться через командную строку. Изменение системного пути показано на рис. 3.16.

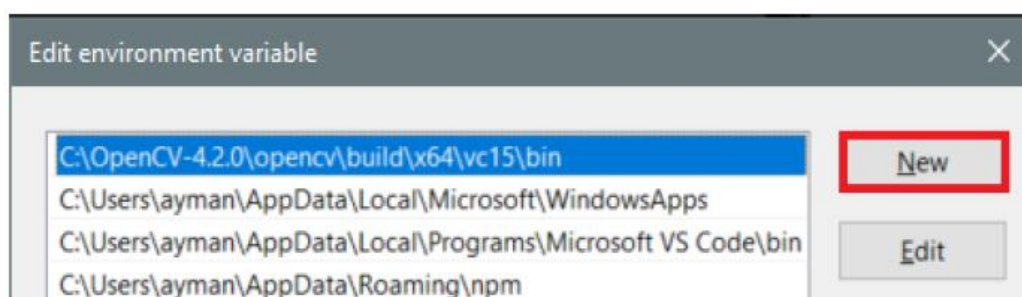


Рисунок 3.16. Изменение системного пути

Следующий шаг – настройка Visual Studio для запуска OpenCV. Для этого необходимо создать новый проект на C++ Console App Template. После создания проекта, меняем значение debug ×86 возле кнопки запуска проекта на debug ×64. Теперь надо поменять настройки проекта, чтобы добавить библиотеки OpenCV. На рис. 3.17 показано местоположение кнопок.

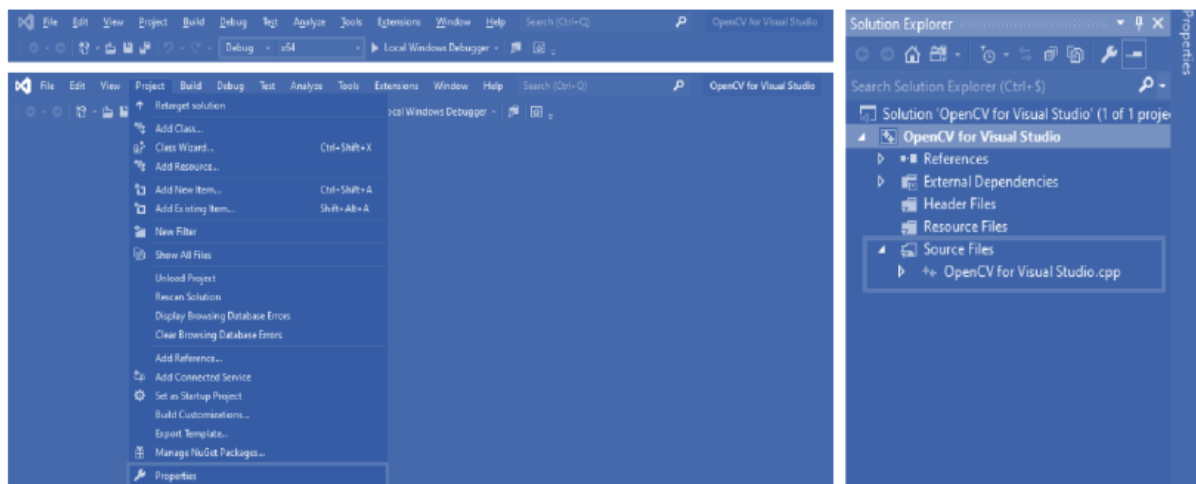


Рисунок 3.17. Окно настроек

Открыв окно настроек, находим VC++ Directories и добавляем пути к файлам include и library OpenCV. Выбор пути к файлам показан на рис. 3.18.

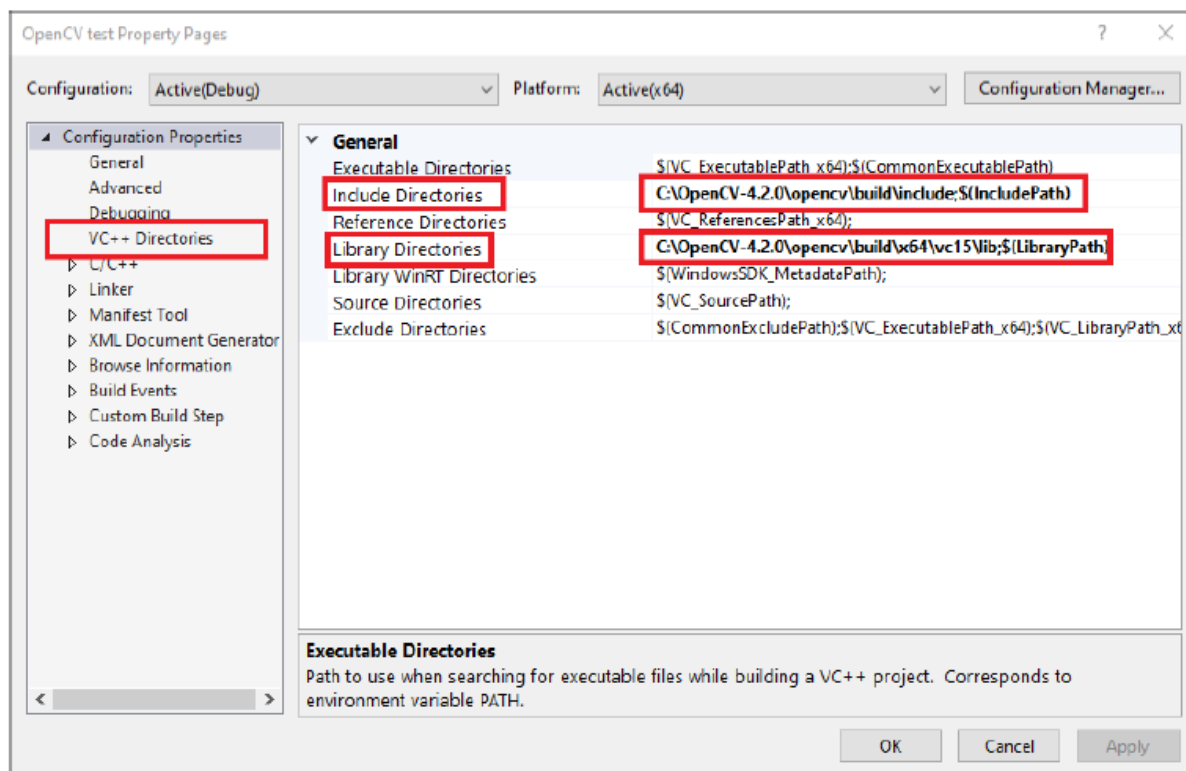


Рисунок 3.18. Добавление файлов OpenCV

Теперь необходимо поменять VC++ project linker, добавив динамическую библиотеку (DLL). На рис. 3.19 подробно показаны шаги добавления динамической библиотеки в среду Visual Studio.

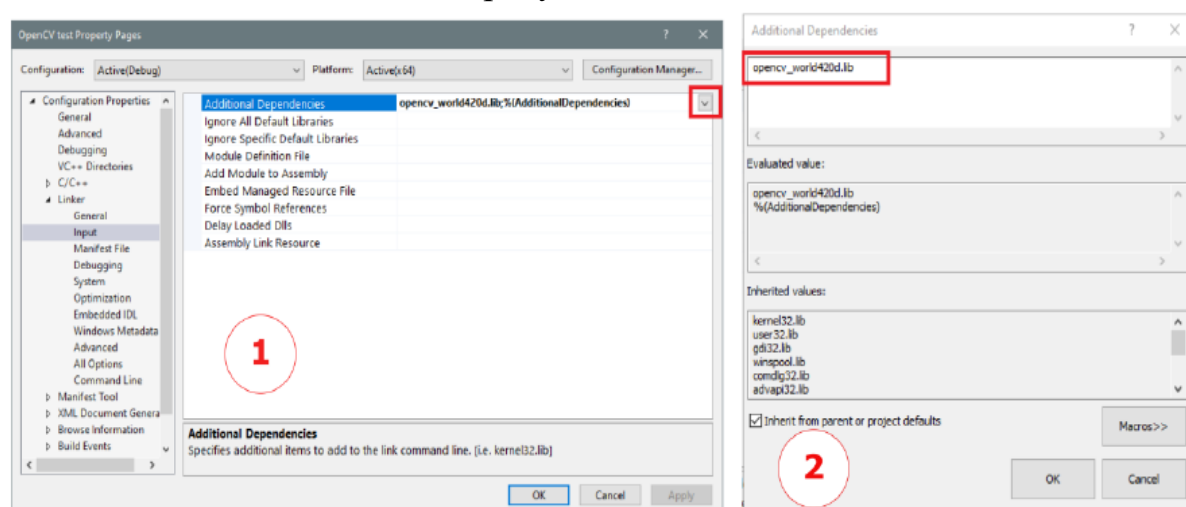


Рисунок 3.19. Добавление DLL

Теперь необходимо запустить код для установки библиотеки OpenCV. На этом процесс установки OpenCV на C++ будет закончен.

3.2.2. Язык программирования Python

Python является высокоуровневым языком программирования общего назначения с автоматическим управлением памятью и динамической строгой типизацией, ориентированный на повышение производительности труда разработчика, качества кода и его читаемости, а также на обеспечение переносимости написанных на нём программ. Необычной особенностью языка является выделение блоков кода пробельными отступами. В языке всё является объектами, он является полностью объектно-ориентированным. На практике редко возникает необходимость обращаться к документации за счёт того, что синтаксис ядра языка минималистичен. Сам же язык используется в том числе для написания скриптов и известен как интерпретируемый [43].

Недостатками языка являются зачастую более высокое потребление памяти написанных на нём программ, и более низкая скорость работы по сравнению с аналогичным кодом, написанным на компилируемых языках типа C или C++.

Python является мультипарадигмальным языком программирования, поддерживающим процедурное, императивное, объектно-ориентированное, структурное, функциональное и метапрограммирование. С помощью динамической типизации решаются задачи обобщённого программирования.

Через декораторы частично поддерживается аспектно–ориентированное программирование, с помощью дополнительных фреймворков обеспечивается более полноценная поддержка. Могут быть реализованы такие методики как логическое и контрактное программирование с использованием библиотек или расширений. Основные архитектурные черты: автоматическое управление памятью, динамическая типизация, механизм обработки исключений, высокоуровневые структуры данных, полная интроспекция, поддержка многопоточных вычислений с глобальной блокировкой интерпретатора (GIL). Поддерживается разбиение программ на модули, которые могут, в свою очередь, объединяться в различные программные пакеты.

В стандартную библиотеку включается большой набор полезных переносимых функций, начиная от функционала для работы с текстом и заканчивая средствами для написания сетевых приложений. Дополнительные возможности, такие как работа с оборудованием, математическое моделирование, разработка компьютерных игр или написание веб–приложений, могут реализовываться посредством обширного количества сторонних библиотек, а также интеграцией библиотек, написанных на Си или C++, при этом и сам интерпретатор Python может интегрироваться в проекты, написанные на этих языках. Также существует специальный репозиторий программного обеспечения, написанного на Python–PyPI. Этот репозиторий стал де–факто стандартом для Python, и предоставляет средства для установки пакетов. По состоянию на 2019 год, в нём содержалось более 175 тысяч пакетов.

Python стал одним из самых известных языков программирования, он используется в веб–разработке, машинном обучении, анализе данных и DevOps, а также в других сферах, включая разработку компьютерных игр. Язык хорошо подходит для обучения студентов программированию, за счёт простого синтаксиса, читабельности и отсутствия необходимости в компиляции, позволяя концентрироваться на изучении алгоритмов, парадигм и концептов. В значительной степени облегчаются экспериментирование и отладка программ, так как язык Python является интерпретируемым. Многие крупные компании используют этот язык, такие как Facebook и Google. В рейтинге популярности языков программирования TIOBE, с показателем 11,03 % по состоянию на апрель 2021 года, язык Python занимает третье место. Python объявлялся *языком года* в 2007, 2010, 2018 и 2020 г. по версии TIOBE. С точки зрения простоты, гибкости и лаконичности язык Python сравнивается с C++ / Java. На рис. 3.20 представлен код «Hello world» на языке Python.

A small rectangular box containing the Python code `print("Hello, world!")`. The word "print" is in green, the opening and closing quotes are in red, and the text "Hello, world!" is in black.

Рисунок 3.20. Синтаксис языка Python в сравнении с языком C++ (см. рисунок 3.15)

Касательно объектно–ориентированного программирования (ООП), в языке Python, в отличие от языков Java и C++, отсутствуют модификаторы доступа к методам класса и полям. Поля и атрибуты у объектов могут создаваться на лету, в ходе исполнения программы, а все методы являются виртуальными. Python, по сравнению с Java, даёт возможность перегружать операторы, что позволяет использовать выражения, близкие к естественным. В совокупности, подход к ООП в Python значительно упрощает процесс программирования, делает код более понятным, и одновременно добавляет гибкости языку. С другой стороны, скорость выполнения кода в языке Python (и других интерпретируемых языках) намного ниже, чем в C++, и ниже, чем в Java. Код на C++ занимает больше строк, но производительнее, чем в Python.

Так, язык Python оказался более гибким чем C++, а язык Java стал компромиссом между производительностью C++ и гибкостью Python.

В Python и Java все объекты создаются в куче, в то время как C++ позволяет создавать объекты как в стеке, так и в куче, в зависимости от используемого синтаксиса. Способ доступа к данным в памяти также влияет на производительность: в Python доступ к данным происходит через хеш–таблицы, в то время как в C++ и Java – по постоянным смещениям в памяти. Использование указателей в C++ довольно сложно, и овладение навыками правильного использования указателей может занять некоторое время.

В отличие от C++, установка пакетов и модулей производится простым способом с помощью команды `Pip Install «имя пакета»`. Это позволяет не тратить время на установку, и сразу перейти к написанию кода, подключая необходимые пакеты по ходу работы, что сильно ускоряет разработку.

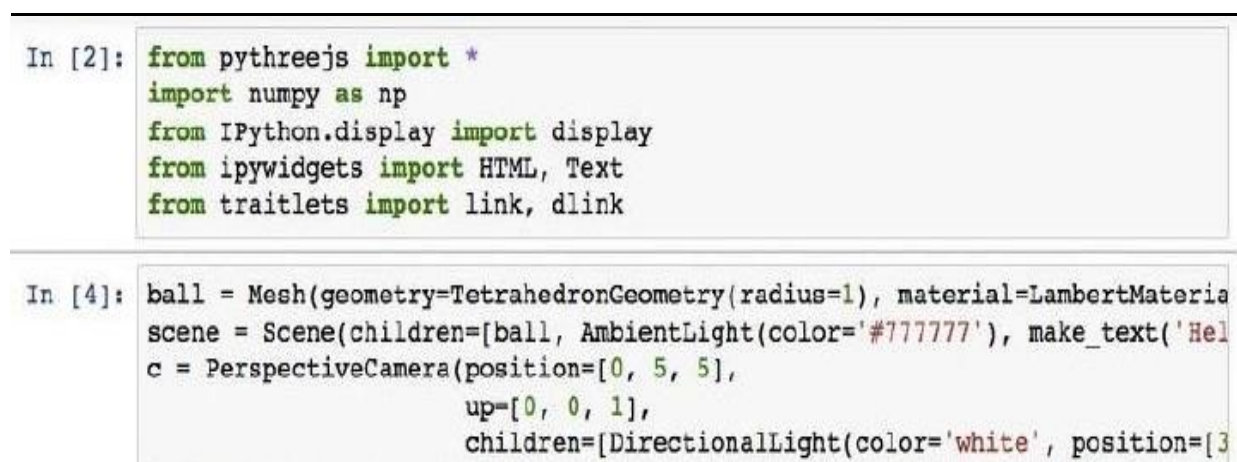
3.2.3. Набор библиотек *Anaconda*

Anaconda включает в себя набор свободных библиотек, объединённых проблематикой машинного обучения и науки о данных, и является дистрибутивом для языков R и Python. Поставка востребованных тематических модулей (SciPy, NumPy, Astropy и др.) осуществляется единым согласованным комплектом, который содержит более 1600 модулей [44].

Anaconda Navigator включена в дистрибутив, и является графическим интерфейсом (GUI), позволяющим устанавливать дополнительные пакеты и запускать приложения без использования командной строки Anaconda Prompt. В Anaconda Navigator, по умолчанию, доступны приложения: Spyder; Jupyter Notebook; Glue; JupyterLab; RStudio; QtConsole; Orange; Visual Studio Code.

3.2.4. Среда разработки Jupyter Notebook

Jupyter Notebook – это мощный инструмент для разработки и просмотра проектов в области Data Science в интерактивном режиме. Он объединяет код и вывод в виде одного полного документа, содержащего текст, математические уравнения и визуализацию. Этот пошаговый подход обеспечивает быстрый и последовательный процесс разработки за счёт немедленного отображения вывода каждого блока. Именно поэтому этот инструмент стал настолько известным в среде Data Science за последнее время. Большая часть он-лайн среды разработчика Kaggle Kernels создана с помощью Jupyter Notebook [45]. Интерфейс интегрированной среды Jupyter Notebook показан на рис. 3.21.



```
In [2]: from pythreejs import *
import numpy as np
from IPython.display import display
from ipywidgets import HTML, Text
from traitlets import link, dlink

In [4]: ball = Mesh(geometry=TetrahedronGeometry(radius=1), material=LambertMaterial)
scene = Scene(children=[ball, AmbientLight(color='#777777'), make_text('Hello')])
camera = PerspectiveCamera(position=[0, 5, 5],
                           up=[0, 0, 1],
                           children=[DirectionalLight(color='white', position=[3
```

Рисунок 3.21. Окно интегрированной среды разработки Jupiter Notebook

Таким образом, авторами была дана оценка программного обеспечения, подходящего для разработки и обучения нейронной сети, а также программы, которая сможет использовать обученную модель. Для разработки данного проекта решено использовать язык программирования Python с дистрибутивом Anaconda, а также интегрированную среду разработки Jupiter Notebook и модули OpenCV и TensorFlow, как основополагающие средства разработчика.

3.3. Разработка программного средства и описание его структуры

3.3.1. Выбор данных для обучения нейронной сети

Программа и модель разрабатывались на языке программирования Python в интегрированной среде, поддерживающей обучение нейронных сетей – Jupyter Notebook (рис. 3.22) с использованием Anaconda (дистрибутивом языков Python и R, включающий набор свободных библиотек, объединенных проблематикой науки о данных и машинного обучения).

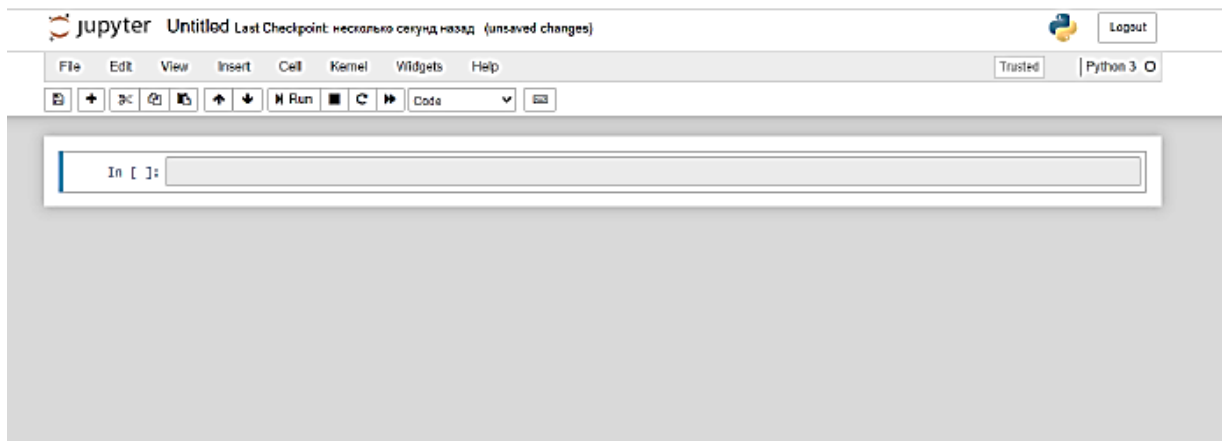


Рисунок 3.22. Рабочая среда Jupyter Notebook

Jupyter Notebook позволяет выполнять часть кода отдельно от основного кода, не теряя при этом данные. На рис. 3.23 показано подключение модулей с помощью оператора Import, а затем в другой ячейке (Cell) код продолжается. Он может быть запущен и отдельно от первой ячейки. Такая возможность позволяет редактировать код в процессе его выполнения, не теряя при этом данные и не перезапуская программу заново, а только ожидая её выполнения. Это ускоряет разработку, позволяя не ждать долгого полного перезапуска всей разработанной программы ради одного участка кода.

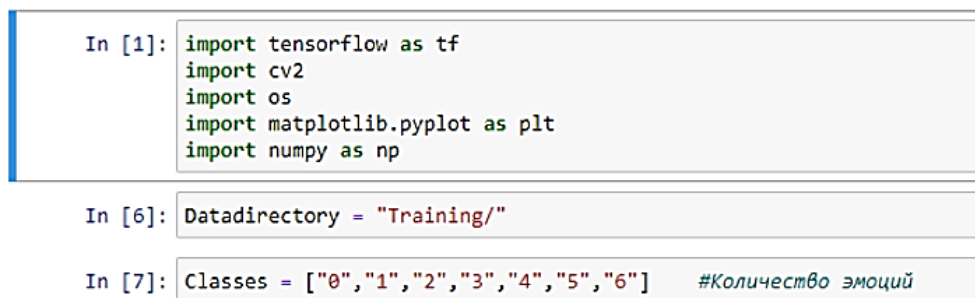


Рисунок 3.23. Ячейки кода

На рис. 3.23 находится начало кода для обучения модели нейронной сети—здесь подключаются модули в первой ячейке. Во второй ячейке задаётся путь к файлам, на которых будет проводится обучение, а в третьей — массив классов. В данном случае, классы—это названия папок, в которых находятся изображения. Для удобства программирования, папки названы семью номерами: 0 — злость (Angry), 1 — отвращение (Disgust), 2 — страх (Fear), 3 — счастье (Happy), 4 — спокойствие (Neutral), 5 — грусть (Sad), 6 — удивление (Surprise).

Следующий шаг — это подсчёт всех изображений и добавление их в массив. Одновременно с этим, к каждому из файлов добавляется цифра класса, к которому они принадлежат. На рис. 3.24 показаны шаги, описанные выше.

```
In [50]: training_Data = []

def create_training_Data():
    for category in Classes:
        path = os.path.join(DataDirectory, category)
        class_num = Classes.index(category)
        for img in os.listdir(path):
            try:
                img_array = cv2.imread(os.path.join(path, img))
                new_array = cv2.resize(img_array, (img_size, img_size))
                training_Data.append([new_array, class_num])
            except Exception as e:
                pass
```

Рисунок 3.24. Создание массива изображений

В таком виде данные для обучения не готовы. Необходимо перемешать их, чтобы нейронная сеть не могла установить зависимость по порядку, в котором они стоят. Ресурсов компьютера, на котором обучалась модель, оказалось недостаточно для обучения. Поэтому, после перемешивания элементов, произошло удаление половины массива, что не пошло на пользу точности, так как эмоции некоторых типов могли вообще не попасть в массив. Обучение проводится много раз, и каждый раз данные перемешиваются.

В следующей ячейке происходит разделение данных на X и Y. Тем самым получается два массива: один содержит изображения, а другой — классы. Массив X конвертируется в массив Numpy, переводя его из массива объектов в массив с одним числовым типом данных. С помощью разделения на 255 каждого пиксела изображения, в массиве X происходит нормализация изображений. Уменьшается диапазон чисел: ранее он был от 0 до 255, а будет от 0 до 1. Массив Y тоже переводится в массив Numpy, так как, из-за присутствия в предыдущем массиве, он получает тип объекта.

На рис. 3.25 показано выполнение перемешивания массива и деление его пополам, из-за недостатка мощности компьютера.

```
In [53]: import random
         random.shuffle(training_Data)

In [54]: training_Data.pop()
         i=0
         while i != len(training_Data)//2:
             D = training_Data.pop()
             i=i+1
```

Рисунок 3.25. Действия над массивом 1

На рис. 3.26 продемонстрировано, каким образом происходит деление на массив изображений X и массив классов Y. Там же показано, как происходит нормализация и приведение массива «Y» к массиву Numpy.

```
In [56]: X = []
         y = []

         for features, label in training_Data:
             X.append(features)
             y.append(label)

         X = np.array(X).reshape(-1, img_size, img_size, 3) ## Конвертация в 4 измерения

In [58]: X = X/255.0; ##Нормализация изображений

In [61]: Y=np.array(y)
```

Рисунок 3.26. Действия над массивом 2

3.3.2. Разработка и обучение модели

Следующим шагом является создание модели для обучения. Для этого за основу взята модель MobileNetV2, которая существенно уменьшит нагрузку на систему при её использовании. На рис. 3.27 показано подключение модели.

```
In [63]: import tensorflow as tf
         from tensorflow import keras
         from tensorflow.keras import layers

In [64]: model = tf.keras.applications.MobileNetV2()
```

Рисунок 3.27. Модель MobileNetV2

Последний этап – это обучение модели. Для этого задаются переменные входа и выхода с именами `base_input` и `base_output`. Они будут использоваться для доступа к модели. Создание переменных показано на рис. 3.28.

```
In [66]: base_input = model.layers[0].input

In [67]: base_output = model.layers[-2].output
```

Рисунок 3.28. Входной и выходной слои

Можно увидеть, что для входа берется первый слой из MobileNetV2, вызываясь через `model.layers[0].input`, ведь нулевой слой является первым. Для выхода берется `model.layers[-2].output`. Обозначение `-2` в этом случае значит, что берётся второй элемент от конца списка.

На рис. 3.29 представлен слой `global_average_pooling2d_1`. Он выбран, чтобы вычленить последний слой, в котором на выход поступает 1000 значений.

block_16_project (Conv2D)	(None, 7, 7, 320)	307200	block_16_depthwise_relu[0][0]
block_16_project_BN (BatchNormal	(None, 7, 7, 320)	1280	block_16_project[0][0]
Conv_1 (Conv2D)	(None, 7, 7, 1280)	409600	block_16_project_BN[0][0]
Conv_1_bn (BatchNormalization)	(None, 7, 7, 1280)	5120	Conv_1[0][0]
out_relu (ReLU)	(None, 7, 7, 1280)	0	Conv_1_bn[0][0]
global_average_pooling2d_1 (Glo	(None, 1280)	0	out_relu[0][0]
predictions (Dense)	(None, 1000)	1281000	global_average_pooling2d_1[0][0]
=====			
Total params: 3,538,984			
Trainable params: 3,504,872			
Non-trainable params: 34,112			

Рисунок 3.29. Слои MobileNetV2

Добавляются необходимые слои, а затем создается собственная модель. Она уже будет выполнять требования, нужные для обучения нейронной сети для разрабатываемого проекта. На рис. 3.30 показано создание модели, которая проходит компиляцию. В следующей ячейке происходит загрузка готовой модели – это та же самая модель, только уже прошедшая этап обучения. Если модель еще ни разу не проходила этап обучения, то эта строка пропускается.

```

In [68]: final_output = layers.Dense(128)(base_output)
         final_output = layers.Activation('relu')(final_output)
         final_output = layers.Dense(64)(final_output)
         final_output = layers.Activation('relu')(final_output)
         final_output = layers.Dense(7, activation='softmax')(final_output) #Слой классификации

In [69]: final_output

Out[69]: <KerasTensor: shape=(None, 7) dtype=float32 (created by layer 'dense_5')>

In [70]: new_model = keras.Model(inputs = base_input, outputs = final_output)

```

Рисунок 3.30. Создание модели

Затем происходит непосредственное обучение модели. Здесь значение Epochs означает количество поколений – то, сколько раз модель обработает данные в массиве, Accuracy – точность, т.е. процентное соотношение правильных ответов сети к неправильным. Параметр Loss – это потери: чем меньше потери, тем лучше модель. Потери рассчитываются на основе обучения и валидации, и их взаимосвязь заключается в том, насколько хорошо модель работает для этих двух наборов. В отличие от точности, потери не определяются процентами: это суммирование ошибок, сделанных в каждом примере, для формирования обучающих или проверочных наборов.

После обучения модель сохраняется либо для дальнейшего обучения, либо уже для использования в программе. На рис. 3.31 показан процесс компиляции новой модели нейронной сети, её обучение и сохранение.

```

In [72]: new_model.compile(loss="sparse_categorical_crossentropy", optimizer = "adam", metrics = ["accuracy"])

In [73]: new_model = tf.keras.models.load_model('Final_model_01d05m')

In [74]: new_model.fit(X,Y, batch_size = 32,epochs = 3)

Epoch 1/3
599/599 [-----] - 2610s 4s/step - loss: 0.5282 - accuracy: 0.0192
Epoch 2/3
599/599 [-----] - 2593s 4s/step - loss: 0.4216 - accuracy: 0.0542
Epoch 3/3
599/599 [=====] - 2602s 4s/step - loss: 0.3649 - accuracy: 0.0762

Out[74]: <tensorflow.python.keras.callbacks.History at 0x26027873160>

In [75]: new_model.save('Final_model_01d05m')

INFO:tensorflow:Assets written to: Final_model_01d05m\assets

```

Рисунок 3.31. Обучение модели нейронной сети

3.3.3. Разработка приложения, использующего модель

После обучения модели необходима программа, с помощью которой можно проверить работоспособность обученной нейронной сети. Язык программирования Python, изначально, не имеет никаких графических опций для разработки приложений с окнами. Чтобы протестировать обученную модель, сначала нужно написать программный интерфейс, в котором будут реализованы кнопки активации функций и окно вывода изображений.

Существует отдельное программное приложение «QT Designer», которое выполняет те же графические функции, что и Visual Studio, но не позволяет ни запустить функции, которые не относятся к интерфейсу, ни вставить их. Наоборот – задается функция интерфейса и вставляется в изначальный код.

Но разрабатывать интерфейс возможно и без использования программы «QT Designer». Это можно осуществить, если подключить модуль PyQT, и напрямую в программном коде указать координаты и размеры интерфейса.

Для его реализации будет использоваться PyQT5 – набор расширений графического фреймворка Qt для языка программирования Python, позволяющий работать с окном, кнопками и другими элементами интерфейса.

На рис. 3.32 представлена блок–схема алгоритма работы интерфейса.

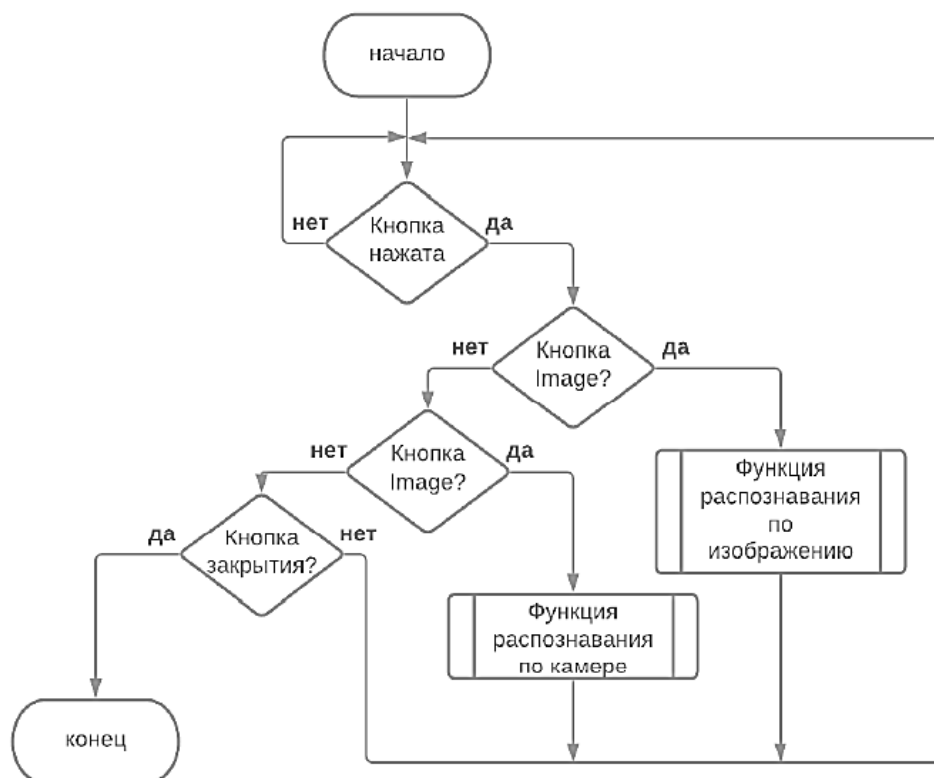


Рисунок 3.32. Блок–схема алгоритма работы интерфейса

На рис. 3.33 представлен вид программы, включающий в себя две кнопки и невидимый, при запуске, Label, в который выводится изображение.

Интерфейс программы минималистичен, так как для разрабатываемой программы, которая, по существу, является прототипом, нет необходимости добавлять ненужные свойства или создавать какие-либо декорации.



Рисунок 3.33. Окно программного приложения

Обе кнопки вызывают свои функции:

- кнопка Image вызывает функцию ImgRecog. Эта функция открывает диалоговое окно выбора файла с типами: jpeg, jpg, png, и, после выбора, сначала определяет лица, вырезает их, а затем выявляет эмоции на изображении.

- кнопка WebCam запускает функцию CamRecog, которая открывает новое окно и определяет эмоции, вычлняя кадры из видеопотока видеокамеры. Принцип работы CamRecog такой же, как и у ImgRecog. Единственная разница между этими двумя функциями – наличие цикла While у функции CamRecog, которая выполняет программу до тех пор, пока не будет нажата клавиша «q»

На рис. 3.34 и 3.35 представлены блок-схемы их алгоритмов:

Для определения эмоций необходимо найти лица. С этой задачей в программе справляется метод Haar Cascade, использующий алгоритм Виолы-Джонса. После нахождения на картинке лиц, их изображения обрезаются, а их координаты сохраняются. На обрезанных изображениях выявляются эмоции с помощью обученной модели нейронной сети. Затем на сохранённые координаты накладывается рамка, обрамляющая лицо. На изображении, немного выше (на 10 пикселей), выводится текстовое сообщение о распознанной эмоции.

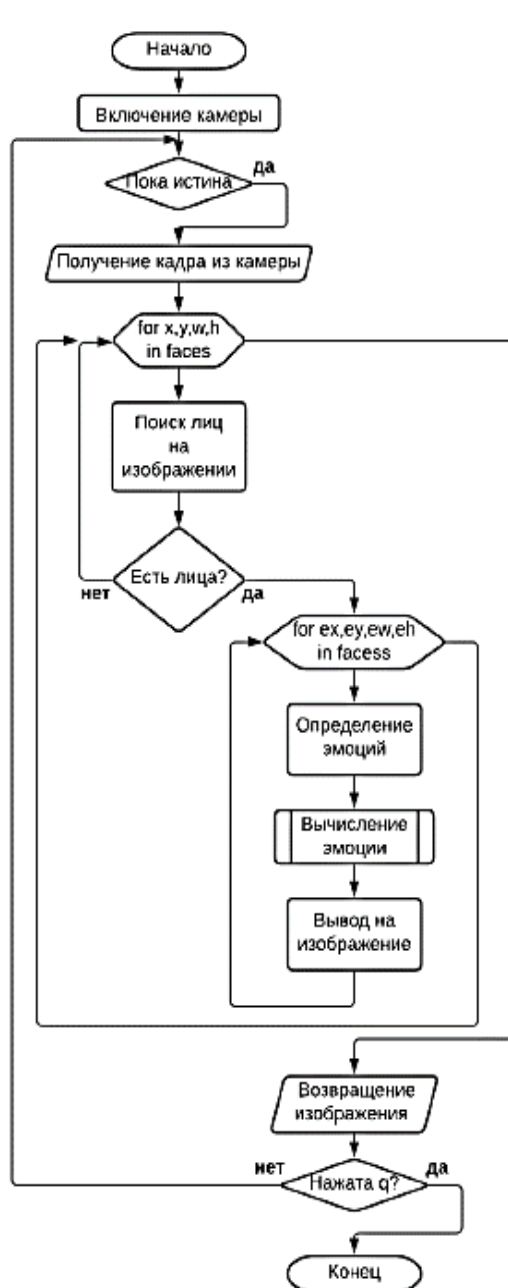


Рисунок 3.34. Блок–схема алгоритма определения эмоций по изображениям

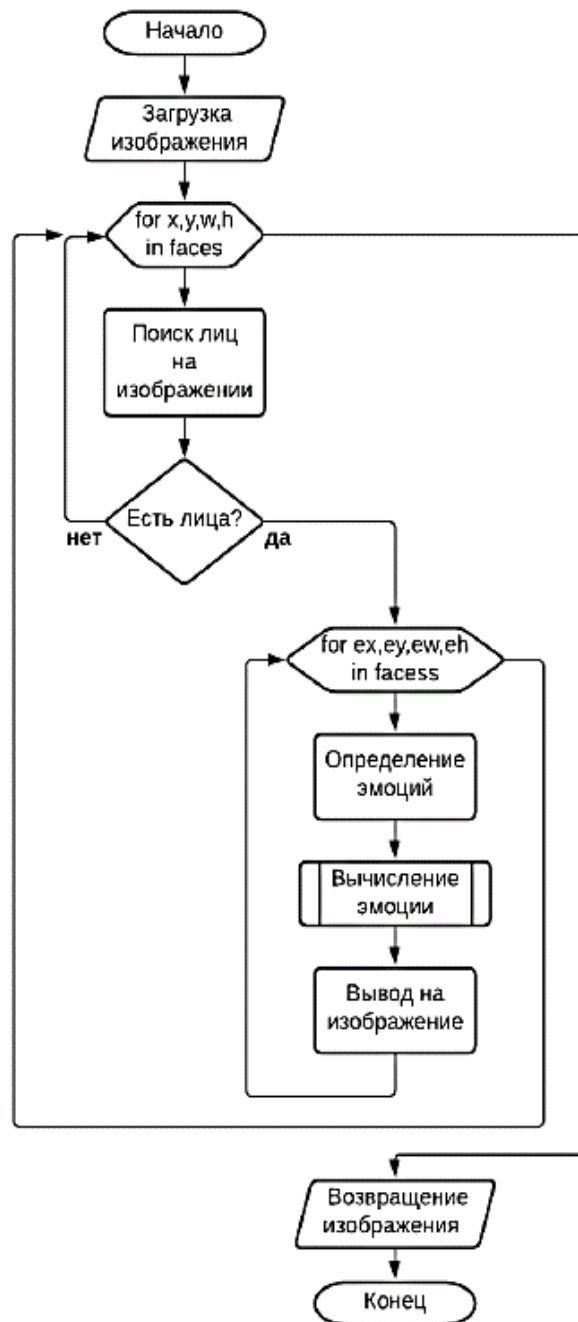


Рисунок 3.35. Блок–схема алгоритма определения эмоций в реальном времени

3.3.4. Тестирование работы программного обеспечения

Для понимания возможностей разработанного программного обеспечения необходимо провести его тестирование. Результаты работы первого теста продемонстрированы на рис. 3.36. Этот тест проводился с использованием изображения одного человека, у которого явно выражена эмоция счастья.

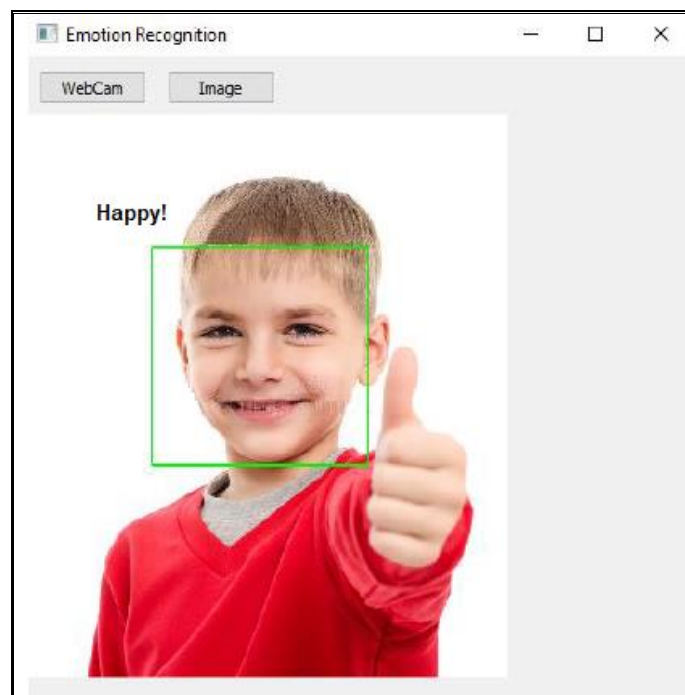


Рисунок 3.36. Результаты теста одного лица с явно выраженной эмоцией

Далее, было протестировано изображение группы людей (рис. 3.37):



Рисунок 3.37. Результаты теста группы лиц с различными эмоциями

Видно, что программа смогла найти все лица людей и определить их эмоции. Также она нашла еще одно лицо на заднем плане фотографии, но определить эмоцию не смогла из-за маленького размера и сильного поворота.

Следующим этапом тестирования стало изображение темнокожего человека. У некоторых существующих программных средств на этом этапе возникали проблемы. Результаты работы теста представлены на рис. 3.38.

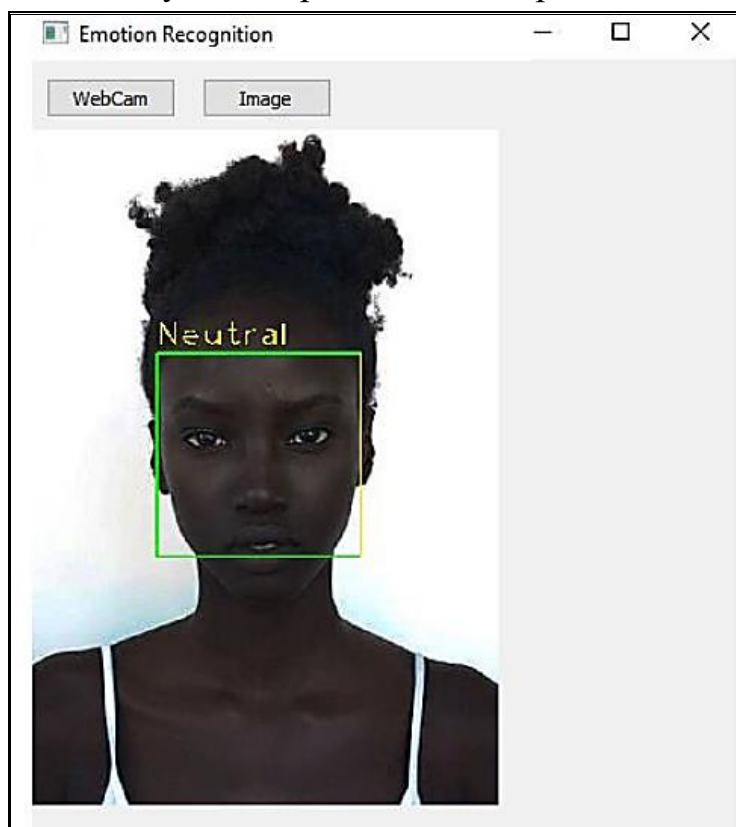


Рисунок 3.38. Результаты теста лица темнокожей девушки

Очевидно, что разработанная программа справилась с задачей.

Так как алгоритм Виолы–Джонса обнаруживает лица, ориентируясь по глазам, рту, носу, расположением бровей, то любое отсутствие одного из этих факторов может повлиять на работу программы. Это же касается и нейронной сети, определяющей эмоции. Для неё важно, в каком положении находятся лицевые признаки, а для темнокожих людей программа может не быть способной обнаружить нос, рот, лоб, брови, или линию волос, не говоря уже об эмоциях, ввиду их плохой различимости на фоне остального лица.

Поэтому далее необходимо проверить, как программа работает на лицах, которые частично закрыты, например, очками. Тест показан на рис. 3.39.

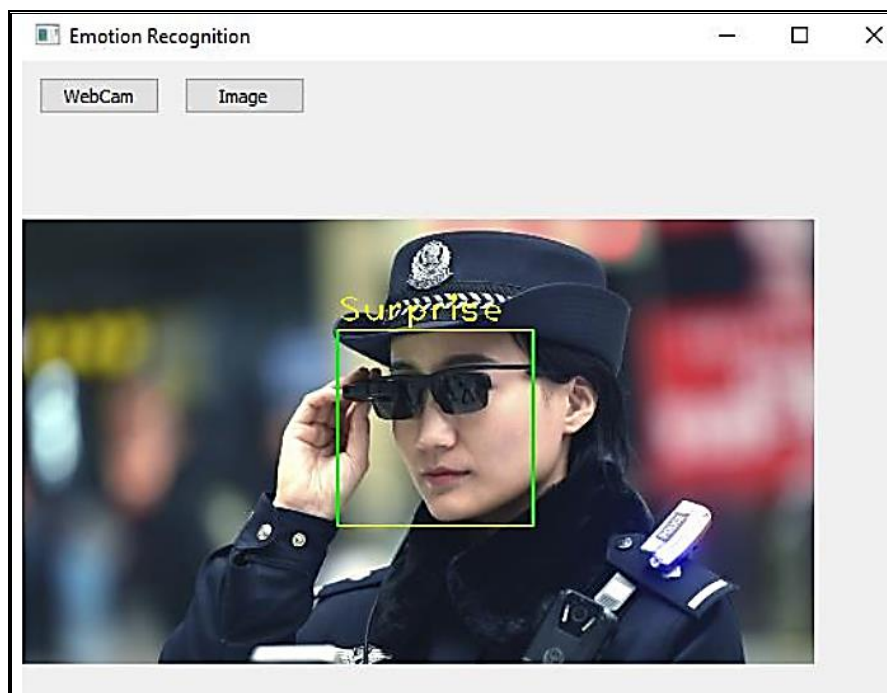


Рисунок 3.39. Результаты теста наполовину закрытого лица

Лицо обнаружено, а на лице определена эмоция, но неправильная. Лицо у девушки явно не является удивленным, а, скорее, спокойным. Это означает, что программа работает некорректно, если не видно выражения глаз.

Некоторые программы не могут определить лица у детей от 0 до 3 лет из-за слишком мягких переходов и неявно выраженных черт лица. Здесь также может происходить ситуация, как с темнокожими людьми. В связи с этим, необходимо проверить программу на возможность определения эмоций у маленьких детей. Результаты теста показаны на рис. 3.40.



Рисунок 3.40. Результаты теста лица маленького ребёнка

Программа не смогла определить лицо ребенка до 3 лет. Был проведен ряд тестов с лицами детей, но обученная нейронная сеть не распознала ни одно. Значит, программа не может работать с лицами маленьких детей.

Следующим шагом является проверка работы алгоритма определения лиц в реальном времени с помощью камеры. После включения камеры, видеопоток поступает на вход программы, где из него вычлняются нужные кадры, которые анализируются, изменяются и выводятся в окно.

Далее представлены четыре кадра, взятые в различные моменты времени, на которых представлены четыре эмоции. Результаты показаны на рис. 3.41.

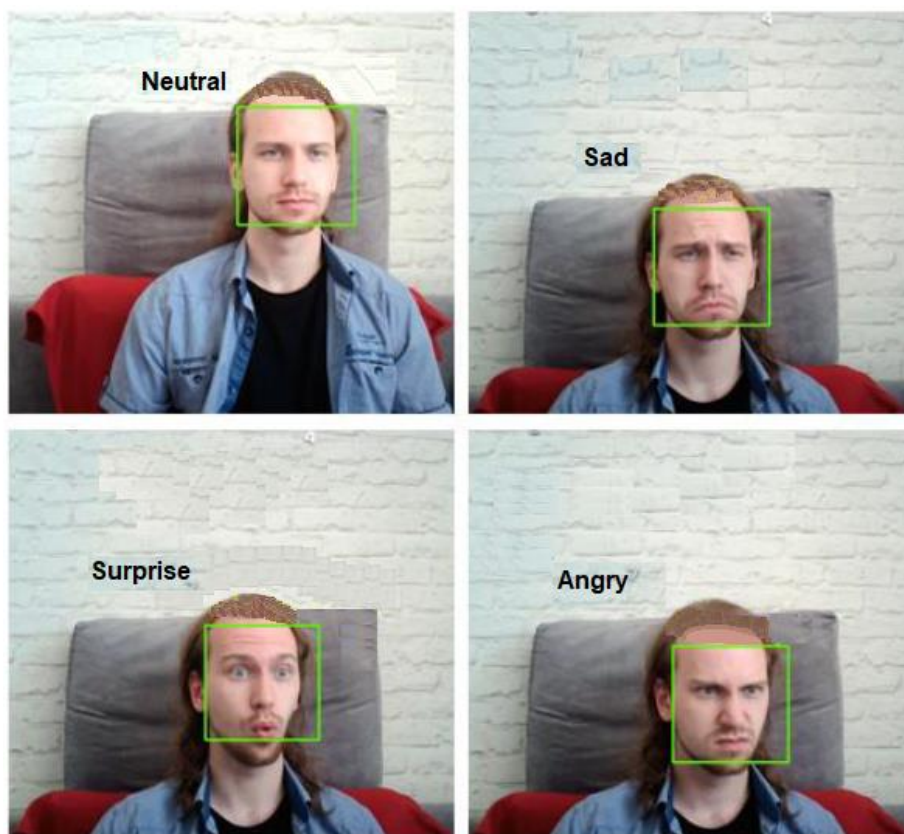


Рисунок 3.41. Результаты теста захвата лиц в видеопотоке

С задачей определения лица в реальном времени, выбирая кадры из видеопотока, программа справилась успешно. Таким образом, обученная нейронная сеть и разработанное программное приложение, использующее эту модель, показали свою эффективность при решении большинства задач распознавания лиц, что было подтверждено в результате тестирования. К его достоинствам можно отнести возможности определения эмоций у темнокожих людей и использования необходимых навыков в комбинации, например, при определении нескольких эмоций на изображении в реальном времени.

4. МОБИЛЬНОЕ ПРИЛОЖЕНИЕ РАСПОЗНАВАНИЯ ЭМОЦИЙ ЧЕЛОВЕКА НА ОСНОВЕ НЕЙРОСЕТЕВЫХ АЛГОРИТМОВ

Внедрение систем распознавания эмоций сегодня ведётся во многих сферах жизни общества. Любая информационная система, взаимодействующая с пользователем напрямую, нуждается не только в анализе смысла команд или действий человека, но также и в понимании его эмоций, т.е. невербального общения. Первенство по применению нейросетей, распознающих эмоции, в России занимают банки. Сбербанк, Альфа–Банк, ВТБ, Росбанк и др., используют эмпатичные интеллектуальные системы, для анализа удовлетворенности клиентов от общения с операторами. Имеются прецеденты использования анализа эмоций клиентов для принятия решений об одобрении кредитов. Такие сервисы, в своих оценках, учитывают паузы в речи, время и громкость общения. Интересной сферой применения нейросетей, распознающих эмоции, является использование голосовых помощников. Массачусетский технологический институт, к примеру, разработал приложение, которое способно определить эмоциональное состояние водителя. Оно следит за уровнем стресса, и стремится снизить его при превышении нормального значения, с помощью регулирования температуры в автомобиле (отопления или кондиционирования), включения успокаивающей музыки или распространения приятных ароматов. Похожее применение систем с эмпатией есть у компании Amazon в фитнес–браслете Halo, который, на основании определения эмоционального состояния пользователя даёт ему советы по проведению расслабляющих процедур или тренировок [46].

Одна из крупнейших областей применения распознавания эмоций – это рекламный рынок. Сейчас, для кастомизации рекламного контента, в основном, используют информацию о запросах пользователя, времени просмотра баннера и др. Этот приблизительный подход, потому что он строится на сложной системе допущений. Уже сейчас, среди крупных сервисов, предоставляющих рекламные площадки, происходит внедрение распознавания эмоций. Например, компания Disney, в своём стриминговом сервисе, по выражению лица пользователя определяет его удовлетворенность контентом. По прогнозам Gartner, к 2024 году больше половины всей рекламы в Интернете будет предоставляться на основе распознавания эмоций. Основным катализатором быстрого роста этой сферы являются голосовые помощники на Web–сайтах, которых, по статистике, для покупок используют уже 50% жителей США и стран Западной Европы.

Многие крупные Интернет–магазины разрабатывают своих «умных ассистентов», чтобы свести весь процесс покупки товаров к простой голосовой команде. Рынок систем для аналитики и распознавания эмоций оценивается более чем в 20 млрд. долларов, и, по расчётам, к 2025 году вырастет вдвое [47].

Развитие точности эмпатии у интеллектуальных консультантов и помощников необходимо, потому что эта отрасль уже сейчас имеет большую конкуренцию. На данный момент боты способны проявлять себя лучше рядовых консультантов call–центров, а в связи с тем, что бот обходится компании в 3–5 раз дешевле, чем сотрудник, все больше компаний делегируют часть функций по общению с клиентами чат–ботам (около 57% компаний на данный момент) – и в первую очередь крупные компании, такие, как МТС, Сбербанк, Тинькофф и др. Бизнесу становится невыгодно и невозможно контролировать деятельность интеллектуальных консультантов и ассистентов вручную. Необходимо развивать у них системы самооценки деятельности. У человека эту функцию выполняет эмпатия, и было бы естественно попробовать воссоздать её и у систем интеллектуальных помощников, в чьи функции входит общение с эмоциональным человеком – клиентом или пользователем [48].

Сейчас существует множество систем, определяющих эмоции человека по голосу, и они достаточно точны, особенно, если для модели необходимо сузить спектр эмоций до положительной или отрицательной. В таком случае алгоритмы работают с точностью до 90%. Но с учётом масштабов областей применения, необходимо стремиться к повышению точности.

В психологии, эмоции принято разделять по длительности, а также по силе их воздействия. Если системе необходимо оценить отношение человека к некоторому объекту, например, к рекламному баннеру или обслуживанию кассира, то необходимо учитывать, что под влиянием долгосрочных эмоций, например, плохого настроения, хронического стресса или воодушевления, реакция на объект может быть усилена, уменьшена, искажена или вообще отсутствовать, при сохранении у человека другого, устойчивого и длительного эмоционального процесса. В таком случае, было бы полезно не полагаться на статистические методы погрешностей, а обучать систему распознавания эмоций анализировать также тот факт, находится ли человек в длительном состоянии подавленности или возбуждения, вызванном посторонними причинами. Как следствие, в окончательной оценке удовлетворенности клиента товаром или услугой нужно учитывать этот психологический фактор.

Кроме того, бывают ситуации, когда необходима мобильность системы распознавания эмоций. Так, в медицине давно подтверждено клинически, что использование эмпатичных систем помогает пациентам с *синдромом аутизма* адаптироваться в обществе и учиться самостоятельно распознавать эмоции.

Аутизм – это расстройство, возникающее вследствие нарушения развития головного мозга. Оно характеризуется выраженным дефицитом социального взаимодействия и общения, ограниченными интересами и повторяющимися действиями. Причины аутизма связаны с генами, влияющими на созревание синаптических связей в головном мозге. По данным из США, в 2020–2021 годах аутизм и расстройства аутистического спектра развития головного мозга диагностированы на официальном уровне у 2 % детей школьного возраста [49].

Поэтому *целью исследования* является создание мобильной системы распознавания эмоций по голосу, которая в режиме реального времени поможет человеку определить, какие чувства испытывает его собеседник, без использования нателных датчиков, компьютера или специальных очков.

4.1. Существующие аналоги алгоритмов распознавания эмоций

В настоящее время системы распознавания эмоций по голосу применяются в самых разных сферах общественной жизни – от оценки качества обслуживания и рекламы, до медицины и обеспечения безопасности. Перечислим основные проблемы, с которыми сталкиваются разработчики подобных систем:

- эмоции субъективны, и люди разных культур воспринимают их по-разному;
- сложность представляет стандартизация голосовых отрывков для анализа.
- выбор, в каких случаях следует делить фразу на отдельные слова, а в каких случаях нужно использовать предложения целиком, не очевиден;
- сбор данных: самый простой метод сбора данных – использование аудио дорожек фильмов, блогов и новостей. Но сложно доказать объективность подобных источников, потому что новости предполагают более нейтральный тон, а блоги – наигранный. Существует проблема ложных новостей (фейков), специально влияющих на эмоции, а что касается фильмов и спектаклей, то сложно судить, насколько сыгранные эмоции соответствуют реальным;
- разметка данных требует больших человеческих и временных ресурсов. Пользователю нужно прослушать базу данных из тысяч голосовых отрывков, промаркировать её, и надеяться на объективность экспертов при оценке эмоций.

Новейший пример использования системы анализа эмоций по голосу применило коллекторское агентство «Сбербанка» – «АктивБизнесКонсалт». Благодаря такой программе коллектор может правильно подобрать тот сценарий разговора, при котором должник вознамерится расплатиться с банком с максимальной эффективностью. По мнению разработчиков, эта система позволяет с вероятностью в 80% спрогнозировать развитие разговора с должником, что сказывается на качестве самого диалога. Кроме того, сервис предназначен для анализа эмоционального выгорания сотрудников.

Разработчики из Сбербанка планируют внедрить еще множество проектов, связанных с распознаванием эмоций. Одно из приоритетных направлений развития – внедрение описанной выше технологии в создание специального чат-бота, который предназначен для работы с должниками. Кроме того, Сбербанк выкупил компанию «Ё-мобиль», для того, чтобы, по мнению экспертов, развивать в России сферу беспилотных автомобилей.

Для рекламы распознавание эмоций внедряется повсеместно. Стартапы Cloverleaf и Affectiva представили электронные ценники с рекламой под названием ShelfPoint. Эти ценники, с помощью встроенных камер, собирают данные об эмоциях покупателей. Такую технологию уже опробовали компании Procter & Gamble, Walmart и другие Интернет-магазины.

Российский стартап Promobot создал робота-консультанта, использующего нейронную сеть, разработанную Neurodata Lab. Подобная нейронная сеть определяет эмоции по видеозаписи лица, аудиозаписи голоса, а также с помощью нательных датчиков, снимающих частоту дыхания и пульс. Этот робот способен взаимодействовать с клиентами, считывать их эмоции и влиять на них для того, чтобы делать персонализированные предложения, удовлетворяя потребности клиентов, чем существенно поднимать продажи.

Компания Promobot хорошо зарекомендовала себя на мировом рынке, что позволило ей начать активно продавать разработанных ею роботов за границу. Основные страны, закупающие новейшие технологии у Promobot – это Соединенные Штаты, Саудовская Аравия, Израиль, Кувейт и Швеция. Роботы этой компании могут выполнять услуги продавцов, консультантов, гидов и консьержей. Естественно, что для взаимодействия с людьми им необходимы инструменты эмпатии и распознавания эмоций клиентов.

В Сбербанке, ВТБ и «Альфа-Банке» тестируют технологию разбора эмоций с видеокамер наблюдения для анализа удовлетворенности клиентов банка. Кроме того, такой сервис способен распознавать опасные ситуации и подавать сигнал о происшествии службе охраны. «Росбанк» совместно с Neurodata Lab протестировал систему определения эмоций покупателей по голосу и речи. В процессе тестирования посетитель звонил в банк, а нейронная сеть рассматривала его эмоциональное состояние и смысл разговора. Кроме того, искусственный интеллект замечал паузы в речи оператора, громкость голоса и время общения с клиентом, что позволило не только обследовать удовлетворенность клиента предоставленным сервисом, но и контролировать службу операторов контакт-центра и профессионализм каждого из них.

Росбанк разработал систему для анализа эмоционального состояния клиента по тексту, что позволяет отвечать клиенту в чате с высокой эффективностью, а в нужный момент заменить чат-бота оператором.

Системы определения эмоций по голосу достаточно просто обмануть. Например, в одной из американских школ ученица протестировала систему распознавания агрессии от голландской компании Sound Intelligence в два этапа. В первый день она в течение нескольких минут издавала угрожающие крики, но система никак не отреагировала на это событие. На следующий день девушка громко кашляла рядом с микрофоном тестируемого устройства, и оно отправило сигнал тревоги на пульт управления [50].

Системы анализа аудио информации часто подразумевают наличие дополнительного оборудования, как минимум, компьютера с подключенным микрофоном, но в некоторых ситуациях это оказывается не конструктивно.

Системы распознавания эмоций активно применяются для адаптации людей, страдающих синдромом аутизма. Существует разработка от компании Affectiva, с подтвержденной эффективностью – умные очки с простым графическим интерфейсом, отображающие на своем экране яркие вспышки света, отражающие эмоции собеседника, что помогает человеку с аутизмом научиться самостоятельно определять чувства других людей. Главный минус этой разработки состоит в том, что разработанные умные очки так и не вышли на уровень массового производства. Кроме того, они требуют подключения к специальной аппаратуре, и, как минимум, к компьютеру или ноутбуку.

Из подобных разработок нужно отметить свёрточную нейронную сеть, созданную учёными из Массачусетского технологического института, для определения *состояния депрессии* человека с помощью анализа его речи. Доказанная точность такой интеллектуальной системы составила 77% [51].

Компания Beyond Verbal создала интеллектуальную информационную систему для анализа ментального здоровья человека. Исследования помогают оценить состояние здоровья пациента. Благодаря использованию системы искусственного интеллекта, анализируются такие особенности голоса, которые невозможно уловить человеческим ухом. Особенность их нейросети заключается в том, что она выделяет из аудиозаписи только голосовые биологические маркеры, которые сообщают многое о состоянии здоровья человека, начиная от выявления стресса и депрессии, и заканчивая диагностикой болезнью сердца [52].

Создание мобильного приложения со сходными функциями способно облегчить распространение такой важной биомедицинской технологии.

Обычные психологические тесты для диагностики аутизма не отличаются высокой точностью, но психологи из университета Северной Каролины научились диагностировать расстройства аутистического спектра крайне высоко точно, даже у шестимесячных детей. Это делается с помощью алгоритма машинного обучения, анализирующего магнито-резонансные карты функциональной связности мозга человека, сделанные в томографе.

Аутизм и расстройства аутистического спектра – это неврологические заболевания развития, которые характеризуются нарушениями в социальной сфере и повторяющимся поведением. Статистически к данному диагнозу причисляют любого шестьдесят восьмого ребенка. Это заболевание не лечится, впрочем, специфические педагогические программы могут помочь детям адаптироваться в обществе и вести самостоятельную жизнь [49].

Поведенческая терапия будет успешна, если довольно заблаговременно начать работать с ребенком. Симптомы начинают обнаруживаться, как правило, уже на втором году жизни. Имеются диагностические признаки, основанные, к примеру, на анализе движения глаз, по которым возможно предсказать расстройство в возрасте от шести месяцев до года. Впрочем, они считаются недостаточно надёжными, потому что на этот признак могут воздействовать и другие заболевания, и эти движения не специфичны для расстройства.

Ранее было показано, что для аутизма характерно непостоянное формирование многообразных площадей мозга, что можно детектировать при помощи магнитно–резонансной томографии. С развитием функциональной МРТ в медицине появилась возможность предопределять функциональную взаимосвязанность между разными участками головного мозга.

Исследуемая выборка включала 59 младенцев из категории высокого риска развития расстройств аутистического спектра, то есть из семей с уже известными случаями заболевания. В возрасте шести месяцев их исследовали с помощью магнитно–резонансной томографии, для того, чтобы составить карту многофункциональной связанности различных участков мозга. Эта методика предполагает, что области, которые в состоянии покоя или действия характеризуются сигналом одной интенсивности, являются функционально связанными. На основе матриц связанности областей полученных изображений нейронная сеть предвредила возможность расстройства у девяти младенцев, и впоследствии эта диагностика подтвердилась [49].

4.2. Объективные характеристики проявления эмоций в речи

Общение людей—это комплексный процесс, включающий в себя *вербальный* уровень – слова и *невербальный* уровень – действия, вызванные эмоциями. Распознавание эмоций (эмпатия) – это древний эволюционный механизм, свойственный всем млекопитающим. Этот процесс интуитивен, и не требует осмысленного анализа [53]. Эмпатия помогает понять настроение и чувства собеседника, и, исходя из этого, регулировать собственное поведение.

Когда прикладной задачей является оценка отношения человека к какому–либо товару или услуге, то для машинного обучения часто используется непрерывный подход с разделением эмоций на положительные и отрицательные. Задача исследования состоит в повышении эффективности такого подхода, добавив в него одновременно и оценку настроения, так как этот параметр влияет на то, насколько ярко человек выражает эмоцию [54].

Ранее использовались термины, относящиеся к психологическому состоянию человека, принятые в русскоязычном IT–сообществе, однако было бы целесообразно уточнить определения, принятые в психологии [55].

Настроение – это продолжительный эмоциональный процесс невысокой интенсивности, образующий общий эмоциональный фон для протекающих психологических процессов. Оно отличается от аффектов, чувств и эмоций.

По эмоциональному тону оно бывает ровным (*эйтимическим*), пониженным (*гипотимическим*), повышенным (*гипертимическим*), тревожным и др.

«Чувство – это эмоциональный процесс, отражающий субъективное оценочное отношение человека к реальным или абстрактным объектам. Чувства отличаются от аффектов, эмоций и настроений» по [55, 56].

В таком случае нужно переформулировать задачу. Когда прикладной задачей является оценка отношения человека к некоторому товару или услуге, подразумевается оценка его чувств. Зачастую, для машинного обучения, используется оценка эмоций – непродолжительных эмоциональных состояний, что уместно не в каждой ситуации. Задача исследования состоит в том, чтобы повысить эффективность стандартного подхода за счёт оценки настроения, так как этот параметр влияет на то, как человек выражает свои чувства.

Основные изменения в вегетативной нервной системе, отражающиеся в человеческой речи, можно отследить по изменению громкости, высоты тона и скорости речи во всём диапазоне этих изменений в течении диалога [57].

Несмотря на то, что эти признаки наиболее точно свидетельствуют об аффективном состоянии (это наименее продолжительное эмоциональное состояние), благодаря возможностям нейронных сетей эффективно решать задачи прогнозирования и распознавания образов, правильно разметив данные для обучения, на основании этих параметров можно точно определить и более сложные эмоциональные состояния человеческой психики.

Эволюционная концепция возникновения эмоций появилась в книге Чарльза Дарвина «Выражение эмоций у человека и животных» в 1872 году. В ней дарвинистский принцип применяется к психическому формированию живого организма, и доказывается, что между действием животного и человека не существует большой разницы. Как показал Дарвин, во внешнем выражении многообразных психологических состояний, в экспрессивно–телесном выражении у антропоидов и слепорожденных детей наблюдается много общих корпоративных реакций. Эти наблюдения легли в базу его теории.

Эмоции, согласно данной теории, возникли в ходе эволюции живых существ как жизненно важные приспособительные механизмы для адаптации организма к условиям и ситуациям его жизни. Физиологические изменения, сопровождающие всевозможные экспансивные состояния, объединенные с соответственными эмоциями движения, по Дарвину, есть не что иное, как рудименты реальных приспособительных реакций организма [57].

Действительно, совокупность психологических состояний человека и высших животных, стоящих, по своему интеллектуальному развитию, близко к человеку, настолько очевидна, что никогда не вызывала сомнений.

Рудиментарная теория эмоций развивает эволюционную мысль о происхождении эмоций, расценивая их со стороны поведенческой сферы. После публикации работ Чарльза Дарвина, ряд психологов (в частности, Герберт Спенсер и его ученики, французские позитивисты – Теодюль Рибо и его школа, а также германская биологически ориентированная психология) стали совершенствовать мысли о биологическом происхождении человеческих эмоций из аффективных и невольных реакций животных [55 – 57].

Предполагается, что эмоции можно рассматривать как остаточные проявления результатов, когда-то сопровождавших биологические реакции. В этой концепции, выразительные движения, сопровождающие наш страх, смотрятся как остаточные реакции при бегстве и обороне, а телодвижения, сопровождающие наш гнев, выглядят как остаточные фрагменты движений, сопровождавших реакцию нападения у наших зоологических предков.

Страх стал рассматриваться как «заторможенное бегство», а гнев – как «заторможенная драка», то есть эмоции позиционируются будто «затухающее эхо» примитивных программ поведения человека, возникших в ходе его эволюции. Закономерность предоставленной концепции приводит к несколько неоднозначным выводам об отмирании эмоций как таковых в ходе будущей эволюции, и не объясняет разнообразие эмоционального мира человека.

Психоанализ обращает внимание на энергетическую составляющую психических процессов, рассматривая в данном плане и эмоциональную сферу. Несмотря на то, что подобный теоретический вариант истолкования эмоций был недостаточно привязан к организации работы мозга, он в заинтересовал многих исследователей, занимавшихся данной проблемой. Согласно воззрению Зигмунда Фрейда, неосознаваемые эмоции являются источником излишка энергии, который он определяет, как *либидо*.

Структурное содержание понятия *либидо* обусловлено остроконфликтной ситуацией, имеющей место в прошлом, и зашифрована на инстинктивном уровне в сознании человека. Однако факты, свидетельствующие о подобной пластичности психики, плохо согласуются с идеей «законсервированного» конфликта. В такой догадке плохо просматривается биологический смысл.

Некогда психоанализ пришёл к выводу, что энергия «бессознательного» не хранится в структурах мозга как «дефект развития», а представляется следствием появления в психике излишка энергии, как результат неидеальной адаптации личности в социуме. Например, австрийский психолог Альфред Адлер считал, что большинству детей изначально присуще чувство личного несовершенства в сравнении со «всемогущими взрослыми», что ведёт к выработыванию у них комплекса неполноценности. Согласно воззрениям Адлера, формирование личности зависит от того, как данный комплекс будет компенсироваться. В патологических вариантах человек стремится победить свой комплекс неполноценности установлением власти над другими людьми.

Концепция американского невропатолога Джеймса Пейпеца, являющаяся примером нейропсихологического подхода к обсуждению эмоций, была создана в 1937 году. Изучая экспансивные расстройства у больных с поражением *гиппокампа* и поясной извилины, Пейпец выдвинул догадку о существовании в человеческом организме единой системы, соединяющей ряд структур мозга, и образующей мозговой субстрат для эмоций. Эта система представляет собой изолированную цепь, и включает в себя: гипоталамус, передневентральное ядро таламуса, поясную извилину, гиппокамп и мамиллярные ядра гипоталамуса. Она получила наименование круга Пейпеца.

Позднее, в 1952 году, конструкция, названная кругом Пейпеца, была названа Полом Мак–Лином *лимбической системой* (название учитывает, что поясная извилина окаймляет основание переднего мозга). Источником побуждения лимбической налаженности представляется гипоталамус, действенность которого модулируют нижележащие текстуры среднего мозга, и посредством таламуса передает возбуждение к поясной извилине коры больших полушарий. По Джеймсу Пейпецу, поясная извилина является субстратом сознательных психологических переживаний. Сигналы от поясной извилины, посредством гиппокампа и мамиллярных тел, вторично достигают гипоталамуса, обеспечивая обратную связь в лимбической системе. Отсюда следует, что индивидуальные переживания, возникающие на уровне коры головного мозга, регулируют висцеральные и моторные проявления эмоций.

В концепции Джеймса Пейпеца имеется ряд противоречий. Например, было доказано экспериментальным путём, что стимуляция гиппокампа человека электрическим током не сопровождается возникновением эмоций (страха, гнева и др.), а больные испытывают при этом только спутанность сознания.

Базисная концепция эмоций выдвинута безотносительно друг от друга американским философом и психологом Уильямом Джеймсом, и датским доктором Карлом Георгом Ланге в 1880–1890 годах. Основанием для концепции явилось изучение соотношения между объективным проявлением активности вегетативной сферы и субъективным чувством переживаемой эмоции. Согласно предоставленной теории, *эмоциональное ощущение* – это проявление в сознании человека функциональных перестроек в организме, совершающихся на уровне вегетативной нервной системы [55 – 57].

Внешнее раздражение активизирует рефлекторные изменения во всём организме – в деятельности сердца, дыхании, кровообращении, тоне мышц. Из-за этого, в сознание человека проецируются различные ощущения, из которых и складывается переживание эмоций. Другими словами, первоначально, под действием внешних стимулов, происходят характерные для эмоций изменения в организме, а затем, как их следствие, возникает сама эмоция. Если вегетативные реакции представляются причиной, а эмоции следствием, в континууме взаимодействия организма со средой, то «мы грустим, потому что плачем, сердимся, потому что наносим удар, боимся, потому что дрожим».

Следует отметить, что в обеих концепциях эмоции рассматриваются несколько односторонне, как признак незначительного комплекта естественных программ поведения. От этого создается впечатление, что ряд процессов нашего сознания лишен эмоционального сопровождения. Соответственно, становится трудно объяснить присущее человеку состояние эмоциональной сферы.

По мнению американского психофизиолога Уолтера Кэннона, в контексте теории о происхождении эмоций Джеймса Ланге, гамму психологических чувств человека невозможно соотнести с относительно незначительным разнообразием вегетативных проявлений. Теория основана на предположении, что наши эмоции являются следствием осознания человеком рефлекторных физиологических изменений в организме. Формирование этих представлений, подкрепленное экспериментальными данными физиологии, дало основу для вывода, что при восприятии событий, возбуждающих наши эмоции, нервные импульсы первоначально поступают в *таламус*, где возбуждение расщепляется. Часть его идет в кору больших полушарий мозга, порождая субъективное переживание эмоций, а другая половина направляется в *гипоталамус*, который распоряжается физическими преобразованиями в организме. Эта догадка представила взаимоотношения между эмоциональной и двигательной сферами и способствовала пониманию сенсорной природы эмоций [53–57].

4.3. Извлечение эмоциональных характеристик из речевого сигнала

Существует множество психологических теорий происхождения эмоций. Обычно термин «эмоция» определяется как переживание человеком своего отношения к чему-либо. Это сразу отражает одну из главных трудностей при классификации эмоций. Эмоции проявляются одновременно во внутренних переживаниях и в поведении, но люди далеко не всегда способны оценить эти параметры у самих себя, более того в зависимости от темперамента и культуры, выражение эмоций человека может сильно различаться.

Существует более объективный подход, интерпретирующий эмоцию физиологический процесс. Доказано, что нейрофизиологический компонент возбуждения влияет на качество переживаемой эмоции. Состояние эмоционального напряжения сопровождается существенными изменениями функций ряда органов и систем. Эмоциональное возбуждение включает в себя: произвольно регулируемые компоненты – мимику, речь и т.д.; и нерегулируемые компоненты эмоций – деятельность сердца, изменение состояния мышечной системы, потоотделение и т.д.

Экспериментальные данные показывают различные нерегулируемые изменения в человеческом организме, сопровождающие различные эмоции:

- радость, при которой наблюдается возбуждение двигательных центров, из-за чего появляются характерные движения (жестикуляция, подпрыгивание, хлопанье в ладоши), усиление кровотока в капиллярах, вследствие чего кожа тела краснеет и теплеет, а внутренние ткани и органы лучше снабжаются кислородом, так как обмен веществ в них начинает происходить интенсивнее.

- гнев, при котором фиксируется сильное увеличение частоты сердечных сокращений, повышение артериального давления и температуры тела;

- страх, при котором наблюдается сильное учащение сердцебиения и значительным понижением температуры тела, и др. [57].

Можно сделать вывод, что объективные физиологические компоненты формирования эмоций достаточно значительны для понимания того, что анализ эмоций универсален для людей различных культур и темперамента.

В научной литературе присутствуют материалы о последовательном формировании в онтогенезе обонятельного, осязательного, тактильного, слухового и зрительного «анализаторов». Из этой концепции следует, что происхождение новой модальности и способность переформировывать более дифференцированные системы усиливает характеристики поведения индивида.

Расценивая итоги своего поведения, человек смотрит на себя *«глазами общества»* и *«отчитывается»* ему. Специфический инструментарий отчёта – речь, в том числе – внутренняя. Обнаружено, что высоко экспансивные оценки чаще связанным с обонянием, а низко экспансивные – со зрением. Это означает, что внешне одно и то же поведение, реализующееся ради преимущества желаемого действия (Approach, положительные эмоции, Э+) или с целью отказа от ненужного действия (Withdrawal, отрицательные эмоции, Э–), поддерживается активацией различных, хотя и перекрывающихся, комплексов нейронов человеческого мозга, и актуализацией нейронных систем, присущих различным доменам персонального жизненного опыта индивида.

При исследовании ЭЭГ–потенциала (электроэнцефалография – это метод исследования головного мозга с помощью регистрации разности электрических потенциалов, возникающих в процессе жизнедеятельности человека) у испытуемых, при дискриминации идентичных по физиологическим чертам слуховых сигналов, в ситуациях получения индивидом положительных и отрицательных эмоций (например, получения или потери денежного вознаграждения), были выявлены надёжные различия между электрическими потенциалами работы мозга, зарегистрированные в сравнимых ситуациях.

Это означает, что изменение соотношения индивида и его окружающей среды в поведении, объединённые с оценкой окончательных его результатов, сравнимо с различными уровнями работы мозга (высоким и низким) и предполагают многообразную систему активности мозга.

Было проведено сравнение ЭЭГ–потенциалов у группы испытуемых, характеристики отчётных усилий которых зависели от осознаваемого или не осознаваемого ими варьирования свойств окружающей среды. Обнаружено несходство параметров мозговой активности и характеристик отчётного действия в ситуациях осознания и неосознания. Эти различия связаны с особенностями системных процессов при оценке окончательных результатов, а также промежуточных результатов, зафиксированных или не отраженных в сознании человека. Полученные результаты обсуждаются с позиций представления о множественности уровней работы мозга, подразумевающего, что каждый из двух выделенных выше уровней сознания С (первый и второй) объединяют категории различающихся уровней сознания и подсознания.

Таким образом, предлагается системно–эволюционный подход к теоретическому решению сложной проблемы сознания и подсознания.

Происхождение опережающего отображения, вместо рефлекторного, обозначает происхождение целей, а, следовательно, индивидуального разделения мира на «плохое» (препятствующее достижению целей человека) и «хорошее» (способствующее их достижению). Это деление объединено с уникальностью видового, индивидуального опыта и текущими потребностями. Утверждается, что те же самые физиологические процессы порождают индивидуальный мир каждого человека, которые характеризуют его жизнь, и создают его причинно–следственные связи. Жизнедеятельность человека связана с формированием в его сознании свойственных ему целей и моделей будущего (неизбежно субъективных) и делением всех происходящих событий в соответствии с индивидуальной шкалой «хорошо»–«плохо».

4.3.1. Мелочастотные кепстральные коэффициенты

В качестве оценочных характеристик для нейронной сети необходимы входные аргументы. Сравнение источников позволяет сделать выводы [58]:

- непосредственно звуковые сигналы, во временной области, заставляют нейронную сеть работать слишком долго и не очень эффективно;
- спектрограммы работают немного быстрее, но ненамного эффективнее;
- мелочастотные кепстральные коэффициенты (mfcc), взятые в качестве характеристики речевых сигналов, показывают себя намного эффективнее.

Спецификация является одной из важных проблем интеллектуального анализа данных. Она осуществляется посредством аналитических моделей, называемых классификаторами. Необходимость систематизации определяется простотой методов её реализации, и высокой интерпретируемостью итогов, по сравнению с результатами других технологий анализа данных.

В настоящее время создано большое количество разнообразных вариантов классификаторов, ради построения которых используются как статистические технологии (логистическая регрессия и дискриминантный анализ), так и технологии машинного обучения (нейронные сети, деревья решений, метод k –ближайших соседей, машины опорных векторов и др.).

Необходимость применения в анализе данных многообразных методов классификации определяется тем, что решаемые с её помощью задачи имеют свои особенности. Например, они могут быть связаны с числом классов или с представлением исходных данных – их объёмом, размерностью и качеством, что требует подбора соответствующего классификатора.

Поэтому подбор классификатора, соответствующего особенностям решаемой задачи анализа, необходим для получения верного решения.

Различные варианты классификаторов имеют свои достоинства и недостатки. Так, классификаторы, использующие *статистические* технологии, обладают высокой математической обоснованностью. Однако при этом они слишком сложны, громоздки и трудоёмки в использовании, требуют знаний вероятностного распределения исходных данных и оценки их параметров (в *параметрических* классификаторах), а также применяют фиксированную структуру модели. Статистические технологии расценивают возможность принадлежности объекта классу, но не объясняют причину такого решения.

Классификаторы, созданные на машинном обучении, не требуют оценки параметров распределения начальных данных, а мера сходства в них формализуется посредством функции расстояния (обычно, евклидова). Такие классификаторы называются *метрическими*. В большинстве случаев, они проще в реализации и использовании, чем *параметрические*, а их результаты более подходят для понимания и интерпретации. Но при этом метрические классификаторы являются *эвристическими* моделями – то есть обеспечивают решение исключительно в ограниченном числе практически значимых случаев, и могут дать неправильное или не единственное решение. Поэтому использовать их результаты необходимо с известной долей осторожности.

Определённым компромиссом между параметрическим и метрическими технологиями является использование для решения задач классификации метода *нейронных сетей* (НС). Этот метод использует непараметрические модели, не требующие предположений о вероятностном распределении данных, но при этом не использует и меры расстояний. Это делает нейронные сети универсальными классификаторами, позволяя получать результаты даже в тех случаях, когда параметрические и метрические классификаторы не обеспечивают приемлемого для пользователя решения.

Мел (от слова мелодия, [58]) – это единица высоты звука, основанная на восприятии этого звука человеческими органами слуха. Амплитудно– частотная характеристика человеческого уха является нелинейной и неравномерной. На рис. 4.1 показаны графические зависимости восприятия человеком звуковых частот в зависимости от громкости (мощности акустического сигнала). Поэтому эмпирически подобранные единицы громкости необходимы для анализа степени *восприятия собеседником* такого явления, как человеческая речь.

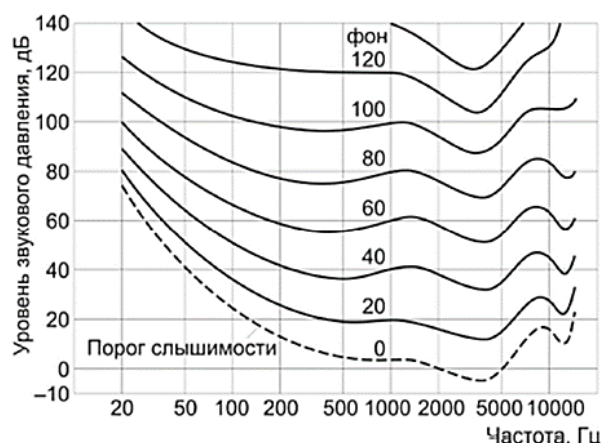


Рисунок 4.1. Кривые восприятия человеком звуковых частот в зависимости от громкости

По опыту, у каждого человека не вызывает сомнений, что в зависимости от громкости звука человеческое ухо воспринимает разные частоты по-разному. Например, низкие частоты, на которых в современной музыке располагаются такие инструменты, как бас-бочка или бас-гитара, становятся хорошо слышны только при достаточно высокой громкости. Высокие частоты, издаваемые флейтой или скрипкой, наоборот, можно различить и при негромком звуке,

Воспринимаемая человеческим слухом высота звука так же нелинейно зависит от его частоты, что показано увидеть на рис. 4.2.

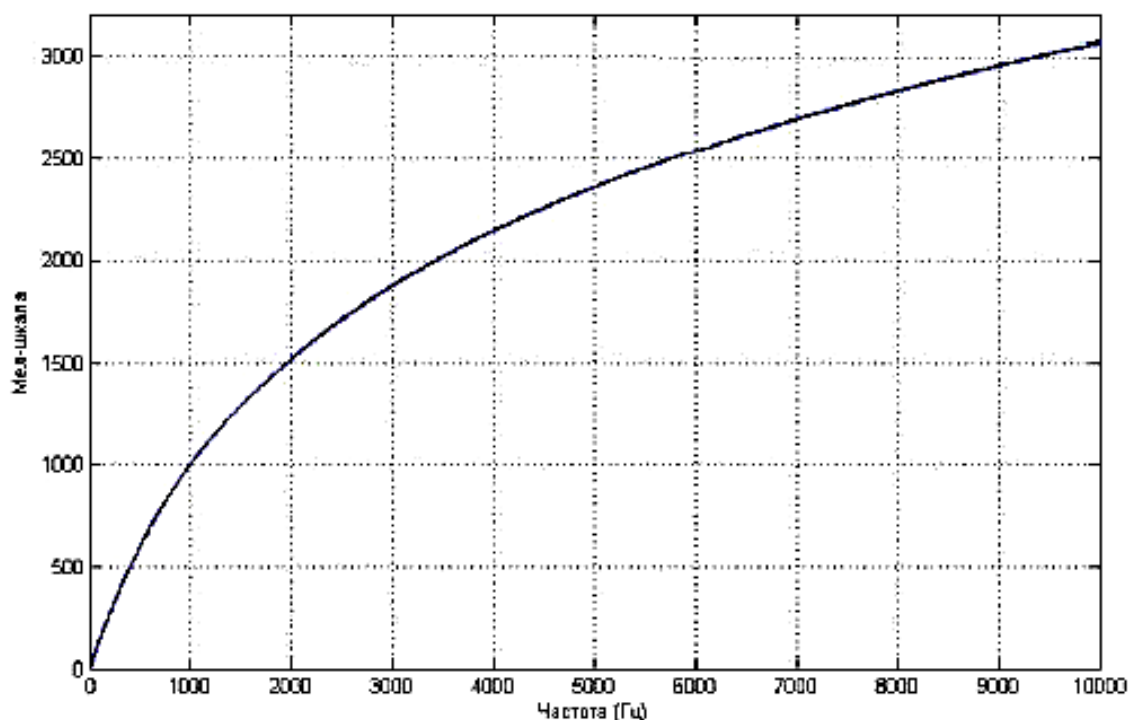


Рисунок 4.2. Кривые восприятия человеком звукового тона в зависимости от частот

Подобные единицы измерения часто используются при решении задач распознавания, так как они позволяют приблизиться к тонким механизмам человеческого восприятия (связанном с работой головного мозга). Такой подход лежит в основе большинства известных систем распознавания речи.

В соответствии с теорией речеобразования, человеческая речь представляет собой акустическую волну, которая излучается системой внутренних органов: легкими, бронхами и трахеей, а затем преобразуется в голосовом тракте. Если предположить, что источники возбуждения и форма голосового тракта относительно независимы, то речевой аппарат человека можно представить в виде совокупности генераторов тоновых сигналов и шумов, а также фильтров.

Рассмотрим преобразование на примере. Воспользуемся записью произнесенного вслух слова «один». На рис. 4.3 показано временное представление этой аудиозаписи.

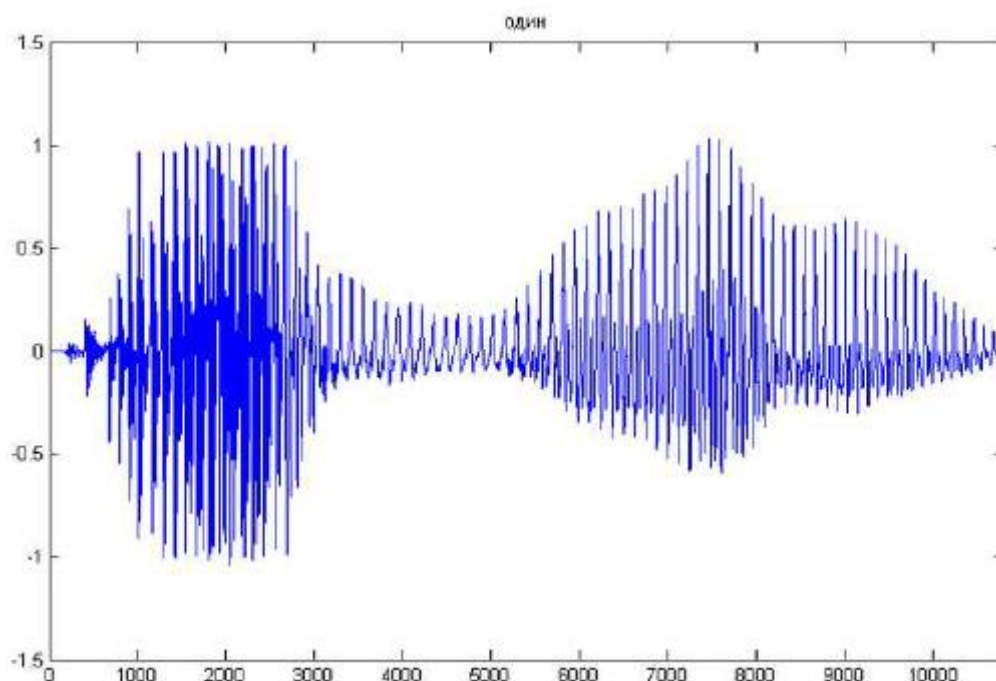


Рисунок 4.3. Временное представление произнесённого вслух слова «один»

Простейшая визуализация звуковых сигналов доступна в библиотеке NumPy языка программирования Python, находящейся в свободном доступе. Для более тонкой настройки можно воспользоваться специализированными программами обработки звука, например, из линейки Ozon.

Создадим спектр исходного сигнала с помощью преобразования Фурье, график спектрограммы которого представлен на рис. 4.4.

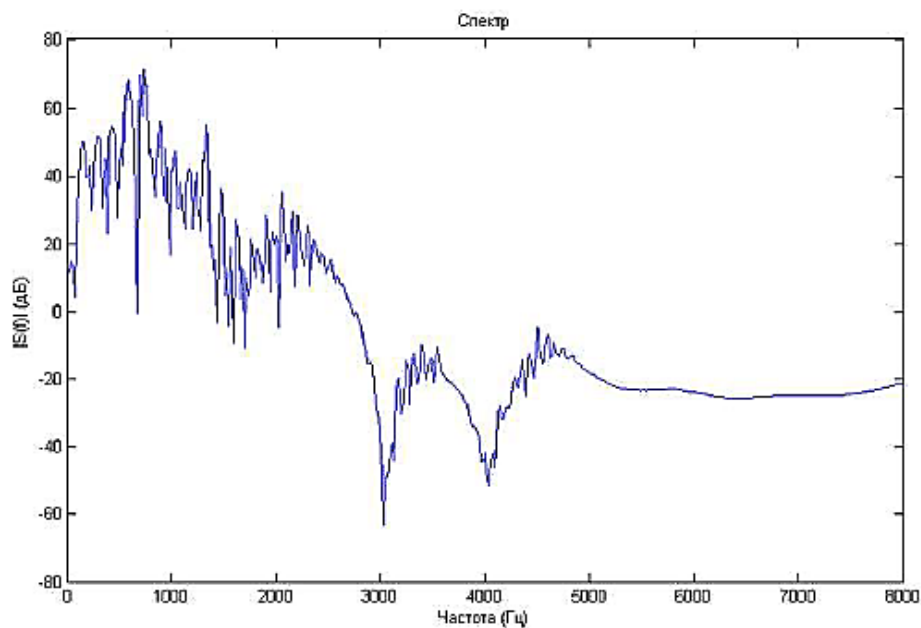


Рисунок 4.4. Спектрограмма слова «один»

Полученный спектр располагается на *мел-шкале*, а затем переводится в частотную шкалу, что продемонстрировано на рис. 4.5.

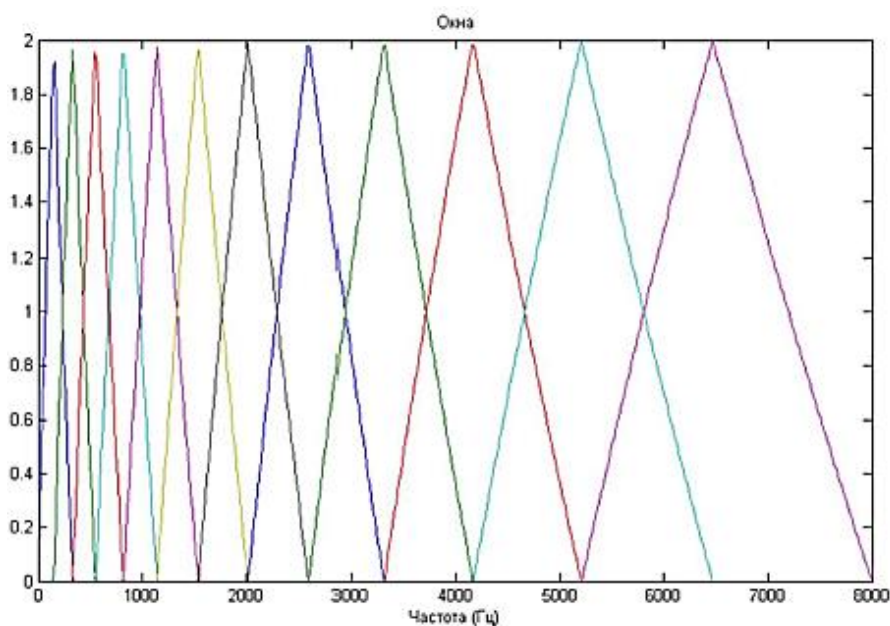


Рисунок 4.5. Частотная шкала слова «один» после мел-преобразования

На графике заметно, что в области высоких частот находится гораздо больше информации для распознавания. Затем находится энергия сигнала и кепстральные коэффициенты. В результате получается мел-кепстральная последовательность такого вида, как показано на рис. 4.6.

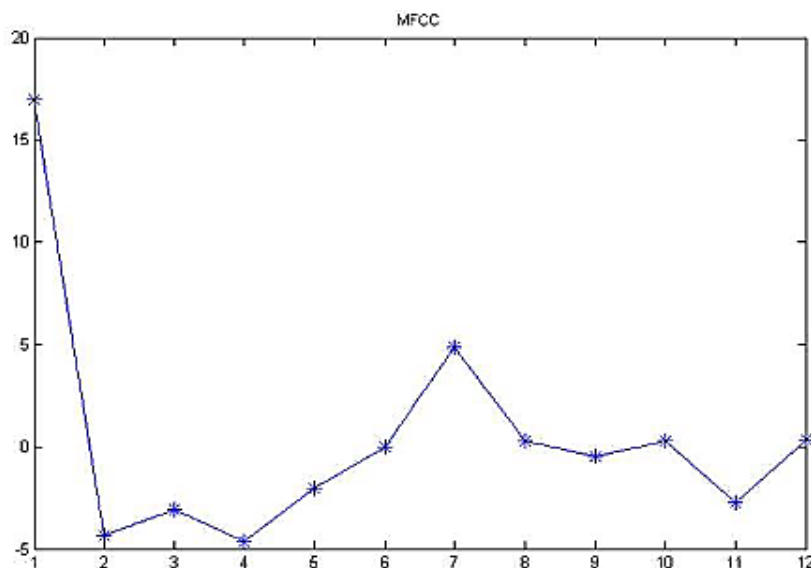


Рисунок 4.6. Мел–кепстральные коэффициенты слова «один»

Получается гораздо меньший набор значений, чем при временном или спектральном представлении данных. Для лучшего результата можно разбить исходное слово на отрезки, и вычислять коэффициенты для каждого из них [59].

4.3.2. Источники данных для обучения нейронной сети

Существует два пути сбора данных для обучения нейронных сетей:

- извлечение аудиоинформации из теле– и радиопередач, аудиокниг, фильмов, блогов, новостей, речи людей и других возможных источников;
- применение наборов данных (датасетов) со структурированными записями.

Если извлекать данные из различных источников самостоятельно, то это позволит использовать гибкую классификацию, выбрать нужный язык и контекст ситуаций, однако получение и обработка достаточного для обучения нейронной сети количества голосовых фрагментов займёт продолжительное время (ведь это может быть несколько тысяч аудиофайлов на разных языках).

Если пользоваться уже готовым набором данных (датасетом), то сначала нужно решить несколько вопросов. Перед тем, как сформировать из готовых датасетов данные для непосредственного обучения своей нейронной сети, нужно знать, какие материалы нужно применять, и где их отыскать.

– Датасет должен являться актуальным и универсальным, другими словами, иметь данные, нужные для решения широкого круга сегодняшних задач. На данный момент англоязычная часть датасета представлена аудиокнигами, русская часть – начиткой текста и живой речью.

– Датасет должен включать исходные данные. Нередко можно повстречать датасеты, которые содержат только аннотации материалов и гиперссылки на исходные аудиофайлы, опубликованные где-то в сети Интернет.

– Датасет должен содержать только легальные материалы и передаваться пользователю после получения им разрешительной лицензии.

– Данные в датасете должны быть высокого качества. Нужно избегать прибавления ненатуральных шумов, помех и аугментаций, а также тщательно следить за тем, чтобы аудио и документ подходили друг другу.

Эти правила выполнить трудно, однако, если они не соблюдаются, то в их отсутствие вся работа теряет смысл. Данные должны обладать высокими качествами, иначе правильно обучить нейронную сеть невозможно.

В реализации правил самым сложным является третий пункт: получить доступ к огромным данным, не нарушив законодательство и права владельцев интеллектуальной собственности, совсем не просто. Для этого нужно изучить все аспекты законов о защите интеллектуальной собственности.

Результат интеллектуальной деятельности (РИД) – это правовой термин, устанавливающий результат проявления творческих, интеллектуальных и духовных процессов человеческого мышления. Например, это фильм, спектакль, картина, рисунок, музыкальное произведение, песня, книга, стихотворение, компьютерная игра, код программы, научное изобретение и т.д. Права на РИД могут принадлежать как одному человеку, так и нескольким сторонам: автору, писателю, переводчику, издательству, композитору, режиссёру, художнику, изобретателю, программисту, и др., а также авторским творческим коллективам. Кроме того, РИД может являться общественным достоянием.

Общественное достояние – это РИД, на которые истек срок защиты авторских прав. Например, в России произведения, как правило, переходят в общественное достояние через 70 лет после смерти автора. Но есть и исключения, иногда полномочия наследуют родственники или компании. Представление об общественном (коллективном) достоянии результатов интеллектуальной деятельности в разных странах своё, поэтому необходимо быть очень внимательными с продукцией зарубежного производства.

РИД, размещенные в глобальной сети Интернет (в социальных сетях, на веб-сайтах, найденные в результатах, полученных в поисковых программах, и т.д.), также охраняются авторским правом, поэтому всегда нужно обращать внимание на лицензионное соглашение, под которой они публикуются.

В большинстве случаев, наименование лицензии указывается рядом с опубликованным результатом интеллектуальной деятельности, либо на ресурсе, где располагается РИД, организовывается отдельная страничка с текстом лицензии. Например, это "Правила цитирования / использования материалов сайта", "Пользовательское соглашение" и т.д., где содержится достаточная для пользователя информация о лицензии. Для создания датасета можно использовать следующие результаты интеллектуальной деятельности:

- полученные законным способом;
- не нарушающие права исходных владельцев;
- не имеющие ограничений на модификацию;
- не имеющие ограничений на использование и распространение.

Так как цель исследования не выходит за рамки стандартного определения эмоций, то целесообразно использовать второй метод.

В открытом доступе существует датасет с названием RAVDESS (The Ryerson Audio Visual Database of Emotional Speech and Song), который состоит из записей 12 актеров и 12 актрис, демонстрирующих следующие эмоции: спокойствие, счастье, грусть, злость, страх, отвращение, удивление.

Это основной спектр эмоций, которые показывает человек при общении, поэтому нет необходимости расширять или ограничивать данный набор. Всего в этой базе данных находится 1440 аудио файлов в формате .wav, средней продолжительностью 2 секунды, с речью актеров на английском языке.

Итак, проведено исследование способов извлечения эмоциональных характеристик из речевого сигнала, рассмотрены источники данных для обучения нейронной сети и изучены мел–кепстральные характеристики речи.

4.4. Разработка оптимального алгоритма распознавания эмоций

Задача определения разновидности звука – того, что он представляет собой (шум ветра, плеск морских волн, звук двигателей самолёта, музыку, пение, крики животных, человеческую речь на каком-либо языке или что-то другое) называется классификацией звука. Модель классификации аудио файлов (нейронная сеть) обучена распознавать различные аудио события.

Существует множество готовых моделей нейронных сетей, подобранных специально для анализа аудио информации. Одна из них – это AudioClassifier из библиотеки TensorFlow Lite от компании Google [59].

Большое количество обучающих материалов, направленность нейронной сети AudioClassifier на использование в мобильных устройствах, с гораздо меньшей производительностью смартфона по сравнению с персональным компьютером, что компенсируется благодаря технологиям Интернета вещей, и гибкость к обновлениям, модификациям и другим изменениям делает эту технологию предпочтительной для задач данного исследования.

AudioClassifier – это класс, использующий нейронную сеть YAMNet. Это глубокая сеть, которая способна распознать 521 класс звуковых событий из корпуса AudioSet–YouTube, на котором она обучена. В YAMNet используется архитектура свертки Mobilenet_v1 с разделением по глубине.

API AudioClassifier библиотеки AudioClassifier можно использовать для создания собственных аудио классификаторов и интегрирования их в мобильное приложение. Это удобно, потому что стандартная модель не содержит данных об эмоциях в речи. Она способна лишь отличить речь от иных звуков. Поэтому мы можем использовать датасет RAVDESS, описанный ранее, для того, чтобы дообучить нейронную сеть YAMNet классифицировать эмоции по речи.

Для этого нужно, с помощью инструментов библиотеки TensorFlow Lite, создать спецификацию базы данных для обучения, преобразовать данные, и, выбрав коэффициенты количества эпох и размера анализируемого образца (сэмпла), провести обучение. Затем, тестируя обученную нейросеть, можно сделать вывод о том, достаточно ли низка получаемая погрешность, и, при необходимости, переобучить нейронную сеть уже с другими параметрами.

Затем, когда результат тестирования станет удовлетворительным, нужно сохранить полученную модель нейронной сети в специальном формате .tflite, чтобы впоследствии использовать её в мобильном приложении.

Наша модель принимает на вход float32 массив float32 Tensor или NumPy, длиной 15600, содержащий 0,975–секундную форму волны, представленную в виде монофонических выборок 16 кГц в диапазоне $[-1.0, +1.0]$.

Модель возвращает float32 тензор формы float32 (1, 521), содержащий предсказанные оценки для каждого из 521 классов в онтологии AudioSet, которые поддерживаются нейронной сетью YAMNet. Индекс столбца (0–520) тензора оценок сопоставляется с соответствующим именем класса AudioSet с помощью карты классов YAMNet, которая доступна в виде связанного файла yamnet_label_list.txt, упакованного в файл модели нейронной сети.

Нейронную сеть YAMNet можно использовать:

- в качестве автономного классификатора звуковых событий, который обеспечивает разумную основу для широкого спектра звуковых событий;
- в качестве высокоуровневого экстрактора функций: выходные данные встраивания 1024-d yamnet могут использоваться в качестве входных функций для другой модели, которую затем можно обучить на небольшом объеме данных для конкретной задачи. Это позволяет быстро создавать специализированные аудио классификаторы, не требуя большого количества размеченных данных, и без необходимости сквозного обучения большой модели нейронной сети.
- параметры модели YAMNet могут использоваться для инициализации части более крупной модели нейронной сети, что позволит быстрее выполнять точную настройку и исследование параметров работы модели [60].

Изначально аудиофайл будет считываться как двоичный файл, который следует преобразовать в числовой тензор. Чтобы загрузить аудиофайл, нужно использовать `tf.audio.decode_wav`, который возвращает аудио в кодировке WAV как тензор, а также указать частоту нужную дискретизации.

Файл WAV содержит данные временных рядов с заданным количеством выборок в секунду. Каждая выборка представляет собой амплитуду аудио сигнала в определенное время. В 16-битной системе, как и файлы в `mini_speech_commands`, значения находятся в диапазоне от -32768 до $+32767$. Частота дискретизации для этого набора данных составляет 16 кГц. Нужно обратить внимание, что `tf.audio.decode_wav` нормализует значения частоты дискретизации до диапазона нормализованных значений $[-1.0, 1.0]$.

Модель имеет дополнительные слои предварительной обработки:

- слой изменения *resizing*, для уменьшения частоты дискретизации входных данных, чтобы модель нейронной сети смогла обучаться быстрее.
- слой *normalization*, для нормализации каждого пиксела в изображении, на основе значений его среднего и стандартного отклонения.

Для уровня *Normalization* сначала необходимо вызвать его метод `adapt`, для обработки обучающих данных, чтобы вычислить суммарную статистику значений его среднего и стандартного отклонения.

Матрица неточностей используется для того, чтобы увидеть, насколько хорошо модель нейронной сети справилась с каждой из команд в наборе тестов.

4.5. Выбор оптимальной среды программирования

В настоящий момент существует две самые известные операционные системы для создания мобильных приложений – Android и IOS [60].

Достоинства операционной системы Android:

- универсальность – возможность работы на любом компьютере;
- массовость применения, что даёт большое сообщество разработчиков;
- языки программирования Java и Kotlin, которые могут быть использованы при написании не только мобильных приложений, но и веб–приложений, а также различных компьютерных игр, применяющихся в смартфонах;
- в ОС Android функционируют не только приложения для смартфонов, но и для умных часов, smart tv, vr/ar; а также для другой носимой электроники;
- среда разработки анализирует программный код и может автоматически переформировывать, а также оптимизировать некоторые его части;
- наличие хорошего фреймворка; что позволяет найти готовые модули под разработку приложений самой разнообразной направленности;
- возможность адаптации платформы под любую специальную задачу;
- исходный код Android открыт для всех желающих; существует платформа Android Open Source Project, где компания публикует все исходные данные, и создатели новых приложений могут этим воспользоваться;
- публикация приложений в Google Play происходит практически мгновенно;
- разумная небольшая плата за размещение приложений в Google Play.

Недостатки операционной системы Android:

- разнообразие устройств и их параметров создает основу для багов разной службы приложений на различных видах смартфонов;
- разнообразие версий операционной системы: в отличие от iOS, у Android остаются актуальными и могут действовать без обновлений устаревшие версии;
- не все обучающие материалы и ответы на форумах актуальны: система существует довольно давно, но при этом аспекты разработки изменяются;
- использование и аккаунт могут быть заблокированы без разьяснения причин, а на жалобы в техническую поддержку откликается бот, а не человек.

Преимущества операционной системы iOS:

- строго ограниченное количество версий смартфона, габаритов экрана и самой операционной системы, отсутствие устаревших версий;
- изменения в операционной системе прозрачны: Apple ежегодно выпускает руководства по разработке приложений для iOS и объясняет, как заменить нужные фрагменты программного кода на более производительные элементы;
- приложений и разработчиков iOS гораздо меньше, и конкуренция ниже.

Несовершенства операционной системы iOS:

- высокая стоимость техники, необходимой для работы;
- необходимость ежегодно продлевать аккаунт для загрузки своей разработки в магазин приложений AppStore для мобильных телефонов iPhone;
- загрузка приложения может занять больше недели, потому что модерация (проверка выполнения правил веб-сайта) производится вручную.

Как следует из проведённого сопоставления, в качестве операционной системы для разработки мобильного приложения логично выбрать Android. Интегрированная среда разработки Android Studio, предназначенная для работы с платформой Android находится в свободном доступе. Требования к аппаратуре Android, необходимой для нормального функционирования программных приложений, опубликованы на официальном веб-сайте Android Studio в графе System Requirements: как минимум 3 ГБ RAM; рекомендуется 8 ГБ RAM; плюс 1 ГБ для Android Emulator; минимальное разрешение экрана 1280 × 800 пикселей. Этого достаточно для запуска и минимальной работы студии, но о высокой производительности и комфортности работы речь не идёт.

Для обеспечения качественной работы наиболее критичными являются пять следующих параметров: производительность и количество аппаратных потоков процессора; количество оперативной памяти; скорость произвольного чтения и произвольной записи дисковой подсистемы; а также скорость мелкоблочного чтения и мелкоблочной записи дисковой подсистемы.

Intel Hyper Threading представляет собой аппаратную технологию Intel, разграничивающую энергоаппаратный поток ядра на два «виртуальных». Благодаря этой технологии, конструкция воспринимает 4-ядерный процессор как 8-ядерный. Такой подход позволяет использовать мощности процессора более эффективно. Подобная технология есть и у процессоров, выпускаемых фирмой AMD. Она включается в настройках BIOS\UEFI системной платы компьютера или ноутбука, где регулируется использование такой опции.

Intel Turbo Boost – распространенная технология компании Intel, которая позволяет автоматически увеличить тактовую частоту процессора на краткий срок, что даёт выигрыш в быстродействии и производительности. В это же время, использование этой технологии «саморазгона» процессора может привести к превышению рабочей температуры ядра процессора, что раскаляет мобильные процессоры до максимальных температур – состояния *Tjunction*.

В смартфонах используются накопители данных, выполненные по технологии *SSD*. Продуктивность их работы со временем падает, благодаря процессу «деградации» накопителей, и тогда используется интерфейс *TRIM*. Технология *Secure Erase* предписывает контроллеру помечать все ячейки памяти как пустые, что, теоретически, должно вернуть продуктивность *SSD* к фабричному состоянию. Но она может привести к потере всех данных!

Мобильные технологии *Genymotion* и *BlueStacks* не требуют аппаратной виртуализации. Однако в этих эмуляторах были изменены многие функции *Android API*, с целью повышения производительности их работы. Это может привести к их нетипичному функционированию, которое никогда не встречается на обычных устройствах. Так, при отладке можно пропустить наличие ошибок (bugs) и необычного поведения программы (features).

Команда *TRIM* интерфейса *ATA* может быть реализована как аппаратно, так и программно. В последнем случае функция вызова команды *TRIM* обеспечивается операционной системой. По многим причинам, рекуррентный вызов может отключиться, что приведёт к резкому падению скорости работы накопителя. Обследовать работоспособность и возобновить вызовы *TRIM* можно с поддержкой специальных утилит от изготовителя накопителя.

Функция *Instant Run* позволяет редактировать байт-код и его функции в уже установленном приложении "на лету". Это существенно экономит время наладки ПО. В новых версиях *Android Studio* интегрирована по умолчанию.

Функции *Gradle Build Configs* используются, если в основной рабочей конфигурации *BuildConfig* или *Flavour* имеются компоненты, используемые исключительно в режимах Release-сборки (например, *Crashlytics*, *Annotation-процессорах* или *Gradle-процедурах*). Для выполнения отладки *Debug* или для проведения тестирования конфигурации их лучше отключить. В случае использования конфигурации пользователя (кастомный *BuildConfig*), следует выключить опцию *PNG Crunching*, так как, по умолчанию, она включена только ради отладки *DebugConfig*. При сборке она применяет компрессию *PNG*.

Параллельная сборка. Если проект содержит несколько самостоятельных модулей (например, несколько App-модулей), то для него актуальна функция «Compile Independent Modules in Parallel», находящаяся в установках «Settings → Build, Execution, Deployment → Compiler». Это позволит более эффективно утилизировать потоки процессора CPU при сборке. Недостатком является больший размер оперативной памяти (кучи, heap size), выделенной запущенной программе, и, как следствие, больший расход оперативной памяти.

Функция *Gradle daemon* позволяет удерживать отдельные экземпляры объектов (инстанс) Jvm и Gradle в оперативной памяти на протяжении трёх часов с момента последней сборки. При этом не нужно тратить время на их инициализацию и расходовать оперативную память (наращивать кучу). При сборке эта функция подсоединяется строкой в опции Gradle, и позволяет периодически проверять зависимости веб-ресурсы, чтобы их устранить.

Плагины (независимо компилируемые программные модули, динамически подключаемые к основной программе и предназначенные для расширения её возможностей), которые могли бы ускорить работу интегрированной среды разработки (IDE) пока не существуют. Однако можно удалить из памяти лишние модули, что повысит быстродействие и уменьшит время запуска программы.

Функция *Power Save Mode* отключает фоновые процессы, протекающие в IDE (индексацию, статический анализ кода, проверку синтаксиса и т. д.). При этом Android Studio работает гораздо быстрее, хотя по своему функционалу не превосходит, например, редактор исходного кода Visual Studio Code.

4.6. Реализация мобильного приложения распознавания эмоций

Приложения в Android Studio состоят из активных форм (Activity), как и в любой другой высокоуровневой программной среде, в ней поддерживается набор базовых кнопок, переключателей и других элементов интерфейса [60].

Для создания приложения нужны две связанные формы: основная, на которой расположен переключатель ввода и длительности обрабатываемого сигнала, и вспомогательная, на которой формируется список результатов анализа работы нейронной сети. Для реализации разрабатываемого проекта потребуется набор библиотек, основные из которых перечислены ниже:

- PackageManager – класс для получения различной информации, относящейся к пакетам приложений, которые в настоящее время установлены на устройстве;
- AudioRecord – класс для работы с микрофоном;
- AudioClassifier – класс для работы с нейронной сетью.

Файл, полученный при *дообучении* нейронной сети, используется в качестве основной модели. Сигналы от микрофона разбиваются на отрезки заданной длины и анализируются с помощью этой модели, а затем значения, относящиеся к классу Speech (речь), если вероятность этого выше 30%, будут формироваться в отдельный список, и выводиться на форму.

Для тестирования программы, через интерфейс USB подключается телефон с включенным режимом разработчика, а затем запускается построение пакета.

В первую очередь необходимо обучить и настроить описанную ранее модель нейронной сети YAMNet реагировать на необходимый класс событий, а именно – на эмоциональные состояния, отражаемые в человеческой речи.

Для этого создаётся отдельный проект, в котором устанавливается нужная библиотека TensorFlow Model Market. Библиотека Model Maker использует трансферное обучение нейронной сети, чтобы упростить процесс обучения модели TensorFlow Lite с использованием настраиваемого набора данных.

Переобучение модели TensorFlow Lite с собственным пользовательским набором данных сокращает объём обучающих данных и время обучения.

Импортируются также библиотеки для обработки звука, воспроизведения и визуализации аудио процессов, которые представлены на рис. 4.7.

```
import tensorflow as tf
import tf.lite.model_maker as mm
from tf.lite.model_maker import audio_classifier
import os

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import itertools
import glob
import random

from IPython.display import Audio, Image
from scipy.io import wavfile
```

Рисунок 4.7. Импорт библиотек, необходимых для обучения модели нейронной сети

При долгосрочном использовании Model Maker для аудио нужно начать со спецификации модели. Это базовая модель, в которой новая модель будет искать информацию для исследования новых классов. Спецификация модели также воздействует на выбор способа, которым набор данных будет преобразован с учётом параметров частоты дискретизации и количества каналов.

YAMNet – это классификатор звуковых событий, обученный на наборе данных AudioSet для моделирования звуковых событий из онтологии AudioSet. Ожидается, что его вход будет на частоте 16 кГц и с 1 каналом. Не следует делать передискретизацию самостоятельно, об этом позаботится Model Maker.

- опция `frame_length` определяет длину любого обучающего образца. В этом случае `EXPECTED_WAVEFORM_LENGTH * 3` с;

- опция `frame_steps` – определяет, насколько далеко находятся обучающие образцы (сэмплы). В этом случае `i`-й отсчет начнется посредством строки `EXPECTED_WAVEFORM_LENGTH * 6` с после `(i-1)`-го отсчета.

Причина установки этих значений состоит в необходимости обойти некоторые ограничения в реальном комплекте данных. Например, в наборе данных о речи актеров, актеры говорят не все время. Они говорят, отдыхают и снова говорят, с шумом между словами. Длинный кадр поможет запечатлеть фразу, но слишком длинный кадр уменьшит количество образцов (сэмплов) для обучения. Аргументы для спецификации можно увидеть на рис. 4.8.

```
spec = audio_classifier.YamNetSpec(  
    keep_yamnet_and_custom_heads=True,  
    frame_step=3 * audio_classifier.YamNetSpec.EXPECTED_WAVEFORM_LENGTH,  
    frame_length=6 * audio_classifier.YamNetSpec.EXPECTED_WAVEFORM_LENGTH)
```

Рисунок 4.8. Спецификация обучающей выборки

`Audio_Classifier` имеет метод `Create`, который создает модель и начинает её обучение. В этой попытке используются все конфигурации по умолчанию. Модель обучается в течение 100 периодов, или эпох (рис. 4.9).

```
batch_size = 128  
epochs = 100  
  
print('Training the model')  
model = audio_classifier.create(  
    train_data,  
    spec,  
    validation_data,  
    batch_size=batch_size,  
    epochs=epochs)
```

Рисунок 4.9. Обучение модели нейронной сети

На рис. 4.10 можно увидеть результат тестирования обученной модели.

```
Evaluating the model  
28/28 [=====] - 13s 432ms/step - loss: 0.9247 - acc: 0.7566  
[0.9246906042098999, 0.7566016316413879]
```

Рисунок 4.10. Результаты тестирования модели

Последний шаг – экспорт модели нейронной сети для её применения на встроженных устройствах или в браузере. Метод `Export` экспортирует оба формата. На этом модель нейронной сети для анализа эмоций готова.

Можно реализовать мобильное приложение. Для этого нужно создать проект в среде разработки Android Studio с минимальной версией Android 4.4 для того, чтобы приложение поддерживалось более чем на 99% устройств с операционной системой Android. В качестве языка программирования для разработки выберем язык Java, потому что, хотя сейчас Google официально рекомендует язык Kotlin как лучший язык для разработки мобильных приложений в Android Studio, фреймворк TensorFlow лучше работает с Java.

Создадим первую форму (activity), на которой будет расположен весь пользовательский интерфейс. Для этого нам понадобятся инструменты:

- *CoordinatorLayout* – это сверхмощное средство разметки отображения элементов *FrameLayout*. *CoordinatorLayout* предназначен для использования в качестве контейнера для взаимодействия с одним или несколькими дочерними отображениями. Указав поведение для дочерних отображений в *CoordinatorLayout*, можно обеспечить множество способов различных взаимодействий в пределах одного родительского отображения, и они также могут взаимодействовать друг с другом. Классы представления указывают поведение по умолчанию, при использовании в качестве дочернего элемента *CoordinatorLayout* с помощью аннотации *CoordinatorLayout.DefaultBehavior*.

- *Switch* – это виджет (видео–приложение) с тумблером, переключающим два состояния. Пользователь может «перетаскивать большой палец» вперед и назад, чтобы выбрать нужный вариант, или просто нажимать тумблер для переключения, как флажок. Свойство `text` управляет текстом, отображаемым в метке переключателя, тогда как текст *off* и *on* управляет текстом на ползунке. Точно так же, `textAppearance` и связанные методы `setTypeface()` управляют шрифтом и стилем текста метки, тогда как `switchTextAppearance` и связанные методы `setSwitchTypeface()` управляют шрифтом бегунка.

RecyclerView (это мини–приложение Android, отображающее коллекцию элементов в списке и позволяющее пользователю её прокручивать) упрощает эффективное отображение больших наборов данных. Нужно предоставить свои данные и определить, как выглядит каждый элемент, а библиотека *RecyclerView* динамически создаст элементы, необходимые пользователю.

RecyclerView перерабатывает эти отдельные элементы. Когда элемент прокручивается на экране, RecyclerView не удаляет его изображение. Вместо этого, оно повторно используется для новых элементов, прокручивающихся на экране. Этот процесс значительно повышает производительность работы приложения, улучшая его отклик на запросы и снижая энергопотребление.

Примечание. RecyclerView—это имя не только приложения, но и библиотеки. На странице RecyclerView в шрифте кода, это имя означает класс в библиотеке.

Ключевые классы. Чтобы создать динамический список, необходимый для разрабатываемого приложения, несколько классов работают вместе.

RecyclerView – это набор представлений объектов ViewGroup, похожий на контейнер, содержащий объекты, соответствующие данным пользователя. Это и есть представление объекта, поэтому нужно добавить RecyclerView в свой макет так же, как и любой другой элемент пользовательского интерфейса.

Каждый отдельный элемент в списке определяется своим объектом – держателем представления. Когда держатель представления создан, с ним не связаны никакие данные. После его создания RecyclerView привязывает его к своим данным. Следует определить его, расширяя RecyclerView.ViewHolder.

RecyclerView запрашивает эти представления и связывает их с данными. Нужно определить адаптер, расширяя RecyclerView.Adapter.

Менеджер по расположению размещает отдельные элементы в списке. Можно использовать один из менеджеров компоновки, предоставляемый библиотекой RecyclerView, или определить собственный. Все менеджеры компоновки основаны на абстрактном классе библиотеки LayoutManager. Можно увидеть, как все части списка сочетаются друг с другом, в образцах приложения RecyclerView (Kotlin) или RecyclerView (Java).

Шаги по реализации пользовательского RecyclerView. При использовании RecyclerView нужно выполнить несколько важных этапов.

Прежде всего, нужно *реши́ть*, как будет *выглядеть разрабатываемый список* или *сетка* элементов. Обычно используется один из стандартных менеджеров компоновки, принадлежащих библиотеке RecyclerView.

Затем следует *спроектировать сценарий* поведения каждого элемента в списке, как он будет выглядеть и вести себя. На основе этого проекта нужно расширить класс ViewHolder. Используемая версия ViewHolder предоставляет все функции для элементов списка. Держатель представления – это оболочка вокруг него, и этим представлением управляет RecyclerView.

Следующий шаг – определить адаптер, связывающий исходные данные с представлениями ViewHolder. Имеются расширенные параметры настройки, адаптирующие RecyclerView к точным потребностям пользователя.

Элементы в RecyclerView упорядочены по классу LayoutManager. Библиотека RecyclerView предоставляет три менеджера компоновки, которые обрабатывают наиболее распространенные ситуации компоновки:

- LinearLayoutManager упорядочивает элементы в одномерном списке.
- GridLayoutManager упорядочивает все элементы в двумерной сетке:

Если сетка расположена вертикально, то GridLayoutManager пытается сделать все элементы в каждой строке одинаковой ширины и высоты, но разные строки в этой сетке могут иметь разную высоту.

Если сетка расположена горизонтально, то GridLayoutManager пытается сделать все элементы в каждом столбце одинаковой ширины и высоты, но разные столбцы в этой сетке могут иметь разную ширину.

StaggeredGridLayoutManager похож на GridLayoutManager, но не требует, чтобы элементы в строке имели одинаковую высоту (для вертикальных сеток) или элементы в столбце имели одинаковую ширину (для горизонтальных сеток). Элементы в строке или столбце могут смещаться друг относительно друга.

Далее, нужно будет *разработать макет* отдельных предметов. Этот макет понадобится при разработке держателя представления. После того, как макет будет разработан, необходимо реализовать свой адаптер и ViewHolder. Эти два класса работают совместно, чтобы определить, как отображаются данные. ViewHolder – это оболочка вокруг View, которая содержит макет для отдельного элемента в списке. Адаптер создает объекты ViewHolder по мере необходимости, а также устанавливает данные для этих представлений. Процесс связывания представлений с их данными называется *привязкой*.

После определения адаптера нужно переопределить 3 ключевых метода:

- *onCreateViewHolder ()*: RecyclerView вызывает этот метод всякий раз, когда ему нужно создать новый ViewHolder. Метод создает и инициализирует ViewHolder и связанный с ним View, но не заполняет содержимое представления, так как ViewHolder еще не привязан к конкретным данным.

- *onBindViewHolder ()*: RecyclerView вызывает этот метод, чтобы связать ViewHolder с данными. Метод извлекает соответствующие данные и использует их для заполнения макета держателя представления. Так, если RecyclerView отображает список имен, то метод может найти соответствующее имя в списке и заполнить виджет TextView держателя представления.

– getItemCount (): RecyclerView вызывает этот метод, чтобы получить размер набора данных. Например, в адресную книгу, это может быть общее количество адресов. RecyclerView использует это, чтобы определить, когда больше нет элементов, которые могут быть отображены.

На рис. 4.11 представлен пример простого адаптера с вложенным ViewHolder, который отображает список данных. В этом случае RecyclerView отображает простой список текстовых элементов. Адаптеру передается массив строк, содержащий текст для элементов ViewHolder.

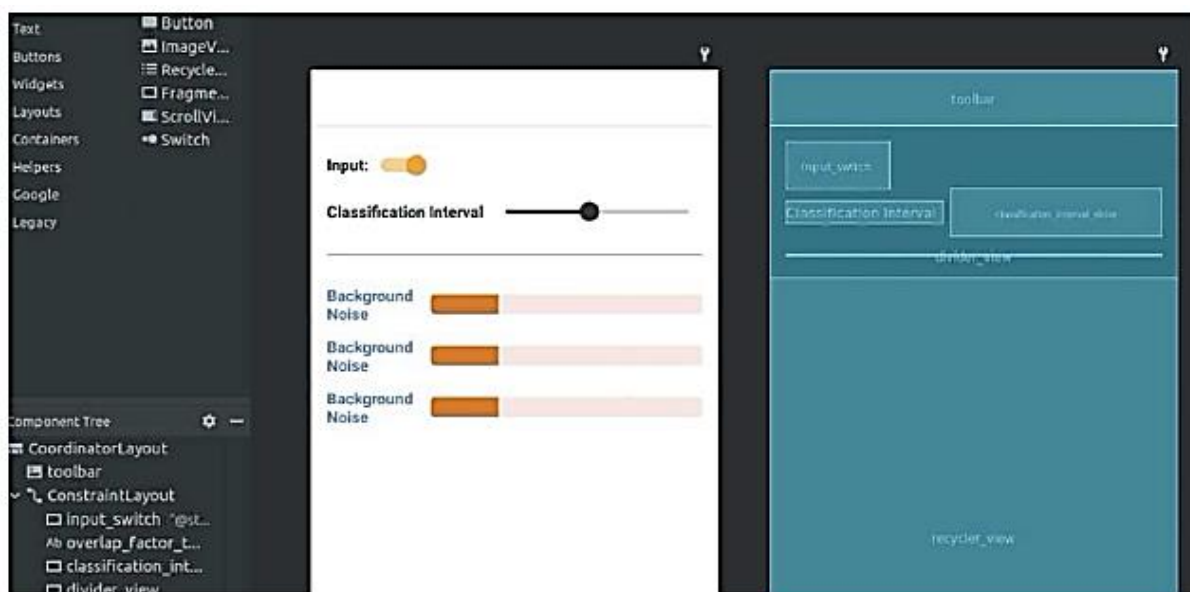


Рисунок 4.11. Основная форма

На вспомогательной форме отображается представление результата – имя класса и его значение схожести с анализируемым сигналом (рис. 4.12).

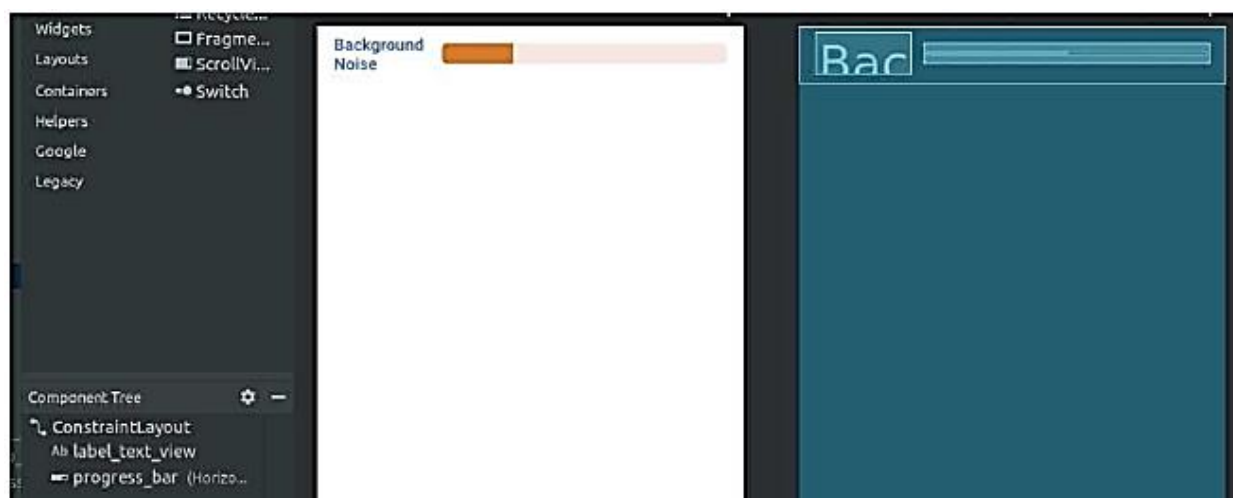


Рисунок 4.12. Вспомогательная форма

В форме скрипта создание формы можно видеть на рис. 4.13.

```
16 <androidx.coordinatorlayout.widget.CoordinatorLayout xmlns:android="http://sc
17     xmlns:app="http://schemas.android.com/apk/res-auto"
18     xmlns:tools="http://schemas.android.com/tools"
19     android:layout_width="match_parent"
20     android:layout_height="match_parent"
21     tools:context=".MainActivity">
22
23     <androidx.appcompat.widget.Toolbar
24         android:id="@+id/toolbar"
25         android:layout_width="match_parent"
26         android:layout_height="?attr/actionBarSize"
27         android:background="@android:color/white"
28         android:elevation="4dp"
29         app:layout_constraintTop_toTopOf="parent" />
30
31     <androidx.constraintlayout.widget.ConstraintLayout
32         android:layout_width="match_parent"
33         android:layout_height="match_parent"
34         android:paddingTop="?attr/actionBarSize">
```

Рисунок 4.13. Скриншоты работы приложения на разных тестовых сигналах

Для работы приложения необходимо указать необходимые разрешения: одно из важных ключевых разрешений RECORD_AUDIO, на использование микрофона, представлено на рис. 4.14 в виде манифеста приложения.

```
ity_main.xml x item_probability.xml x MainActivity.kt x AndroidManifest.xml x
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="org.tensorflow.lite.examples.soundclassifier">

    <uses-permission android:name="android.permission.RECORD_AUDIO" />

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="TFL Sound Classifier"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

Рисунок 4.14. Основные разрешения в манифесте приложения

Если переключатель ввода находится в активном положении, то активируется функция классификации. Все данные, обрабатываемые внутри приложения, необходимо преобразовать к типу `Int`, а данные, обрабатываемые нейросетью – к типу `Double`. Кроме ввода с микрофона, обрабатываются также и данные с переключателя интервала образца (сэмпла). Можно указать временной интервал – от 100 до 1000 миллисекунд.

Также используется опция *Handler* – механизм, позволяющий работать с очередью сообщений. Он привязан к конкретному потоку (`Thread`) и работает с его очередью. `Handler` умеет помещать сообщения в очередь. При этом он ставит самого себя в качестве получателя этого сообщения. И когда приходит время, система достает сообщение из очереди и отправляет его адресату на обработку. Результирующий программный код можно увидеть на рис. 4.15.

```
// Input switch to turn on/off classification
keepScreenOn(inputSwitch.isChecked)
inputSwitch.setOnCheckedChangeListener { _, isChecked ->
    if (isChecked) startAudioClassification() else stopAudioClassification()
    keepScreenOn(isChecked)
}

// Slider which control how often the classification task should run
classificationIntervalSlider.value = classificationInterval.toFloat()
classificationIntervalSlider.setLabelFormatter { value: Float ->
    "${value.toInt()} ms"
}
classificationIntervalSlider.addOnChangeListener { _, value, _ ->
    classificationInterval = value.toLong()
    stopAudioClassification()
    startAudioClassification()
}
}

// Create a handler to run classification in a background thread
val handlerThread = HandlerThread("backgroundThread")
handlerThread.start()
handler = HandlerCompat.createAsync(handlerThread.looper)

// Request microphone permission and start running classification
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
    requestMicrophonePermission()
} else {
    startAudioClassification()
}
```

Рисунок 4.15. Скриншоты работы приложения на различных тестовых сигналах

Функция классификации запускается с ранее обученной моделью нейронной сети. В качестве входного тензора ей подается аудио с микрофона, записанное в переменную Record, а затем соответствующие аудио классы передаются в список, который выводится на экран другой частью кода. А в качестве вывода нейросеть выводит текстовый файл со списком классов, которые характеризуют обрабатываемую нейронную сеть.

Единственное условие: вероятность определения класса должна быть выше минимальной заданной. В условиях она определяется в 30%.

Функцию классификации можно увидеть на рис. 4.16.

```
private fun startAudioClassification() {
    // If the audio classifier is initialized and running, do nothing.
    if (audioClassifier != null) return;

    // Initialize the audio classifier
    val classifier = AudioClassifier.createFromFile(this, MODEL_FILE)
    val audioTensor = classifier.createInputTensorAudio()

    // Initialize the audio recorder
    val record = classifier.createAudioRecord()
    record.startRecording()

    // Define the classification runnable
    val run = object : Runnable {
        override fun run() {
            val startTime = System.currentTimeMillis()

            // Load the latest audio sample
            audioTensor.load(record)
            val output = classifier.classify(audioTensor)

            // Filter out results above a certain threshold, and sort them descendingly
            val filteredModelOutput = output[0].categories.filter {
                it.score > MINIMUM_DISPLAY_THRESHOLD
            }.sortedBy {
                -it.score
            }
        }
    }
```

Рисунок 4.16. Функция классификация обученной модели нейронной сети

Далее нужно прописать обработчики ошибок, предусматривающие прекращение записи, и отсутствие сообщений. Для этого:

- в основном потоке создаётся Handler;
- в потоке загрузки файлов происходит обращение к Handler, и с его помощью в очередь помещается сообщение для него же самого;
- система рассматривает это сообщение, обнаруживает, что адресат – Handler, и отправляет это сообщение на обработку в тот же самый Handler;
- Handler, получив сообщение, обновляет TextView.

На рис. 4.17 показана описанная выше обработка потоков.

```
val finishTime = System.currentTimeMillis()

Log.d(TAG, "Latency = ${finishTime - startTime}ms")

// Updating the UI
runOnUiThread {
    probabilitiesAdapter.categorylist = filteredModelOutput
    probabilitiesAdapter.notifyDataSetChanged()
}

// Rerun the classification after a certain interval
handler.postDelayed(this, classificationInterval)
}
}

// Start the classification process
handler.post(run)

// Save the instances we just created for use later
audioClassifier = classifier
audioRecord = record
}

private fun stopAudioClassification() {
    handler.removeCallbacksAndMessages(null)
    audioRecord?.stop()
    audioRecord = null
    audioClassifier = null
}
```

Рисунок 4.17. Обработка потоков в Handler

Затем, запустим разработанное приложение на подключенном по USB устройстве Android и протестируем его на нескольких образцах (сэмплах).

Для того, чтобы использовать утилиту Android Debug Bridge (ADB), предназначенную для отладки устройств на ОС Android, с устройством, подключенным по USB, необходимо разрешить USB-отладку в системных настройках смартфона или планшета, в разделе «Параметры разработчика». На некоторых устройствах этот раздел по умолчанию скрыт.

Необходимо найти пункт "Номер сборки" и щёлкнуть по нему 7 раз. Должно появиться окно, оповещающее о том, что активирован режим разработчика. Теперь в настройках должен появиться раздел параметров разработчика. Включим в нём опцию "Отладка USB".

На рис. 4.18 показаны результаты тестирования приложения.



Рисунок 4.18. Результаты тестирования приложения на различных тестовых сигналах

Разработанное приложение тестировалось на аудио файлах из базы данных, которая использовалась для обучения модели нейронной сети, но не вошедших в обучающую выборку. Результаты соответствуют ожидаемым.

Итак, мобильное приложение, предназначенное для распознавания эмоций человека по голосу, с помощью извлечения эмоциональных характеристик из речевых сигналов с помощью нейросетевых технологий, успешно реализовано.

5. НЕЙРОННЫЕ СЕТИ ДЛЯ МОНИТОРИНГА И РАСПОЗНАВАНИЯ ОБЪЕКТОВ, ПРЕДСТАВЛЯЮЩИХ УРОЗУ, НА ИЗОБРАЖЕНИИ

Трудно вообразить современных людей без использования гаджетов – смартфонов, умных часов, носимой электроники, Bluetooth–наушников, и др. Почти у каждого девайса есть видеокамера, с помощью которой можно не только проводить фото и видео съемку, но и находить объекты с заданными параметрами, благодаря возможностям обученных нейронных сетей.

В настоящее время широко используются технологии машинного зрения. Это одна из важнейших областей науки о больших данных, которая является частью технологии искусственного интеллекта, пытающаяся воспроизвести возможности человеческого зрения. Более того, машинное зрение стало частью жизни каждого человека. Сознательно или неосознанно, мы все используем различные функции, в основе которых лежат методы компьютерного зрения. Например, мы используем разблокировку по лицу в наших смартфонах, машинное зрение используется в нейронных сетях, а, следовательно, имеется возможность обучать их для решения более сложных задач, при выполнении которых затрачивается намного меньше времени, чем может человек.

Актуальность таких исследований обусловлена тем, что использование компьютерного зрения при нахождении объектов, возможно, представляющих угрозу, на изображении, может уменьшить роль человеческого фактора в тех случаях, когда реакция и точность очень важны при классификации объекта.

Целью исследования является реализация алгоритма, способного находить, определять и распознавать объект, а также его классифицировать, то есть, показать маску объекта и вероятность соответствия классификации.

Объектом исследования является алгоритмы распознавания объектов на изображении, *предметом исследования* – алгоритмы локализации и классификации объектов на изображении с применением сверточных нейронных сетей.

Для достижения поставленной цели определены *следующие задачи*:

- изучение принципов работы свёрточных нейронных сетей;
- понимание особенностей, достоинств и недостатков нейронных сетей;
- поиск наиболее подходящих алгоритмов решения поставленной задачи;
- обучение модели свёрточной нейронной сети на специальных образцах;
- тестирование и проверка работы обученной свёрточной нейронной сети.

5.1. Анализ принципов работы искусственных нейронных сетей и их применение для распознавания объектов на изображении

5.1.1. Обзор методов распознавания объектов на изображении

В настоящем исследовании нужно разработать алгоритм, который должен распознавать объекты на изображении, при том, что они могут принадлежат разным классам. Современный мир окружен гигантскими массами цифровой визуальной информации. Для анализа и классификации разрушительного океана визуальной информации, на которой могут быть изображены объекты, представляющие собой угрозу, необходимы методы анализа изображений.

В частности, полезными могут оказаться методы автоматического анализа семантического контента изображений или видео. Содержание изображения определяет значимость большинства потенциальных применений. Одним из важных аспектов контента изображения являются объекты, находящиеся на нём. Поэтому существует необходимость в методах распознавания объектов.

Распознавание объектов является важной задачей обработки изображений и компьютерного зрения. Оно связано с определением идентичности объекта, наблюдаемого на изображении, по набору известных тегов. Люди могут легко распознать любой объект в реальном мире без каких-либо усилий, однако вычислительные устройства сами по себе не могут распознавать объекты.

Алгоритмические описания задачи распознавания объектов реализуются на компьютерах, что является сложной задачей. Необходимо разработать новые, менее сложные и эффективные методы распознавания объектов.

Многие успешные подходы к решению проблемы общего обнаружения объектов используют представление объектов с помощью набора локальных дескрипторов, описывающих содержимое изображения. Глобальные функции обеспечивают лучшее распознавание. Используются также функции цвета и формы. Однако наиболее эффективным подходом к решению данной задачи является использование сверточных нейронных сетей.

5.1.2. Анализ принципа работы искусственной нейронной сети

Термин "*нейрон*" происходит от основной функциональной единицы нервной системы человека, а именно нейрона или нервных клеток человека, присутствующих в мозге и других частях тела человека.

Нейронная сеть – это группа алгоритмов, которые удостоверяют лежащую в основе взаимосвязь в наборе данных, аналогичных человеческому мозгу. Нейронная сеть помогает изменить входные данные таким образом, чтобы сеть давала наилучший результат без изменения процедуры вывода.

Структура искусственной нейронной сети в сравнении с биологической нервной сетью очень похожа, она пытается её скопировать для подражания нейронным связям человеческого мозга. Можно утверждать, что структура искусственных нейронных сетей аналогична биологической нейронной сети. Так как человеческий мозг – это сеть из миллиардов плотно связанных нейронов, которые очень сложны, нелинейны и имеют триллионы синапсов, то в основе нейронной сети лежат те же биологические аналоги и в основном состоит из дендритов, аксона, тела клеток, синапсов, сома и ядра.

Дендриты несут ответственность за получение входных данных от других нейронов, а аксон отвечает за передачу данных от одного нейрона на другой. Молекулярное и биологическое оборудование нейронных сетей живых организмов основано на электрохимической сигнализации.

Нейроны создают электрические импульсы только при соблюдении определенных условий. Некоторые элементы из нейронной структуры мозга присутствуют при рождении организма, в то время как другие части нейронной сети развиваются вследствие обучения, особенно на ранних стадиях жизни, чтобы адаптироваться к окружающей среде (новые входы).

Механизм работы искусственной нейронной сети (ИНС) выглядит как направленный граф, узлами которого и являются нейроны, а дугами служат связи с весами между ними. Структура показана на рис. 5.1.

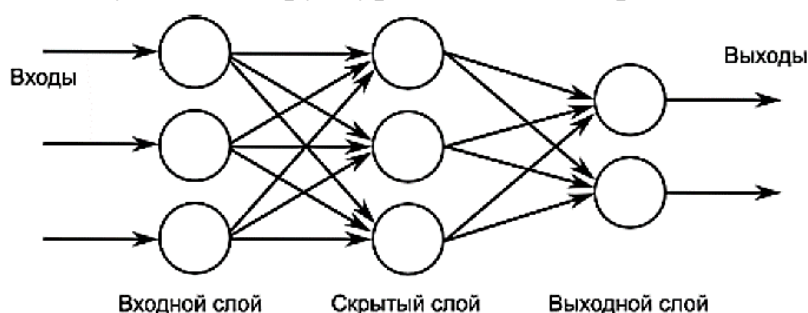


Рисунок 5.1. Структура искусственной нейронной сети

Искусственные нейронные сети состоят из слоёв, соединенных с блоками ввода и вывода, называемых нейронами. Однослойная ИНС называется *персептроном* [61]. В ней может присутствовать несколько скрытых слоев.

Входные блоки (рецепторы), веса соединений, суммирующая функция, вычислительные и выходные блоки (эффекторы) составляют *искусственный нейрон*. Модель персептрона, как однослойной ИНС, показана на рис. 5.2

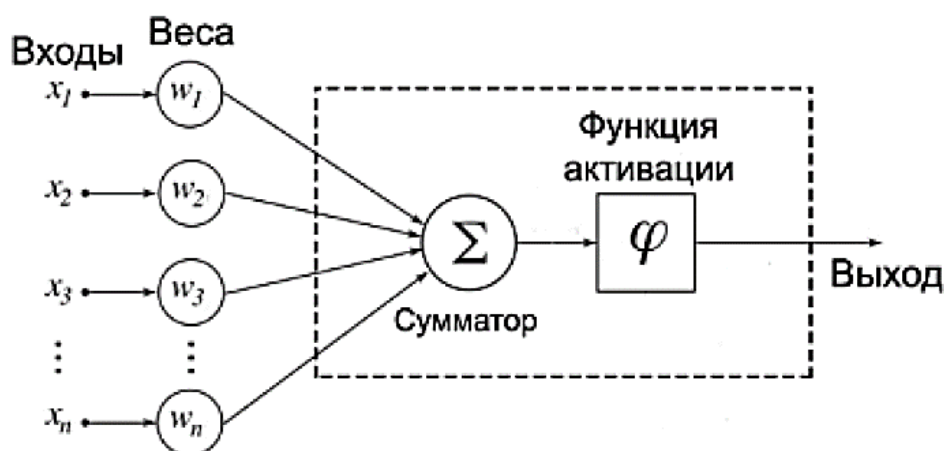


Рисунок 5.2. Модель персептрона, как однослойной искусственной нейронной сети

Хотя искусственные нейроны работают значительно медленнее, чем кремниевые логические элементы, их массивная взаимосвязь компенсирует низкую скорость. Значение веса связи – это сила указанной связи между нейронами. Веса связи инициализируются и корректируются случайным образом, с помощью алгоритма оптимизации, для сопоставления агрегаций входных стимулов с желаемой конфигурацией выходной функции нейрона.

Искусственные нейронные сети работают по тому же принципу, как и их биологическое аналоги. Их можно рассматривать как взвешенные направленные графы, где нейроны можно сравнить с узлами, а связь между двумя нейронами – как взвешенные ребра графа. Обработывающий элемент нейрона получает множество сигналов (как от других нейронов своей искусственной нейронной сети, так и в качестве входных сигналов из внешнего мира).

Сигналы могут быть модифицированы в принимающем синапсе, а взвешенные входы суммируются в обрабатывающем элементе. Если сигнал пересекает допустимый порог, то он поступает на вход другим нейронам (или на выход во внешний мир), и процесс повторяется. Обычно веса представляют силу взаимосвязи между нейронами. Функция активации — это передаточная функция, которая используется для получения желаемого результата для решаемой задачи. Например, желаемый результат равен нулю или единице, в случае двоичного классификатора. Сигмовидная функция также может быть использована в качестве функции активации.

Существует множество разновидностей функций активации: линейная регрессия, логистическая регрессия, функция идентичности, биполярная функция, бинарная сигмоида, биполярная сигмоида, гиперболическая касательная функция, сигмоидальная гиперболическая функция и др.

Основные функции активации представлены на рис. 5.3.

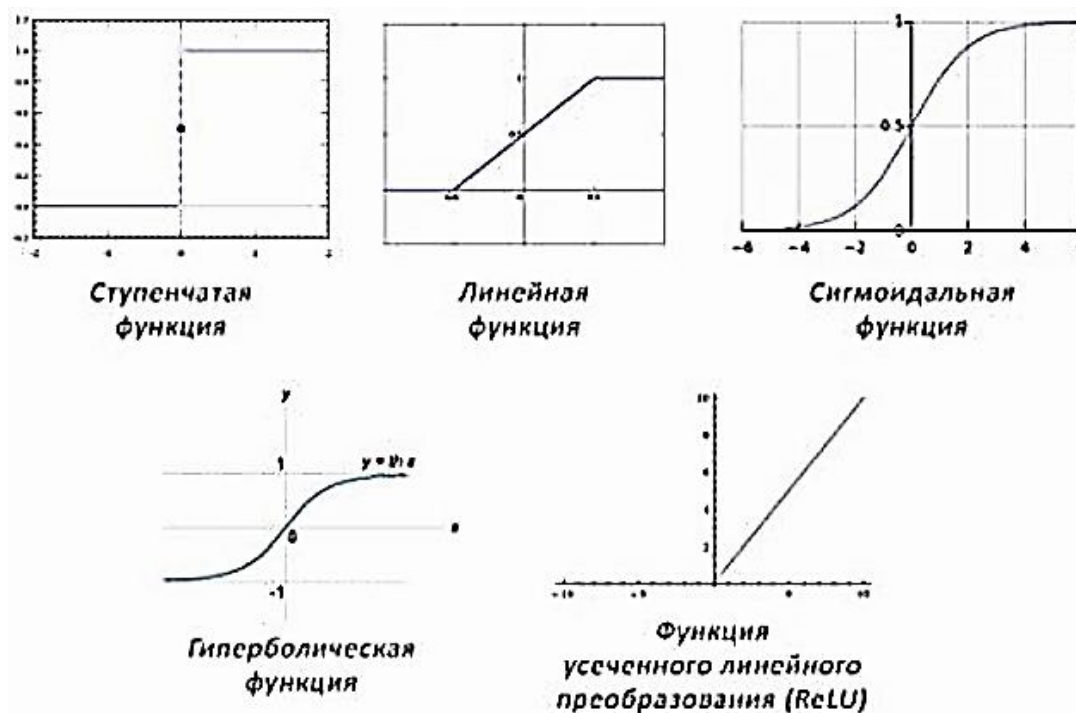


Рисунок 5.3. Основные разновидности функций активации

Искусственные нейронные сети специально разрабатываются для конкретной задачи, например, двоичной классификации, многоклассовой классификации, распознавания образов и др. Веса синоптических связей нейронных сетей корректируются в процессе обучения. Если биологическая нейронная сеть содержит 86 миллиардов нейронов и триллионы синапсов, то искусственная нейронная сеть состоит из не более, чем 1000 нейронов [62]. Если сравнить биологические нейронные сети с искусственными, то для ИНС необходимо большое количество энергии и вычислительных ресурсов.

5.1.3. Анализ методов обучения искусственных нейронных сетей

Как отмечалось ранее, искусственные нейронные сети созданы по образцу биологических нейронных сетей мозга живых организмов. Искусственный нейрон имитирует поведение биологического нейрона путем сложения значений входных сигналов, которые он получает.

Если такое значение превышает некий порог, то нейрон посылает собственный сигнал на свой выход, который затем принимается другими нейронами. Однако нейрон не должен относиться к каждому из своих входов с одинаковым весом. Каждый из его входов может быть скорректирован путем умножения на некоторый весовой коэффициент. Скажем, если бы вход А был вдвое важнее входа В, то вес входа А был бы равен двум. Вес может быть и отрицательным, если значение этого входа не представляет важности.

Каждый нейрон связан с другими нейронами в сети через синаптические связи, значения которых взвешены, и сигналы, распространяющиеся по сети, усиливаются или ослабляются этими значениями веса. Процесс обучения нейронной сети включает в себя регулировку этих весовых значений таким образом, чтобы конечный выход сети давал правильный ответ. Простейшая версия искусственной нейронной сети, основанная на персептроне Фрэнка Розенблатта, имеет три слоя нейронов. Первый – это входной слой. Он принимает входные значения – например, пиксели фотографии. Выходы этого первого слоя нейронов подключены к среднему слою, называемому «скрытым». Выходы этого «скрытого слоя» нейронов затем подключаются к последнему выходному слою, который и дает ответ на вопрос, чему обучили сеть. Например, сеть может быть обучена распознавать фотографии кошек. Выходной слой сети будет иметь два выхода: «кошка» или «не кошка». Учитывая набор фотографий, которые человек пометил как «кошка» или «не кошка», сеть обучается, регулируя веса таким образом, чтобы при появлении новой немаркированной фотографии кошки, она выдавала более 90 процентов вероятности, что это кошка, или менее 10 процентов, если это не кошка.

Нейронные сети могут классифицировать объекты и более, чем на две категории, например, они распознают рукописные цифровые символы от 0 до 9, 26 букв латинского алфавита или звуки музыки, относящиеся к 7 нотам.

Персептроны были ограничены тем, что имели только один средний «скрытый» слой нейронов. Хотя их создатель Фрэнк Розенблатт понимал, что наличие большего количества внутренних скрытых слоев полезно для решения многих задач, он не нашел способа обучить такую нейронную сеть.

Только после того, как в 1980 году специалист по объединению нейронных сетей Джеффри Хинтон разработал алгоритм, известный как «обратное распространение сигнала», для обучения искусственных нейронных сетей с несколькими скрытыми слоями, эта проблема была решена.

Искусственные нейронные сети с большим количеством скрытых слоев известны под названием «многослойные персептроны», или как «глубокие» нейронные сети, отсюда и появился термин «глубокое» обучение. Количество скрытых слоев и ответ на вопрос, сколько нейронов должна иметь такая сеть, и является её архитектурой, а определение оптимальной архитектуры для конкретной задачи сейчас является процессом проб и ошибок, и находится ближе к искусству, чем к науке. В самой основе проектирования нейронных сетей лежит большой простор для творческой изобретательности человека.

В приведенном примере использовался помеченный набор данных для определения того, изображена ли на картинке кошка, или нет. Обучение с данными, помеченными человеком, представляет собой так называемое *контролируемое обучение*, поскольку оно контролируется человеческими метками. Большая часть современных систем глубокого обучения работает на основе подобных систем с контролем, и именно здесь личные предубеждения в предварительно помеченных данных могут исказить работу сети.

Существует два других вида машинного обучения. *Неконтролируемое обучение* даёт сети немаркированные данные и требует от неё самостоятельно попытаться найти закономерности и кластеры сходства в предметах, а человек уже после этого процесса анализирует и даёт названия кластерам, которые нашла нейронная сеть. Неконтролируемое обучение может иногда сочетаться с контролируемым обучением (для предварительного обучения сети, которая затем обучается на маркированных данных), что значительно сокращает время обучения при использовании только контролируемого обучения.

Третий тип машинного обучения называется *обучением с подкреплением*. Такое обучение часто используется в компьютерных играх. Вместо того, чтобы получать внешние данные о выигрышах и проигрышах, система сама генерирует эти данные, играя сама с собой снова и снова, становясь с каждым разом все лучше. Обучение с подкреплением было вдохновлено идеями о том, как дети учатся совершать хорошие поступки с помощью вознаграждений и избегать плохих поступков с помощью наказаний. Например, для создания компьютерной игры AlphaGo (го – китайская интеллектуальная игра, сродни шахматам и шашкам), впервые в мире выигравшей матч у профессионального игрока – чемпиона мира Ли Седоля, компания Google DeepMind в 2015 году, использовала искусственные нейронные сети с комбинацией контролируемого машинного обучения, а также обучения с подкреплением.

В настоящее время широко применяется алгоритм обучения нейронных сетей, известный как *метод градиентного спуска*. Здесь под градиентом понимается числовое вычисление, необходимое для настройки параметров сети так, чтобы её выходное отклонение было сведено к минимуму. Принцип работы метода градиентного спуска представлен на рис. 5.4.

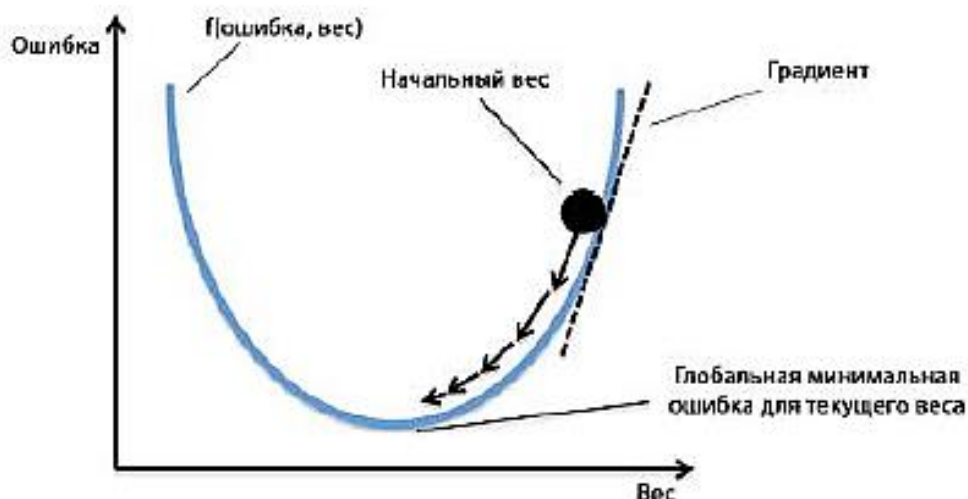


Рисунок 5.4. Принцип градиентного спуска

Алгоритм имеет несколько версий работы в зависимости от количества выборок, которые вводятся в нейронную сеть для каждой итерации.

Пакетный градиентный спуск означает, что все доступные данные вводятся одновременно. Это сопровождается высоким риском *застревания данных*, так как градиент рассчитывается с использованием всех выборок, где вариации минимальны. Для сети важно иметь вход с некоторой случайностью.

Метод стохастического градиентного спуска предполагает, что на каждой итерации вводится одна случайная выборка. Градиент рассчитывается только для конкретной выборки, что подразумевает введение желаемой случайности и усложняет возможность застревания. Недостатком метода является его низкая скорость работы, так как требуется множество итераций. Кроме того, он не использует преимущества доступных вычислительных ресурсов.

Метод стохастического *мини-пакетного градиентного спуска* предполагает, что вместо подачи в сеть отдельных выборок на каждой итерации вводится N случайных элементов. Это сохраняет преимущества метода, а также ускоряет обучение за счёт распараллеливания операций. Процесс показан на рис. 5.5.

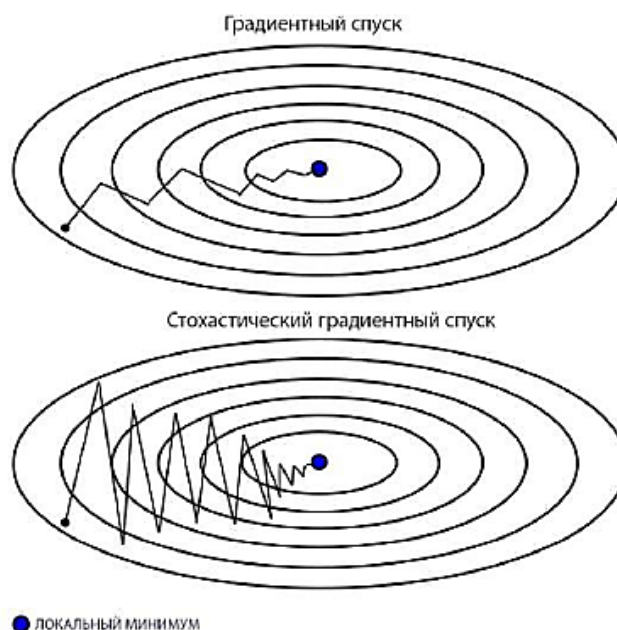


Рисунок 5.5. Процесс обучения нейронной сети при использовании различных вариантов метода градиентного спуска

В данном проекте использован метод стохастического *мини-пакетного градиентного спуска*. Выбирается N случайных элементов, что обеспечивает хороший баланс между требованиями к случайности и времени обучения, а также приемлемо для доступной памяти графического процессора GPU.

5.1.4. Особенности сверточной нейронной сети

Сверточная нейронная сеть (СНС, от англ. convolutional neural network, CNN), представляет собой специализированный тип модели нейронной сети, использующей *свёртку* вместо общего матричного умножения, по крайней мере в одном из своих скрытых слоев. Это тип искусственной нейронной сети, используемой для распознавания и обработки изображений в виде пиксельных данных. СНС используются для работы с двумерными данными изображений, хотя могут работать с одномерными и трёхмерными данными.

Как известно, нейронная сеть получает на входе исходные данные, и начинает преобразования благодаря скрытым слоям, которые представляют собой наборы нейронов. В них каждый нейрон связан с нейронами предыдущих слоёв, в которых они функционируют независимо друг от друга. Крайний слой является выходным слоем, и он же выдаёт результат в виде оценки классов. Архитектура сверточной нейронной сети показана на рис. 5.6.

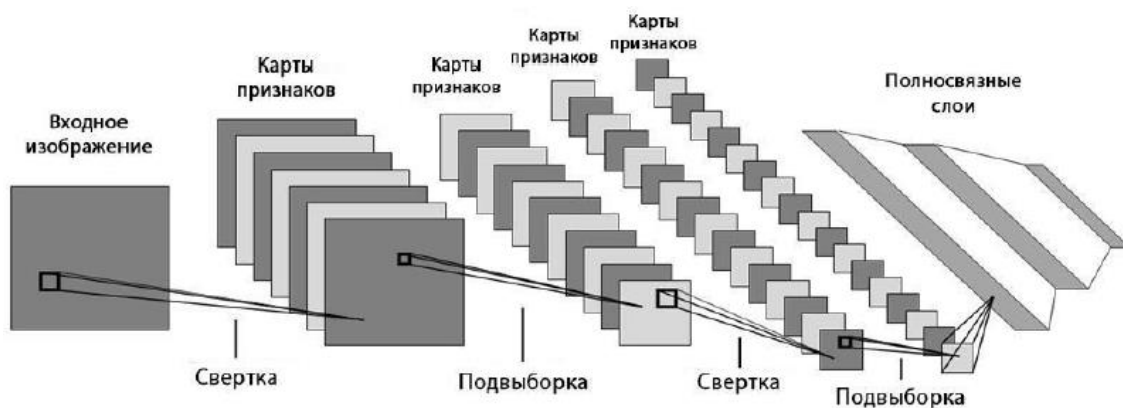


Рисунок 5.6. Архитектура свёрточной нейронной сети

В СНС центральное место занимает свёрточный слой, который и даёт сети ее название. Этот слой выполняет операцию, называемую *свёрткой*.

В контексте свёрточной нейронной сети, свертка –это линейная операция, включающая в себя умножение набора весов с входными данными, подобно традиционной нейронной сети. Так как метод был создан для двумерного ввода данных, умножение производится между массивом входных данных и двумерным массивом весов, который называется *фильтром* или *ядром*.

Операция свёртки представлена на рис. 5.7.

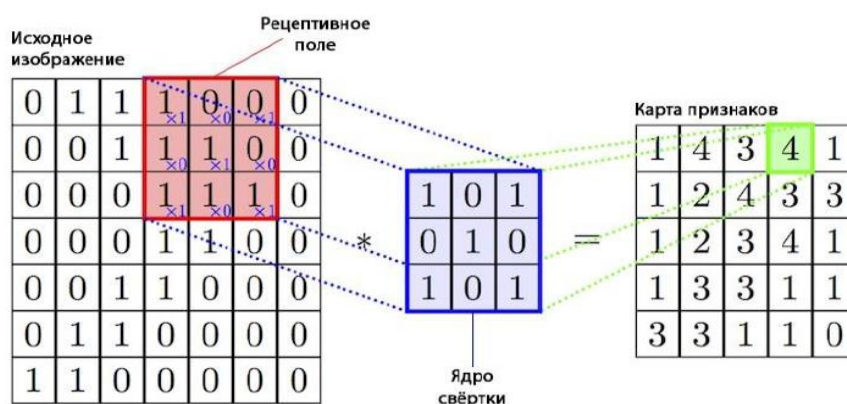


Рисунок 5.7. Операция свертки

Фильтр, как двумерный массив весов, всегда меньше входных данных. Тип умножения, применяемого между участком входного сигнала (размером с фильтр) и самим фильтром, является *точечным произведением*, т. е. поэлементным умножением между участком входного сигнала, размером с фильтр, и самим фильтром, которое затем суммируется, всегда приводя к одному значению. Поскольку в результате операции получается одно значение, операцию часто называют *скалярным произведением*.

Использование *фильтра* меньшего размера, чем входной, является преднамеренным, поскольку это позволяет одному и тому же фильтру (как двумерному набору весов) умножаться на входной массив несколько раз, в разных точках на входе. В частности, фильтр последовательно применяется к каждой перекрывающейся части, или участку слоя свёрточной нейронной сети, размером с фильтр входных данных, слева направо, и сверху вниз [63].

Это систематическое применение одного и того же фильтра к участку изображения. Если фильтр предназначен для обнаружения определенного типа объекта во входных данных, то его последовательное применение ко всему входному изображению даёт возможность обнаружить объект в любом месте изображения. Эта способность называется *инвариантностью перевода*, например, возникает интерес к тому, присутствует ли вообще на конкретном изображении какая-либо функция, а не в каком месте она присутствовала.

Результат однократного умножения фильтра на входной массив представляет собой одно значение. Поскольку фильтр применяется к входному массиву несколько раз, результатом является двумерный массив выходных значений, которые представляют собой фильтрацию входных данных. Выходной двумерный массив этой операции является *картой объектов* [64].

После создания карты объектов, каждое значение в такой карте можно передавать через нелинейную функцию, такую, как ReLU, так же, как это делается для выходов полностью подключенного слоя. Фильтр (ядро), перед применением к входным данным, переворачивается. Технически, свёртка, при использовании свёрточных нейронных сетей, по сути, является *перекрестной корреляцией*, хотя в процессе глубокого машинного обучения свёрточных нейронных сетей эта операция называется операцией свёртки.

Таким образом, имеются входные данные, такие, как изображение значений пикселей, и фильтр, представляющий собой набор весов. Фильтр систематически применяется к входным данным, для создания карты объектов.

Уровень объединения отвечает за уменьшение пространственного размера свернутого элемента. Это сделано для уменьшения требований к вычислительной мощности, необходимой для обработки данных, за счёт уменьшения размеров. Есть два типа объединения при свёртке: *Max Pooling* и *Average Pooling*.

При *максимальном объединении (Max Pooling)* ищется максимальное значение пиксела из части изображения, покрытой ядром. Операция также действует как подавитель шума – т.е. полностью отбрасывает шумные активации и выполняет устранение шума вместе с уменьшением размерности.

При среднем объединении (Average Pooling) ищется средняя величина всех значений из той части изображения, которая покрыта ядром. Выполняется уменьшение размерности, которое действует, как механизм подавления шума.

Для задачи данного проекта объединение Max Pooling работает намного эффективнее, чем Average Pooling, и является предпочтительным. Пример выполнения подвыборки по максимальному значению показан на рис. 5.8.



Рисунок 5.8. Пример выполнения подвыборки по максимальному значению

Идея применения операции свёртки к массивам пикселов изображений не является новой и уникальной для сверточных нейронных сетей. Это распространенный метод, используемый в задачах компьютерного зрения.

Пример операции свертки для цветного изображения показан на рис. 5.9.

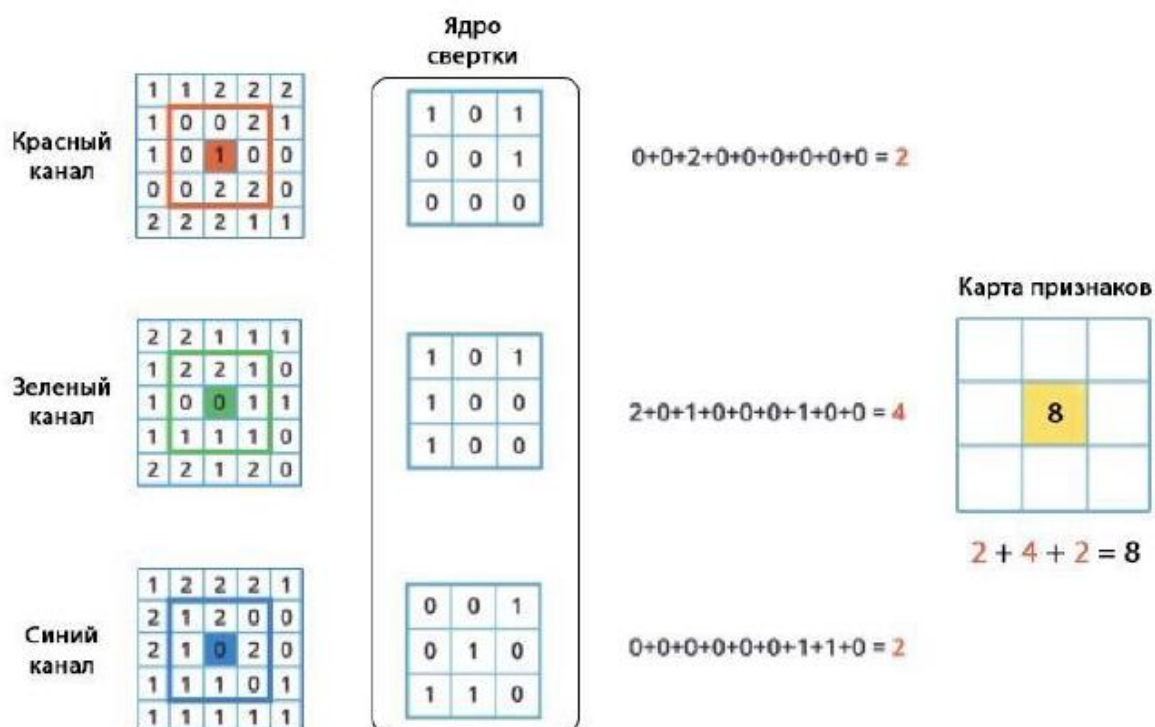


Рисунок 5.9. Свертка цветного изображения

При сжатии картинки и выполнении свёртки следует запомнить информацию о границах изображения, содержащего нули, ширина которого зависит от размера ядра свертки. Эта операция показана на рис. 5.10.

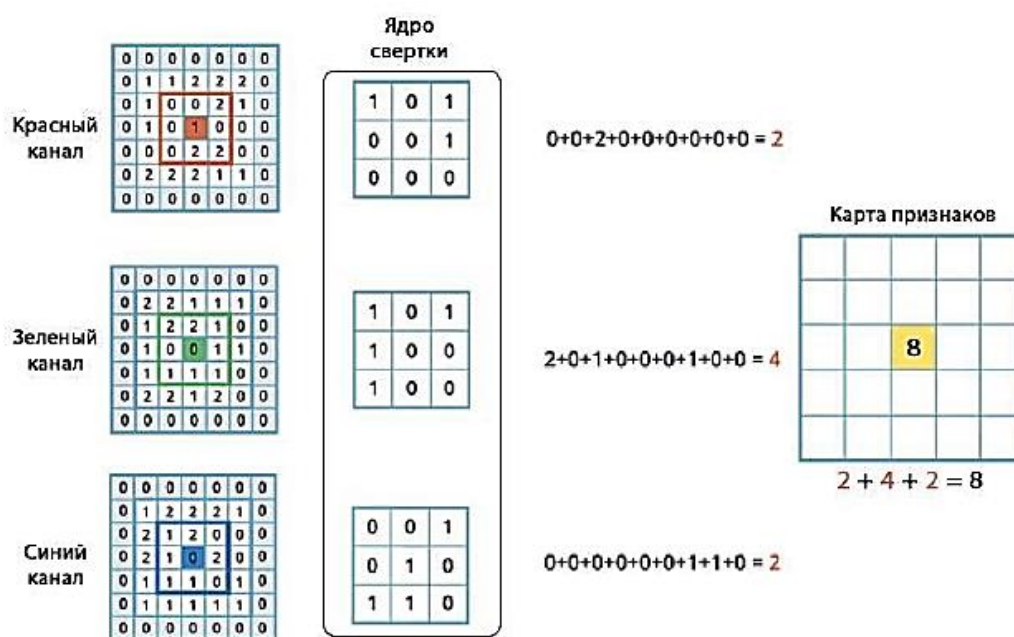


Рисунок 5.10. Свертка цветного изображения с применением заполнения

Целью свёртки является извлечение функций высокого уровня, таких, как нахождение границы (кромки) входного изображения. При этом нельзя ограничиваться только свёрточным слоем. Преобразование отвечает за захват низкоуровневых функций – нахождение границ, цвета и ориентации градиента.

С помощью дополнительных слоев архитектура нейронной сети адаптируется к функциям высокого уровня, формируя сеть, имеющую возможность распознавания изображений в исходном наборе данных.

Добавление полностью связанного слоя – это способ изучения нелинейных комбинаций высокоуровневых функций, представленных выходными данными сверточного слоя. Слой с полным подключением позволяет исследовать возможность применения нелинейной функции в этом пространстве.

После преобразования входного изображения в подходящую форму для многоуровневого персептрона, нужно сгладить изображение в вектор–столбец [65]. Сглаженный вывод подается в нейронную сеть с прямой связью, и к каждой итерации обучения применяется метод обратного распространения ошибки. В течение ряда эпох модель способна различать доминирующие и низкоуровневые функции в изображениях, а также классифицировать их.

Исторически сложилось так, что фильтры были разработаны экспертами по компьютерному зрению вручную, а затем были применены к изображению. Это позволяло получить карту объектов, или результат применения фильтра, что в некоторой степени облегчало процесс анализа изображения [65].

До 2015 года задачи распознавания изображений, например, человеческих лиц, обычно выполнялись с помощью трудоёмких программ с ручным кодированием, которые определяли черты лица, такие, как глаза, брови и нос. Многие приложения оптического распознавания символов (Optical Character Recognition, OCR) или распознавания лиц вообще не использовали машинное обучение. Быстрое увеличение вычислительной мощности, широкая доступность больших наборов данных, графических процессоров и программ глубокого машинного обучения привели к тому, что в середине 2010-х годов сверточные нейронные сети стали стремительно прогрессировать, и смогли обеспечить гораздо более высокую точность, чем традиционные методы. Внезапно, в академических и промышленных кругах, они стали стандартом для решения почти всех задач, связанных с компьютерным зрением.

В настоящее время средний пользователь смартфона, вероятно, имеет в своем кармане одно или два приложения, использующие свёрточные нейронные сети – концепция, которая была бы немыслима в 2010 году.

Таким образом, проанализированы принципы работы искусственных нейронных сетей, проведён обзор методов распознавания объектов на изображении, исследованы принципы работы свёрточной нейронной сети и методы её обучения. Чтобы решить задачу о нахождении объектов на изображении, требуется выбрать наиболее подходящий метод нахождения объектов с применением сверточных нейронных сетей [66].

5.2. Анализ программных средств реализации

Необходимо обосновать выбор языка программирования, интегрированной среды разработки приложения, используемые библиотеки и фреймворки.

5.2.1. Выбор среды для разработки приложения

Главным инструментом разработки приложения стала интегрированная единая среда разработки (Integrated Development Environment, IDE) от компании JetBrains на базе IntelliJ IDEA PyCharm для языка программирования Python.

Эта среда разработчика имеет следующие преимущества [67 – 75]:

- большое число *плагинов* (независимо компилируемых программных модулей, динамически подключаемых к основной программе для расширения и/или применения её возможностей), библиотек, разнообразных расширений, которые облегчают и ускоряют разработку программных приложений;
- возможность *рефакторинга* (оптимизации, подчистки уже готового кода программы с целью упрощения его обслуживания, понимания и расширения, при этом сделанные изменения не должны влиять на функционал программы);
- удобный текстовый редактор программного кода;
- предварительная проверка созданного приложения на ошибки;
- возможность дополнения созданного программного кода;
- большое количество руководств, для облегчения начала работы новичкам;
- имеет средство для отладки программ (Debug System) для поиска ошибок;
- легкая навигация по проекту, удобные функции поиска и замены кода;
- возможность поддержки интеграции систем управления базами данных;
- удобная для пользователя установка библиотек и виртуального окружения.

5.2.2. Программная платформа (фреймворк) TensorFlow

TensorFlow – это библиотека с открытым исходным кодом, разработанная компанией Google для приложений глубокого машинного обучения, также поддерживающая и традиционное машинное обучение. Изначально платформа TensorFlow была разработана для больших численных вычислений, без учёта глубокого машинного обучения, однако эта библиотека оказалась очень полезной и для решения задач глубокого обучения нейронных сетей. По этой причине компания Google выложила её в открытый доступ [67 – 69].

TensorFlow принимает данные в виде многомерных массивов более высокой размерности, называемых *тензорами*. Такие многомерные массивы очень удобны при работе с большими объёмами данных (Big Data).

TensorFlow работает на основе графов потоков данных, которые имеют узлы и ребра. Поскольку механизм выполнения имеет форму графов, то выполнять код библиотеки TensorFlow гораздо проще распределенным образом, на кластере компьютеров, при использовании графических процессоров.

До появления подобных программных платформ (фреймворков), механизм кодирования в машинном обучении был гораздо сложнее.

Библиотека TensorFlow предоставляет собой высокоуровневый интерфейс программирования приложений (Application Programming Interface, API) как совокупность инструментов и функций для создания новых приложений. Поэтому для подготовки нейронной сети, конфигурирования нейрона или его программирования, в TensorFlow сложное кодирование не требуется. Все эти задачи автоматически выполняет библиотека. Кроме того, TensorFlow имеет интеграцию с языками программирования Java и Kotlin.

Приложения глубокого обучения очень сложны, а процесс обучения требует большого количества вычислений. Это занимает много времени из-за большого размера данных, и включает в себя несколько итерационных процессов, умножение матриц, математические вычисления, и т. д.

Графические процессоры (GPU) популярны в контексте компьютерных игр, где нужно, чтобы экран и изображение имели высокое разрешение. Они изначально были разработаны для этой цели, однако для разработки приложений глубокого обучения GPU также широко используются.

Одним из преимуществ TensorFlow является возможность поддержки как графических, так и традиционных вычислительных процессоров, что обеспечивает более высокое быстродействие при компиляции, чем другие библиотеки глубокого машинного обучения, такие, как Keras и Torch.

TensorFlow получает много преимуществ от поддержки коммерческой компании A-List, принадлежащей Google. Это не только способствует быстрому развитию проекта, но и создаёт множество важных приложений на основе TensorFlow, которые упрощают его развертывание и использование:

- Tensor Processing Unit (кристалл TPU) для повышения производительности работы в облаке Google. Это интегральная схема ускорителя искусственного интеллекта, разработанная компанией Google специально для глубокого машинного обучения нейронных сетей, в частности, с использованием собственного программного обеспечения TensorFlow от Google

- для обмена моделями нейронных сетей, созданных с рамками (границами);
- версии фреймворка для браузеров и мобильных устройств; и др.

Недостатком является то, что некоторые детали реализации библиотеки TensorFlow затрудняют получение полностью детерминированных результатов обучения модели нейронной сети для некоторых учебных заданий.

Иногда модель, обученная в одной IDE, немного отличается от модели, обученной в другой системе, даже если им были переданы одни и те же данные.

Причины могут быть разные: например, важен фактор, как и где формируются и присваиваются случайные числа; или недетерминированное поведение при использовании графических процессоров. Эти проблемы можно решить: команда разработчиков TensorFlow рассматривает возможность создания дополнительных элементов управления, чтобы повлиять на детерминизм (предопределённость обучения нейронной сети) в процессе работы.

5.2.3. Обоснование выбора языка программирования

Для написания программного приложения было решено использовать язык программирования Python, являющийся объектно–ориентированным, высокоуровневым, интерпретируемым кроссплатформенным языком. Это мощный компьютерный язык, поддерживающий множество парадигм программирования, высоко производительный и хорошо оптимизированный. Его достоинствами являются универсальность; лёгкость в обучении; удобство в чтении кода; существование множества библиотек, облегчающих разработку нового программного обеспечения. Недостатком Python является довольно медленное выполнение программ (как и у всех интерпретаторов), в сравнении с низкоуровневыми компилируемыми С–подобными языками [67 – 75].

На рис. 5.11 показан рейтинг языков программирования 2021 года.

Rank	Language	Type	Score
1	Python▼	🌐 📱 🖥️	100.0
2	Java▼	🌐 📱 🖥️	95.3
3	C▼	📱 🖥️ ⚙️	94.6
4	C++▼	📱 🖥️ ⚙️	87.0
5	JavaScript▼	🌐	79.5

Рисунок 5.11. Рейтинг наиболее актуальных языков программирования 2021 года

Очевидно, что в настоящее время Python очень актуален и популярен среди разработчиков, и входит в топ языков программирования. Программный код на языке Python понятен и хорошо воспринимается даже новичками. Работа в нём не требует особенных навыков, а наличие большого количества библиотек значительно облегчает работу программисту. Среда разработки этого языка PyCharm и библиотека TensorFlow также поддерживают машинное обучение.

5.3. Выбор и реализация алгоритма для нахождения объектов

5.3.1. Описание принципа работы алгоритма *Single Shot Detector (SSD)*

Проблема нахождения объектов на изображении – это задача машинного обучения, в рамках которой выполняется определение наличия или отсутствия объекта определённого домена на изображении, нахождение границ этого объекта в системе координат пикселей исходного изображения. В зависимости от алгоритма обучения, объект может характеризоваться координатами ограничивающей рамки, ключевыми точками и/или контуром объекта.

Алгоритм Single Shot Detector (SSD) представляет собой алгоритм обнаружения объекта, напрямую предсказывающий координаты и категории ограничивающей рамки. Процесс регистрации объекта не генерируется.

Для сравнения, традиционный метод обнаружения объектов различного размера работает так: он преобразовывает изображения в реальные размеры, затем обрабатывает их по отдельности, и, наконец, объединяет результаты. Алгоритм SSD может достичь того же эффекта, комбинируя карты характеристик различных свёрточных слоев нейронной сети [71, 72].

Основная сетевая структура алгоритма – это VGG16, свёрточная нейронная сеть, содержащая 16 слоев. Сеть состоит из двух частей:

Первая часть сети выделяет характерные признаки в изображении. Она состоит из чередующихся каскадов свёртки и подвыборки. Сначала идут два каскада: «свёртка–свёртка–подвыборка», а затем три каскада «свёртка–свёртка–свёртка–подвыборка». Размер узла свёртки во всех слоях свёрточной нейронной сети составляет 3×3 . При подвыборке производится выбор максимального значения (Max Pooling) из квадрата размером 2×2 .

Вторая часть сети отвечает за классификацию объекта на изображении по выделенным на предыдущем этапе признакам. Эта часть содержит 3 полносвязных слоя. В первых двух слоях находится 4096 нейронов, а в последнем – 1000 нейронов (по количеству классов объектов).

На вход нейронная сеть VGG16 получает изображение размером 224×224 пиксела и 3 канала цвета (красный, зеленый и синий). На выходе сеть выдает вероятности (в формате *One hot encoding*) того, что на изображении находится тот или иной класс объекта [73–75].

Сеть VGG16 преобразовывает два полностью связанных слоя в свёрточный слой, а затем добавляет четыре сетевых структуры со свёрточной слоистой структурой. Структура представлена на рис. 5.12.

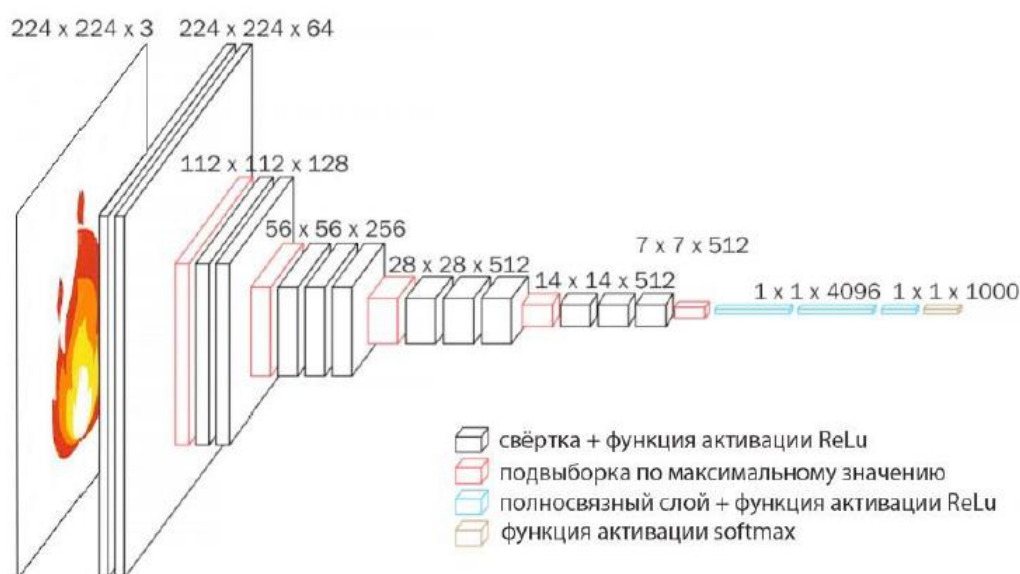


Рисунок 5.12. Структура сверточной нейронной сети VGG16

Выходные данные пяти различных свёрточных слоев свертываются с помощью двух ядер свертки 3×3 соответственно. Доверие для выходной классификации, каждое поле по умолчанию генерирует 21 достоверность, локализация для регрессии вывода, каждое поле по умолчанию генерирует 4 значения координат (x, y, w, h). Кроме того, эти пять свёрточных слоёв также генерируют блок по умолчанию (с помощью сгенерированных координат) через слой PriorBox. Дано количество полей, по умолчанию, для каждого из пяти описанных ранее свёрточных слоёв. Наконец, первые три результата вычислений объединяются, и передаются в слой потерь.

Алгоритму SSD требуется только одно входное изображение и поля истинности каждого объекта во время обучения. Базовая сетевая структура основана на свёрточной нейронной сети VGG16. После предварительного обучения в наборе данных ImageNet используются два новых свёрточных слоя, вместо *fc6* и *fc7*, а также происходит небольшое изменение в *pool5*.

Для поиска объектов на изображении используются методы регрессии ограниченного прямоугольника, с использованием модифицированного алгоритма обнаружения объекта *MultiBox* и метода классификации. Схема модуля обнаружения объекта на изображении показана на рис. 5.13.

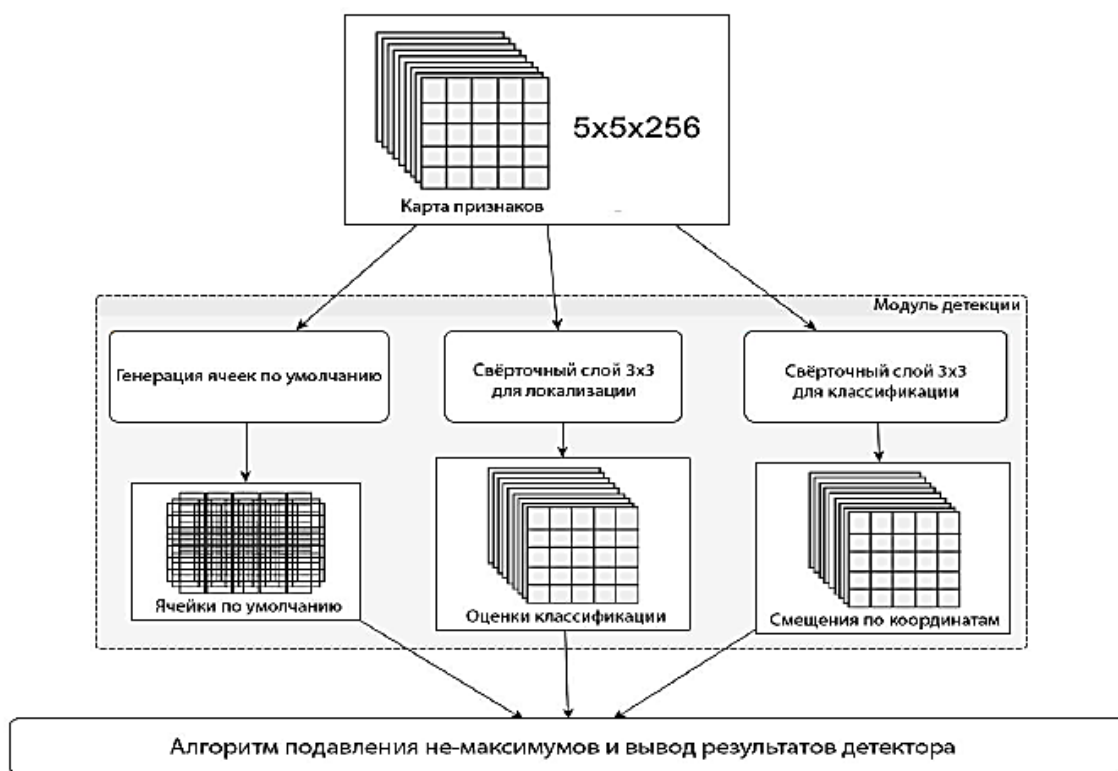
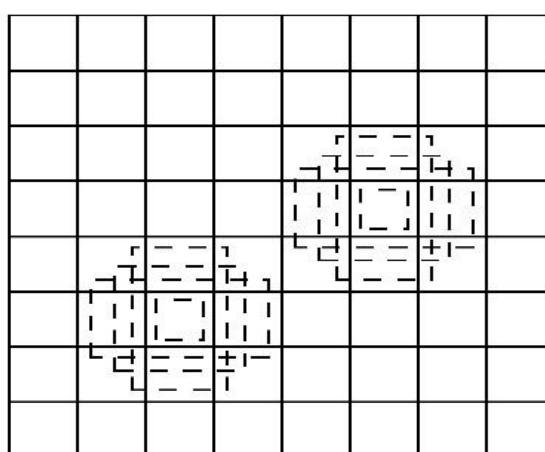


Рисунок 5.13. Схема модуля детектирования алгоритма SSD

Эти ограничительные рамки распределены по всей карте объектов и связаны с её конкретными точками [70–73]. Ограничительные рамки играют важную роль в алгоритме SSD, как те, которые наиболее точно соответствуют фактическому местоположению обнаруженного объекта на изображении.

Пример расположения ограничительных рамок, по умолчанию, на карте признаков размером 8×8 продемонстрирован на рис. 5.14.



Карта признаков 8×8

Рисунок 5.14. Пример расположения рамок по умолчанию на карте признаков

Задачей обучения архитектуры нейронной сети в алгоритме SSD является минимизация функции потерь, складывающаяся из ошибок локализации и классификации. Расчёт функции потерь показан в формуле:

$$L(x, c, l, g) = \frac{1}{N} \cdot (L_{conf}(x, c) + \alpha L_{loc}(x, l, g)) \quad (5.1)$$

где N – количество ограничивающих рамок объектов по умолчанию;

α – значение, необходимое для кросс-валидации объекта, равное 1;

$L_{conf}(x, c)$ – ошибка классификации; $\alpha L_{loc}(x, l, g)$ – ошибка локализации.

Схема распознавания обнаруженных объектов на изображении с помощью алгоритма SSD (Single Shot Detector) представлена на рис. 5.15.

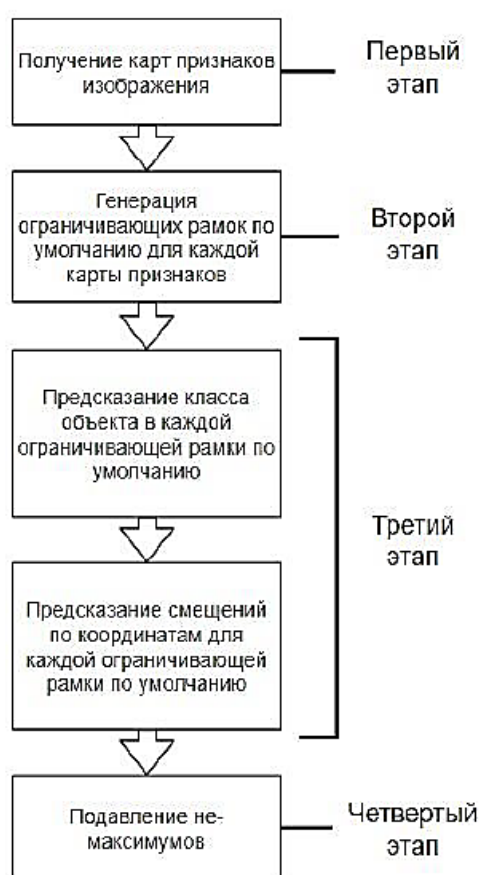


Рисунок 5.15. Схема работы алгоритма SSD (Single Shot Detector)

Итак, рассмотрены известные методы и классы, применяемые при реализации алгоритма SSD. Для распознавания объектов на изображении найден оптимальный для реализации данного проекта алгоритм Single Shot MultiBox Detector. Описаны ключевые этапы его работы, а также рассмотрено его взаимодействие с платформой разработки – библиотекой TensorFlow.

5.4. Обучение свёрточной нейронной сети, и оценка точности реализации алгоритма нахождения объекта

5.4.1. Создание набора изображений для обучения нейронной сети

Для того, чтобы свёрточная нейронная сеть обнаруживала искомые объекты на изображении, нужно обучить её выполнять такой поиск. Первым делом нужно создать набор данных (Dataset), на которых сеть будет обучаться – то есть, настраивать свои весовые коэффициенты. Датасет состоит из двух подборок данных, первая из которых – тренировочная, и на её информации происходит обучение, а вторая подборка данных предназначена для проверки работоспособности и точности работы обучаемой нейронной сети [61 – 65].

При распознавании объектов на изображении нужно выбрать классы, которые будет искать разрабатываемая программа. Эти объекты были собраны вручную, а именно – был сформирован набор данных из 200 изображений.

Чтобы нейронная сеть определяла объекты намного точнее, объекты в тренировочном наборе данных должны быть изображены в разных ракурсах, повернуты в различных плоскостях и находиться под разным освещением.

Далее, с помощью программы LabelImg на изображениях расставляются специальные тэги, а также название классов в формате Pascal VOC. Вся информация должна иметь необходимые данные для обучения [65 – 67].

LabelImg – это программное средство, бесплатный инструмент с открытым исходным кодом для графической маркировки изображений. Он написан на языке Python и использует фреймворк PyQt для графического интерфейса. Эта среда разработки является оптимальной для задач данного проекта.

Следующим этапом является добавление наибольшей части изображений в тренировочный набор данных (Dataset). Все оставшиеся изображения передаются в проверочный набор данных. Примеры изображений, содержащихся в обучающем наборе данных, продемонстрированы на рис. 5.16.

Собрав и обработав изображения с помощью программы LabelImg, следует перейти к обучению нейронной сети с полученным набором данных.

5.4.2. Обучение модели для распознавания объектов на изображении

Для обучения свёрточной нейронной сети необходимо применить несколько основных параметров, одним из таких является *эпоха* (англ. epoch) – это процесс прохождения через нейронную сеть всех тренировочных данных.

Немаловажным параметром также является *батч* (англ. batch) или *пакет*. Так как одна эпоха слишком велика для компьютера, то набор данных (*dataset*) делят на маленькие партии (*batches*). Это небольшой набор примеров, при помощи которых при обучении нейронной сети вносятся необходимые корректировки. Прекращать обучение модели СНС можно только тогда, когда достигнута необходимая точность её работы [63].

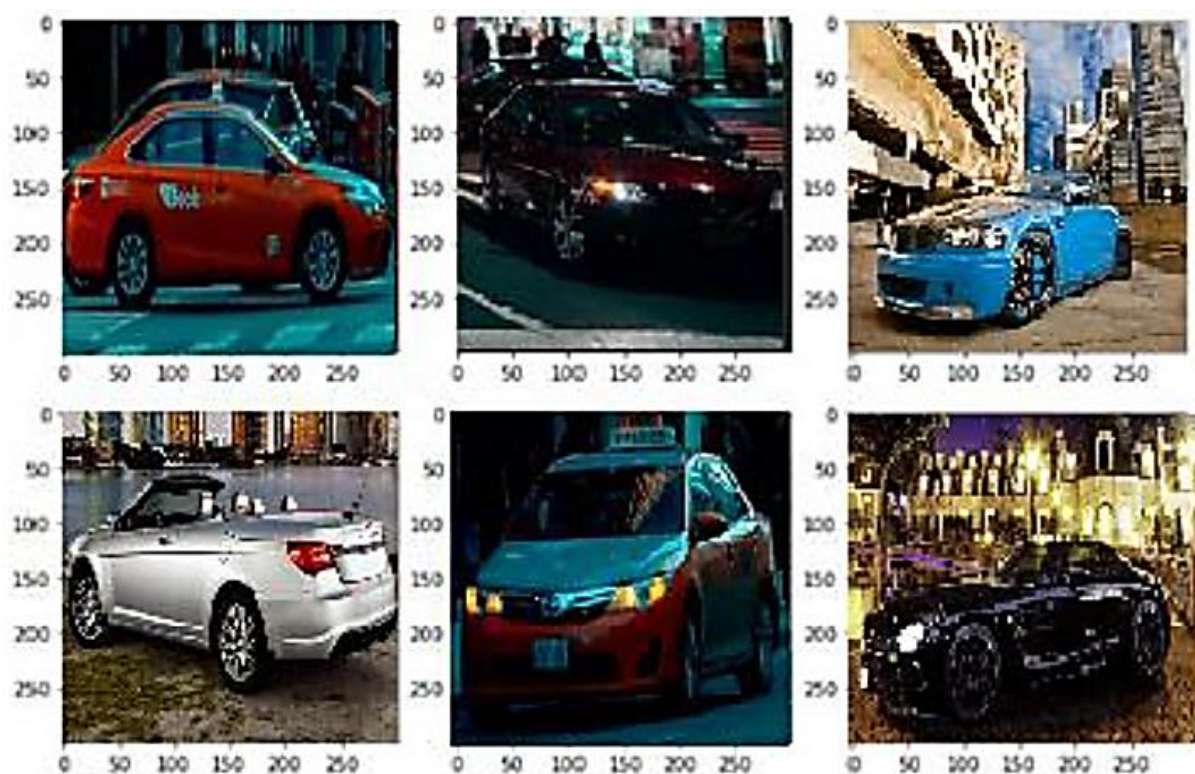


Рисунок 5.16. Пример данных (изображений) из обучающего набора данных

Модель нейронной сети считается обученной только тогда, когда она достаточно точно находит объекты как на тех изображениях, которые входили в обучающий набор данных, так и на тех, которые в этот датасет не входили. Если она работает хорошо только на данных из обучающего набора, то это говорит о том, что необходимо провести её *переобучение* [61 – 65].

Одной из основных проблем нейронных сетей является их переобучение. Необходимость этого процесса показывает, что свёрточная нейронная сеть теряет способность к обобщению. Причины этого заключаются в следующем:

- использовалось недостаточное количество обучающих данных;
- слишком велико число корректируемых параметров, используемых в сети;
- несовершенство архитектуры СНС и необходимость её корректирования.

Эту проблему можно решить: нужно вовремя прекратить обучение сети и использовать уже сохраненные весовые коэффициенты, точность которых достаточно высока. В дальнейшем, нужно найти причины и исправить ошибки.

Обучение свёрточной нейронной сети разрабатываемого проекта было выполнено с использованием бесплатного сервиса от Google, а именно Google Colab Notebook, в котором для командной оболочки используется язык Python, и есть возможность получить доступ к графическим процессам. Результаты обучения нейронной сети на десяти эпохах представлены на рис. 5.17.

Epoch: 1	Time: 2e+02s	Train loss: 5.2744	Train conf: 3.5243	Train loc: 0.6726	Val loss: 3.2448	Val conf: 2.5898	Val loc: 0.6550
Epoch: 2	Time: 2e+02s	Train loss: 4.2240	Train conf: 2.5017	Train loc: 0.6467	Val loss: 3.0579	Val conf: 2.4188	Val loc: 0.6392
Epoch: 3	Time: 2e+02s	Train loss: 4.0390	Train conf: 2.3309	Train loc: 0.6347	Val loss: 2.8330	Val conf: 2.2031	Val loc: 0.6300
Epoch: 4	Time: 2e+02s	Train loss: 3.9137	Train conf: 2.2171	Train loc: 0.6253	Val loss: 2.8345	Val conf: 2.2152	Val loc: 0.6193
Epoch: 5	Time: 2e+02s	Train loss: 3.8785	Train conf: 2.1922	Train loc: 0.6171	Val loss: 2.7481	Val conf: 2.1429	Val loc: 0.6053
Epoch: 6	Time: 2e+02s	Train loss: 3.8071	Train conf: 2.1389	Train loc: 0.6012	Val loss: 2.6774	Val conf: 2.0873	Val loc: 0.5901
Epoch: 7	Time: 2e+02s	Train loss: 3.7719	Train conf: 2.1203	Train loc: 0.5866	Val loss: 2.6505	Val conf: 2.0809	Val loc: 0.5695
Epoch: 8	Time: 2e+02s	Train loss: 3.7283	Train conf: 2.0969	Train loc: 0.5684	Val loss: 2.6397	Val conf: 2.0814	Val loc: 0.5583
Epoch: 9	Time: 2e+02s	Train loss: 3.7216	Train conf: 2.1083	Train loc: 0.5524	Val loss: 2.8003	Val conf: 2.2532	Val loc: 0.5471
Epoch: 10	Time: 2e+02s	Train loss: 3.6329	Train conf: 2.0317	Train loc: 0.5414	Val loss: 2.5064	Val conf: 1.9792	Val loc: 0.5272

Рисунок 5.17. Результат обучения нейронной сети на 10 эпохах

Как видно на рис. 5.18, ошибка нахождения объектов на обоих типах данных уменьшается. Изменение общей ошибки для обучаемой свёрточной нейронной сети за 10 эпох также показано на рис. 5.18.

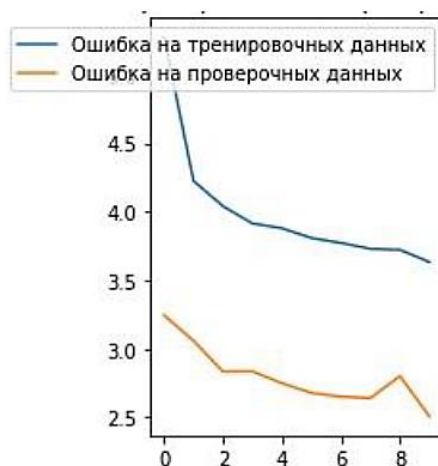


Рисунок 5.18. Изменение ошибки нейронной сети за 10 эпох

Так как набор данных достаточно мал, что обусловлено мощностью используемого компьютера, обучение производилось до тех пор, пока общая ошибка стала приемлемо малой. Наименьшая ошибка, по данным обучения и тестирования достигла 0,4 и 0,5, что говорит, что переобучать модель не нужно.

5.4.3. Программная реализация нейронной сети

После подключения необходимых для работы библиотек, и указания нужного типа выходных данных, была произведена загрузка ранее обученной через Google Colab модели СНС в память ПК, что показано на рис. 5.19.

```
detection_graph = tf.Graph()
with detection_graph.as_default():
    od_graph_def = tf.GraphDef()
    with tf.gfile.GFile(model_file_name, 'rb') as fid:
        serialized_graph = fid.read()
        od_graph_def.ParseFromString(serialized_graph)
        tf.import_graph_def(od_graph_def, name='')

```

Рисунок 5.19. Загрузка предварительно обученной модели свёрточной нейронной сети

Далее загружаются метки классов изображений, после чего картинка преобразуется в нужный массив. Загрузка меток классов показана на рис. 5.20.

```
image_file_name = 'street.jpeg'
image = Image.open(image_file_name)
(im_width, im_height) = image.size
image_np = np.array(image.getdata()).reshape((im_height, im_width, 3)).astype(np.uint8)
plt.figure(figsize=(12, 8))
plt.imshow(image_np)

```

Рисунок 5.20. Загрузка метки классов

После предыдущего этапа добавляется размерность массива и запускается процедура поиска объектов на изображении, с помощью описанного ранее алгоритма. Пример кода запуска поиска объектов показан на рис. 5.21.

```
with detection_graph.as_default():
    with tf.Session() as sess:
        ops = tf.get_default_graph().get_operations()
        all_tensor_names = {output.name for op in ops for output in op.outputs}
        tensor_dict = {}
        for key in [
            'num_detections', 'detection_boxes', 'detection_scores',
            'detection_classes', 'detection_masks'
        ]:
            tensor_name = key + ':0'
            if tensor_name in all_tensor_names:
                tensor_dict[key] = tf.get_default_graph().get_tensor_by_name(
                    tensor_name)
        if 'detection_masks' in tensor_dict:
            detection_boxes = tf.squeeze(tensor_dict['detection_boxes'], [0])
            detection_masks = tf.squeeze(tensor_dict['detection_masks'], [0])
            real_num_detection = tf.cast(tensor_dict['num_detections'][0], tf.int32)
            detection_boxes = tf.slice(detection_boxes, [0, 0], [real_num_detection, -1])
            detection_masks = tf.slice(detection_masks, [0, 0, 0], [real_num_detection, -1, -1])
            detection_masks_reframed = utils_ops.reframe_box_masks_to_image_masks(
                detection_masks, detection_boxes, image_np.shape[0], image_np.shape[1])
            detection_masks_reframed = tf.cast(
                tf.greater(detection_masks_reframed, 0.5), tf.uint8)
            tensor_dict['detection_masks'] = tf.expand_dims(
                detection_masks_reframed, 0)
        image_tensor = tf.get_default_graph().get_tensor_by_name('image_tensor:0')

```

Рисунок 5.21. Часть кода запуска поиска объектов

Следующим этапом является визуализация результата: выводится исходное изображение, на котором объекты выделяются прямоугольниками и масками. Рядом с каждой рамкой выводится название класса объекта и вероятность, с которой объект принадлежит этому классу. код показан на рис. 5.22.

```
vis_util.visualize_boxes_and_labels_on_image_array(  
    image_np,  
    output_dict['detection_boxes'],  
    output_dict['detection_classes'],  
    output_dict['detection_scores'],  
    category_index,  
    instance_masks=output_dict.get('detection_masks'),  
    use_normalized_coordinates=True,  
    line_thickness=8)  
plt.figure(figsize=(24, 16))  
plt.imshow(image_np)
```

Рисунок 5.22. Вывод результата

После загрузки нужного изображения, на котором изображено множество объектов, запускается разработанная программа распознавания объектов. Пример изображения с несколькими объектами показан на рис. 5.23.



Рисунок 5.23. Загрузка метки классов

Результатом работы программы является изображение, на котором видны нужные объекты классов *Car* и *Person*, с рамкой – маской, или ограничивающим прямоугольником, и со значением вероятности, с которой объект соответствует нужному классу. Пример результирующего изображения показан на рис. 5.24.



Рисунок 5.24 – Загрузка метки классов

Очевидно, что сегментация и классификация объектов таких классов как, как *Car* и *Person* происходит довольно быстро, точно и без ошибок.

В результате выполнения данного проекта было разработано программное средство для ПК или ноутбука под управлением ОС Windows. Был реализован функционал, позволяющий находить объекты на изображении: обнаруживать и распознавать класс объекта; отображать рамку – маску и прямоугольник вокруг объекта, вычислять вероятность соответствия объекта своему классу;

Выполнен обзор методов распознавания объектов на изображении.

Для сегментации и классификации найденных объектов на изображении был использован алгоритм Single Shot MultiBox Detector, в основе которого лежат сверточные нейронные сети. Этот метод является оптимальным, так как с его помощью можно найти объекты за один полный проход нейронной сети. При этом алгоритм является достаточно быстрым и точным.

6. МОНИТОРИНГ ДЕСТРУКТИВНОГО ПОВЕДЕНИЯ ОБУЧАЮЩИХСЯ ПРИ ПРОВЕДЕНИИ АТТЕСТАЦИОННЫХ МЕРОПРИЯТИЙ

Сегодня в мире много внимания уделяется искусственному интеллекту, будоражащему энтузиастов из различных сфер науки и техники. За последнее десятилетие он получил стремительное развитие, что обусловлено увеличением вычислительных мощностей компьютеров, нужных для обучения нейронных сетей. Искусственный интеллект представляет собой систему или машину, способную имитировать поведение или функции мыслительных способностей человека, чтобы выбирать оптимальное решение на основе полученного опыта, в ходе её постепенного обучения для выполнения различных задач [76].

Одними из наиболее актуальных и достаточно быстро развивающихся областей искусственного интеллекта являются глубокое машинное обучение и компьютерное зрение. Эти задачи достаточно близко связаны друг с другом.

Машинное обучение (Machine Learning) – это один из обширных разделов ИИ, изучающий методы построения алгоритмов, способных самообучаться за счёт применения решений множества сходных задач. В 1952 году *Артур Самуэль* разработал первую программу способную к самообучению. Автор дал определение термину машинного обучения, которое позже, в более точной формулировке, определил *Т.М. Митчелл*. В 1957 году была предложена первая в мире модель нейронной сети, которая реализовывала алгоритмы машинного обучения. В настоящее время разрабатываются разнообразные системы на основе машинного обучения, предназначенные для *Интернета Вещей*, концепции «*Умного дома*», беспилотных транспортных средств и др. Близким к машинному обучению направлением является компьютерное зрение, в котором также используются алгоритмы, направленные на самообучение.

Компьютерное зрение (Computer Vision) – это область искусственного интеллекта, связанная с анализом видео изображений, в результате которого компьютер способен распознавать объекты и классифицировать их. Это направление относится к молодым, но бурно развивающимся, хотя начало у него было положено ещё в 60–70 годах, когда компьютеры смогли управлять обработкой больших объёмов данных, таких, как изображения. На данный момент компьютерное зрение так же, как и машинное обучение, применяется во многих отраслях человеческой деятельности. В симбиозе эти области способны решать достаточно большое количество современных проблем [77].

В 2020 году мировому сообществу пришлось столкнуться с глобальной пандемией, что сказалось на многих отраслях человеческой деятельности. Это, в свою очередь, ещё в большей степени привело к развитию систем на основе искусственного интеллекта, и в частности, в направлениях машинного обучения и компьютерного зрения. Одной из наиболее затронутых пандемией актуальных отраслей оказалось образование. Кардинальным изменением в этой сфере стал переход на электронный или дистанционный режим обучения, вследствие чего начали применяться и аттестационные мероприятия в режиме онлайн.

Вспоминая самостоятельные, контрольные, зачётные и экзаменационные работы в очном формате, в голову приходит чуткий взор преподавателя, всматривающегося в каждого обучающегося, чтобы он не списывал материалы со шпаргалки. Такая методика сдачи аттестационных работ существует до сих пор, однако она далека от идеала. Помимо всего прочего, формат приёма аттестационных работ во время электронного или дистанционного обучения существенно отличается от традиционного и является более затруднительным.

Преподавателю необходимо следить за каждым обучающимся, однако этот процесс усложняется в связи с ограничением видимой зоны. Так как в большинстве многопользовательских приложений для видеоконференцсвязи предусмотрены совсем небольшие окна отображения, в которых обычно видны лишь голова и плечи обучающегося, то возникает проблема с обнаружением нечестного поведения человека, сдающего аттестацию. В связи с этим возникает задача обнаружения нечестного поведения человека при сдаче зачёта или экзамена в дистанционном или электронном форматах, что решается посредством систем онлайн–мониторинга (прокторинга).

Онлайн–прокторинг – это система наблюдения, или мониторинга, автоматически отслеживающая запрещённое деструктивное поведение или действия обучающегося участника, сдающего аттестационные работы в удалённом формате или с применением дистанционных технологий.

Во время сдачи академических тестов проводится наблюдение по трём каналам: это видеопоток, аудиопоток и запись экрана рабочего пространства. Для обработки и выявления аномального поведения школьника или студента, проходящего аттестационные мероприятия, применяются алгоритмы на основе машинного обучения и компьютерного зрения. При возникновении подозрительного нечестного деструктивного поведения обучающегося, или действий, вызванных экзаменуемым, система должна зафиксировать их и сообщить о нарушениях экзаменатору или выделенному специалисту.

Обычно за процессом прокторинга наблюдает специально обученный сотрудник учебного заведения – *проктор*. Его задачей является выявление нарушений и отслеживание каких-либо ошибок в системе мониторинга. Но такую роль может играть и сам экзаменатор, пройдя специальное обучение по использованию всей системы мониторинга или отдельных её компонентов.

Для проведения подобного типа проведения зачётов или экзаменов требуется несколько компонентов: персональный компьютер или ноутбук с функциями поддержки веб-камеры и микрофона, стабильное Интернет-соединение и пакет программного обеспечения, если используется настольная версия системы, или же веб-браузер, если используется веб-приложение. Перечень таких компонентов сейчас имеется у большинства обучающихся, и затруднений с использованием данной системы возникать не должно.

Подводя итоги, можно сказать, что принцип подобного метода сдачи зачётов или экзаменов заключается в осуществлении как можно более честного проведения аттестационного мероприятия. Для реализации этого метода используются средства машинного обучения и компьютерного зрения, которые в свою очередь отслеживают и сообщают о каких-либо нарушениях.

Таким образом, разработка импортозамещающей компьютерной системы онлайн-прокторинга на основе машинного обучения и компьютерного зрения, предназначенной для образовательных учреждений России, в которых проводятся аттестационные мероприятия, является очень актуальной.

Объектом исследования являются системы онлайн-мониторинга.

Предметом исследования выступают методы и технологии, позволяющие отслеживать различные деструктивные действия в режиме реального времени.

Цель исследования заключается в разработке комплекса программных модулей, отслеживающих поведение обучающихся во время аттестации.

Для осуществления цели нужно решить *следующие задачи*:

1. Изучить теоретические сведения о различных системах мониторинга.
2. Проанализировать существующие методы мониторинга (отслеживания) поведения обучающегося в исследуемых системах прокторинга.
3. Разработать оптимальный комплекс модулей, необходимых для решения поставленной задачи, и определить нужные ресурсы для их реализации.
4. Осуществить программную реализацию разрабатываемого комплекса.

6.1. Аналитический обзор систем онлайн–прокторинга

6.1.1. Общие сведения о системах онлайн–прокторинга

Онлайн прокторинг – это процедура дистанционного сопровождения и контроля во время сдачи онлайн тестирований, зачётов или экзаменов, за процессом которого следит модератор – *проктор*. В качестве наблюдателя, или проктора, может выступать как специально обученный сотрудник учебного заведения, или преподаватель, так и отдельно подготовленный специалист. У него есть возможность следить за действиями обучающихся во время зачёта или экзамена из трёх источников: видео–аудио–потока, с помощью веб–камеры и микрофона, и с помощью трансляции рабочего стола.

Такой метод позволяет уменьшить, и, в идеальном случае, исключить использование шпаргалок, возможность нечестного и деструктивного поведения, а также подтвердить личность обучающегося, независимо от его местоположения, относительно других лиц, сдающих экзамен, и наблюдателя.

На сегодняшний день в России слово *прокторинг* мало кому известно. Опыт использования данного формата очень мал и не всегда положителен. Поэтому стоит обратиться к истории появления данного формата в целом.

Первая система с подобной технологией была изобретена в США в 2008 году. Авторами стали Дон Касснер и Джарред Морган. Перед ними стояла задача в разрешении дилеммы, состоящей в предоставлении надлежащего контроля во время сдачи экзаменов и непрактичностью дальних поездок в учебные заведения. Касснер и Морган приняли решение о разработке нового инновационного, доступного и надёжного способа надзора за процессом экзаменов, который отвечал бы всем академическим требованиям. Данная проблема была решена разработкой системы ProctorU, которая, в свою очередь, содержала необходимый комплекс средств для обеспечения более честного проведения тестирования в режиме онлайн. В будущем, по мере развития технологий, стали появляться более усовершенствованные аналоги и типы систем онлайн–прокторинга, которые используют в своей основе методы и алгоритмы машинного обучения и компьютерного зрения [78].

6.1.2. Классификация систем онлайн–прокторинга

По мере развития компьютерных технологий, системы прокторинга совершенствовались, и в них стали применяться алгоритмы искусственного интеллекта. Сегодня в мире существует несколько типов систем прокторинга, и каждая из них имеет как свои плюсы, так и минусы.

А) Проктор–человек. Данный тип прокторинга подразумевает под собой отсутствие программного модуля. Наблюдение и идентификация проводится в ручном режиме, и проктор сам фиксирует и помечает нарушения. Он может находиться непосредственно вместе с преподавателем, который проводит экзамен, либо наблюдать за тестированием без участия преподавателя. Недостатки: стоимость часа работы специалиста–проктора, проводящего наблюдение (если это не преподаватель) достаточно высока. Затраченное количество денег напрямую зависит от числа задействованных прокторов, количества экзаменов и времени, затраченного на каждый из них. Этот минус является основной причиной низкой популярности данного типа прокторинга. Масштабируемость прокторинга такого типа зависит от количества прокторов и наблюдаемых учащихся, сдающих зачёты и экзамены. Организация процесса при большом количестве прокторов и учащихся достаточно сложна. Кроме того, существует и проблема человеческого фактора: иногда встречаются весьма недобросовестные и непрофессиональные, а то и коррумпированные прокторы, что может сказаться на качестве полученных результатов.

Б) Проктор–компьютер. Это один из самых сложно реализуемых видов прокторинга, в котором используются направления искусственного интеллекта. В процессе наблюдения участвует только компьютер, без вмешательства человека. Модуль ИИ сам идентифицирует личность обучающегося, следит за его действиями, направлением его взгляда, анализирует посторонние шумы и фиксирует нарушение поведения. По окончании тестирования программа собирает отчёты с результатами и формирует оценочную базу. Сложность обуславливается тем, что реализация и настройка данной система очень проблематична ввиду огромного количества данных и коррекции условий, так как программа зачастую может быть излишне конструктивна и принимает многие действия человека, проходящего анализ, за обман или мошенничество.

Недостатки: возможные неточности и ошибочные результаты. В связи с этим рекомендуется присутствие проктора–человека, так как на данном этапе технологического развития даже самый совершенный искусственный интеллект может допускать ошибки. Поэтому данный тип прокторинга используется редко, хотя и имеет наименьшую стоимость среди других.

В) Проктор компьютер человек, синхронный метод. Это комбинированный тип, наиболее точный. Идентификация личности может проходить как с помощью программного модуля, так и в ручном режиме с помощью проктора.

Задачей проктора в целом является обеспечение наибольшей прозрачности хода экзамена. Он может информировать студентов о нарушениях, которые замечает система, или советовать, чего не стоит делать, чтобы не вызвать к себе излишних подозрений. В ходе экзамена ведётся запись по трём каналам: с помощью веб-камеры, рабочего стола и микрофона. Эту информацию в дальнейшем можно проанализировать вручную, если были допущены ошибки в работе проктора-человека или компьютерной системы. Проктор имеет право остановить или вовсе прервать экзамен у всех участников (или у отдельного лица) если эти люди не соблюдают регламент проведения аттестационных работ в онлайн режиме, и демонстрируют деструктивное поведение.

Недостатки: наибольшая стоимость из всех существующих типов прокторинга, а также невозможность предотвращения нарушений.

Г) *Проктор компьютер-человек – асинхронный метод.* Также является комбинированным типом, но с постпроверкой полученных результатов. Идентификация личности и весь остальной процесс проводится программно. Система прокторинга записывает все действия каждого участника по трём каналам, отслеживает запрещённые действия и помечает канал участника, где было зафиксировано нарушение. После окончания процесса тестирования, данные проверяет преподаватель, который в свою очередь подтверждает или опровергает наличие каких-либо серьёзных нарушений. Такой тип прокторинга менее затратен по сравнению с синхронным методом, так как не требует дополнительных ресурсов в виде большого количества прокторов.

Недостаток: нужно время на ручную перепроверку помеченных данных.

6.1.3. Анализ аналогов и применяющихся в них методов отслеживания

Системы онлайн-прокторинга представляют собой большие программные продукты, включающие множественный функционал. Схема работы у всех систем похожа. Все этапы работы систем онлайн-прокторинга представлены на рис. 6.1. Обучающийся студент, сдавая экзамен, может находиться дома.

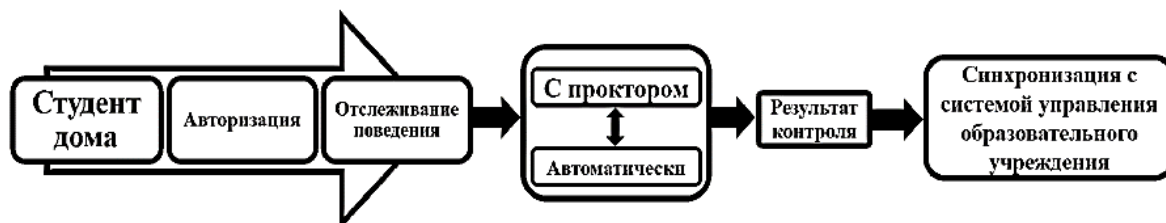


Рисунок 6.1. Этапы работы системы онлайн-прокторинга

Так как целью проекта является разработка комплекса программных модулей, отслеживающих поведение обучающихся во время аттестационных мероприятий (тестирования, зачётов и экзаменов), то для разработки оптимального комплекса необходимо провести анализ уже существующих систем онлайн–прокторинга и определить, какие из компонентов используются в этих системах. Краткий анализ существующих решений, каким–либо образом связанных с отслеживанием поведения людей, представлен в таблице 6.1.

Таблица 6.1. Анализ популярных систем онлайн–прокторинга

№	Наименование модуля	ProctoRU	Responds	Proctorio	Examus
1.	Доступны ли живые наблюдатели	Да	Нет	Нет	Да
2.	Возможность общения с наблюдающими	Да	Н/д	Да	Н/д
3.	Возможность просмотра видеозаписи	Нет	Да	Да	Да
4.	Требуется ли веб–камера	Да	Да	Да	Да
5.	Возможность хранения видеозаписи	Да	Да	Да	Да
6.	Требуется ли студенческий билет	Да	Да	Да	Н/д
7.	Возможность распознавания лиц	Нет	Нет	Да	Да
8.	Отслеживание направления взгляда	Нет	Нет	Да	Да
9.	Оценка положения головы	Нет	Нет	Нет	Нет
10.	Подсчёт людей, находящихся в кадре	Да	Нет	Нет	Нет
11.	Обнаружение посторонних предметов	Да	Нет	Нет	Нет
12.	Обнаружение подмены лица	Нет	Нет	Нет	Нет
13.	Обнаружение открытого рта	Нет	Нет	Нет	Нет

По данным, предоставленным в открытом доступе сети Интернет, можно сделать вывод, что наиболее функциональными системами онлайн–прокторинга, с точки зрения отслеживания поведения экзаменуемых студентов, является зарубежная система «Proctorio» и его отечественный аналог система «Examus». Однако, как в них, так и в остальных системах, отсутствуют некоторые модули, связанные с анализом лицевой части человека, что всё–таки может сказываться на качестве проводимого тестирования. В большей степени эта проблема вызвана сложностью реализации самих модулей, а также их интеграции в готовые системы. В ходе анализа менее известных аналогов также обнаружено отсутствие многих важных компонентов. В связи с этим было принято решение разработать недостающие компоненты, составляющие полный модуль, отвечающий за отслеживание поведения экзаменуемого студента. Модуль, представленный на рис. 6.1, содержит следующие функции:

- обнаружение лица и лицевых ориентиров учащегося студента;
- обнаружение подмены лица экзаменуемого студента на другое лицо;
- отслеживание направления взгляда и положения головы студента;
- обнаружение в кадре посторонних лиц и запрещённых предметов;
- отслеживание состояния открытого рта, что говорит о разговорах.

Итак, рассмотрены основные теоретические сведения, касающиеся систем онлайн–прокторинга. Существует несколько вариантов данных систем, но у каждого из них есть свои плюсы и минусы. Выявлено, что основной задачей подобных систем является отслеживание поведения и действий обучающихся во время аттестационных мероприятий (тестирования, зачётов или экзаменов) и уменьшение вероятности присутствия нечестного и деструктивного поведения студента во время проведения аттестационных мероприятий в режиме онлайн. Проанализированы подобные системы, существующие в Российском образовании, и на их основе определена оптимальная структура разрабатываемого комплекса программных модулей онлайн–прокторинга.

6.2. Алгоритмическое конструирование разрабатываемой системы

Исходя из поставленной цели, необходимо разработать структуру будущего программного комплекса. Так как в дальнейшем компоненты должны иметь возможность интеграции в другие системы, это следует учесть при разработке.

Разрабатывая структуру комплекса, следует опираться на определённые принципы построения архитектуры программных приложений, а именно:

- *эффективность*: программное средство должно выполнять свои функции;
- *расширяемость*: должна быть возможность добавлять в систему новые функции, не нарушая её функционал и основную структуру;
- *гибкость*: быстрое и удобное внесение дополнительного функционала;
- *масштабируемость*: возможность распараллеливания процесса создания программного обеспечения для сокращения времени разработки;
- *код и сопровождение*: в дальнейшем над программным решением могут работать и другие специалисты, для чего нужно правильно структурировать программный код, сделав его читаемым и понятным для других людей;
- *тестируемость*: у программного средства должна быть возможность тестирования кода на выявления ошибок и коррекции функционала.
- *повторное использование*: должна быть обеспечена возможность использования разработанных модулей в разных программных системах.

Для дальнейшей разработки и поддержания работы полноценного приложения использована *инкрементальная модель*. Она представляет собой разработку по частям: проект реализуется и тестируется инкрементно, то есть каждый раз с небольшими добавлениями. Схема модели показана на рис. 6.2.

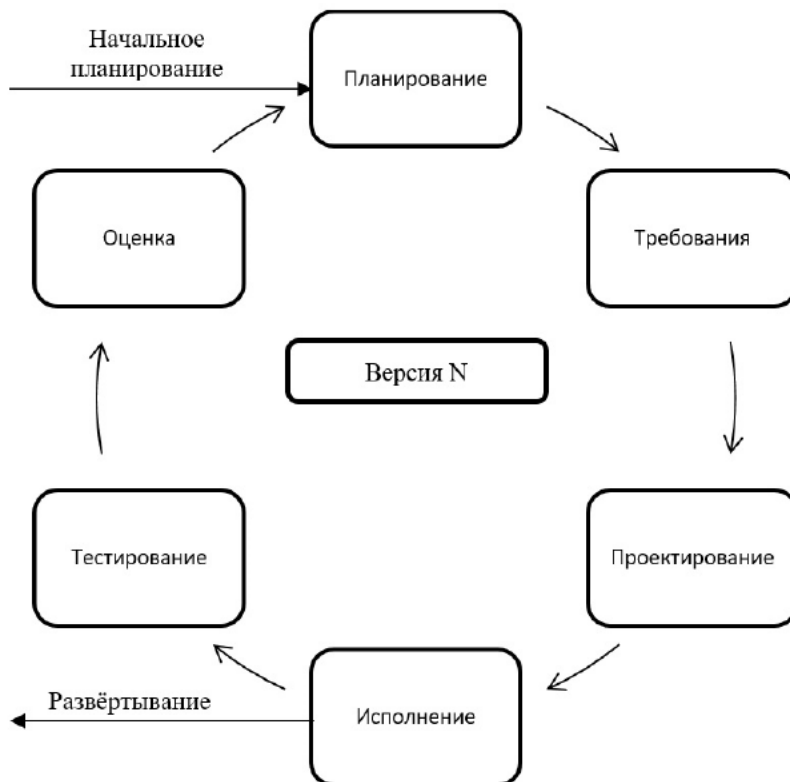


Рисунок 6.2. Схема инкрементальной модели

В первоначальной версии комплекса реализован весь запланированный функционал. В дальнейшем, каждый из компонентов, входящий в данный комплекс, будет корректироваться и дорабатываться по мере необходимости, а также к нему будет добавляться новый функционал и новые модули.

6.2.1. Структура программного комплекса

Структура разрабатываемого программного комплекса должна быть простой и не слишком перегруженной, для того, чтобы в дальнейшем специалист, работающий с этим программным обеспечением, мог с лёгкостью интегрировать в систему необходимый функционал. Кроме того, должна быть возможность как расширения функционала уже имеющихся компонентов, так и создания новых. Опираясь на эти требования и инкрементальную модель, была разработана следующая схема, представленная на рис. 6.3.

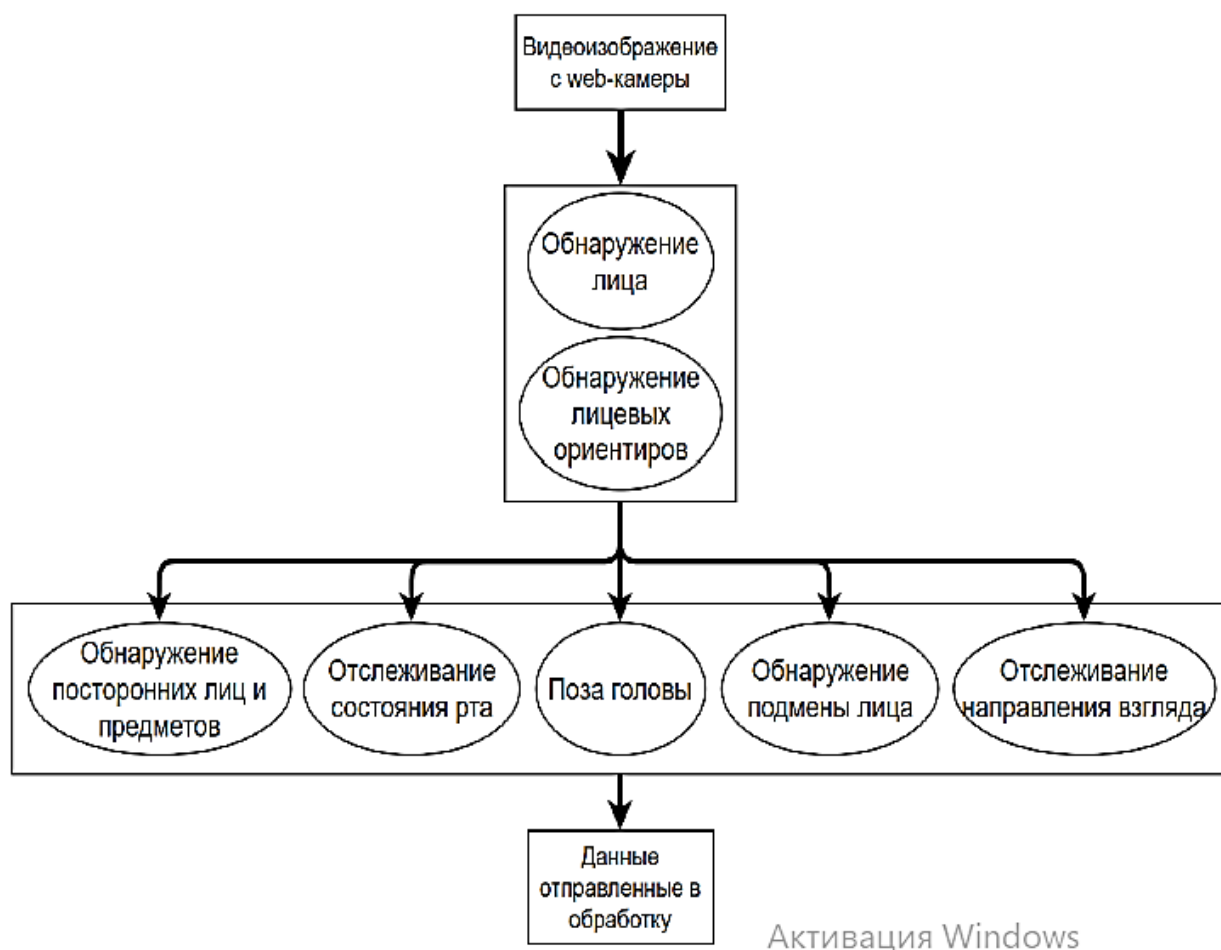


Рисунок 6.3. Схема работы комплекса модулей

Изучив эту схему, можно сделать вывод, что для функционирования компонентов отслеживания различного состояния и поведения объекта наблюдения нужно получать исходные данные от компонента обнаружения лица и лицевых ориентиров, на вход которым подаётся видеоизображения с веб-камеры. Затем полученное видеоизображение будет обрабатываться различными модулями и вести оповещение о поведении студента на экзамене, либо передаваться в обработку полноценной системой.

6.2.2. Анализ библиотек и моделей распознавания образов на изображении

Исходя из цели исследования, первоочередной задачей является реализация одного из основополагающих аспектов компьютерного зрения – распознавание образов (в нашем случае – лица экзаменуемого студента). Для достижения цели необходимо провести анализ существующих библиотек и моделей машинного обучения по распознавания лиц на изображении (рис. 6.4).

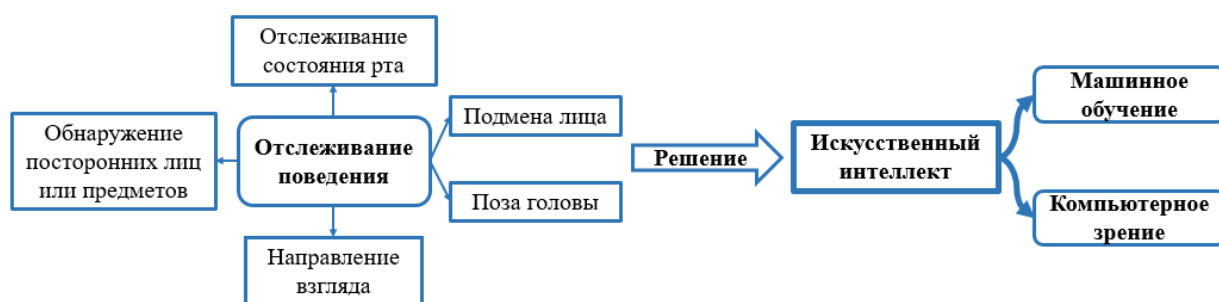


Рисунок 6.4. Этапы создания системы онлайн–прокторинга

Так как на данный момент существует достаточно большое количество подобных библиотек, то было принято решение провести анализ наиболее актуальных и точных решений. В ходе сравнительного анализа были выявлены преимущества и недостатки, описанные в таблице 6.2.

Таблица 6.2. Анализ библиотек по распознаванию образов на изображении

Тип применяемого детектора	Работа в режиме реального времени	Производительность, CPU, fps	Производительность, GPU, fps	Качество детектирования
OpenCV DNN	+	27	Нет данных	Высокое
Open CV HaarCascade	+	29	Нет данных	Низкое
DLib HOG	+	39	Нет данных	Среднее
Dlib CNN	–	–	62	Высокое
+	17	21	Высокое	

По результатам анализа было выявлено, что библиотека Dlib со сверточными нейронными сетями (CHC, или Convolutional Neural Network, CNN) является самой точной, но она не работает в режиме реального времени на процессоре, а это фундаментальное условие для создания комплекса.

Библиотеки MTCNN и Dlib HOG имеют похожие результаты работы, но обе имеют недостатки, связанные с большими изображениями у первой библиотеки (MTCNN) и маленькими изображениями и трудностями распознавания некоторых частей лица у второй библиотеки (Dlib HOG).

Библиотеки OpenCV и HaarCasade имеют худшие результаты в обработке как отдельных изображений, так и видеоматериалов, и предпочтение отдано библиотеке компьютерного зрения OpenCV с интеллектуальным модулем машинного обучения «Глубокие нейронные сети» (Deep Neural Network, DNN).

OpenCV – это библиотека с открытым исходным кодом, использующая сотни алгоритмов компьютерного зрения. Она имеет модульную структуру, что означает включение нескольких статических или общих библиотек. В доступе есть все необходимые модули для работы с изображением и видео.

Достоинством библиотеки является модуль *Deep Neural Network*, который может поддерживать загрузки разных моделей нейронных сетей из разных фреймворков. Модуль не поддерживает тонкую настройку, но хорошо оптимизирован под процессоры Intel, что позволяет получить хорошую производительность при работе с видео в режиме реального времени [79].

Для того, чтобы обнаружить лицо и его ключевые точки на видео, нужно использовать определённую модель нейронной сети, которая способна давать хорошие результаты. Модель позволяет задавать параметры предварительной обработки входного изображения и создает сеть из файла с обученными весами и архитектурой, устанавливает ввод предварительной обработки и выполняет прямой проход в модуле *Deep Neural Network*.

Существует множество готовых моделей нейронных сетей. Для обнаружения лица была выбрана модель «*Opencv_face_detector_uint8.pb*» и конфигурационный файл «*Opencv_face_detector.pbtxt*», поставляемые вместе с модулем DNN *OpenCV*, а также модель «*Caffemodel*» как альтернатива.

Для составления карты ключевых лицевых ориентиров использована модель из библиотеки глубокого обучения *Tensorflow*. Для обнаружения посторонних лиц и предметов на видеоизображении в режиме реального времени применена модель *YOLOv3* фреймворка *DarkNet*.

6.2.3. Алгоритм работы программного комплекса

Для того, чтобы все модули взаимодействовали между собой, но и в тоже время являлись самостоятельными компонентами, реализован компонент с двумя общими модулями – обнаружения ключевых лицевых ориентиров и обнаружения лица. Эти модули используются для реализации необходимого функционала, посредством передачи данных в другие модули.

Обнаружение лица. В выбранном модуле библиотеки *OpenCV* метод поиска лиц основаны на алгоритмах Хаара и *AdaBoost*. Однако они дают не слишком хорошие результаты в скорости и точности обнаружения лиц при различных условиях. Решением этой проблемы стало применение моделей глубоких нейронных сетей на основе фреймворков *Tensorflow* и *Caffe* [80].

Так как модуль DNN имеет возможность поддержки различных фреймворков и моделей нейронных сетей, то в разрабатываемых модулях использоваться будет именно DNN. Модели нейронных сетей дают хорошие результаты по скорости и качеству обнаружения в различных условиях.

Идеей этого модуля будет применение глубокого машинного обучения, вместе со средствами библиотеки компьютерного зрения, для обнаружения на видеоизображении человеческих лиц. В дальнейшем, функции и данные, полученные в этом модуле, будут использоваться в других модулях, на уровень ниже. Алгоритм работы содержит следующее [81]:

- 1) с модуля Deep Neural Network (DNN) на вход программного средства подаётся предварительно обученная модель нейронной сети;
- 2) внутренние алгоритмы, по определённым принципам, обнаруживают область изображения, в которой может находиться человеческое лицо;
- 3) область с найденным лицом копируется, вырезается, выравнивается и уходит на обработку для дальнейшего использования.

Обнаружение лицевых ориентиров. Так как в реальных условиях сделать фронтальное изображение лица человека проблематично, то обычно, после этапа обнаружения лица выполняется дополнительный этап, называемый *выравниванием лица*. В этом процессе на изображении лица необходимо обнаружить характерные точки и отметить определённые области, такие как брови, нос, глаза (уголки глаз), рот (уголки рта) и т.д. Такой метод при обнаружении ключевых точек называется *локализацией лицевых ориентиров*.

Для обнаружения лицевых ориентиров в этом модуле используется модель нейронной сети, построенная с помощью фреймворка Tensorflow.

Алгоритм работы состоит из следующих этапов [81]:

- обнаружение координат лица, которые действуют как область интереса пользователя (Region of Interest – ROI) и извлекаются из изображения;
- преобразование выделенного региона в квадратную форму размером 128×128 пикселей, и передача этой формы в модель нейронной сети;
- модель нейронной сети возвращает 68 ключевых точек, полученных при экстремальных ракурсах, углах наклона и повороте лица, которые могут быть нормализованы до исходного размера изображения.

Полученные ключевые точки лица и функции из данного модуля будут использоваться в модулях на ступень ниже в разработанной структуре.

Обнаружение подмены лица. Принцип работы этого модуля заключается в использовании возможностей различных цветовых пространств, таких как YCrCb и CIE $L^*u^*v^*$, и создании гистограммы в этих каналах.

Цветовое пространство YCrCb или $Y'CrCb$ – это семейство цветовых пространств, использующихся для передачи цветных изображений в компонентном видео и цифровой фотографии. Y – компонент яркости, C_b и C_r являются синим и красным цветоразностными компонентами.

Цветовое пространство CIE $L^*u^*v^*$ было принято, как стандарт Международной комиссии по освещению (CIE) в 1976 году, в качестве простого для вычисления преобразования цветового пространства CIE XYZ 1931 года, стремящегося обеспечить единообразие восприятия. Стандарт широко используется для приложений компьютерной графики, имеющих дело с цветными огнями. Параметр L соответствует яркости цвета, u отвечает за переход от зеленого к красному, v – за переход от синего к фиолетовому.

Цветная гистограмма представляет собой распределение цветовой композиции в изображении. Она демонстрирует появление различных типов цветов и количество пикселей в каждом типе цветов. Связь между гистограммами цвета и яркости заключается в том, что гистограмма цвета может быть выражена как «три гистограммы яркости», каждая из которых показывает распределение яркости каждого отдельного цветового канала красный / зеленый / синий. Гистограммы объединяются в *вектор признаков* $FV = (Y, Cr, Cb, L, u, v)$ размером 1536, который передаётся как входные данные дополнительному классификатору дерева для целей обучения нейронной сети. На рис. 6.5 показана схема работы метода.



Рисунок 6.5. Схема работы метода обнаружения подмены лица

После работы модуля распознавания лица, интересующая пользователя область изображения извлекается, и затем преобразуется в требуемые цветовые пространства. Гистограммы вычисляются и объединяются в вектор признаков, в соответствии с требованиями модели нейронной сети.

Далее, модель предсказывает, является ли человеческое лицо, найденное на изображении, живым, или же фото/видео подделкой (фейком). На рис. 6.6 продемонстрированы примеры выстроенных гистограмм цветовых пространств, упомянутых выше, по поддельному и подлинному изображению.

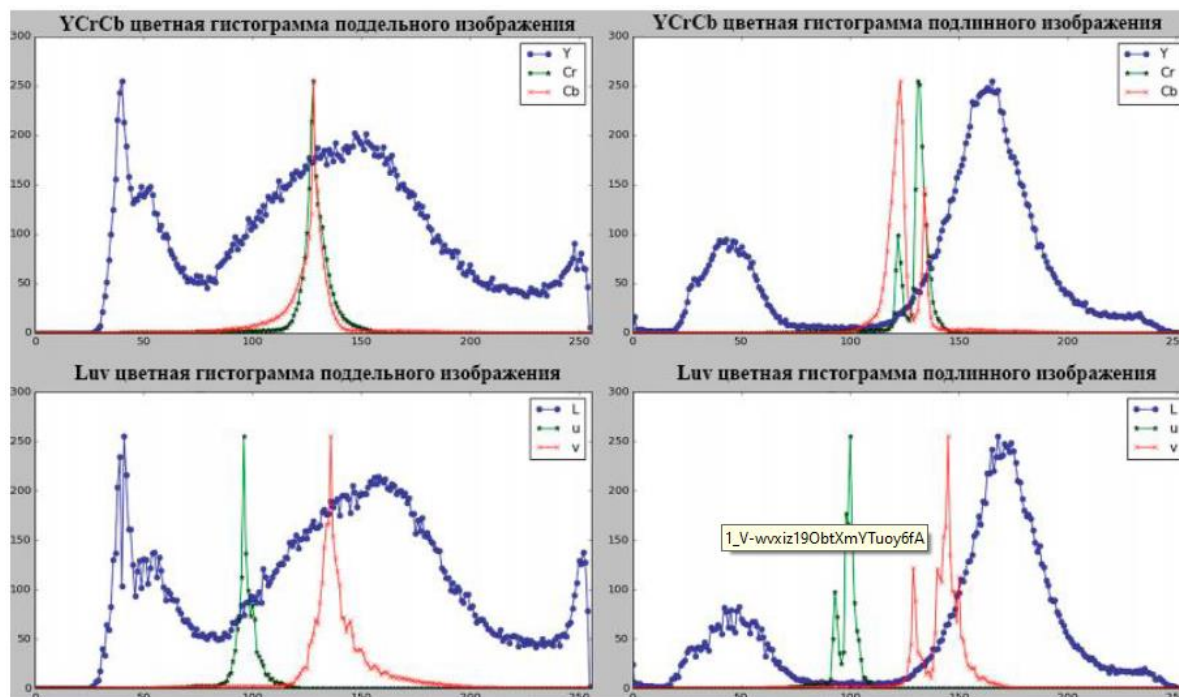


Рисунок 6.6. Пример гистограмм с подлинным и поддельным изображением

Модель нейронной сети, используемая в данном методе, обучалась на базе данных для атак с повторением, содержащей в себе около 1300 видео 50 клиентов при различных условиях освещения. Она имеет как печатные фотографии, так и фото– и видеоизображения лиц клиентов, полученных на цифровых устройствах. Программный модуль разрабатывается с учётом обработки видеоизображений, выполненных с веб–камеры.

Отслеживание направления взгляда. Для решения подобной задачи также необходимо получить лицевые ориентиры. На рис. 6.7 изображены ключевые точки лица, возвращаемые моделью лицевых ориентиров из модуля, описанного выше. Эти точки лица будут использованы в разрабатываемом комплексе.

Далее, имея набор ориентиров, на изображении необходимо найти глаза. Данные, полученные в ходе работы модуля обнаружения лицевых ориентиров, хранятся в массиве ключевых точек. Нужно хранить координаты (x, y) точек левого и правого глаза из этого массива, и передавать их в функцию, которая будет обрисовывать их на маску, используя средства библиотеки OpenCV.

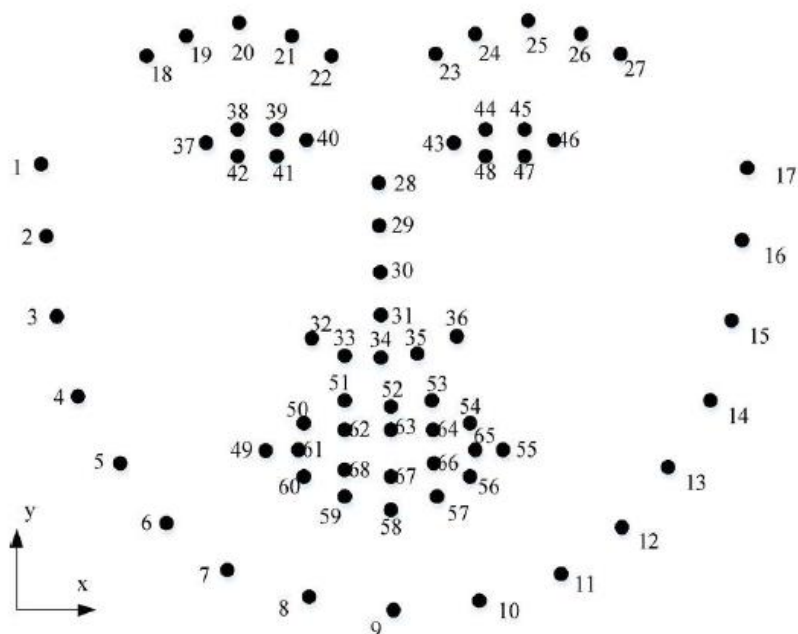


Рисунок 6.7. Форма лица с 68 ключевыми точками

В результате, получится изображение с областью между точками, заполненной определённым цветом. Далее, необходимо отделить глазное яблоко от остальной части глаза, которая находится на маске, и преобразовать результат в оттенки серого цвета для определения порога, который используется для создания двоичной маски. Следующий шаг будет состоять в нахождении оптимального порогового значения, при котором можно отделить глазное яблоко от остальной части глаза, а затем найти его центр. Для этого реализован специальный *трекбар* (TrackBar, в виде ползунка), управляющий пороговым значением, для ручного режима настройки, в зависимости от освещения. Конечным этапом станет использование найденных контуров глазных яблок и нахождение их центра для определения направления взгляда.

Таким образом, в определённой функции модуля находится так называемый центр глаза, и в зависимости от положения найденного контура, от этого центра будет определяться направление взгляда человека.

Отслеживание положения головы. В основе идеи оценки позы человека и положения его головы находится поворот стандартной 3D-модели на определённый угол, до тех пор, пока 2D-проекция трёхмерных точек на модели не совпадёт с точками на тестируемом изображении. На рис. 6.8 продемонстрирован принцип конвертации 3D-модели на 2D-проекцию.

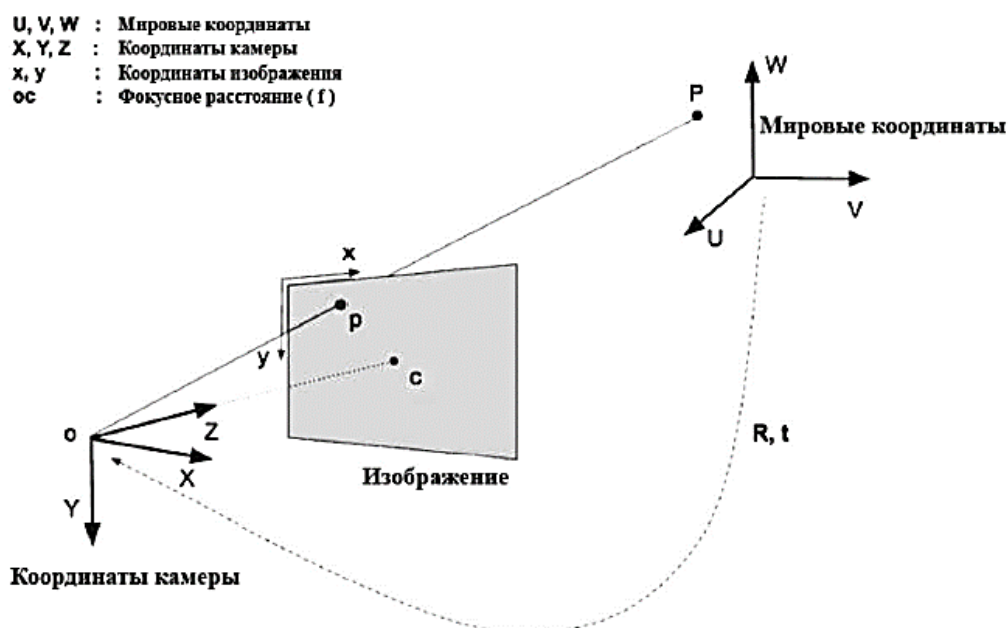


Рисунок 6.8. Проекция 3D-модели на 3D-изображение

Первым этапом в этом модуле является нахождение лица на изображении с помощью модуля «Обнаружение лица». Второй этап – получение координат лицевых ориентиров. Для этого используется модель нейронной сети модуля «Нахождение лицевых ориентиров». Далее, нужно получить координаты человеческого лица, являющегося областью интереса, извлечь их из видеоизображения, преобразовать в квадратную форму размером 128×128 и передать в модель нейронной сети, которая возвращает 68 ключевых точек. Эти точки будут нормализованы до исходного размера видеоизображения. На третьем этапе необходимо рассчитать трёхмерную позу человека на изображении, для чего необходимо знать следующую информацию [82]:

1) 2D координаты нескольких точек. Необходимо знать координаты (x, y) местоположения важных точек на изображении. Это координаты черт лица человека – его подбородка, уголков глаз, кончика носа, уголков рта.

2) 3D расположение этих точек на изображении. Для решения этой задачи была использована произвольная система отсчёта:

- кончик носа: $(0, 0, 0, 0, 0, 0)$; подбородок: $(0, 0, -330, 0, -65, 0)$;
- левый угол левого глаза: $(-225.0, 170.0, -135.0)$;
- правый угол правого глаза: $(225.0, 170.0, -135.0)$;
- левый угол рта: $(-150.0, -150.0, -125.0)$;
- правый угол рта: $(150.0, -150.0, -125.0)$;

3) внутренние параметры камеры.

Взяв стандартные трёхмерные координаты данных лицевых ориентиров, следует оценить их рациональные и поступательные векторы на кончике носа. Для более точной оценки используются внутренние параметры камеры: фокусное расстояние и оптический центр. После получения необходимых векторов, необходимо спроецировать эти трёхмерные точки на двумерную поверхность, которая и является видеоизображением.

Обнаружение посторонних лиц и объектов. Этот модуль является как наиболее сложным в реализации, так и наиболее требовательным в вопросах производительности работы программного обеспечения. Для обнаружения посторонних лиц и объектов будет использоваться предварительно обученная модель нейронной сети и веса YOLOv3, обученные на наборе данных COCO. Эта модель может точно классифицировать 80 объектов в 3D масштабе, используя для этого свёрточную нейронную сеть DarkNet-53, составляющую основу системы обнаружения объектов YOLO в режиме реального времени [83].

Система YOLO использует абсолютно новый подход по сравнению с другими аналогами. Она применяет единственную нейронную сеть ко всему изображению. Эта сеть делит изображение на сетку с ячейками размером $S \times S$. Каждая из ячеек может содержать несколько объектов для распознавания. Кроме того, каждая ячейка прогнозирует количество ограничительных рамок / якорных рамок (Bounding box / Anchor boxes) и доверительное значение (Confidence value) для каждой области, ограниченной рамкой. Это значит, что значение определяет вероятность нахождения какого-либо объекта в области.

Если ячейка сетки не имеет какого-то определённого объекта, то доверительное значение этой области должно быть низким. Каждая ячейка отвечает за предсказание вероятности класса объекта (ячейка содержит только вероятность нахождения объекта в ней). К примеру, если ячейка предсказывает изображение собаки, то это не означает, что картинка собаки действительно находится в ней. Это означает только то, что если присутствует объект, то он с какой-то вероятностью является изображением собаки. На рис. 6.9 показана комбинация методов, применяющихся в YOLO [83].

В результате работы алгоритма, системой YOLO будет определяться индекс класса объекта на видеоизображении, и, если какой-либо из индексов будет равен какому-либо из нарушающих объектов – подаётся оповещение.

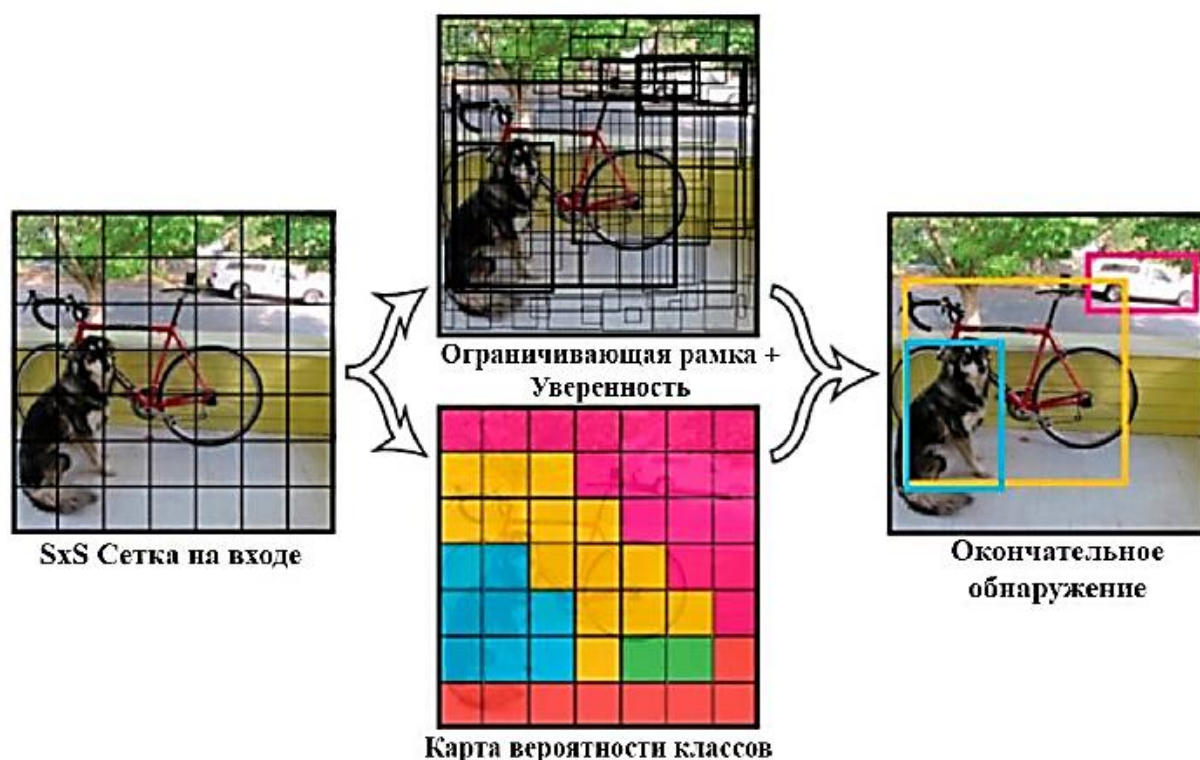


Рисунок 6.9. 3D схема методов обнаружения объектов в системе YOLO

Отслеживание состояния рта. Такой модуль, по своему принципу работы, очень схож с модулем отслеживания направления взгляда студента. В нём также используются ключевые точки лица, что обусловлено тем фактом, что объект наблюдения должен сидеть прямо перед камерой (как во время аттестационного мероприятия) для фиксирования расстояния от камеры до губ. Это необходимо для того, чтобы алгоритм мог рассчитывать расстояние между внешними и внутренними точками губ (5 внешних пар и 3 внутренние пары точек). На рис. 6.10 такие пары точек представлены с нумерацией.

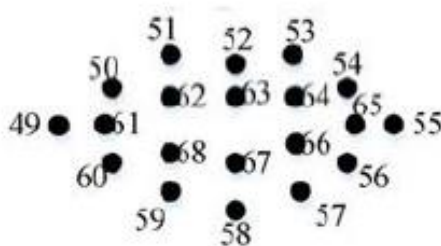


Рисунок 6.10. Нумерация внутренних и внешних пар точек на губах

Если расстояние между парами точек после фиксации увеличивается и превышает определённое значение, то модуль фиксирует нарушение и выводит оповещение о недопустимых действиях студента на экзамене.

Нужно отметить, что нумерация ключевых точек может изменяться при использовании различных языков программирования. Так, в языке Python нумерация индексов начинается с 0, потому обрабатываться будут элементы массива с номера 48 до 67. Данные могут отправляться на постобработку на сервер, для составления отчётов или для дальнейшего использования.

Итак, произведён анализ существующих библиотек по распознаванию объектов на изображении, были выявлены их достоинства и недостатки. Определены оптимальные библиотеки языка Python, пригодные для разработки программного комплекса. Разработана структура и функционал каждого из модулей, описаны алгоритмы работы каждого модуля.

6.3. Программное конструирование системы онлайн–прокторинга

Для программной реализации комплекса потребуется определить, какой из языков программирования лучше подойдёт для решения поставленной задачи. Также необходимо определить и список используемых библиотек.

6.3.1. Выбор языка программирования и среды разработки

В настоящее время большинство библиотек, фреймворков и моделей нейронных сетей поддерживаются языком программирования Python. Он является наиболее распространённым языком для работы с нейронными сетями, так как большая часть низкоуровневых библиотек и фреймворков написаны на высокоуровневых языках C и C++, имеющих высокое быстродействие и оптимальное использование памяти. Python хорошо подходит для описания слоёв нейронной сети, загрузки данных, запуска обработки и отображения результатов. Ещё одним его достоинством является тот факт, что большинство моделей нейронных сетей, опубликованных в сети Интернет, написаны именно на Python, что облегчает работу с ними при разработке новых программ [84].

С помощью расширения Cython можно оптимизировать код Python и реализовать в нём специфические черты, относящиеся к C/C++. Стоит упомянуть, что у Python имеются библиотеки CUDA и cuDNN, позволяющие использовать ресурсы графических процессоров (GPU). Средой разработки для реализации проекта была выбрана интегрированная среда PyCharm, которая предоставляет средства для анализа кода и графической отладки, а также поддерживает различные инструменты разработчика на разных платформах.

6.3.2. Используемые библиотеки

В ходе исследования было принято решение использовать [84 – 86]:

OpenCV – библиотеку алгоритмов компьютерного зрения с открытым исходным кодом, предназначенную для обработки различных изображений, и содержащую численные алгоритмы общего назначения.

Tensorflow – библиотеку с открытым исходным кодом, предназначенную для глубокого машинного обучения, а также для решения задач построения и обучения нейронных сетей, с целью автоматизации классификации и нахождения образов, достигая качества человеческого восприятия.

Keras – открытая нейросетевая библиотека, написанная на языке Python, и являющаяся надстройкой над фреймворком Tensorflow. Предназначена для упрощения использования интерфейсов и предоставления оперативности в работе с нейронными сетями глубокого машинного обучения [85].

NumPy – библиотека с открытым исходным кодом, созданная для работы с многомерными массивами и высокоуровневыми математическими функциями.

Math – основной модуль, предназначенный для работы с математическими функциями в языке программирования Python, совместимый с библиотеками.

Joblib – инструмент для облегчения конвейерной обработки в Python, прозрачного кэширования функций на диск и простых параллельных вычислений.

Для увеличения производительности работы программ могут быть использованы специальные программно–аппаратные средства, использующие ресурсы графических процессоров, от компании Nvidia – CUDA и cuDNN.

6.3.3. Программная реализация

Основными модулями в программном комплексе являются модули «Обнаружение лица» и «Обнаружение лицевых ориентиров».

В модуле «Обнаружение лица» устанавливаются пути к файлам модели и конфигурационному файлу. Далее идёт поиск лица и отрисовка области.

Функции, содержащиеся в модуле «Обнаружение лица»:

- `get_face_detector (modelFile, configFile, quantized)`: эта функция производит инициализацию и поиск графических файлов, содержащих изображения лиц;
- `find_faces (img, model)`: функция осуществляет поиск лица с применением модели и конфигурационного файла (конфиг–файла);
- `draw_faces (img, model)`: функция отрисовки области по найденному лицу.

В модуле «Обнаружение лицевых ориентиров» устанавливается путь к модели и конфиг-файлам. Происходит поиск ключевых точек и их отрисовка на области лица. Функции модуля «Обнаружение лицевых ориентиров»:

- `get_landmark_model (saved_model)`: получение модели лицевых ориентиров;
- `get_square_box (box)`: функция отвечает за получение квадратной области;
- `move_box (box, offset)`: функция отвечает за перемещение квадратной области;
- `detect_marks (img, model, face)`: отвечает за обнаружение ориентиров;
- `draw_marks (image, marks, color (0, 255, 0))`: отрисовка лицевых ориентиров.

Указанные функции и параметры, которые они возвращают, используются другими модулями. Так, в модуле «Отслеживание направления взгляда» используются модель и ориентиры, получаемые в модулях на ступень выше. Они инициализируются и применяются в цикле с обработчиком веб-камеры.

Функции, содержащиеся в модуле «Отслеживание направления взгляда»:

- `eye_on_mask (mask, side, shape)`: создание региона интереса на маске, включая размер глаз, нахождение крайних точек каждого глаза;
- `find_eyeball_position (end_points, cx, cy)`: функция, отвечающая за нахождение на изображении глаза человека позиции глазного яблока;
- `contouring (thresh, mid, img, end_points, right = False)`: обнаружение самого большого контура лица на изображении, разделённого на пополам, а затем – нахождение положения глаз человека на этом контуре;
- `process_thresh (thresh)`: обработка порогового значения направления глаз;
- `print_eye_pos (img, left, right)`: нахождение стороны, в которую смотрит глаз;
- `nothing (x)`: создание трекбара для изменения порогового значения.

Модуль «Обнаружение подмены лица» содержит одну функцию и цикл, в котором обрабатываются данные из функции и потоковое видео с веб-камеры:

- `calc_hist (img)`: отвечает за вычисление гистограммы изображения и его нормализацию для охвата цветового диапазона от 0 до 255.

Назначение модуля «Отслеживание положения головы» сводится к задаче проецирования 3D-точек трёхмерного изображения на двумерное 2D-изображение с помощью рациональных и поступательных векторов на кончике носа. В данном модуле содержатся следующие функции:

- `get_2d_point (rotation_vector, translation_vector, camera_matrix, val)`: функция возвращает трёхмерные 3D-точки в виде двумерных 2D-точек;

- `draw_annotation_box (img, rotation_vector, translation_vector, camera_matrix, color = (255, 255, 0), line_width = 2)`: функция отрисовывает поля аннотации;
- `head_pose_points (img, rotation_vector, translation_vector, camera_matrix)`: функция возвращает координаты (x, y) для отрисовки полей аннотации.

Далее следует цикл, обрабатывающий точки в поле аннотации по условиям, заданным заранее, и вывод результатов отслеживания положения головы.

Модуль «*Отслеживание состояния рта*» работает по тому же принципу, что и модуль отслеживания взгляда, но не требует обработки порогового значения. Модуль имеет два цикла, имеющих следующее назначение:

1. Фиксацию расстояния от камеры до рта экзаменуемого студента (с функцией обнаружения лица и лицевых ориентиров);
2. Обработку и отслеживание изменения расстояния между контрольными точками в области рта, что отслеживает подозрительные движения губ.

Модуль «*Обнаружение посторонних лиц и предметов*» работает на другой модели, так как с его помощью необходимо производить обнаружение не только лиц нескольких человек, но и других предметов (например, запрещённого на экзамене мобильного телефона) с чем хуже справляются прошлые модели. Для этого используется нейронная сеть DarkNet и модель YOLOv3 [83].

В этом модуле содержится большое количество функций, поэтому далее будут перечислены только основные из них, со своими параметрами [84]:

- `weights_download (out='models /yolov3.weights')`: функция отвечает за загрузку весов нейронной сети из директории, или из сети Интернет;
- `load_darknet_weights (model, weights_file)`: загрузка весов в нейронную сеть;
- `draw_outputs (img, outputs, class_names)`: функция отрисовки выхода;
- `darknetConv (x, filters, kernel_size, strides=1, batch_norm=True)`: функция обработки доверительного значения в нейронной сети Darknet;
- `darknet (name=None)`: основная функция, создающая нейронную сеть;
- `yoloConv (filters, name=None)`: доверительное значение системы YOLO;
- `yolo_boxes (pred, anchors, classes)`: функция, отвечающая за получение ограничительных рамок для фиксации найденного изображения.

Далее, в цикле обрабатывается видеопоток, в котором фиксируются объекты, находящиеся в файле классов. Если какой-либо из индексов объектов, обнаруженных на изображении, совпадает с индексом объекта в файле, то система оповещает об этом, также, как и если ни одного из объектов наблюдения, входящих в классы обнаружено не было [85, 86].

6.3.4. Тестирование программного средства и сравнение результатов

Актуальность применения систем онлайн–прокторинга подтверждается, кроме ранее указанных факторов, тестированием систем распознавания лиц в режиме реального времени (рис. 6.11). При вынужденном переходе на удалённый формат аттестационных мероприятий, вероятность нечестного поведения лиц, не обладающих достаточным уровнем подготовки, достаточно высока. Поэтому проведение зачётов и экзаменов в режиме он–лайн весьма эффективно.

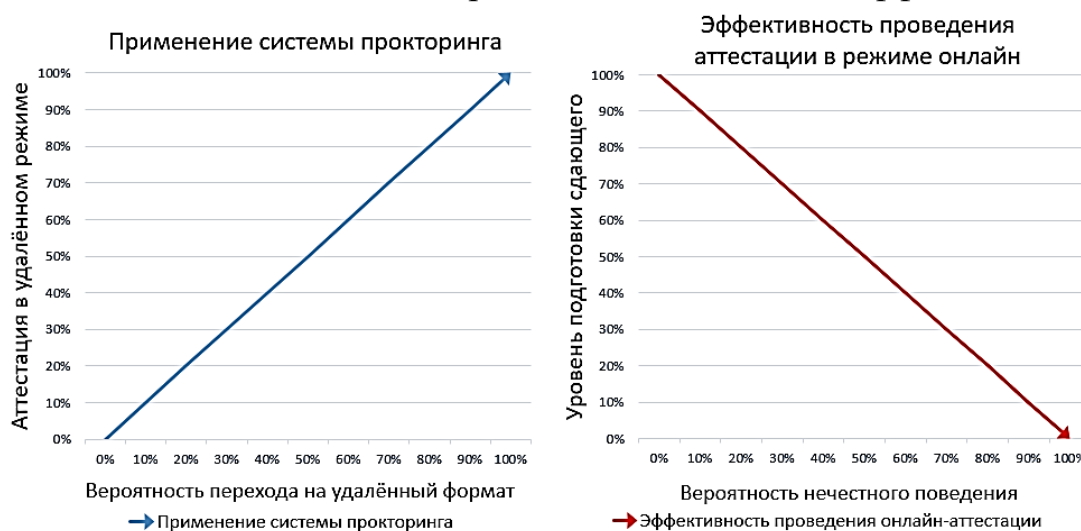


Рисунок 6.11. Тестирование актуальности применения систем онлайн–прокторинга

На рис. 6.12 показан результат тестирования различных модулей систем (Haar, Dnn, Mtcnn, DLib) распознавания лиц в режиме реального времени.

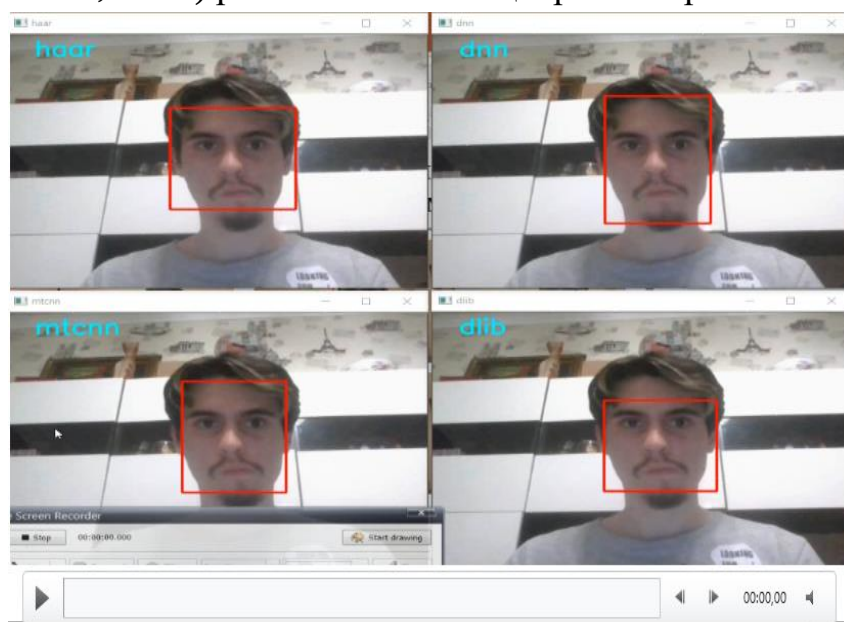


Рисунок 6.12. Тестирование систем распознавания лиц в режиме реального времени

На рис. 6.13 показаны фазы работы программного комплекса

3 ФАЗЫ РАБОТЫ КОМПЛЕКСА:

- 1) Сбор данных
- 2) Анализ и распознавание ориентиров
- 3) Распознавание частей лица через веб-камеру



Рисунок 6.13. Схема реализуемого комплекса системы распознавания лиц

На рис. 6.14 продемонстрированы разработанные программные модули.



В таблице 6.3 показаны преимущества разработанного комплекса он-лайн прокторинга – мониторинга поведения экзаменуемого студента.

Особенности модулей прокторинга	Собственная разработка	Proctoru	Respondus	Proctorio	Alproctor	Examus
Проверка подмены лица	+	-	-	+	-	-
Оценка позы головы	+	-	-	-	-	-
Отслеживание направления взгляда	+	-	-	+	-	+
Распознавание посторонних лиц и предметов	+	+	-	-	-	+
Обнаружение открытого рта	+	-	-	+	-	-

7. НЕЙРОННАЯ СЕТЬ ДЛЯ РАСПОЗНАВАНИЯ ДЕСТРУКТИВНЫХ ЧЕРТ ХАРАКТЕРА ЧЕЛОВЕКА ПО ЕГО РУКОПИСНОМУ ПОЧЕРКУ

Идея возможности изучать характер человека по его рукописному почерку существует уже давно. Известно, что идентификация личности по почерку тесно связана с индивидуальностью каллиграфии – с уникальностью проявления характера человека в его способе начертания букв. Эта характеристика является персональной для разных людей. Установлено, что на каждом этапе обучения каллиграфии, а затем во время закрепления навыков письма, существует некая зависимость между способами начертания каллиграфических символов и внешними и внутренними факторами, которые воздействуют на людей [87].

Методы психологического исследования характера человека по почерку применяются на протяжении многих веков. Однако *графология* часто вызывала сомнения в научности своих методов. Тем не менее, психология развивается быстрыми темпами. В Интернете существуют Web–сайты, где все желающие пользователи могут изучить свой психологический тип и свойства нервной системы людей по почерку, с помощью разработанных новых методик.

Существуют и соответствующие компьютерные программы. Тем не менее, до сих пор методы составления психологического портрета человека остаются закрытыми и не автоматизированными, а ведь рукописный почерк, как и биометрия, является уникальным источником информации, позволяющим дать характеристику его владельцу с психологической стороны, основываясь не на чьём-то субъективном мнении, а на объективных свойствах личности, отраженных в письме. Однако ни одна из программ не работает с рукописным почерком напрямую. Подобные системы получают данные о каллиграфии какого-либо человека, оцениваемые пользователем, что влечёт за собой существенные недостатки: субъективную оценку пользователя, ввиду его недостаточной подготовки в вопросах анализа рукописного текста [87].

Пользователь не всегда может полноценно оценить уровень того или иного свойства. Если определить наклон почерка, типы которого четко определены, достаточно легко, то с таким свойством, как округлость букв, для определения которой необходимо вычислить количество дугообразных и угловатых элементов и рассчитать их доленое соотношение, возникают трудности. Даже профессиональные *графологи* порой могут не согласиться на оценке какой-нибудь особенности каллиграфии, которая вызывает споры.

Графология, как одно из направлений науки о психологии личности, основанная на определении черт характера человека по его почерку, развивается уже больше 500 лет, несмотря на скептическое отношение к ней многих учёных. Однако именно в век компьютерных технологий она может сделать большой скачок вперед: сегодня для анализа почерка уже можно применить новые методы искусственного интеллекта, в том числе нейронные сети. Методы цифровой обработки изображения сегодня используются повсеместно, а просканированные экземпляры рукописного почерка людей можно хранить в специализированных базах данных. Всё это вместе взятое, даёт возможность систематизировать собранные экземпляры рукописного почерка, и привести их к виду, удобному для анализа. Машинное обучение (Machine Learning) сегодня – одна из самых быстро развивающихся областей информатики. Благодаря широкому распространению цифровых устройств, о машинном обучении заговорили как о революционном способе решения задач, ранее не поддающихся решению, таких, как классификация и анализ графических данных, распознавание изображений (рукописного почерка), определение психологических характеристик личности, в том числе – деструктивных, прогнозирование возможно опасного поведения человека, и многих других.

Выявление потенциально преступных наклонностей человеческой личности очень важно, например, в криминалистике, так как это позволит полиции применять своевременные меры по профилактике преступлений.

Объектом исследования проекта являются нейросетевые технологии распознавания черт характера человека по его рукописному почерку.

Предметом исследования являются алгоритмы и методы искусственного интеллекта, предназначенные для распознавания черт характера человека по его рукописному почерку, и определения степени деструктивности личности.

Целью проекта является создание программного средства распознавания черт характера человека по его рукописному почерку, а также определение степени деструктивности его наклонностей. Задачи исследования включают: анализ имеющихся модулей распознавания черт характера по рукописному почерку; формулирование основных требований к разработке модуля, обзор основных программных средств, предназначенных для разработки модуля, а также разработка алгоритма распознавания рукописного почерка и создания программного модуля, реализующего этот алгоритм.

7.1. Анализ существующих систем и их применение

7.1.1. Обзор систем других приложений

Графология – это учение, согласно которому существует устойчивая связь между рукописным почерком и индивидуальными особенностями человеческой личности. Задача состоит в определении таких особенностей.

На текущий момент времени на российском рынке программного обеспечения отсутствует программное средство, способное работать с оригиналом почерка, несмотря на то, что компьютерные технологии предоставляют многие возможности. Например, существует возможность сканирования образцов рукописного текста и дальнейшая их обработка для приведения к виду, удобному для анализа. Разработка программного средства, позволяющего анализировать рукописный текст, является своевременной.

В ходе поиска аналогов были выявлено несколько сайтов, позволяющих пройти тест на изучение своего же рукописного почерка. Подобные системы не дают компьютеру возможность самому обработать рукописный текст. Вместо этого, Web–сайты предлагают пользователю самостоятельно оценить характеристики своего рукописного почерка и выбрать заранее заготовленные, весьма поверхностные варианты свойств собственного характера. Например, на рис. 7.1 можно увидеть интерфейс подобного сайта *Инсайт–клиника*.

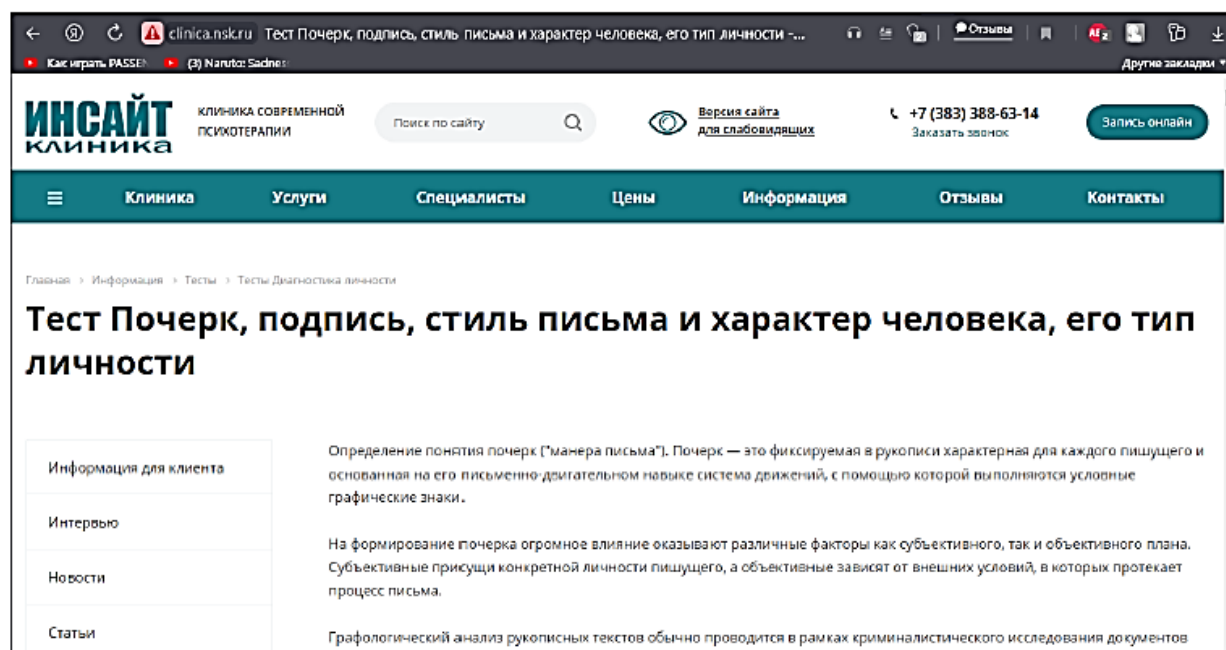


Рисунок 7.1. Внешний вид интерфейса Web–сайта «Инсайт–клиника»

Если пользователь способен оценить характеристики своего рукописного почерка самостоятельно, то этот сайт ему вполне подходит. Инсайт–клиника предлагает пройти тест, состоящий из 7 вопросов, на каждый из которых предоставляется от 5 вариантов ответов. Вопросы являются характеристиками написанного текста, будь то структура почерка или же координация письма. После прохождения теста предлагается заполнить таблицу, в которой в дальнейшем будут систематизированы результаты, то есть выписаны коды полученных ответов. После этого нужно наложить шаблон психологических типов и сделать вывод. При наложении кода психотипа на сформированную таблицу, необходимо искать соответствия. Максимальное несоответствие составляет 7 единиц. В дальнейшем, пользователю необходимо, с помощью указанной на сайте методички, и сформированной таблицы с результатами, провести анализ почерка, попутно выявляя черты собственного характера, относящиеся к тем или иным свойствам рукописного текста [88].

Инсайт–клиника также предлагает пройти краткий неформальный тест для анализа своего почерка. В тесте предлагается ответить на большое количество вопросов, а затем провести подсчёт баллов, набранных за каждый ответ. Имея точное число баллов и сопоставив с предлагаемым диапазоном, можно составить некий психологический портрет собственной личности.

Интересной альтернативой выступает Web–сайт «*JungLand.ru*», имеющий автоматический подсчёт баллов в ходе онлайн–тестирования и выдачу результатов, интерфейс которого представлен на рис. 7.2.

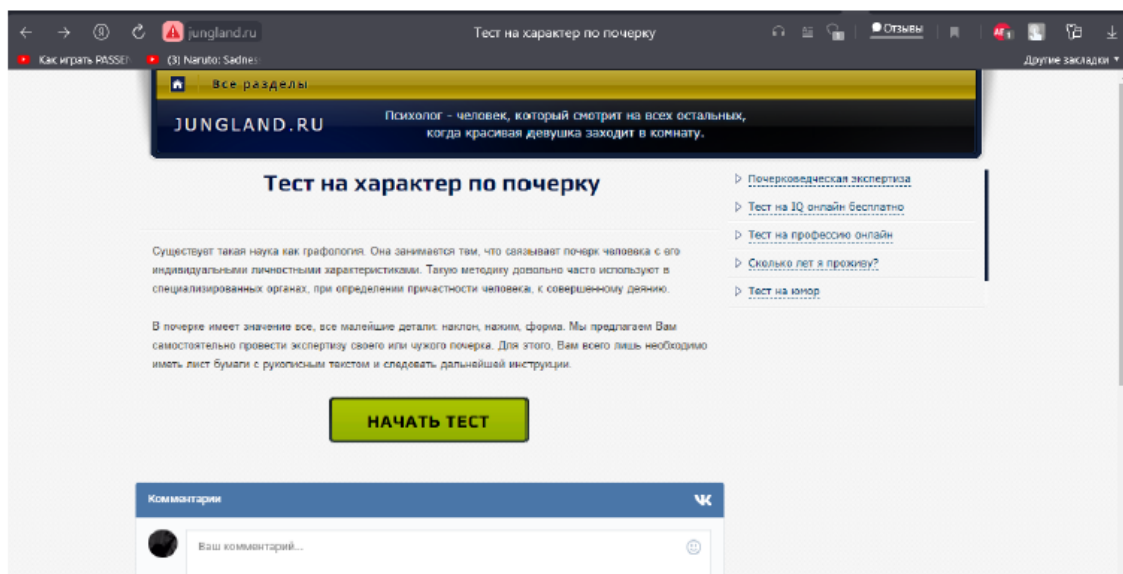


Рисунок 7.2. Внешний вид интерфейса Web–сайта «JungLand.ru»

Как и у предыдущего сервиса, здесь отсутствует автоматизация процесса анализа рукописного текста, написанного пользователем, и возможность работы с оригиналом почерка. Автоматизация процесса анализа рукописного текста на этом сайте не планируется и в дальнейшем. «JungLand» предлагает пройти онлайн–тестирование, состоящее из 7 вопросов об основных характеристиках рукописного текста, таких, как: направление и наклон почерка; размер букв; размашистость и сила нажима. После прохождения, теста сервис выдаст результат в виде полной характеристики рукописного почерка пользователя. Результат будет достаточно приблизительным, поверхностным и спорным, так как пользователю неизвестна методика анализа его почерка [89].

Ещё один Web–сайт «Onedio», можно увидеть на рисунке 1.3.

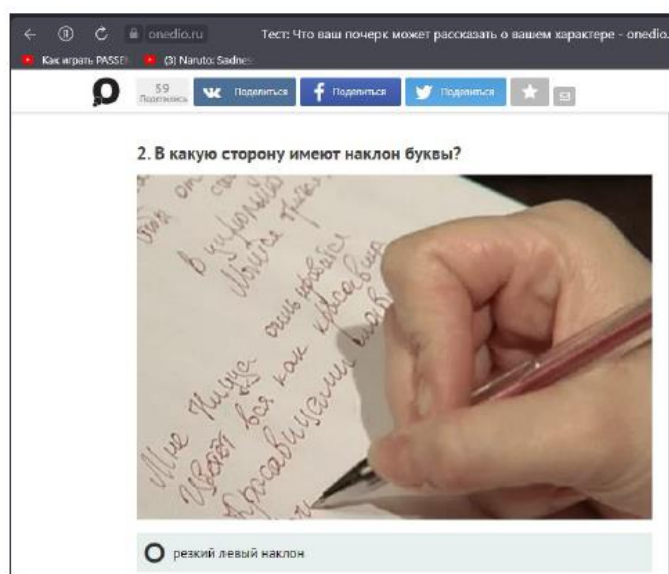


Рисунок 7.3. Внешний вид интерфейса Web–сайта «Onedio»

Сервис «Onedio», так же, как и «JungLand», предоставляет собой онлайн–тестирование с автоматическим подсчетом и выдачей результата.

В отличие от «JungLand», «Onedio» даёт пользователю возможность посмотреть на визуальный пример текста, связанного с одной из характеристик, чтобы он смог понять, о каком свойстве его почерка идет речь в тесте.

7.1.2. Анализ основных возможностей приложений

Графологический анализ – это процесс выявления индивидуально–психологической вариативности почерка [87]. Исходя из этого, возникает необходимость главной функции, объединяющей все рассмотренные сервисы и предмет исследования – автоматизации процесса графологического анализа.

Данной возможностью не обладают рассмотренные ранее сервисы такого типа. Однако это не единственное требование, необходимое для корректного функционирования графологического анализа. Кроме него, требуется [88]:

- работа с оригиналом рукописного текста, написанного пользователем;
- получение информации о дополнительных характеристиках рукописного почерка пользователя, таких, как: нажим, угол базовой линии, размер письма, межстрочный интервал, давление пера, наклон букв и др.;
- строгое соответствие каждой характеристике почерка – одного значения характеристики личности, подтверждаемого графологическим анализом.

Этот перечень является минимальным набором средств и возможностей для нормального функционирования разрабатываемого программного средства, дополняя собой другие функции графологического анализа. Предлагаемое программное средство обладает рядом достоинств в следующих случаях:

- при необходимости работы с оригиналом текста;
- для обучаемости программного средства за счёт нейронных сетей, глубокого машинного обучения и увеличения наборов данных (датасетов);
- в случае автоматизированности процесса графологического анализа;
- для учёта характеристик оригинального рукописного текста (почерка автора) с помощью встроенных функций графологического анализа;
- в случае предварительной автоматической обработки текста с помощью встроенных библиотек, предназначенных для обработки символов.

В дальнейшем, программное средство можно переобучить для решения других задач, помимо распознавания черт характера. Его можно использовать в криминалистике, для решения задач судебно-почерковедческой экспертизы.

7.1.3. Сравнительная характеристика приложений

Для понимания специфики работы и построения наиболее объективной картины сервисов графологического анализа нужно провести сравнительный анализ их характеристик. Он позволит понять проблематику наилучшим образом, выявить недостатки, которые необходимо исправить и достоинства, которые стоит улучшить. Анализ таких характеристик приведён в таблице 7.1, где последняя колонка, содержит информацию о разрабатываемой авторами системе. Все представленные программные продукты, кроме авторского, не поддерживают автоматизированность графологического анализа.

Однако, для удобства пользования программным средством, это необходимо, так как сервисы, присутствующие на рынке, используют лишь тестирование от лица конечного пользователя, что, в свою очередь, не может гарантировать корректность выдаваемого психологического портрета.

Таблица 7.1 – Сравнение сервисов графологического анализа

Критерий	Инсайт – клиника	JungLand	Onedio	Разработка авторов ML– Graphology
Автоматизированность графологического анализа	Нет	Нет	Нет	Да
Работа с оригиналом рукописного текста	Нет	Нет	Нет	Да
Анализ характеристик рукописного почерка	Нет	Нет	Нет	Да
Открытость исходного кода ресурса (Open Source)	Нет	Нет	Нет	Да
Бесплатность версии	Да	Да	Да	Да

В табл. 7.1 последняя колонка, содержит информацию о разработанной авторами системе – предмете исследования. Все представленные продукты, кроме авторского, не поддерживают автоматизированность графологического анализа. Однако это необходимо для удобства пользования программным средством, так как сервисы, присутствующие на рынке, используют лишь тестирование от лица конечного пользователя, что, в свою очередь, не может гарантировать корректность выдаваемого психологического портрета.

Открытость исходного кода (Open Source) может позволить сторонним разработчикам помочь в улучшении предлагаемого программного средства и Интернет–ресурса, и создать свои собственные модификации [89].

7.2. Выбор программных средств системы графологического анализа

7.2.1. Программная среда PyCharm

PyCharm является одной из самых востребованных интегрированных сред программирования (IDE) языка программирования Python. Эта среда разработана компанией JetBrains, известной своей разработкой IDE для языков Java и Kotlin–«IntelliJ IDEA», и IDE для языка JavaScript–«WebStorm».

Также существует возможность поддержки Web-разработки со стороны высокоуровневого Web-фреймворка Django языка Python, что является дополнительным преимуществом. Эта программная платформа позволяет быстро создавать безопасные и поддерживаемые Web-сайты. Django обладает множеством функций Web-разработки, что значительно облегчает работу.

Множество факторов делают PyCharm самой полной и всесторонней интегрированной средой разработки для работы с языком Python. Доступный как кроссплатформенное приложение, PyCharm совместим с платформами Linux, MacOS и Windows, и обеспечивает поддержку как версий Python 2 (2.7), так и Python 3 (3.5 и выше). PyCharm поставляется с множеством модулей, пакетов и инструментов, чтобы ускорить разработку программ [90].

PyCharm можно настроить в соответствии с требованиями разработки, и личными предпочтениями разработчика. Кроме анализа кода, PyCharm имеет: графический отладчик; интегральный модульный тестер; интеграцию систем управления версиями (VCSs); поддержку науки о данных с помощью диспетчера пакетов с открытым исходным кодом Anaconda, который широко используется для анализа данных, научных вычислений и предиктивной аналитики.

Помимо этих функций, PyCharm предлагает следующие возможности:

- генерацию кода, для создания специфических кодовых конструкций;
- справочную информацию по программному коду, с целью мгновенного доступа к документации программного интерфейса приложения (API), советы по использованию различных программных сущностей и т. д.;
- помощь в импорте недостающих и отсутствующих библиотек;
- быстрые исправления для оптимизации программного кода;
- языковые инструменты для процесса быстрой разработки, отладки, запуска, тестирования и развертывания программных приложений;
- языковые инъекции для взаимодействия с поддерживаемыми языками программирования внутри атрибутов, тегов или строковых литералов;
- живые шаблоны для расширения и внедрения аббревиатур в сложные языковые кодовые конструкции, нужные при написании программного кода.

7.2.2. Язык программирования Python

Python — язык программирования высокого уровня, это интерпретатор, созданный для производства легкого в написании и читаемости кода и для обеспечения кроссплатформенности программного обеспечения [91].

В языке Python реализованы все основные парадигмы современного программирования: функциональное, структурное, аспектно–ориентированное, объектно–ориентированное и императивное программирование [92].

К основным особенностям языка можно отнести: наличие динамических типов объектов; автоматизированное управление всей выделенной памятью; механизмы отслеживания и обработки исключений; реализацию вычисления многопоточных алгоритмов; структуры данных высокого уровня; модульность.

Существует, поддерживаемый профессиональным сообществом, каталог программного обеспечения, написанного на языке Python – PyPI. Репозиторий содержит программные пакеты, готовые к установке в операционные системы. Он стал стандартом для разработчиков на языке Python. По состоянию на конец 2021 года, в нём содержится более 300 тысяч программных пакетов [93].

7.2.3. Библиотека NumPy

Эта библиотека для языка Python, предназначенная для взаимодействия с массивами. NumPy сама частично написана на языке Python, но большинство фрагментов, требующих быстрых вычислений, написаны на языках C или C++. Библиотека имеет возможности для использования её в области линейной алгебры, преобразований Фурье, работы с матрицами и др. [94].

NumPy создана в 2005 году Т. Олифантом. Проект является открытым исходным кодом, а, следовательно, его можно использовать без ограничений. В Python есть *списки*, которые служат *массивами*, но они обрабатываются достаточно медленно. NumPy может предоставить объект массива, который работает до 50 раз быстрее, чем традиционные *списки–массивы* Python.

Объект `array` в NumPy называется `ndarray`, он предоставляет множество вспомогательных функций, которые делают работу с ним достаточно легкой. Подобные массивы часто используются в науке о больших данных (big data), где особо важны скорость обработки и выделяемые ресурсы компьютера.

Массивы NumPy хранятся в памяти компьютера в одном непрерывном месте, в отличие от *списков–массивов* языка Python. Поэтому вычислительные процессы могут манипулировать ими с высокой эффективностью. Такое поведение в информатике называется *локальностью отсчёта*. Это основная причина, по которой NumPy работает намного быстрее, чем списки. Кроме того, он оптимизирован для работы с новейшими архитектурами процессоров.

7.2.4. Библиотека *Math*

Язык Python предоставляет модуль, специально разработанный для математических операций более высокого уровня. Модуль *Math* – это важный компонент, предназначенный для работы с математическими операциями, поставляемый в комплекте со стандартным выпуском Python. Большинство функций этого модуля представляют и продолжают линию математических функций платформы C. Поскольку его основные функции написаны на *CPython*, то модуль *Math* эффективен и соответствует стандарту C.

Практические применения модуля *Math* включают [93]:

- расчёт комбинаций и перестановок с использованием факториалов;
- вычисление высоты полюса с помощью тригонометрических функций;
- расчёт радиоактивного распада с помощью экспоненциальной функции;
- расчёт кривой подвесных мостов с помощью гиперболических функций;
- решение кубических, интегральных и дифференциальных уравнений;
- моделирование периодических функций, таких, как звуковые и световые волны, с использованием тригонометрических функций, и др.

Модуль *Math* содержит множество предопределенных констант. Наличие доступа к этим константам даёт ряд преимуществ: не нужно кодировать их вручную, что экономит много времени; они обеспечивают согласованность всего программного кода. Модуль включает в себя известные математические константы и важные значения: *Pi*; *Tau*; Euler's number; *Infinity*; *NaN* и др.

7.2.5. Библиотека *OpenCV*

Это библиотека с исходным кодом, предназначенная для обработки и классификации изображений. *OpenCV* имеет модульную структуру [94]:

- ядро – модуль, предназначенный для определения основных структур информации, в том числе – многомерных массивов и начальных функций, используемых всеми остальными модулями [95];
- модуль обработки изображений – графический компонент, включающий в себя линейную и нелинейную выборку изображений, геометрические трансформирование цветового диапазона, гистограммы и т. д.;
- видеоанализ – компонент, основывающийся на алгоритмах оценивания движения, разницы фона и отслеживания движения объектов;

- базовые алгоритмы многовидовой геометрии, калибровки одиночной камеры и стереокамеры, оценивания положения и местонахождения объекта, алгоритмы стерео последовательности и элементы 3D–реконструкции;
- основные детекторы признаков, дескрипторы и их сопоставители;
- обнаружение объектов и экземпляров predetermined классов.

7.3. Алгоритмическое конструирование программного средства

Оптимальная модель работы приложения представлена на рис. 7.4.

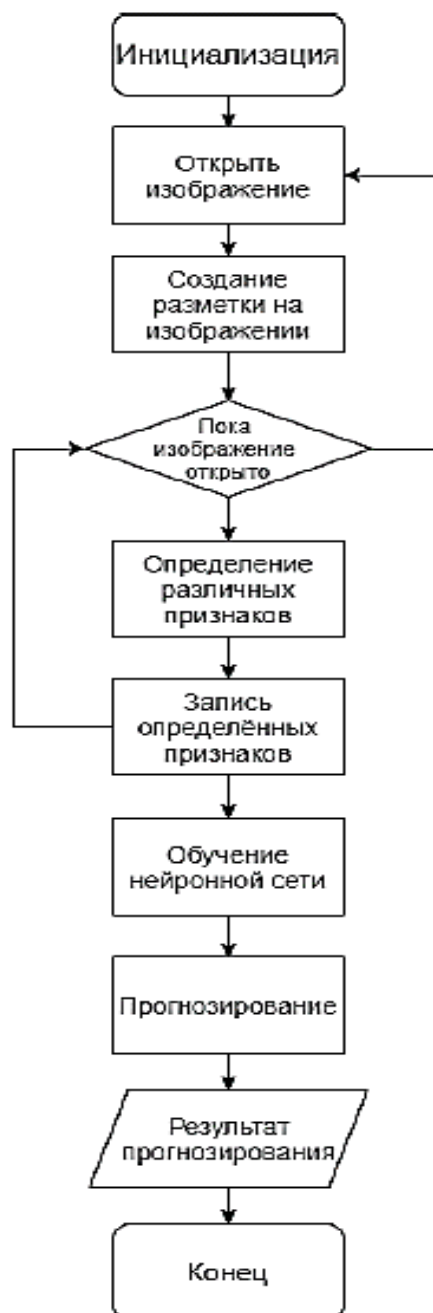


Рисунок 7.4. Алгоритм работы программного средства

Работа программного приложения заключается в следующем: первым делом загружается фотография оригинала текста, написанного рукописным почерком некоего человека. Далее, необходимо провести создание разметки изображения. Для этого потребуется использовать две функции вычисления горизонтальной и вертикальной проекций соответственно.

Функции выполняют подсчет пикселей в горизонтали строк и столбцов в вертикали, затем возвращают список, содержащий сумму пикселей в строках и столбцах. После создания разметки рукописного текста нужно определить различные индивидуальные признаки начертания букв. Это такие функции, как определение базового угла; верхнего поля; размера письма; межстрочного интервала; интервала между словами; давления пера; наклона текста и др.

Все эти признаки, вместе взятые, формируют некое психоаналитическое соотношение между рукописным почерком его владельца и особенностями психики его личности, что выражается в стиле его письма (рис. 7.5).



Рисунок 7.5. Графологическое соотношение, демонстрирующее устойчивую связь между рукописным почерком и индивидуальными особенностями личности

В процессе работы функций определения характеристик почерка также используются функции обработки изображения. Например, часто применяются функции двусторонней фильтрации, медианной фильтрации (для удаления шумов), изменения цветового пространства, поиска и выравнивания контуров изображения по горизонтали и по вертикали, и т. д.

Таким образом, после определения признаков, значения приводятся к дискретным величинам, и записываются для дальнейшего обучения нейронной сети. Используя результаты вычислений, применяется метод прогнозирования черт характера автора рукописного текста. Далее, на основе всех перечисленных выше значений, выполняется графологический анализ личности автора.

В качестве примера можно привести образец рукописного почерка российской императрицы Елизаветы Петровны, найденный в её личной переписке, и результат графологического анализа её личности (рис. 7.6):



Якошья димитри
верная раба
Елизавета

Скачущие, кривые, объёмистые, но тонкие, изящные, изысканные буквы, характеризуют личность и глубину души автора подписи. Высокомерность, капризность, но в то же время благородство, доброта и веселый нрав императрицы выражают и определяют буквы всей её подписи, от заглавной буквы до окончания подписи.

Рисунок 7.6. Рукописный текст и результаты графологического анализа характеристик личности российской императрицы Елизаветы Петровны

7.4. Разработка и кодирование программного средства

При создании программного средства прежде всего были подобраны наборы данных (датасеты), содержащие изображения различных способов рукописного написания русских букв, и результаты психологического анализа, формирующие графологические соотношения. Полученные зависимости демонстрируют устойчивую связь между рукописным почерком человека и его индивидуальными особенностями – личности, написавшей этот текст.

Затем, используя файл с исходными данными для обучения нейронной сети, формируется список значений, пригодный для обучения нейронной сети. Для начала, используя метод `isfile()`, выполняется проверка того, является ли указанный путь к файлу существующим. Далее, извлекаются все функции, путём вызова инструкции `extract`, и сохраняется значение необработанных функций инструкцией `raw_feature_list`. Эти действия показаны на рис. 7.7.


```

import os
import extract

os.chdir("images");
files = [f for f in os.listdir('.') if os.path.isfile(f)]
os.chdir("../")

page_ids = []
if os.path.isfile("raw_feature_list"):
    print ("Info: raw_feature_list already exists.")
    with open("raw_feature_list", "r") as label:
        for line in label:
            content = line.split()
            page_id = content[-1]
            page_ids.append(page_id)

with open("raw_feature_list", "a") as label:
    count = len(page_ids)
    for file_name in files:
        if(file_name in page_ids):
            continue
        features = extract.start(file_name)
        features.append(file_name)
        for i in features:
            label.write("%s\t" % i)
        print>>label, ''
        count += 1
        progress = (count*100)/len(files)
        print str(count)+' '+file_name+' '+str(progress)+'%'
    print "Done!"

```

Рисунок 7.7. Запись необработанных значений

Далее, нужно сформировать список дискретных значений для дальнейшего обучения классификатора. Они открываются в файле `raw_feature_list`, с использованием метода `split()`, нумеруются значения в строке, и для каждой присваивается определенная функция характеристики рукописного почерка. Функции определены в файле `categorize`. После использования функций дискретные значения и комментарии записываются в список `feature_list`. Код, соответствующий этим действиям, представлен на рис. 7.8.

Для вычисления дискретных значений были использованы функции, описанные в файле `categorize`. Код функций представлен на рис. 7.9, *a* и *b*. Важным файлом является `extract`. Он содержит основные функции для работы с обработкой изображения при помощи библиотеки OpenCV [95].

Важным файлом является `extract`. Он содержит основные функции для работы с обработкой изображения при помощи библиотеки OpenCV [96].

Функция *двусторонней фильтрации*, представленная на рис. 7.10, использует метод `bilateralFilter()`. Этот метод фильтрации очень эффективен для удаления шума, а также сохраняет острые края изображения [97].

```

import categorize

if os.path.isfile("feature_list"):
    print ("Error: feature_list already exists.")

elif os.path.isfile("raw_feature_list"):
    print ("Info: raw_feature_list found.")
    with open("raw_feature_list", "r") as raw_features, open("feature_list", "a") as features:
        for line in raw_features:
            content = line.split()

            raw_baseline_angle = float(content[0])
            raw_top_margin = float(content[1])
            raw_letter_size = float(content[2])
            raw_line_spacing = float(content[3])
            raw_word_spacing = float(content[4])
            raw_pen_pressure = float(content[5])
            raw_slant_angle = float(content[6])
            page_id = content[7]

            baseline_angle, comment = categorize.determine_baseline_angle(raw_baseline_angle)
            top_margin, comment = categorize.determine_top_margin(raw_top_margin)
            letter_size, comment = categorize.determine_letter_size(raw_letter_size)
            line_spacing, comment = categorize.determine_line_spacing(raw_line_spacing)
            word_spacing, comment = categorize.determine_word_spacing(raw_word_spacing)
            pen_pressure, comment = categorize.determine_pen_pressure(raw_pen_pressure)
            slant_angle, comment = categorize.determine_slant_angle(raw_slant_angle)

            features.write("%s\t" % str(baseline_angle))
            features.write("%s\t" % str(top_margin))
            features.write("%s\t" % str(letter_size))
            features.write("%s\t" % str(line_spacing))
            features.write("%s\t" % str(word_spacing))
            features.write("%s\t" % str(pen_pressure))
            features.write("%s\t" % str(slant_angle))
            features.write("%s\t" % str(page_id))

```

Рисунок 7.8. Инициализация необходимых полей

Гауссовский метод работает хуже, так как он не учитывает одинаковую интенсивность пикселей, а также является ли пиксель крайним.

```

1  def determine_baseline_angle(raw_baseline_angle):
2      comment = ""
3      # falling
4      if(raw_baseline_angle >= 0.2):
5          baseline_angle = 0
6          comment = "DESCENDING"
7      # rising
8      elif(raw_baseline_angle <= -0.3):
9          baseline_angle = 1
10         comment = "ASCENDING"
11     # straight
12     else:
13         baseline_angle = 2
14         comment = "STRAIGHT"
15
16     return baseline_angle, comment
17

```

Рисунок 7.9, а. Часть функций Categorize

```

18 def determine_top_margin(raw_top_margin):
19     comment = ""
20     # medium and bigger
21     if (raw_top_margin >= 1.7):
22         top_margin = 0
23         comment = "MEDIUM OR BIGGER"
24     # narrow
25     else:
26         top_margin = 1
27         comment = "NARROW"
28
29     return top_margin, comment
30
31 def determine_letter_size(raw_letter_size):
32     comment = ""
33     # big
34     if (raw_letter_size >= 18.0):
35         letter_size = 0
36         comment = "BIG"
37     # small
38     elif (raw letter size < 13.0):

```

Рисунок 7.9, б. Часть функций Categorize

```

21 def bilateralFilter(image, d):
22     image = cv2.bilateralFilter(image,d,50,50)
23     return image

```

Рисунок 7.10. Двусторонняя фильтрация

Функция *инвертирования двоичного порога* представлена на рис. 7.11. Она используется для изменения цветового пространства, а конкретно – для преобразования в оттенки серого. Также используется метод *threshold* для установки порогового значения пикселей [95–97].

```

31 def threshold(image, t):
32     image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
33     ret,image = cv2.threshold(image,t,255,cv2.THRESH_BINARY_INV)
34     return image

```

Рисунок 7.11. Инвертирование двоичного порога

Следующая функция *эрозии объектов* на изображении предназначена для размывания объектов переднего плана. Обычно она выполняется на бинарных изображениях. На вход этой функции подается два параметра: *первым* является исходное изображение, а *вторым* – структурный элемент или ядро, которое и определяет характер результата операции эрозии.

```

def erode(image, kernelSize):
    kernel = np.ones(kernelSize, np.uint8)
    image = cv2.erode(image, kernel, iterations=1)
    return image

```

Рисунок 7.12. Использование функции эрозии изображения

Функции *разметки изображения* представлены на рис. 7.13. Для подсчета суммы пикселей на изображении и дальнейшем использовании этих значений, следует воспользоваться функциями горизонтальной и вертикальной проекции. Функция горизонтальной проекции подсчитывает сумму пикселей в каждой строке и возвращает список–массив. Функция вертикальной проекции также подсчитывает сумму пикселей, но в столбцах, и также возвращает список.

```

98
99 def horizontalProjection(img):
100     (h, w) = img.shape[:2]
101     sumRows = []
102     for j in range(h):
103         row = img[j:j+1, 0:w] # y1:y2, x1:x2
104         sumRows.append(np.sum(row))
105     return sumRows
106
107 def verticalProjection(img):
108     (h, w) = img.shape[:2]
109     sumCols = []
110     for j in range(w):
111         col = img[0:h, j:j+1] # y1:y2, x1:x2
112         sumCols.append(np.sum(col))
113     return sumCols
114

```

Рисунок 7.13. Горизонтальная и вертикальная проекции

Функция *извлечения давления пера* представлена на рис. 7.14. Реализация этой функции заключается в преобразовании цветового пространства в оттенки серого, затем – в инвертировании изображения, пиксел за пикселом. Далее, происходит применение двусторонней фильтрации и использование двоичного порога до нуля. Наконец, нужно сложить все нулевые значения пикселей в изображении и разделить их на общее количество, чтобы найти среднее значение пикселей во всем изображении. Далее, основываясь на концентрации темных пикселей, можно сделать вывод о среднем давлении пера.

Далее, необходимо запустить файл *train_predict*, который с помощью библиотеки *Skicit Learn* обучает классификаторы SVM распознаванию черт характера человеческой личности по рукописному почерку.

Для прогнозирования потребуется загрузить исследуемое изображение, для чего потребуется ввести имя файла, хранящего оригинал изображения. После загрузки изображения, программное средство, с помощью описанных выше функций сможет обработать, а затем и спрогнозировать, исходя из обучения, какие черты характера присущи автору рукописного текста.

```

def barometer(image):

    global PEN_PRESSURE

    image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    h, w = image.shape[:]
    inverted = image
    for x in range(h):
        for y in range(w):
            inverted[x][y] = 255 - image[x][y]

    filtered = bilateralFilter(inverted, 3)

    ret, thresh = cv2.threshold(filtered, 100, 255, cv2.THRESH_TOZERO)

    total_intensity = 0
    pixel_count = 0
    for x in range(h):
        for y in range(w):
            if(thresh[x][y] > 0):
                total_intensity += thresh[x][y]
                pixel_count += 1

    average_intensity = float(total_intensity) / pixel_count
    PEN_PRESSURE = average_intensity

    return

```

Рисунок 7.14. Извлечение давления пера

Для примера нужно загрузить фотографию с изображением рукописного почерка. Фотография почерка представлена на рис. 7.15.

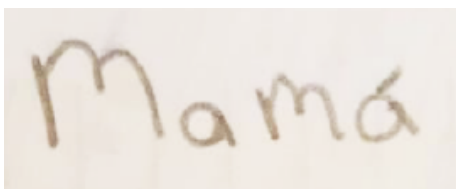


Рисунок 7.15. Пример изображения рукописного почерка

После загрузки фотографии с образцом рукописного почерка, программа выдает графологический анализ – результат, показанный на рисунке 7.16.

В результате работы программы очевидно, что характеристики почерка, показанного на фотографии, демонстрируют присущие человеку – владельцу почерка, следующие черты характера: наличие умственной энергии и силы воли; скромность; недисциплинированность (недостаток самодисциплины); некоммуникабельность; нелюбовь к шумным собраниям, склонность к одиночеству, и, вследствие этого, некоторая социальная изоляция.


```

[suiseiseki@Asus_N580GD ml-graphology-master]$ python train_predict.py
Info: label_list found.
Classifier 1 accuracy: 1.0
Classifier 2 accuracy: 1.0
Classifier 3 accuracy: 1.0
Classifier 4 accuracy: 1.0
Classifier 5 accuracy: 1.0
Classifier 6 accuracy: 1.0
Classifier 7 accuracy: 1.0
Classifier 8 accuracy: 1.0
Enter file name to predict or z to exit: mam.png
Baseline Angle: DESCENDING
Top Margin: NARROW
Letter Size: SMALL
Line Spacing: BIG
Word Spacing: BIG
Pen Pressure: LIGHT
Slant: IRREGULAR
Emotional Stability: [0.]
Mental Energy or Will Power: [1.]
Modesty: [1.]
Personal Harmony and Flexibility: [0.]
Lack of Discipline: [1.]
Poor Concentration: [0.]
Non Communicativeness: [1.]
Social Isolation: [1.]
-----
Enter file name to predict or z to exit: z
[suiseiseki@Asus_N580GD ml-graphology-master]$ 

```

Рисунок 7.16. Результат работы разработки авторов – программы ML_Graphology

Судя по почерку, его автору не присущи такие черты характера, как: эмоциональная устойчивость; личная гармония и гибкость; зато очевидны невнимательность и проблемы с концентрацией на каком-то одном деле.

Таким образом, разработано импортозамещающее программное средство распознавания черт характера, в том числе и деструктивных наклонностей, человеческой личности по его рукописному почерку, на основе нейросетевых технологий *ML_Graphology*. ПО обеспечивает такие почерковедческие функции, как определение: базового угла написания букв; верхнего поля текста; размера письма; межстрочного интервала; межсимвольного интервала и интервала между словами; давления пера, угла наклона текста и др.

Поскольку авторами было установлено, что на данный момент на российском рынке не представлено ни одного аналога подобного Web-приложения, то разработка приобретает особую ценность.

Демонстрация работоспособности разработанного программного средства позволяет сделать вывод о возможности его применения, в том числе, и в криминалистике, в процессе судебно-почерковедческой экспертизы.